Ciência de Dados em R

Curso-R

Última atualização: 14/07/2021

Contents

Sobre **5**

1 Instalação **11**

1.1 Instalação do R 11

1.2 Instalação do RStudio 13

1.3 Instalação de softwares adicionais 13

2 RStudio **25**

2.1 Telas 25

2.2 Atalhos 27

2.3 Projetos 28

2.4 Git e versionamento 33

2.5 Cheatsheets 34

2.6 Addins 36

2.7 Snippets 36

3 R Básico **39**

3.1 Pedindo Ajuda 39

3.2 R como calculadora 43

3.3 Objetos e funções 45

3.4 Data frames 48

3.5 Classes 49

3.6 Vetores 52

3.7 Testes lógicos 56

3.8 Valores especiais 61
3.9 Listas 64
3.10 Mais sobre data frames 67
3.11 Mais sobre funções 74
3.12 Controle de Fluxo 79
3.13 Outros tópicos 86

4 Pacotes **97**
4.1 Instalação de pacotes 97
4.2 Tidyverse 99

5 Importação **105**
5.1 Caminhos 105
5.2 O pacote readr 106
5.3 Os pacotes readxl e writexl 115
5.4 haven 118

6 Pipe **121**
6.1 O operador pipe 122
6.2 Outros operadores 125

7 Manipulação **129**
7.1 Tibbles 131
7.2 O pacote dplyr 135
7.3 O pacote tidyr 170
7.4 O pacote stringr 182
7.5 O pacote lubridate 193
7.6 O pacote forcats 199

8 Visualização **217**
8.1 O pacote ggplot2 217
8.2 Extensões do pacote ggplot2 259

9 Relatórios **265**

9.1 Markdown 266

9.2 R Markdown 272

10 Programação funcional (purrr) **285**

10.1 Iterações básicas 285

10.2 Iterações intermediárias 288

10.3 Iterações avançadas 289

10.4 Miscelânea 293

10.5 Exercícios 296

11 R avançado **299**

11.1 NSE 299

12 Miscelânea **305**

12.1 Arrumando banco de dados: o pacote janitor 305

13 Respostas **311**

13.1 R básico 311

13.2 Importação 337

13.3 Pipe 339

13.4 O pacote dplyr 340

13.5 O pacote stringr 353

13.6 O pacote lubridate 358

13.7 O pacote forcats 364

13.8 O pacote ggplot2 367

13.9 Programação funcional (purrr) 386

13.10NSE 391

Referências **399**

Sobre

Este livro está em **construção** e em **revisão aberta**. Fique à vontade para corrigir qualquer tipo de erro que encontrar no nosso material.

Todo o conteúdo deste livro é gratuito e aberto.

Autoria:

- Athos Damiani
- Beatriz Milz
- Caio Lente
- Daniel Falbel
- Fernando Correa
- Julio Trecenti
- Nicole Luduvice
- William Amorim

A Curso-R

A Curso-R é o braço de Educação da R6 Consultoria, uma empresa de treinamentos e consultoria em Ciência de Dados e linguagem de programação R. Surgimos em 2015, com cursos de introdução ao R no Programa de Cursos de Verão do Instituto de Matemática e Estatística da Universidade de São Paulo (IME-USP).

- Conheça os nossos cursos: https://www.curso-r.com/cursos/
- Conheça o nosso blog: https://www.curso-r.com/blog/
- Conheça o restante do nosso material: https://www.curso-r.com/material/
- Saiba mais sobre a Curso-R: https://www.curso-r.com/sobre/

O R e o RStudio

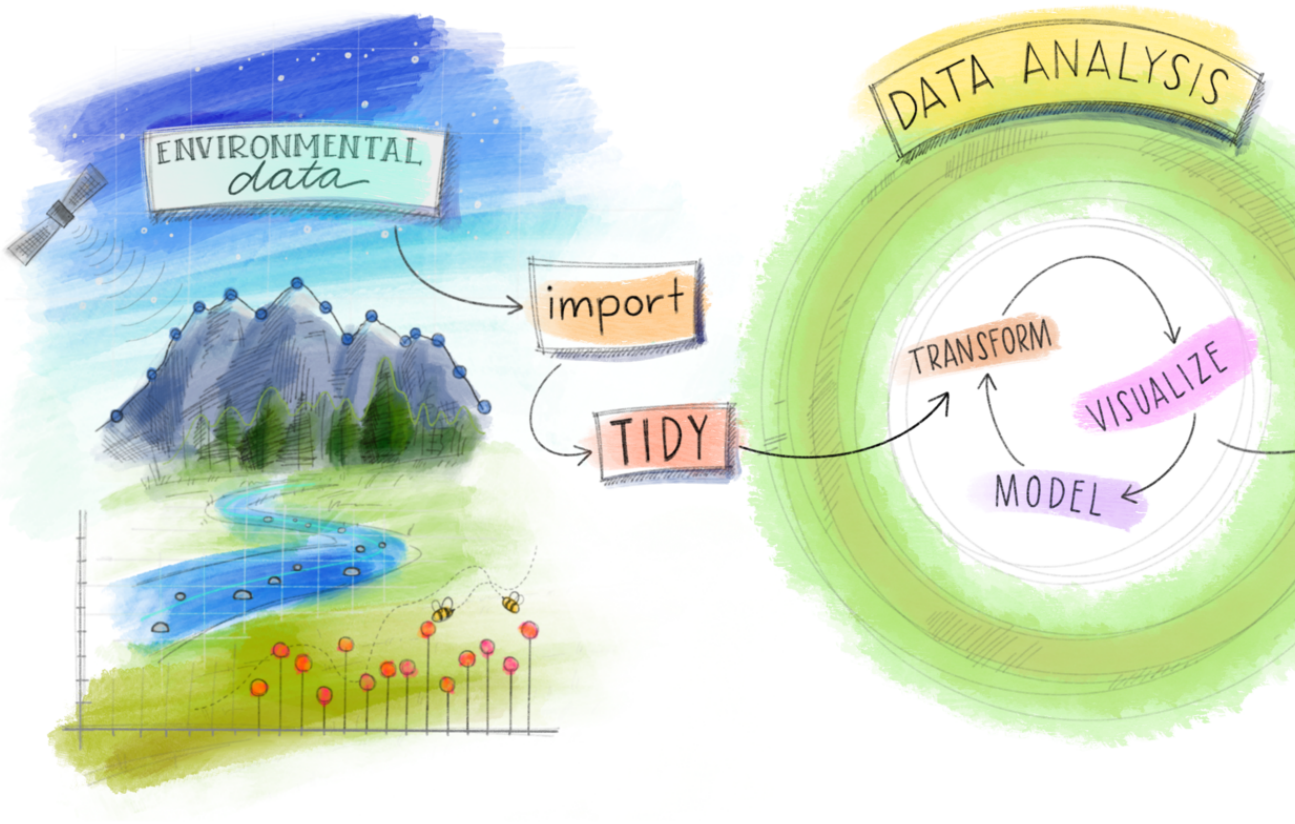
O R é uma linguagem de programação *open source* para análise de dados que fornece uma grande variedade de ferramentas estatísticas e gráficas.

O RStudio é o ambiente de desenvolvimento mais utilizado para programação em R. Ele possui diversas ferramentas que facilitam a utilização do R para se fazer análise de dados.

Este livro

Este livro tem como objetivo guiar o leitor por todas as etapas de uma análise de dados, utilizando o R como ferramenta. Nele, você encontrará ferramentas para importar, arrumar, manipular, visualizar e modelar bases de dados. E também ferramentas para comunicar e automatizar resultados.

\begin{figure}



{

}

\caption{Arte por Allison Horst (**?**). Veja nas Referências onde encontrá-la.}
\end{figure}

Comunidade

Seja bem-vinda(o) ao #rstats.

Compartilhe seu desenvolvimento! Poste seus avaços nas redes sociais,

compartilhe suas dúvidas em fóruns, procure e participe dos grupos e comunidades de programadores.

- Comunidade R-Ladies: Global e Brasil

- Fórum de dúvidas da Curso-R: https://discourse.curso-r.com/

- Grupo de divulgação da Curso-R no Telegram: https://t.me/r6cursor

- Grupo R Brasil no Telegram: https://t.me/rbrasiloficial

\begin{figure}

R
#rstats
#RLadies
#dataviz
WELCOME
Horst '18

{

}

\caption{Arte por Allison Horst (**?**). Veja nas Referências onde encontrá-la.}
\end{figure}

Agradecimentos

- Haydee Svab - Revisão do conteúdo inicial sobre Markdown.
- Sugestões feitas durante transmissões na Twitch: AugustoCL_ e pouchewar .

Chapter 1

Instalação

Nesta seção, abordaremos como instalar o R e o RStudio no Linux e no Windows. Também discutiremos sobre a instalação de pacotes no R.

1.1 Instalação do R

A instalação padrão do R é feita a partir do CRAN, uma rede de servidores espalhada pelo mundo que armazena versões idênticas e atualizadas de códigos e documentações para o R.

> Sempre que for instalar algo do CRAN, utilize o servidor (*mirror*) mais próximo de você.

1.1.1 No Windows

Para instalar o R no Windows, siga os seguintes passos:

1. Acesse o CRAN: https://www.r-project.org/
2. No menu à esquerda, encontre a opção **Download** e clique em **CRAN**.
3. Escolha a opção de servidor (*mirror*) mais próxima de você.
4. Clique na opção **base**.
5. Na nova página, clique em **Download R x.x.x for Windows*, sendo x.x.x o número da versão que será baixada. Se você teve algum problema com o download, tente escolher outro servidor no passo 3.

6. Feito o download, clique duas vezes no arquivo baixado e siga as instruções para instalação.

   Na etapa de escolher a pasta de destino da instalação, se você escolher um local que não esteja dentro da sua pasta de usuário, você precisará de acesso de administrador. Se escolher uma pasta dentro da sua paste de usuário, não precisará.

1.1.2 No Linux

Como a instalação no Linux depende da distribuição utilizada e, em geral, pessoas que utilizam Linux são mais experientes, vamos informar apenas as coordenadas até as instruções/arquivos. Se você tiver alguma dificuldade durante o processo, por favor envie a sua dúvida para a nossa comunidade. Faremos o possível para ajudar.

1. Acesse o CRAN: https://cran.r-project.org/
2. No menu à esquerda, encontre a opção **Download** e clique em **CRAN**.
3. Escolha a opção de servidor (*mirror*) mais próxima de você.
4. Clique em *Download R for Linux*.
5. Clique no link referente à distribuição que você utiliza.
6. Siga as instruções contidas na página para instalar o R. Se você teve algum problema com o download, tente escolher outro servidor no passo 3.

1.1.3 No MacOS

Para instalar o R no MacOS, siga os seguintes passos:

1. Acesse o CRAN: https://www.r-project.org/
2. No menu à esquerda, encontre a opção **Download** e clique em **CRAN**.
3. Escolha a opção de servidor (*mirror*) mais próxima de você.
4. Na nova página, clique em **Download R for (Mac) OS X*.
5. Clique na versão do R que você quer baixar (geralmente queremos baixar a mais recente). O objetivo aqui é baixar um arquivo do tipo "R-x.x.x.pkg", sendo x.x.x o número da versão que vamos instalar. Se você teve algum problema com o download, tente escolher outro servidor no passo 3.
6. Feito o download, clique duas vezes no arquivo baixado e siga as instruções para instalação.

1.2 Instalação do RStudio

Agora vamos instalar a versão *open source* do RStudio, a IDE que utilizaremos para escrever e executar códigos em R.

Para instalar o RStudio no Windows, siga os seguintes passos:

1. Acesse a página de downloads da RStudio: https://rstudio.com/products/rstudio/download/#download

- Se você tiver acesso administrador, baixe a versão referente ao seu sistema operacional que está na lista de *All Installers*.
- Se você não tiver acesso de administrador, baixe a versão referente ao seu sistema operacional que está na lista de *Zip/Tarballs*.

Instalando se você for administrador

2. Clique duas vezes no arquivo que você baixou da página do RStudio e siga as instruções de instalação.

Instalação se você não for administrador

2. Descompacte o arquivo baixado no passo anterior. Geralmente isso pode ser feito clicando no arquivo compactado com o botão direito do mouse e clicando na opção *descompactar* ou *extrair*.
3. Após a descompactação ter sido finalizada, você terá uma pasta chamada: `RStudio-x.x.x`, em que x.x.x é o número da versão baixada. Abra essa pasta e entre na subpasta com nome `bin`.
4. Procure pelo arquivo chamado `rstudio` e clique duas vezes. Isso abrirá o RStudio. Recomendamos fixar o programa na barra de tarefas para não precisar repetir essa etapa sempre que for abrir o programa.

Observação: se você excluir a pasta que extraímos, o RStudio irá parar de funcionar.

1.3 Instalação de softwares adicionais

Os softwares descritos anteriormente são suficientes para realizar as tarefas básicas em R. Porém, principalmente quando tratamos de relatórios com R, alguns softwares adicionais podem ser necessários em casos específicos! A seguir citamos alguns destes softwares, em quais casos eles são necessários e/ou úteis, e instruções sobre como instalá-los.

1.3.1 LaTeX

O LaTeX é um software para renderização de documentos PDF e também uma linguagem de programação. É bastante usada na academia para produção de artigos científicos.

No R, o LaTeX aparece quando queremos gerar relatórios e apresentações em PDF, a partir de um documento RMarkdown. Na prática, o RMarkdown é transformado para LaTeX através do software *Pandoc*[1], que por sua vez é processado para gerar o PDF. Portanto, para gerar um relatório em PDF através do RMarkdown, precisamos ter alguma ferramenta de LaTeX instalada.

A recomendação das pessoas que desenvolvem o RMarkdown é a utilização do `tinytex`, uma distribuição leve do LaTeX.

```
install.packages('tinytex')
tinytex::install_tinytex()  # instalar o TinyTeX
```

Caso você encontre algum problema para instalar o TinyTex, também é possível instalar outros editores de LaTex. A seguir estão apresentadas algumas opções!

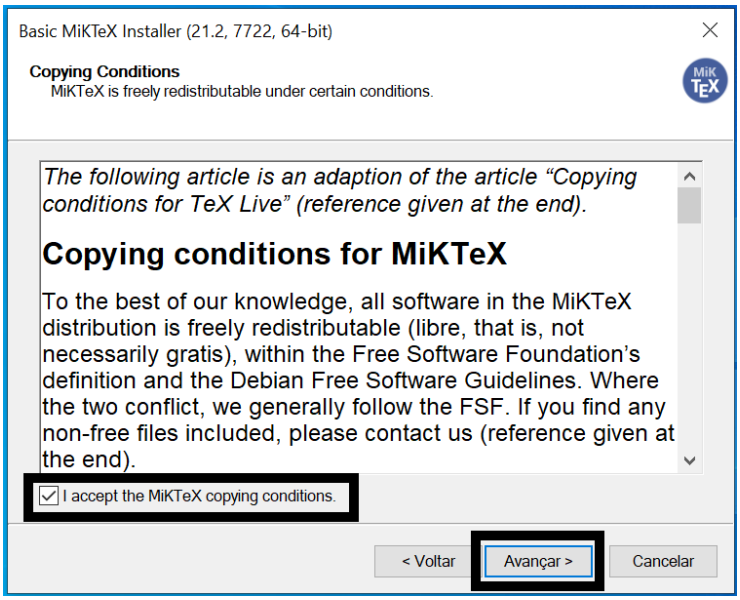
1.3.1.1 No Windows

Uma distribuição de LaTeX que apresenta versão para Windows é o MiKTeX.

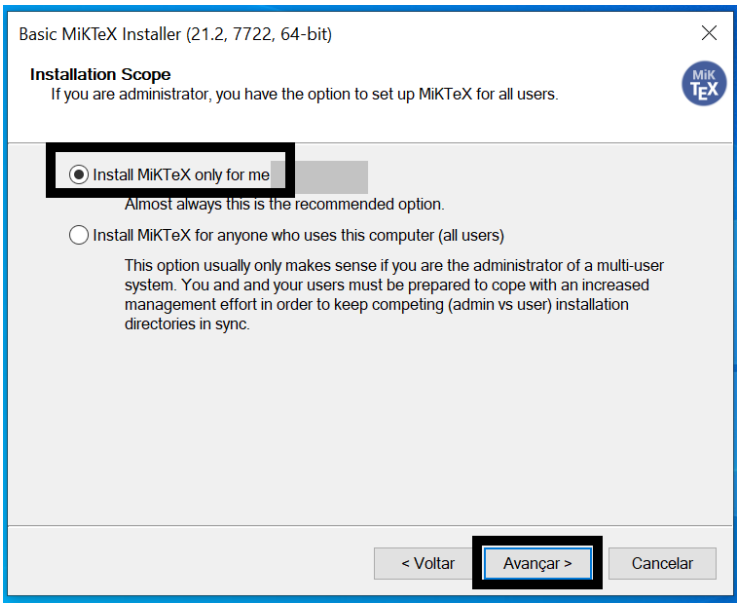
1. Para instalá-lo, primeiramente faça o download do arquivo `.exe` através deste link, escolhendo a versão de instalação para Windows.

2. Abra o arquivo baixado. Uma janela irá abrir, e a primeira etapa de instalação é a "Copying Conditions". Nesta etapa você deve selecionar a opção **"I accept the MiKTeX copying conditions"** e clicar em **"Avançar"**.

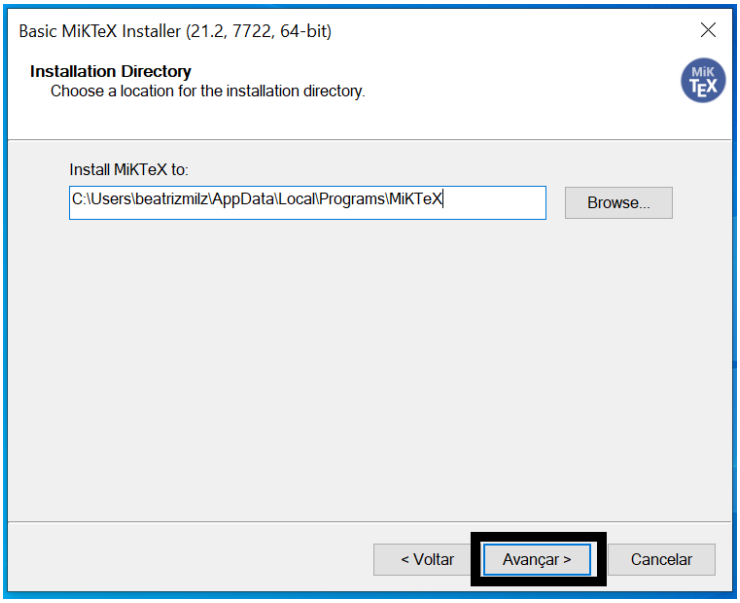
[1] O Pandoc já vem dentro do próprio RStudio, então não é necessário instalar este *software*.



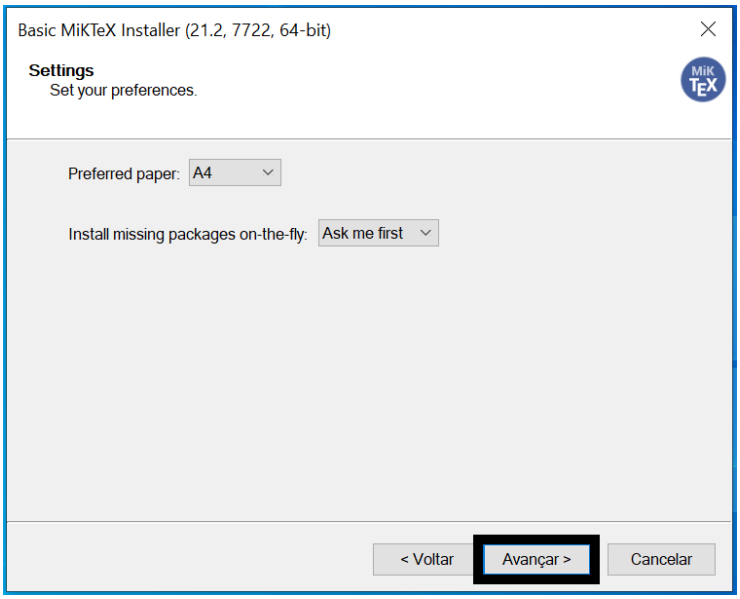
3. A próxima etapa da instalação, chamada de "Installation Scope", você deve escolher em qual usuário o software deve ser instalado. Selecione a primeira opção (**"Install MiKTeX only for me.") para que o MiKTeX seja instalado apenas no seu usuário atual no computador. Essa é a opção recomendada. Então clique em "Avançar"**.



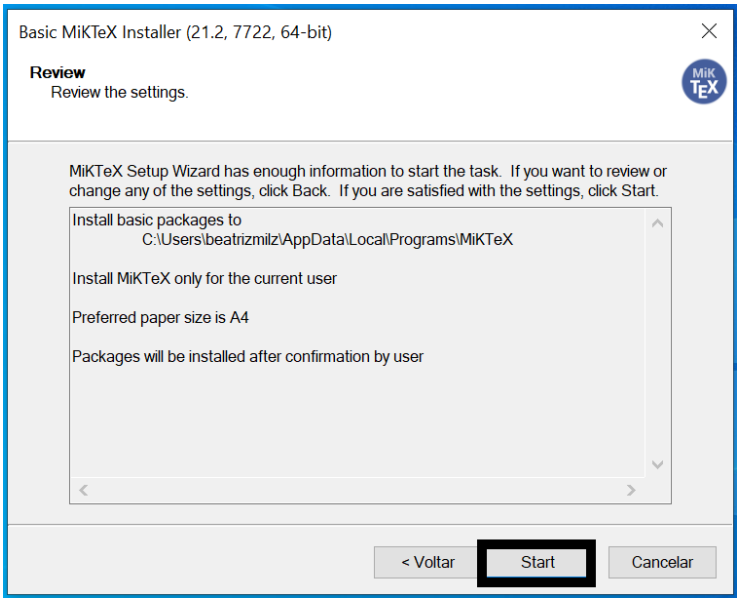
4. Na etapa “Installation Directory”, escolha o diretório em que o MiKTeX será instalado. Caso tenha dúvidas sobre isso, não altere nada e utilize o caminho padrão de instalação. Então clique em **“Avançar”**.



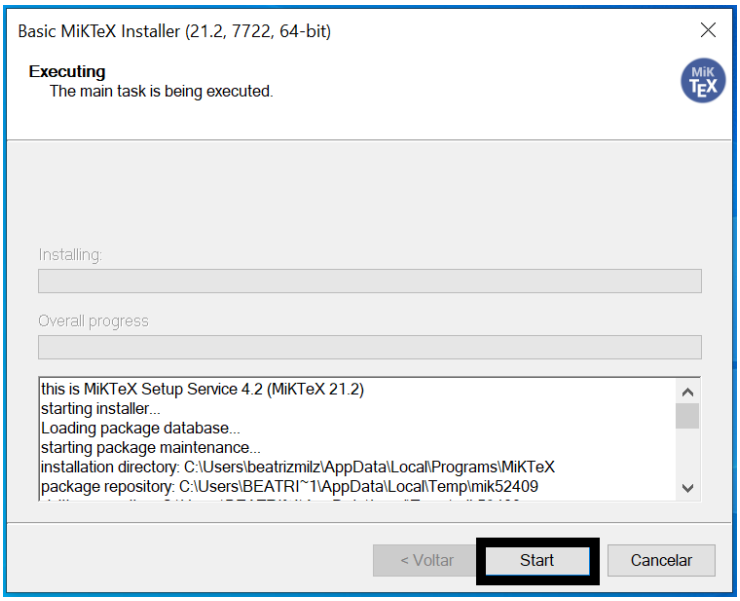
5. Na etapa “Settings”, utilize as configurações padrão e clique em **“Avançar”**.



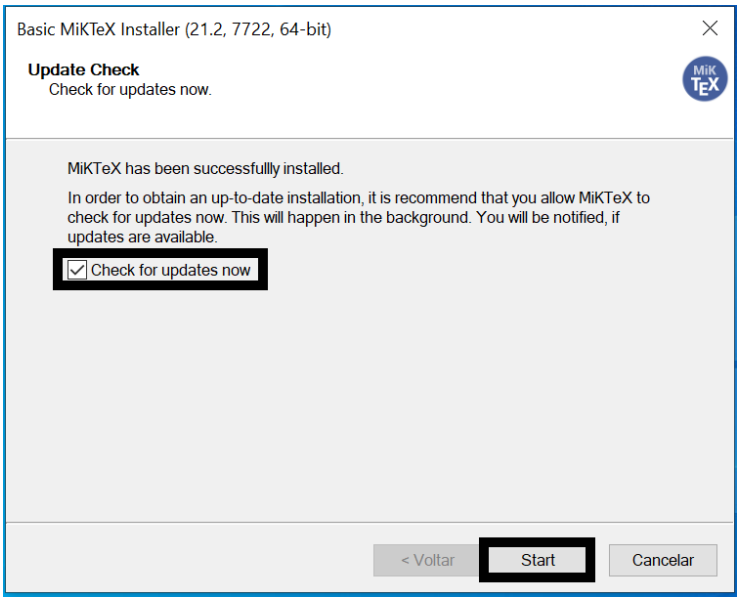
6. Na etapa “Review”, é apresentado as configurações escolhidas anteriormente. Clique em **Start** para que a instalação seja começada.



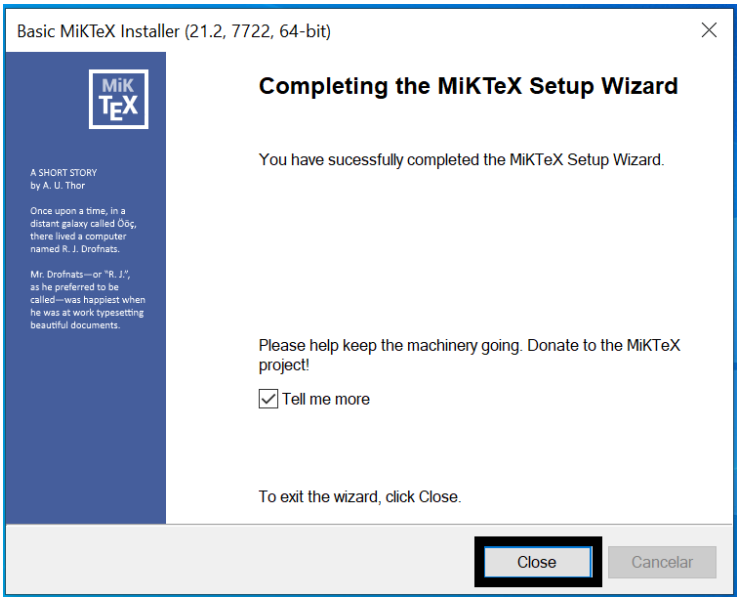
7. O processo de instalação será realizado, e pode demorar alguns minutos. Duas barras verdes aparecerão para demonstrar a porcentagem da instalação já realizada. Quando essa etapa terminar, as barras ficarão cinza e o você deverá clicar no botão **Start**.



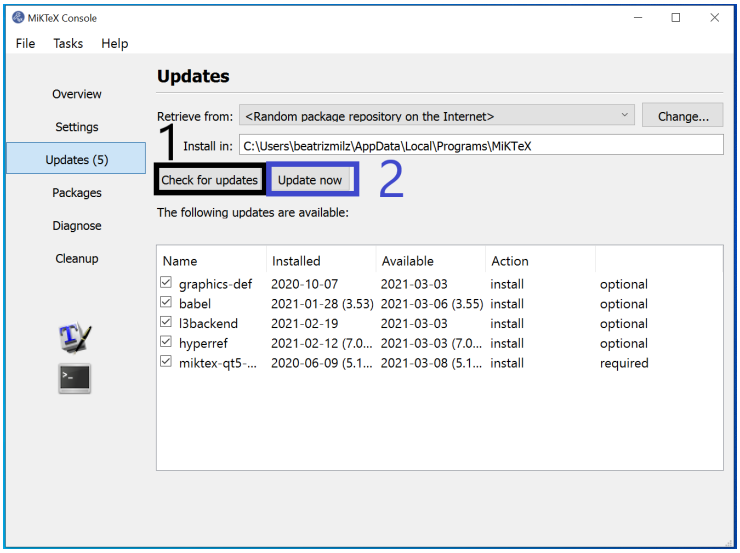
8. Na etapa “Update Check”, apresentará a opção de buscar por atualizações. Mantenha o botão **“Check for updates now”** selecionado, e clique no botão **Start**.



9. A próxima janela mostrará uma mensagem dizendo que a instalação foi completa. Clique no botão **Close** para fechar essa janela.



10. Abra o MiKTeX console através do menu Iniciar do Windows. No menu lateral, escolha a opção "Updates". Clique em **"Check for updates"**, e o MiKTeX buscará informações sobre as possíveis atualizações necessárias. Então clique em **"Update now"**, para que as atualizações sejam baixadas e instaladas. Essa etapa também pode demorar um pouco!



11. Após instalar as atualizações, o MiKTeX avisará que ele precisa ser fechado. Abra-o novamente. Após este processo, o MiKTex estará pronto para uso.

1.3.1.2 No Linux

No linux, a forma mais direta de garantir que temos o LaTeX na máquina com todas as dependências é instalando o `texlive-full`. No Ubuntu, basta rodar

```
sudo apt install texlive-full
```

> **Importante**: o texlive-full é uma fora "bruta" de instalar tudo do LaTeX, e pode ocupar mais do que 5GB do seu disco! Então tome cuidado e procure alternativas, como as descritas neste site.

1.3.1.3 No MacOS

Uma distribuição de LaTeX que apresenta versão para MacOS é o MikTex. Para instalá-lo, primeiramente faça o download do arquivo `.dmg` através deste

link. Este tutorial apresenta todos os passos de instalação no MacOS, porém abaixo também vamos descrever os passos:

1. Faça o download do arquivo `.dmg` através deste link, escolhendo a versão de instalação para MacOS.
2. Abra o arquivo baixado. Uma janela irá abrir, e você deve arrastar o ícone "MiKTeX Console" para a pasta "Applications" (como mostrado na animação abaixo, sendo necessário arrastar apenas uma vez).
3. No Launchpad (menu de arquivos instalados), procure o ícone do "MiKTeX Console" e abra-o. Você também pode usar a pesquisa do MacOS (utilizando o atalho Command + Barra de espaço), pesquisando por MiKTeX para abrir o software.
4. Ao abrir pela primeira vez, o MiKTeX solicitará que você selecione uma opção para que a instalação seja completada. Selecione **"Finish private setup"**, e isso fará com que o MiKTeX esteja disponível apenas no seu usuário do computador. Após selecionar essa opcão, ele terminará a configuração, e aparecerá uma mensagem dizendo que o MiKTex será reiniciado. Espere até que ele reinicie. Ao final, aparecerá uma mensagem dizendo que é necessário fazer atualizações no MiKTeX.
5. No menu lateral, escolha a opção "Updates". Clique em **"Check for updates"**, e o MiKTex buscará informações sobre as possíveis atualizações necessárias. Então clique em **"Update now"**, para que as atualizações sejam baixadas e instaladas.
6. Após instalar e atualizar, o MiKTeX está pronto para uso!

1.3.2 Office

No R, os arquivos do pacote office também são gerados a partir do Pandoc. Por isso, não é necessário instalar nenhum software adicional para gerar os documentos.

No entanto, para abrir documentos do Office, como `.pptx` e `.docx`, é necessário ter o Office ou alguma alternativa aberta instalados.

Além disso, por conta das particularidades desses tipos de documentos, muitas vezes é necessário instalar pacotes adicionais, que não vêm com o `{rmarkdown}`. Abaixo, segue uma lista de pacotes usualmente utilizados nesse contexto:

- `{officedown}`.
- `{officer}`, que é uma dependência do `{officedown}`.
- `{flextable}`, para geração de tabelas.

- `{svglite}`, para renderizar gráficos em SVG.

  Uma dica adicional: É possível gerar gráficos *editáveis* nos documentos Office! Dessa forma, você pode gerar o relatório bruto e depois editar manualmente. Para isso, siga o tutorial deste vídeo do YouTube. Referência: link.

1.3.2.1 No Windows

O LibreOffice pode ser instalado através deste link, escolhendo o sistema operacional "Windows" e clicando em "Baixar" e executando o arquivo baixado.

1.3.2.2 No Linux

No linux, a forma mais fácil de abrir documentos provenientes do Office é utilizando o LibreOffice:

```
sudo apt install libreoffice
```

O LibreOffice possui algumas limitações quando comparado ao software da Microsoft. Por exemplo, as tabelas que saem bonitas no Office podem ficar desformatadas no LibreOffice.

o LibreOffice 7.1 não está ainda no aptitude. Se quiser instalar essa versão, é possível baixar o arquivo `.deb` no site da ferramenta.

1.3.2.3 No MacOS

O sistema operaciona MacOS disponibiliza o iWork, que é composto pelas ferramentas Numbers, Pages e Keynote. Esses programas possibilitam abrir arquivos provenientes do Office.

Outra opção é a utilização do LibreOffice.

1. Primeiro, faça o download do arquivo `.dmg` de instalação, através deste link, escolhendo o sistema operacional "MacOS" e clicando em "Baixar".
2. Após baixar o arquivo, abra-o. Uma janela irá abrir (como na imagem a seguir). Você deve arrastar o ícone "LibreOffice" para a pasta "Applications" (como mostrado na animação abaixo, sendo necessário arrastar apenas uma vez).
3. O LibreOffice será instalado e você poderá utilizá-lo abrindo através do Launchpad (menu de arquivos instalados), ou também utilizando a pesquisa do Mac (utilizando o atalho Command + Barra de espaço), pesquisando por LibreOffice para abrir o software.

1.3.3 Pagedown e Chrome

O `{pagedown}` é uma alternativa recente para produzir arquivos em HTML utilizando o `paged.js`, uma biblioteca em JavaScript que torna um arquivo HTML muito parecido com um PDF. Com a ajuda do Chrome, é possível, inclusive, exportar o arquivo para PDF.

O `{pagedown}`, portanto, apresenta o melhor de dois mundos: a praticidade da extensão HTML e a portabilidade de arquivos PDF. Dessa forma, é possível gerar documentos PDF com formatos agradáveis sem depender do LaTeX.

No entanto, para gerar documentos PDF, é preciso utilizar um navegador. O navegador recomendado pelo `{pagedown}` é o Chrome, tanto que o pacote já vem com a função `pagedown::chrome_print()`, que pode ser utilizada para transformar um arquivo HTML ou RMarkdown em PDF.

> Se você já tiver o Chrome instalado na sua máquina, provavelmente tudo funcionará sem problemas. Se não tiver e não quiser instalar, é possível instalar somente o headless Chrome.
>
> No linux, uma alternativa é instalar o projeto Chromium, que é quase que totalmente open source.

1.3.4 Renderizando HTML widgets

Os HTML widgets são formas de ligar bibliotecas do JavaScript com pacotes do R. Isso permite gerar relatórios dinâmicos em HTML diretamente do R. O pacote `{htmlwidgets}` possui um conjunto de melhores práticas para criação dessas soluções.

No entanto, é comum utilizar HTML widgets em relatórios estáticos. Para isso, o R precisa de uma estratégia para tirar fotos estáticas dos widgets. Isso é realizado através do pacote `{webshot}`.

Você pode instalar o `{webshot}` executando o código abaixo:

```
install.packages("webshot")
```

Para que o `{webshot}` funcione, no entanto, também é necessário ter o PhantomJS instalado na máquina. O PhantomJS é basicamente um navegador que roda totalmente no plano de fundo, possibilitando a captura de tela.

Para instalar o PhantomJS, execute o código abaixo:

```
webshot::install_phantomjs()
```

1.3.5 Blogdown

Para criar blogs utilizando o pacote `{blogdown}`, também é necessário instalar o Hugo, programa que renderiza sites estáticos.

Felizmente, essa tarefa é bem tranquila pois o `{blogdown}` já vem com um helper para isso. Basta executar o código abaixo e reiniciar a sua sessão do R para que o Hugo seja instalado:

```
blogdown::install_hugo()
```

1.3.6 Ferramentas de desenvolvimento

Em algumas situações (como por exemplo, no desenvolvimento de pacotes), é solicitado a instalação de ferramentas de desenvolvimento. Essas ferramentas dependem do sistema operacional utilizado.

No caso do desenvolvimento de pacotes, você pode verificar se já possui as ferramentas de desenvolvimento instaladas usando a seguinte função: `devtools::has_devel()`. Caso a mensagem retornada seja *"Your system is ready to build packages!"*, significa que você já tem as ferramentas necessárias instaladas.

```
devtools::has_devel()
#> Your system is ready to build packages!
```

1.3.6.1 No Windows - Rtools

O software Rtools é apenas necessário para computadores que utilizem o sistema operacional Windows. Para instalá-lo, clique neste link e faça o download do arquivo instalador referente à sua versão do Windows (32 ou 64 bits). Abra o arquivo baixado para realizar a instalação.

1.3.6.2 No Linux - r-base-dev

No Linux, caso seja necessário, você também pode instalar o pacote `r-base-dev`. Por exemplo, no Ubuntu você pode instalá-lo executando o seguinte código no terminal:

```
sudo apt install r-base-dev
```

1.3.6.3 No MacOS - Xcode command line tools

O Xcode command line tools é uma ferramenta de desenvolvimento oficial da Apple. Para instalar essa ferramenta, primeiramente é preciso acessar o Terminal. Você pode encontrar o Terminal procurando na lista de programas instalados (*Launchpad*), ou então na opção de busca (usando o atalho: **command + barra de espaço**) e escrevendo Terminal.

No terminal, escreva o seguinte código e aperte Enter, o que iniciará a instalação:

```
xcode-select --install
```

Siga as instruções apresentadas para instalar.

Caso a seguinte mensagem apareça, significa que você já tem a ferramenta instalada: `error: command line tools are already installed, use "Software Update" to install updates`.

Caso queira confirmar que a instalação foi bem sucedida, execute no terminal o código a seguir. Caso a mensagem de retorno seja um caminho (por exemplo: `/Library/Developer/CommandLineTools`), significa que a instalação foi bem sucedida, e esse é o caminho onde a ferramenta foi instalada.

```
xcode-select -p
```

1.3.7 Git

O Git é um programa para linha de comando que possibilita realizar o controle de versões dos arquivos em um diretório (ou seja, em uma pasta). Para instalar esse sistema de controle de versões distribuído, acesse a página sobre Download do Git, escolha o seu sistema operacional, siga as instruções apresentadas para instalação.

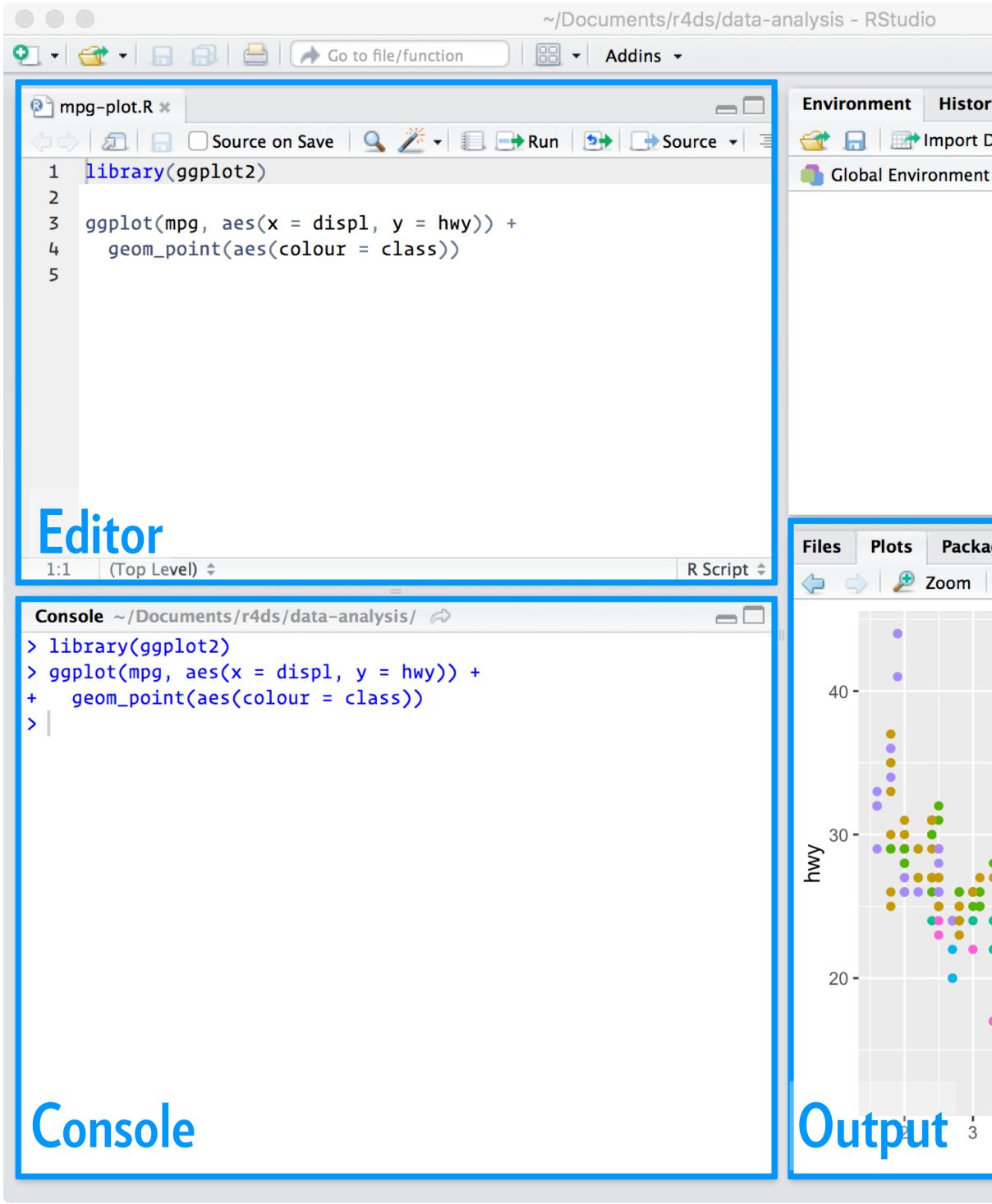
Chapter 2

RStudio

O RStudio é sem dúvidas o mais completo ambiente de desenvolvimento para programação em R. Descubra aqui as funcionalidades do RStudio que nos permitem escrever códigos e analisar resultados de forma muito mais eficiente.

2.1 Telas

Ao abrir o RStudio, você verá 4 quadrantes. Observe a figura abaixo.


~/Documents/r4ds/data-analysis - RStudio
Go to file/function
Addins
mpg-plot.R
Source on Save
Run
Source
1 library(ggplot2)
2
3 ggplot(mpg, aes(x = displ, y = hwy)) +
4 geom_point(aes(colour = class))
5
Editor
1:1
(Top Level)
R Script
Environment
Histor
Import D
Global Environment
Files
Plots
Packa
Zoom
Console ~/Documents/r4ds/data-analysis/
> library(ggplot2)
> ggplot(mpg, aes(x = displ, y = hwy)) +
+ geom_point(aes(colour = class))
>
Console
40
30
20
hwy
Output
3

Esses quadrantes representam o **editor**, o **console**, o **environment** e o **output**. Eles vêm nesta ordem, mas você pode organizá-los da forma que preferir acessando a seção *Pane Layout* da opção `Global options...` no menu `Tools`.

O editor e o console são os dois principais painéis do RStudio. Passaremos a maior parte do tempo neles.

- **Editor/Scripts**: é onde escrevemos nossos códigos. Repare que o RStudio colore algumas palavras e símbolos para facilitar a leitura do código.
- **Console**: é onde rodamos o código e recebemos as saídas. O R vive aqui!

Os demais painéis são auxiliares. O objetivo deles é facilitar pequenas tarefas que fazem parte tanto da programação quanto da análise de dados, como olhar a documentação de funções, analisar os objetos criados em uma sessão do R, procurar e organizar os arquivos que compõem a nossa análise, armazenar e analisar os gráficos criados e muito mais.

- **Environment**: painel com todos os objetos criados na sessão.
- **History**: painel com um histórico dos comandos rodados.
- **Files**: mostra os arquivos no diretório de trabalho. É possível navegar entre diretórios.
- **Plots**: painel onde os gráficos serão apresentados.
- **Packages**: apresenta todos os pacotes instalados e carregados.
- **Help**: janela onde a documentação das funções serão apresentadas.
- **Viewer**: painel onde relatórios e dashboards serão apresentados.

2.2 Atalhos

Conhecer os atalhos do teclado ajuda bastante quando estamos programando no RStudio. Veja os principais:

- **CTRL+ENTER**: avalia a linha selecionada no script. O atalho mais utilizado.
- **ALT+-**: cria no script um sinal de atribuição (<-). Você o usará o tempo todo.
- **CTRL+SHIFT+M**: (%>%) operador *pipe*. Guarde esse atalho, você o usará bastante.
- **CTRL+1**: altera cursor para o script.
- **CTRL+2**: altera cursor para o console.
- **CTRL+ALT+I**: cria um chunk no R Markdown.
- **CTRL+SHIFT+K**: compila um arquivo no R Markdown.
- **ALT+SHIFT+K**: janela com todos os atalhos disponíveis.

No MacBook, os atalhos geralmente são os mesmos, substituindo o **CTRL** por **command** e o **ALT** por **option**.

2.3 Projetos

Uma funcionalidade muito importante do RStudio é a possibilidade de criar **projetos**.

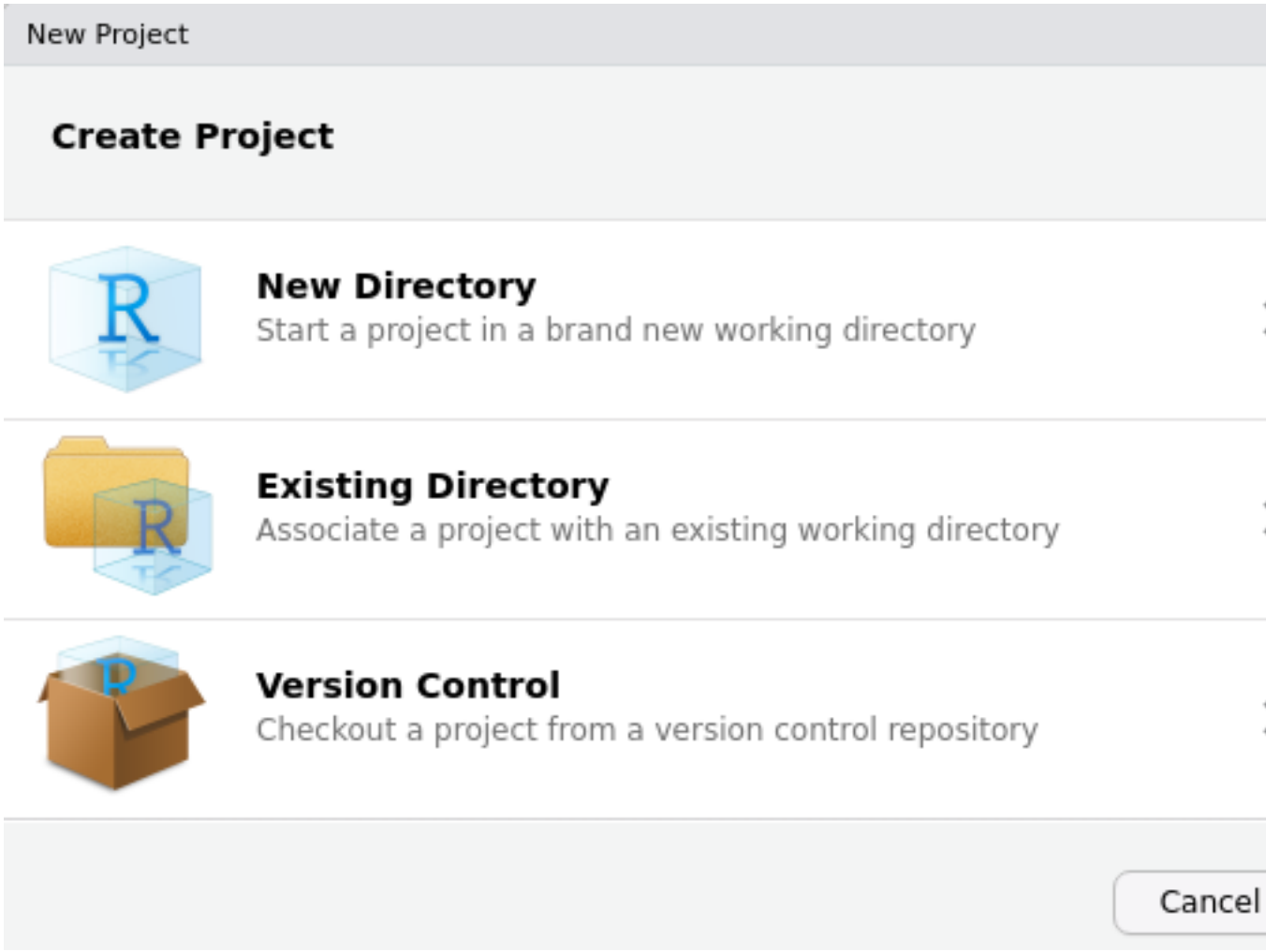
Um projeto nada mais é do que uma pasta no seu computador. Nessa pasta, estarão todos os arquivos que você usurá ou criará na sua análise.

A principal razão de utilizarmos projetos é **organização**. Com eles, fica muito mais fácil importar bases de dados para dentro do R, criar análises reprodutíveis e compartilhar o nosso trabalho.

> Você que está começando agora no R, já se habitue a criar um novo projeto para cada nova análise que for fazer.

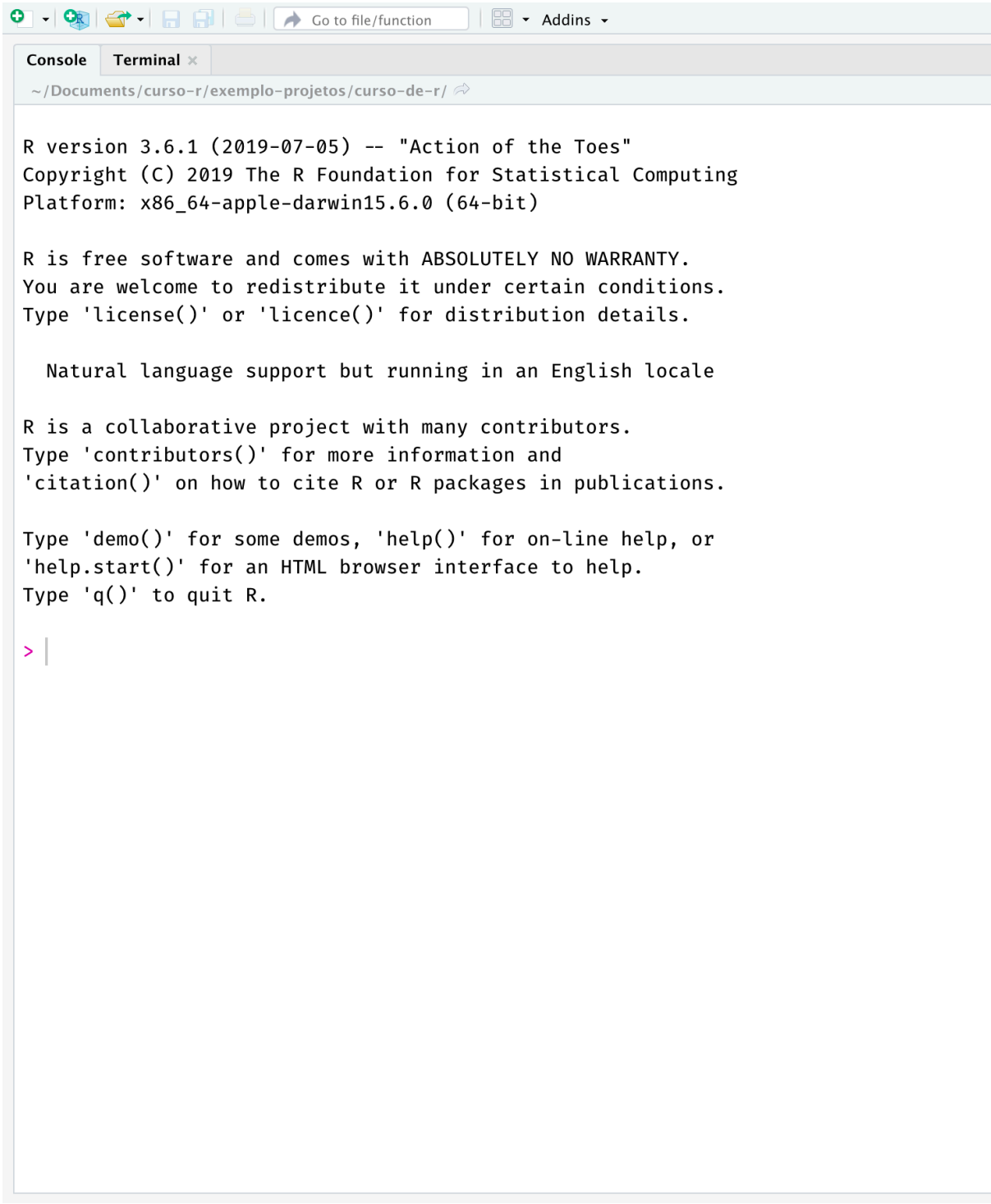
Para criar um projeto, clique em `New Project...` no Menu `File`. Na caixa de diálogo que aparecerá, clique em `New Directory` para criar o projeto em uma nova pasta ou `Existing Directory` para criar em uma pasta existente.

Se você tiver o `Git` instalado, você também pode usar projetos para conectar com repositórios do Github e outras plataformas de desenvolvimento. Para isso, basta clicar em `Version Control`.



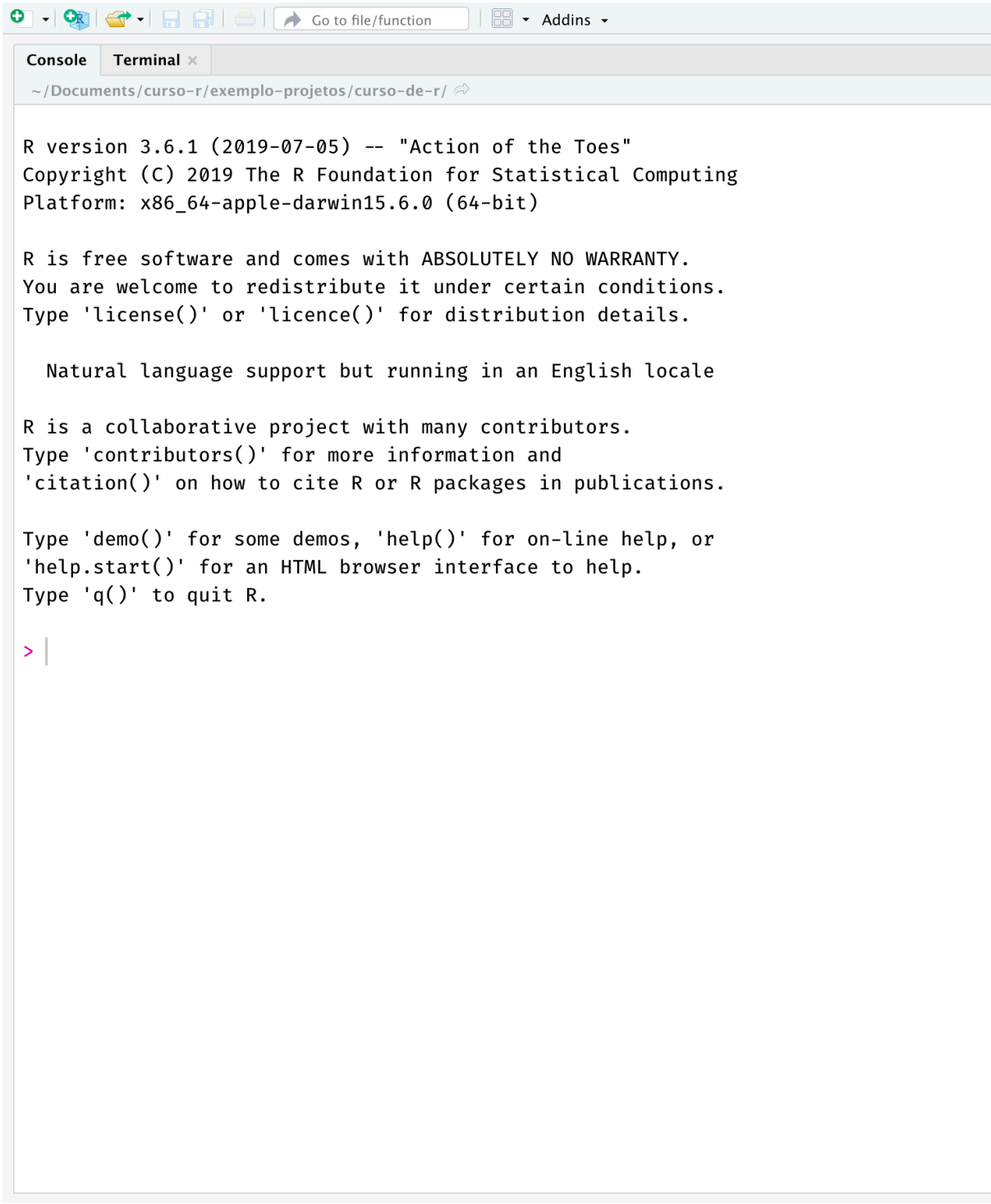
Criando um projeto, o RStudio criará na pasta escolhida um arquivo `nome-do-projeto.Rproj`. Você pode usar esse arquivo para iniciar o RStudio já com o respectivo projeto aberto.

Quando um projeto estiver aberto no RStudio, o seu nome aparecerá no canto superior direito da tela. Na aba **Files**, aparecerão todos os arquivos contidos no projeto.


Go to file/function
Addins
Console
Terminal
~/Documents/curso-r/exemplo-projetos/curso-de-r/
R version 3.6.1 (2019-07-05) -- "Action of the Toes"
Copyright (C) 2019 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin15.6.0 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>

Você pode criar livremente novas pastas dentro da pasta do projeto. Por padrão, o R sempre começará a procurar arquivos na pasta raiz do projeto (é a pasta que contem o `nome-do-projeto.Rproj`).

Uma maneira fácil de navegar entre projetos é utilizar o menu disponibilizado quando clicamos no nome do projeto. Veja a figura a seguir.


Go to file/function
Addins
Console
Terminal
~/Documents/curso-r/exemplo-projetos/curso-de-r/
R version 3.6.1 (2019-07-05) -- "Action of the Toes"
Copyright (C) 2019 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin15.6.0 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>

Nesse menu, além de podermos criar novos projetos ou abrir projetos já existentes, também temos um acesso rápido a projetos abertos recentemente. Basta clicar em qualquer um deles para trocar de projeto, isto é, deixar de trabalhar em uma análise e começar a trabalhar em outra.

A seguir, apresentamos algumas estruturas de organização de projetos no RStudio.

Estrutura 1. Por extensão de arquivo.

```
nome_do_projeto/
  - .Rprofile   # códigos para rodar assim que abrir o projeto
  - R/          # Código R, organizado com a-carrega.R, b-prepara bd.R, c-vis.R, d-modela, ...
  - RData/      # Dados em formato .RData
  - csv/        # Dados em .csv
  - png/        # gráficos em PNG
  - nome_do_projeto.Rproj
```

Estrutura 2. Típico projeto de análise estatística.

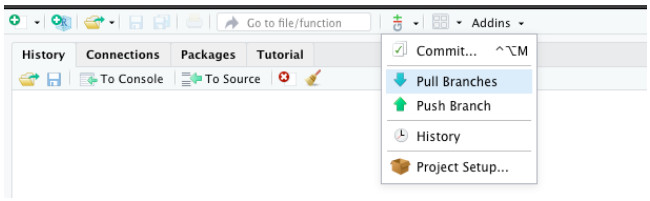
```
project/
  - README.Rmd   # Descrição do pacote
  - set-up.R     # Pacotes etc
  - R/           # Código R, organizado com 0-load.R, 1-tidy.R, 2-vis.R, ...
  - data/        # Dados (estruturados ou não)
  - figures/     # gráficos (pode ficar dentro de output/)
  - output/      # Relatórios em .Rmd, .tex etc
  - project.Rproj
```

Estrutura 3. Pacote do R.

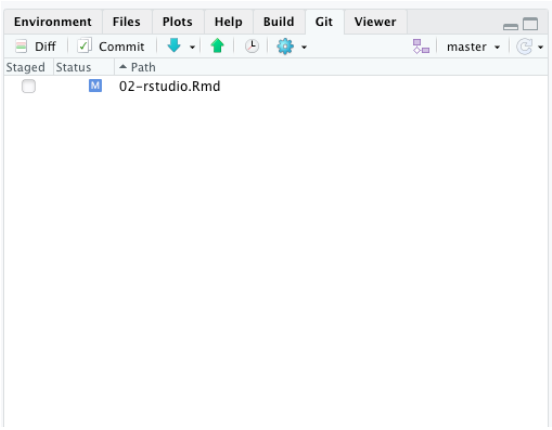
```
project/
  - README.md    # Descrição do pacote
  - DESCRIPTION  # Metadados estruturados do pacote e dependências
  - NAMESPACE    # importações e exportações do pacote
  - vignettes/   # Relatórios em .Rmd
  - R/           # Funções do R
  - data/        # Dados estruturados (tidy data)
  - data-raw/    # Dados não estruturados e arqs 0-load.R, 1-tidy.R, 2-vis.R, ...
  - project.Rproj
```

2.4 Git e versionamento

O RStudio possui funcionalidades para quem trabalha com o programa *Git* para versionar arquivos.



Além disso, quando você inicia o Git na pasta raiz de um projeto, o RStudio criará uma nova abinha chamada *Git*, onde você pode confirir os arquivos modificados que estão à espera de um *commit*.



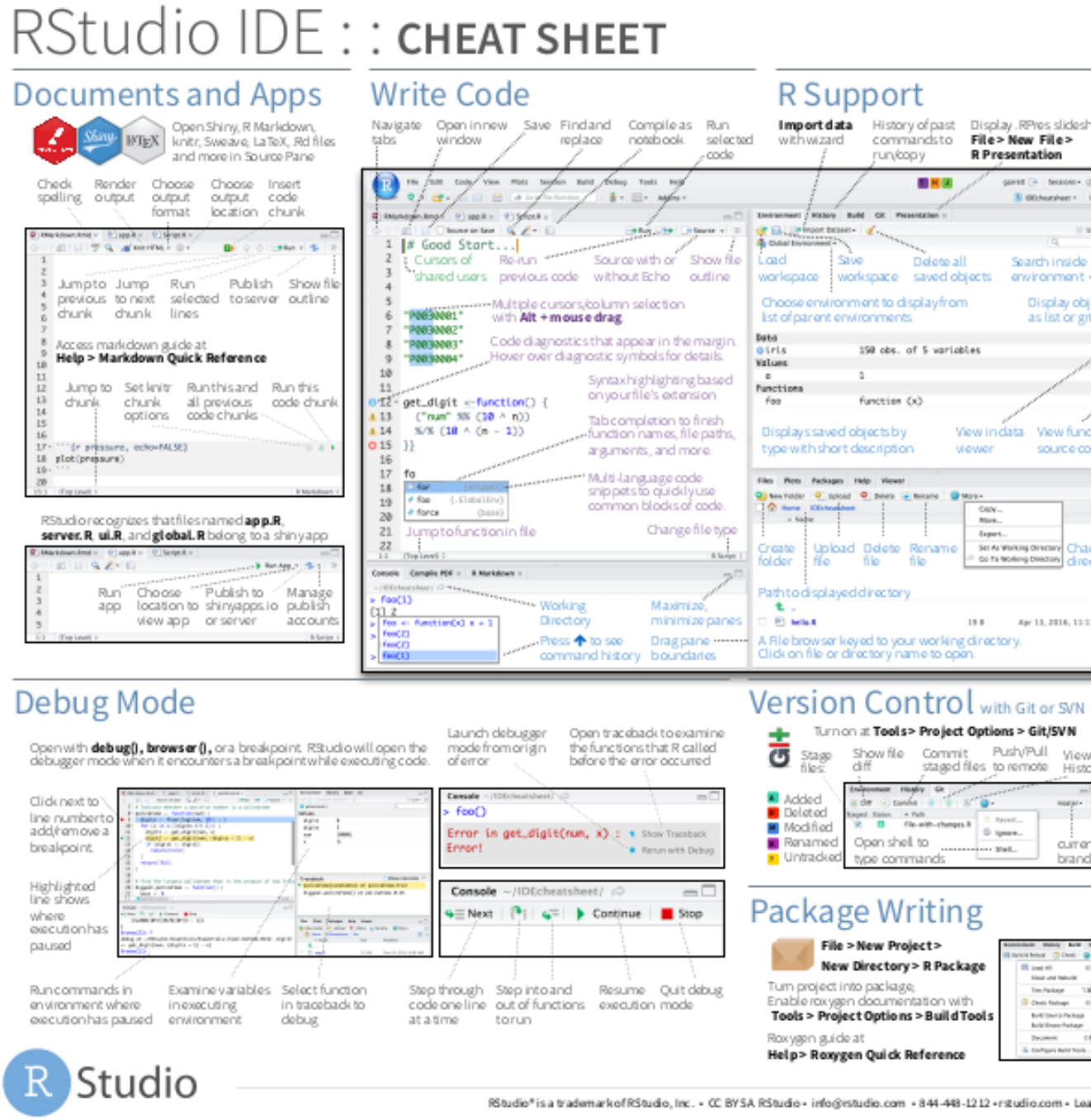
Para saber mais sobre Git e versionamento de arquivos, leia esse excelente capítulo do Zen do R.

2.5 Cheatsheets

O RStudio tem à disposição algumas *folhas de cola*, as **cheatsheets**. Elas trazem um resumão de como utilizar diversos pacotes e até o próprio RStudio.

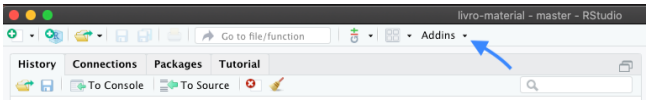
Para acessá-las, basta clicar no menu `Help` e então em `Cheatsheets`.

Outra forma de acessá-las é através do site da RStudio, onde existem muitas opções de **cheatsheets**, incluindo algumas traduzidas para o Português.


RStudio IDE : : CHEAT SHEET
Documents and Apps
Open Shiny, R Markdown, knitr, Sweave, LaTeX, Rd files and more in Source Pane
Check spelling
Render output
Choose output format
Choose output location
Insert code chunk
Jump to previous chunk
Jump to next chunk
Run selected lines
Publish to server
Show file outline
Access markdown guide at
Help > Markdown Quick Reference
Jump to chunk
Set knitr chunk options
Run this and all previous code chunks
Run this code chunk
RStudio recognizes that files named app.R, server.R, ui.R, and global.R belong to a shiny app
Run app
Choose location to view app
Publish to shinyapps.io or server
Manage publish accounts
Write Code
Navigate tabs
Open in new window
Save
Find and replace
Compile as notebook
Run selected code
Cursors of shared users
Re-run previous code
Source with or without Echo
Show file outline
Multiple cursors, column selection with Alt + mouse drag
Code diagnostics that appear in the margin. Hover over diagnostic symbols for details.
Syntax highlighting based on your file's extension
Tab completion to finish function names, file paths, arguments, and more.
Multi-language code snippets to quickly use common blocks of code.
Jump to function in file
Change file type
Working Directory
Maximize, minimize panes
Press ↑ to see command history
Drag pane boundaries
R Support
Import data with wizard
History of past commands to run/copy
Display .RPres slideshows
File > New File > R Presentation
Load workspace
Save workspace
Delete all saved objects
Search inside environment
Choose environment to display from list of parent environments
Display objects as list or grid
Displays saved objects by type with short description
View in data viewer
Create folder
Upload file
Delete file
Rename file
Path to displayed directory
A File browser keyed to your working directory. Click on file or directory name to open.
Debug Mode
Open with debug(), browser(), or a breakpoint. RStudio will open the debugger mode when it encounters a breakpoint while executing code.
Launch debugger mode from origin of error
Open traceback to examine the functions that R called before the error occurred
Click next to line number to add/remove a breakpoint
Highlighted line shows where execution has paused
Error in get_digit(num, x) :
Error!
Show Traceback
Rerun with Debug
Console ~/IDEcheatsheet/
Next
Continue
Stop
Run commands in environment where execution has paused
Examine variables in executing environment
Select function in traceback to debug
Step through code one line at a time
Step into and out of functions to run
Resume execution
Quit debug mode
Version Control with Git or SVN
Turn on at Tools > Project Options > Git/SVN
Stage files
Show file diff
Commit staged files
Push/Pull to remote
Added
Deleted
Modified
Renamed
Untracked
Open shell to type commands
Package Writing
File > New Project > New Directory > R Package
Turn project into package. Enable roxygen documentation with Tools > Project Options > Build Tools
Roxygen guide at
Help > Roxygen Quick Reference
RStudio
RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com

2.6 Addins

Addins são cápsulas de código R que podem ser executados interativamente a partir do menu `Addins`.

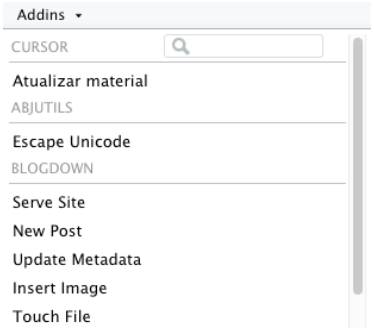


Eles servem para fazer diversos tipos de tarefas, como identar seu código, ajudar nas tarefas de copiar e colar, insirir códigos no seu script e muito mais.

Os addins são criados pelos pacotes que instalamos, então cada pacote vai nos disponibizar addins que facilitam algum tipo de tarefa.

Veja aqui um exemplo de um addin que formata um código e aqui um addin que filtra uma base.

O pacote `{CursoR}` da Curso-R, por exemplo, tem um addin que auxilia quem faz nossos cursos a atualizar o material de cada aula.



2.7 Snippets

Snippets são atalhos que podemos criar para gerar pedaços rotineiros de código.

O RStudio já vem com vários snippets criados. Você pode visualizar os snippts existentes ou criar novos acessando `Tools > Global Options... > Code > Editting > Snippets > Edit snippets`.

No arquivo de texto aberto, basta seguir o padrão dos snippets já existentes para criar novos. Você também pode editar os snippets padrão do RStudio.

```
snippet ei
  else if (${1:condition}) {
    ${0}
  }

snippet fun
  ${1:name} ← function(${2:variables}) {
    ${0}
  }

snippet for
  for (${1:variable} in ${2:vector}) {
    ${0}
  }

snippet while
  while (${1:condition}) {
    ${0}
  }

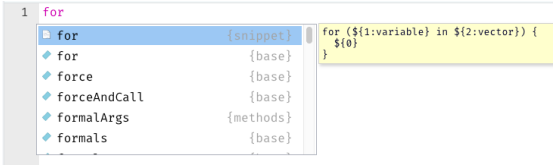
snippet switch
  switch (${1:object},
    ${2:case} = ${3:action}
  )

snippet apply
  apply(${1:array}, ${2:margin}, ${3: ... })

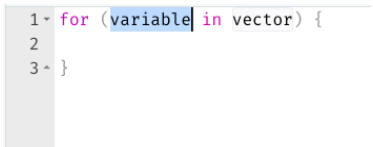
snippet lapply
  lapply(${1:list}, ${2:function})

snippet sapply
  sapply(${1:list}, ${2:function})
```

Cada snippet terá um *nome*, que deverá ser utilizado no script para ativar o código guardado por trás.



Basta escrever o nome, apertar a teclar `TAB` e *voilà*.



Chapter 3

R Básico

Introduziremos aqui os principais conceitos de programação em R. Indicamos a leitura deste capítulo a quem nunca teve contato com uma linguagem de programação ou a quem gostaria de entender um pouco melhor a estrutura de objetos, funções e classes do R.

Os tópicos discutidos aqui são especialmente importantes para entendermos o que é um *data frame*, a nossa base de dados dentro do R, e quais operações estão sendo realizadas por trás das cortinas quando estivermos filtrando suas linhas ou modificando suas colunas. Também são importantes para começarmos a criar as nossas próprias funções, o que deixa nossos códigos muito mais organizados, eficientes e *compartilháveis*.

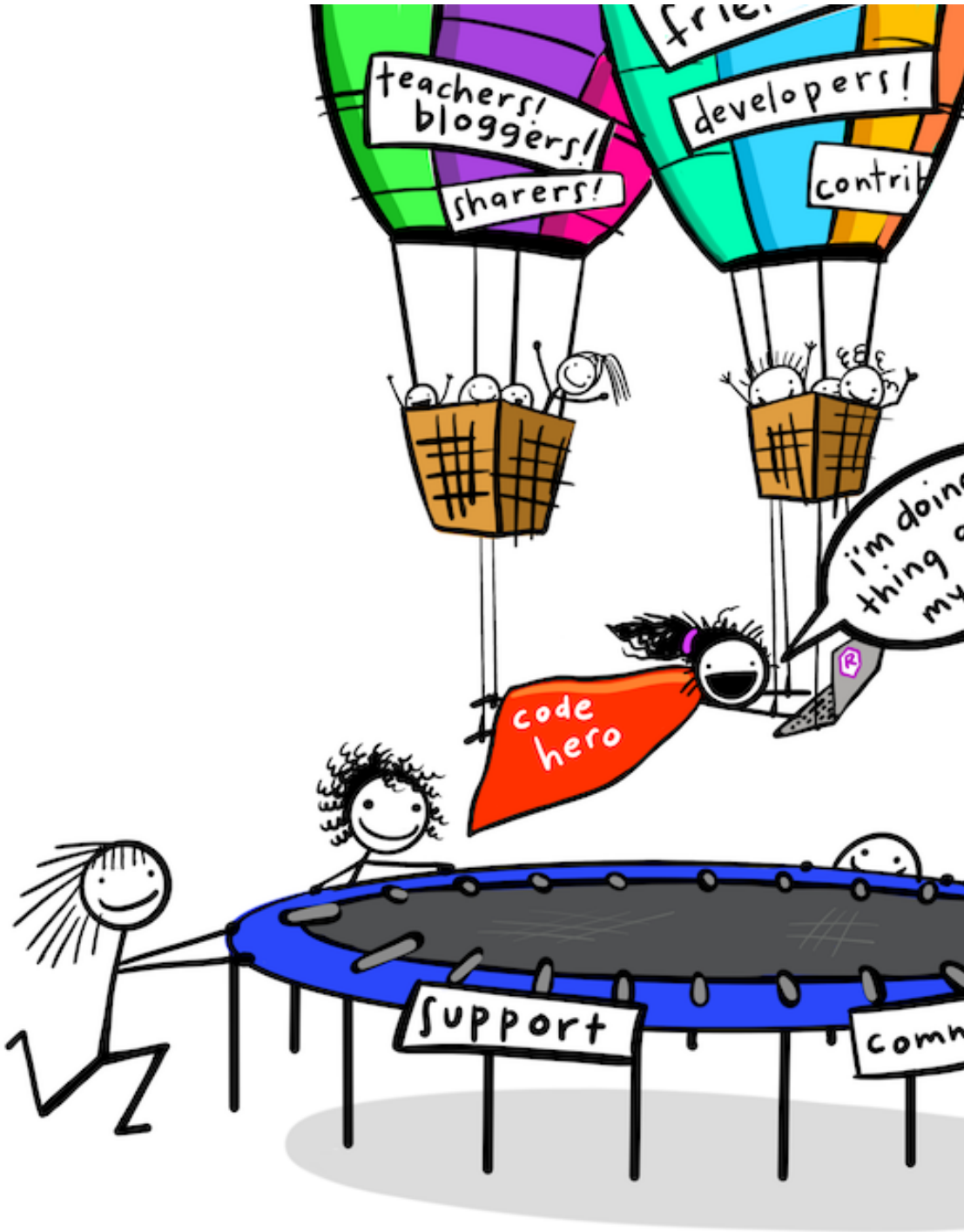
3.1 Pedindo Ajuda

A linguagem `R` é bem intuitiva. É possível fazer bastante coisa à base da tentativa e erro. Além disso, grande parte do conhecimento é escalável, isto é, aprender a utilizar uma função é meio caminho andado para aprender todas as outras funções que operam de forma semelhante[1].

No entanto, a intuição não infalível, e recorrentemente vamos precisar de ajuda para rodar alguma função ou descobrir como fazer alguma tarefa no R. Felizmente, a comunidade R é bem ativa e existem vários lugares para buscar respostas. Nesta seção, vamos apresentar as principais maneiras algumas dessas maneiras.

\begin{figure}

[1] Essa ideia é um dos princípios por trás do `tidyverse`.


teachers!
bloggers!
sharers!
developers!
code
hero
support

{

}

\caption{Arte por Allison Horst (**?**). Veja nas Referências onde encontrá-la.}
\end{figure}

No R, há quatro principais entidades para se pedir ajuda:

- Help/documentação do R
- Google
- Stack Overflow
- Coleguinha

A busca por ajuda é feita preferencialmente, mas não necessariamente, na ordem acima.

3.1.1 Documentação do R

A documentação do R serve para você aprender a usar uma determinada função. Se você não sabe o que é uma função, não se preocupe. Discutiremos esse tópico nas Seções 3.3 e 3.11.

Você pode acessar a documentação de uma função[2] das seguintes maneiras:

```
?mean
help(mean)
```

Algumas dicas:

1. Leia a seção *Usage* para ter noção de como usar a função.
2. Os parâmetros da função estão descritos em *Arguments*.
3. Os exemplos no final são particularmente úteis.
4. Caso essa função não atenda às suas necessidades, a seção *See Also* sugere funções relacionadas.

Alguns pacotes possuem tutorias de uso mais completos. Esses textos são chamados de `vignettes` e podem ser acessados com a função `vignette(package = 'nomeDoPacote')`. Por exemplo, `vignette(package = 'dplyr')`.

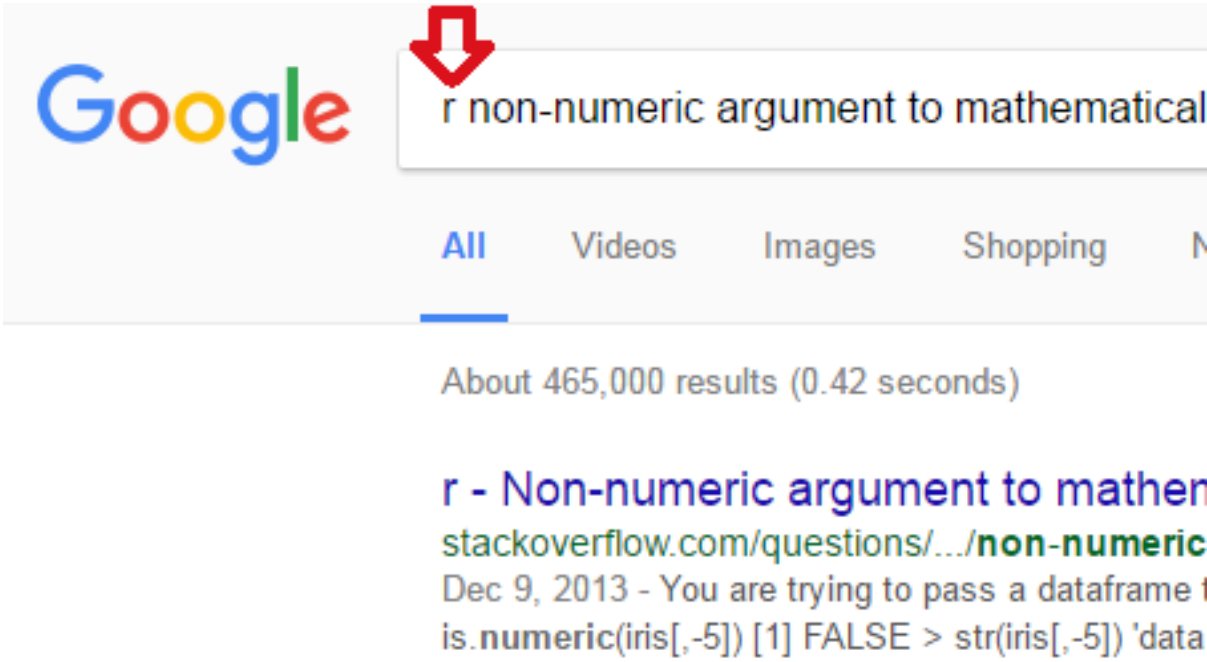
[2]Bases de dados presentes em pacotes também têm documentação, e geralmente é possível encontrar o significado de cada variável nela. Por exemplo, `help(mtcars)`.

3.1.2 Google

Há uma comunidade gigantesca de usuários de R gerando diariamente uma infinidade de conteúdos e discussões. Não raramente, você irá encontrar discussões sobre o seu problema simplesmente o descrevendo no Google. Pesquisas em inglês aumentam consideravelmente a chance de encontrar uma resposta.

Quando você recebe um erro na tentativa de rodar algum código no R e não sabe o que está errado, uma boa estratégia é pesquisar a mensagem de erro no Google. Essa deve ser sua primeira tentativa para resolver o problema. Repare na imagem abaixo o 'r' adicionado na busca. Isso ajuda bastante a encontrar uma solução.

```
log("5")
## Error in log("5"): non-numeric argument to mathematical function
```



3.1.3 Stack Overflow

O Stack Overflow e o Stack Overflow em Português são sites de Pergunta e Resposta amplamente utilizados por todas as linguagens de programação, e o

R é uma delas. Nos EUA, chegam até a usar a reputação dos usuários dentro da plataforma como diferencial no currículo!

Provavelmente o Google lhe indicará uma página deles quando você estiver procurando ajuda. E quando todas as fontes possíveis de ajuda falharem, o Stack Overflow lhe dará o espaço para **criar sua própria pergunta**.

Um ponto importante: como fazer uma *boa pergunta* no Stack Overflow?

No site, existe um tutorial com uma lista de boas práticas, que se encontra aqui. Resumindo, as principais dicas são

- ser conciso;
- ser específico;
- ter mente aberta; e
- ser gentil.

Porém, no caso do R, há outro requisito que vai aumentar muito sua chance de ter uma boa resposta: **exemplinho minimal e reprodutível**.

- Ser **minimal**: usar bancos de dados menores e utilizar pedaços de códigos apenas suficientes para apresentar o seu problema. Não precisa de banco de dados de um milhão de linhas e nem colocar o seu código inteiro para descrever a sua dúvida.
- Ser **reprodutível**: o seu código deve rodar fora da sua máquina. Se você não fornecer uma versão do seu problema que rode (ou que imite seu erro), as pessoas vão logo desistir de te ajudar. Por isso, nunca coloque bancos de dados que só você tem acesso. Use bancos de dados que já vem no R ou disponibilize um exemplo (possivelmente anonimizado) em `.csv` na web para baixar. E se precisar utilizar funções de algum pacote, especifique os pacotes que você usou.

3.2 R como calculadora

O papel do **Console** no R é executar os nossos comandos. Ele avalia o código que passamos para ele e devolve a saída correspondente — se tudo der certo — ou uma mensagem de erro — se o seu código tiver algum problema.

Vamos começar com o exemplo mais simples possível:

```
1 + 1
## [1] 2
```

Nesse caso, o nosso comando foi o código `1 + 1` e a saída foi o valor `2`.

Quando compilamos? Quem vem de linguagens como o C ou Java espera que seja necessário compilar o código em texto para o código das máquinas (geralmente um código binário). No R, isso não é necessário. O R é uma linguagem de programação dinâmica que interpreta o seu código enquanto você o executa.

Tente agora jogar no console a expressão: `2 * 2 - (4 + 4) / 2`.

Pronto! Você já é capaz de pedir ao R para fazer qualquer uma das quatro operações aritméticas básicas. A seguir, apresentamos uma lista resumindo como fazer as principais operações no R.

```
# adição
1 + 1
## [1] 2

# subtração
4 - 2
## [1] 2

# multiplicação
2 * 3
## [1] 6

# divisão
5 / 3
## [1] 1.666667

# potência
4 ^ 2
## [1] 16

# resto da divisão de 5 por 3
5 %% 3
## [1] 2

# parte inteira da divisão de 5 por 3
5 %/% 3
## [1] 1
```

Repare que as operações e suas precedências são mantidas como na matemática, ou seja, divisão e multiplicação são calculadas antes da adição e subtração. E os parênteses nunca são demais!

Uma outra forma de executar uma expressão é escrever o código em um **script**, deixar o cursor em cima da linha e usar o atalho `Ctrl + Enter`.

Assim, o comando é enviado para o **Console**, onde é diretamente executado. Essa operação é chamada de **avaliar**, **executar** ou **rodar** o código.

Se você digitar um comando incompleto, como `5 +`, e apertar `Enter`, o R mostrará um `+`, o que não tem nada a ver com a adição da matemática. Isso significa que o R está esperando que você enviar **mais** algum código para completar o seu comando. Termine o seu comando ou aperte `Esc` para recomeçar.

```
> 5 -
+
+ 5
[1] 0
```

Se você digitar um comando que o R não reconhece, ele retornará uma mensagem de erro.

NÃO ENTRE EM PÂNICO!

Ele só está avisando que não conseguiu interpretar o comando. Você pode digitar outro comando normalmente em seguida.

```
> 5 % 2
Error: unexpected input in "5 % 2"
> 5 ^ 2
[1] 25
```

Exercícios

1. Qual a diferença entre o R e o RStudio?

2. Podemos usar o RStudio sem o R? E o R sem o RStudio?

3. Precisamos compilar nossos códigos de R?

4. Calcule o número de ouro no R. Dica: o número de ouro é dado pela expressão $\frac{1+\sqrt{5}}{2}$.

5. Por que é preferível escrevermos sempre o nosso código no *script* e não no *Console*?

3.3 Objetos e funções

O R te permite salvar valores dentro de um **objeto**. Um objeto é simplesmente um nome que guarda um valor. Para criar um objeto, utilizamos o operador `<-`.

No exemplo abaixo, salvamos o valor `1` em `a`. Sempre que avaliarmos o objeto `a`, o R vai devolver o valor 1.

```
# Salvando `1` em `a`
a <- 1

# Avaliando o objeto `a`
a
## [1] 1
```

Existem algumas regras para dar nomes aos objetos. A mais importante é: o nome deve começar com uma letra[3]. O nome pode conter números, mas não pode começar com números. Você pode usar pontos `.` e underlines `_` para separar palavras.

```
# Permitido

x <- 1
x1 <- 2
objeto <- 3
meu_objeto <- 4
meu.objeto <- 5

# Não permitido

1x <- 1
_objeto <- 2
meu-objeto <- 3
```

Atenção!

O R **diferencia letras maiúsculas e minúsculas**, isto é, `b` é considerado um objeto diferente de `B`. Rode o exemplo abaixo e observe que dois objetos diferentes são criados no **Environment**.

```
b <- 2
B <- 3

b
## [1] 2
B
## [1] 3
```

[3]Ou com um ponto.

O objeto mais importante para o cientista de dados é, claro, a base de dados. No R, uma base de dados é representa por objetos chamados de *data frames*. Na próxima seção, vamos entender o que são esses objetos.

Enquanto objetos são *nomes* que guardam *valores*, funções no R são *nomes* que guardam um **código de R**. A ideia é muito simples: sempre que você rodar uma função, o código que ela guarda será executado e um resultado nos será devolvido.

A sintaxe para usar uma função é a seguinte:

```
nome_da_funcao(arg1, arg2, argn)
```

Entre parênteses, após o nome da função, temos o que chamamos de *argumentos*. Uma função pode ter qualquer número de argumentos e eles são sempre separados por vírgula.

Basicamente, uma função recebe seus argumentos, executa uma ação sobre ou a partir deles e devolve um resultado. Por exemplo

```
sum(1, 2)
## [1] 3
```

A função `sum()` recebeu os argumentos `1` e `2`, somou os dois valores e devolveu o resultado dessa operação: o valor `3`.

Falaremos mais sobre funções na Seção 3.11.

Exercícios

1. Qual a diferença entre os códigos abaixo?

```
# Código 1
33 / 11

# Código 2
divisao <- 33 / 11
```

2. Multiplique a sua idade por meses e salve o resultado em um objeto chamado `idade_em_meses`. Em seguida, multiplique esse objeto por 30 e salve o resultado em um objeto chamado `idade_em_dias`.

3. Por que o nome `meu-objeto` não pode ser utilizado para criar um objeto? O que significa a mensagem de erro resultante?

```
meu-objeto <- 1
## Error in meu - objeto <- 1: object 'meu' not found
```

3.4 Data frames

Os *data frames* são de extrema importância no R, pois são os objetos que guardam os nossos dados. Eles são equivalentes a uma tabela do SQL ou uma planilha do Excel.

A principal característica de um *data frame* é possuir linhas e colunas[4]. Veja o exemplo abaixo:

```
mtcars
##                    mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C         17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Merc 450SE        16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 450SL        17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 450SLC       15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Fiat 128          32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic       30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla    33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona     21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Dodge Challenger  15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## AMC Javelin       15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Camaro Z28        13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Pontiac Firebird  19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Fiat X1-9         27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2     26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa      30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

[4] Você pode construir um *data frame* vazio, com 0 linha e 0 coluna. No entanto, a *estrutura* de linhas e colunas estará presente.

```
## Ford Pantera L     15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Ferrari Dino       19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Maserati Bora      15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## Volvo 142E         21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

O `mtcars` é um *data frame* nativo do R que contém informações sobre diversos modelos de carros. Ele possui 32 linhas e 11 colunas[5]. Se você quiser saber mais sobre o `mtcars`, veja a documentação dele rodando `?mtcars` no **Console**.

Nos próximos capítulos, os *data frames* serão o nosso principal objeto de estudo. Aprenderemos a selecionar, criar e modificar colunas, filtrar e ordenar linhas, juntar dois *data frames* e, a partir deles, construiremos gráficos e ajustaremos modelos.

Mas, da mesma forma que é muito mais fácil aprendermos a fritar um ovo após entendermos o que é o fogo, uma frigideira e um ovo, vamos estudar nas próximas seções quais são as estruturas que formam os *data frames* e como manipulá-las.

3.5 Classes

A classe de um objeto é muito importante dentro do R. É a partir dela que as funções e operadores conseguem saber exatamente o que fazer com um objeto.

Por exemplo, podemos somar dois números, mas não conseguimos somar duas letras (texto):

```
1 + 1
## [1] 2


"a" + "b"
## Error in "a" + "b": non-numeric argument to binary operator
```

O operador `+` verifica que `"a"` e `"b"` não são números (ou que a classe deles não é numérica) e devolve uma mensagem de erro informando isso.

Observe que para criar texto no R, colocamos os caracteres entre aspas. As aspas servem para diferenciar *nomes* (objetos, funções, pacotes) de *textos* (letras e palavras). Os textos são muito comuns em variáveis categóricas e são popularmente chamados de *strings* no contexto de programação.

[5] A primeira "coluna" representa apenas o *nome* das linhas (modelo do carro), não é uma coluna da base. Repare que ela não possui um nome, como as outras. Essa estrutura de nome de linha é própria de *data frames* no R. Se exportássemos essa base para o Excel, por exemplo, essa coluna não apareceria.

```
a <- 10

# O objeto `a`, sem aspas
a
## [1] 10

# A letra (texto) `a`, com aspas
"a"
## [1] "a"
```

Para saber a classe de um objeto, basta rodarmos `class(nome-do-objeto)`.

```
x <- 1
class(x)
## [1] "numeric"

y <- "a"
class(y)
## [1] "character"

class(mtcars)
## [1] "data.frame"
```

As classes mais básicas dentro do R são:

- *numeric*
- *character*
- *logical*

Geralmente serão utilizados como sinônimos:

- texto, string, *character*, caracteres
- número, valor real, *numeric*, *double*
- lógico, *logical*, booleano, valor TRUE/FALSE

Veja alguns exemplos:

```
# numeric
1
0.10
0.95
pi

# characters
```

```
"a"
"1"
"positivo"

# logical
TRUE
FALSE
```

Um objeto de qualquer uma dessas classes é chamado de **objeto atômico**.

Esse nome se deve ao fato de essas classes não se misturarem, isto é, para um objeto ter a classe `numeric`, por exemplo, todos os seus valores precisam ser numéricos.

Mas como atribuir mais de um valor a um mesmo objeto? Para isso, precisamos criar **vetores**.

3.5.1 Exercícios

1. Guarde em um objeto chamado `nome` uma *string* contendo o seu nome completo.

2. Guarde em um objeto chamado `cidade` o nome da cidade onde você mora. Em seguida, guarde em um objeto chamado `estado` o nome do estado onde você mora. Usando esses objetos, resolva os itens abaixo:

- **a.** Utilize a função `nchar()` para contar o número de caracteres em cada cada string.
- **b.** Interprete o resultado do seguinte código:

```
paste(cidade, estado)
```

- **c.** Interprete o resultado do seguinte código:

```
paste(cidade, estado, sep = " - ")
```

- **d.** Desafio. Como você reproduziria o mesmo resultado do item (b) sem utilizar o argumento `sep`?
- **e.** Qual a diferença entre as funções `paste()` e `paste0()`?

3.6 Vetores

Vetores são estruturas muito importantes dentro R. Em especial, pensando em análise de dados, precisamos estudá-los pois cada coluna de um *data frame* será representada como um vetor.

Vetores no R são apenas **conjuntos indexados de valores**. Para criá-los, basta colocar os valores separados por vírgulas dentro de um `c()`.

```
vetor1 <- c(1, 5, 3, -10)
vetor2 <- c("a", "b", "c")

vetor1
## [1]   1   5   3 -10
vetor2
## [1] "a" "b" "c"
```

Os objetos `vetor1` e `vetor2` são vetores.

Uma maneira fácil de criar um vetor com uma sequência de números é utilizar o operador `:`.

```
# Vetor de 1 a 10
1:10
##  [1]  1  2  3  4  5  6  7  8  9 10

# Vetor de 10 a 1
10:1
##  [1] 10  9  8  7  6  5  4  3  2  1

# Vetor de -3 a 3
-3:3
## [1] -3 -2 -1  0  1  2  3
```

Quando dizemos que vetores são conjuntos *indexados*, isso quer dizer que cada valor dentro de um vetor tem uma **posição**. Essa posição é dada pela ordem em que os elementos foram colocados no momento em que o vetor foi criado. Isso nos permite acessar individualmente cada valor de um vetor.

Para isso, colocamos o índice do valor que queremos acessar dentro de colchetes `[]`.

```
vetor <- c("a", "b", "c", "d")

vetor[1]
## [1] "a"
```

```
vetor[2]
## [1] "b"
vetor[3]
## [1] "c"
vetor[4]
## [1] "d"
```

Você também pode colocar um conjunto de índices dentro dos colchetes, para pegar os valores contidos nessas posições:

```
vetor[c(2, 3)]
## [1] "b" "c"
vetor[c(1, 2, 4)]
## [1] "a" "b" "d"
```

Essa operação é conhecida como *subsetting*, pois estamos pegando subconjuntos de valores de um vetor.

Se você tentar acessar uma posição do vetor que não existe, ele retornará `NA`, indicando que esse valor não existe. Discutiremos o que são `NA`'s na Seção 3.8.

```
vetor[5]
## [1] NA
```

Um vetor só pode guardar um tipo de objeto e ele terá sempre a mesma classe dos objetos que guarda. Para saber a classe de um vetor, rodamos `class(nome-do-vetor)`.

```
vetor1 <- c(1, 5, 3, -10)
vetor2 <- c("a", "b", "c")

class(vetor1)
## [1] "numeric"
class(vetor2)
## [1] "character"
```

Se tentarmos misturar duas classes, o R vai apresentar o comportamento conhecido como **coerção**.

```
vetor <- c(1, 2, "a")

vetor
## [1] "1" "2" "a"
class(vetor)
## [1] "character"
```

Veja que todos os elementos do vetor se transformaram em texto. Agora temos um vetor com o texto `"1"`, o texto `"2"` e o texto `"a"`. Como um vetor só pode ter uma classe de objeto dentro dele, classes mais fracas serão sempre reprimidas pelas classes mais fortes. Como regra de bolso: caracteres serão sempre a classe mais forte. Então, sempre que você misturar números e texto em um vetor, os números virarão texto.

Falaremos bastante de coerção nas próximas seções e capítulos, trazendo exemplos de quando ela ajuda e de quando ela atrapalha.

De forma bastante intuitiva, você pode fazer operações com vetores.

```
vetor <- c(0, 5, 20, -3)
vetor + 1
## [1]  1  6 21 -2
```

Ao rodarmos `vetor1 + 1`, o R subtrai `1` de cada um dos elementos do vetor. O mesmo acontece com qualquer outra operação aritmética.

```
vetor - 1
vetor / 2
vetor * 10
```

Você também pode fazer operações que envolvem mais de um vetor:

```
vetor1 <- c(1, 2, 3)
vetor2 <- c(10, 20, 30)

vetor1  + vetor2
## [1] 11 22 33
```

Neste caso, o R irá alinhar os dois vetores e somar elemento a elemento. Esse tipo de comportamento é chamado de **vetorização**. Isso pode ficar um pouco confuso quando os dois vetores não possuem o mesmo tamanho. Tente adivinhar qual será a saída do código a seguir:

```
vetor1 <- c(1, 2)
vetor2 <- c(10, 20, 30, 40)

vetor1 + vetor2
## [1] 11 22 31 42
```

Embora estejamos somando dois vetores de tamanho diferentes, o R não devolve um erro (o que parecia ser a resposta mais intuitiva). O R alinhou os dois vetores e, como eles não possuíam o mesmo tamanho, o primeiro foi

repetido para ficar do mesmo tamanho do segundo. É como se o primeiro vetor fosse na verdade `c(1, 2, 1, 2)`. Esse comportamento é chamado de **reciclagem**.

Embora contra-intuitiva, a reciclagem é muito útil no R graças a um caso particular muito importante. Quando somamos `vetor + 1` no nosso primeiro exemplo, o que o R está fazendo por trás é transformando o `1` em `c(1, 1, 1, 1)` e realizando a soma vetorizada `c(0, 5, 20, -3) + c(1, 1, 1, 1)`. Isso porque o número `1` nada mais é do que um vetor de tamanho 1, isto é, `1` é igual a `c(1)`.

Usaremos esse comportamento no R o tempo todo e é muito importante a reciclagem para termos certeza de que o R está fazendo exatamente aquilo que gostaríamos que ele fizesse.

Um outro caso interessante de reciclagem é quando o comprimento dos vetores não são múltiplos um do outro.

```
vetor1 <- c(1, 2, 3)
vetor2 <- c(10, 20, 30, 40, 50)

vetor1 + vetor2
## Warning in vetor1 + vetor2: longer object length is not a multiple of shorter
## object length
## [1] 11 22 33 41 52
```

Neste caso, duas coisas aconteceram:

1. O R realizou a conta, repetindo cada valor do primeiro vetor até que os dois tenham o mesmo tamanho. No fundo, a operação realizada foi `c(1, 2, 3, 1, 2) + c(10, 20, 30, 40, 50)`.
2. Como essa operação é ainda menos intuitiva e raramente desejada, o R devolveu um aviso dizendo que o comprimento do primeiro vetor maior não é um múltiplo do comprimento do vetor menor.

Exercícios

1. Guarde em um objeto a sequência de números de 0 a 5 e resolva os itens abaixos.

- **a.** Use subsetting para fazer o R devolver o primeiro número dessa sequência. Em seguida, faça o R devolver o último número da sequência.
- **b.** Multiplique todos os valores do vetor por -1. Guarde o resultado em um novo objeto chamado `vetor_negativo`.

2. Crie um vetor com o nome de tres `frutas`, guarde em um objeto chamado frutas e resolva os itens abaixo.

- **a.** Utilize a a função `length()` para verificar o tamanho do vetor.
- **b.** Inspecione a saída de `paste("eu gosto de", frutas)` e responda se o tamanho do vetor mudou.

3. O que é reciclagem? Escreva um código em R que exemplifique esse comportamento.

4. O que é coerção? Escreva um código em R que exemplifique esse comportamento.

5. Por que a coerção pode ser um problema na hora de importarmos bases de dados para o R?

6. Use a função `sum()` para somar os valores de 1 a 100.

7. Considere o vetor booleano a seguir:

```
dolar_subiu <- c(TRUE, TRUE, FALSE, FALSE, TRUE, FALSE, TRUE)
```

Este vetor tem informação de uma semana (7 dias, começando no domingo) indicando se o dólar subiu (TRUE) ou não subiu (FALSE) no respectivo dia. Interprete o resultado dos códigos abaixo:

- **a.** `length(dolar_subiu)`
- **b.** `dolar_subiu[2]`
- **c.** `sum(dolar_subiu)`
- **d.** `mean(dolar_subiu)`

3.7 Testes lógicos

Poder fazer qualquer tipo de operação lógica é um dos motivos pelos quais programar nos deixar mais eficientes. Dê bastante atenção a elas, pois usaremos comparações lógicas o tempo todo!

Uma operação lógica nada mais é do que um teste que retorna **verdadeiro** ou **falso**. No R (e em outras linguagens de programação), esses valores dois valores recebem uma classe especial: `logical`.

O verdadeiro no R vai ser representado pelo valor `TRUE` e o falso pelo valor `FALSE`. Esses nomes no R são **reservados**, isto é, você não pode chamar nenhum objeto de TRUE ou FALSE.

```
TRUE <- 1
## Error in TRUE <- 1 : invalid (do_set) left-hand side to assignment
```

Checando a classe desses valores, vemos que são lógicos[6]. Eles são os únicos possíveis valores dessa classe.

```
class(TRUE)
## [1] "logical"
class(FALSE)
## [1] "logical"
```

Agora que conhecemos o `TRUE` e `FALSE`, podemos explorar os teste lógicos. Começando pelo mais simples: vamos testar se um valor é igual ao outro. Para isso, usamos o operador `==`.

```
# Testes com resultado verdadeiro
1 == 1
## [1] TRUE
"a" == "a"
## [1] TRUE

# Testes com resultado falso
1 == 2
## [1] FALSE
"a" == "b"
## [1] FALSE
```

Também podemos testar se dois valores são diferentes. Para isso, usamos o operador `!=`.

```
# Testes com resultado falso
1 != 1
## [1] FALSE
"a" != "a"
## [1] FALSE

# Testes com resultado verdadeiro
1 != 2
## [1] TRUE
"a" != "b"
## [1] TRUE
```

Para comparar se um valor é maior que outro, temos à disposição 4 operadores:

[6] Também conhecidos como valores binários ou booleanos

```
# Maior
3 > 3
## [1] FALSE
3 > 2
## [1] TRUE

# Maior ou igual
3 > 4
## [1] FALSE
3 >= 3
## [1] TRUE

# Menor
3 < 3
## [1] FALSE
3 < 4
## [1] TRUE

# Menor ou igual
3 < 2
## [1] FALSE
3 <= 3
## [1] TRUE
```

Um outro operador muito útil é o `%in%`. Com ele, podemos verificar se um valor está dentro de um conjunto de valores (vetor).

```
3 %in% c(1, 2, 3)
## [1] TRUE
"a" %in% c("b", "c")
## [1] FALSE
```

Nós começamos essa seção dizendo que usaremos testes lógicos o tempo todo. O motivo para isso é que eles fazem parte de uma operação muito comum na manipulação de base de dados: os **filtros**.

No Excel, por exemplo, quando você filtra uma planilha, o que está sendo feito por trás é um teste lógico.

Falamos anteriormente que cada coluna das nossas bases de dados será representada dentro do R como um vetor. O comportamento que explica a importância dos testes lógicos na hora de filtrar uma base está ilustrado abaixo:

```
minha_coluna <- c(1, 3, 0, 10, -1, 5, 20)

minha_coluna > 3
## [1] FALSE FALSE FALSE  TRUE FALSE  TRUE  TRUE

minha_coluna[minha_coluna > 3]
## [1] 10  5 20
```

Muitas coisas aconteceram aqui, vamos por partes.

Primeiro, na operação `minha_coluna > 3` o R fez um excelente uso do comportamento de reciclagem. No fundo, o que ele fez foi transformar (reciclar) o valor `3` no vetor `c(3, 3, 3, 3, 3, 3, 3)` e testar se `c(1, 3, 0, 10, -1, 5, 20) > c(3, 3, 3, 3, 3, 3, 3)`.

Como os operadores lógicos também são vetorizados (fazem operações elemento a elemento), os testes realizados foram `1 > 3`, `3 > 3`, `0 > 3`, `10 > 3`, `-1 > 3`, `5 > 3` e, finalmente, `20 > 3`. Cada um desses testes tem o seu próprio resultado. Por isso a saída de `minha_coluna > 3` é um vetor de verdadeiros e falsos, respectivos a cada um desses 7 testes.

A segunda operação traz a grande novidade aqui: podemos usar os valores `TRUE` e `FALSE` para selecionar elementos de um vetor!

A regra é muito simples: **retornar** as posições que receberem `TRUE`, **não retornar** as posições que receberem `FALSE`. Portanto, a segunda operação é equivalente a:

```
minha_coluna[c(FALSE, FALSE, FALSE, TRUE, FALSE, TRUE, TRUE)]
## [1] 10  5 20
```

O vetor lógico filtra o vetor `minha_coluna`, retornando apenas os valores maiores que 3, já que foi esse o teste lógico que fizemos.

Essa é a *mágica* que acontece por trás de filtros no R. Na prática, não precisaremos usar colchetes, não lembraremos da reciclagem e nem veremos a cara dos `TRUE` e `FALSE`. Mas conhecer esse processo é muito importante, principalmente para encontrar problemas de código ou de base.

A seguir, apresentamos uma tabela com os principais operadores lógicos:

Para finalizar, listamos na tabela abaixo os principais operadores lógicos.

Operador	Descrição
x < y	x menor que y?
x <= y	x menor ou igual a y?
x > y	x maior que y?
x >= y	x maior ou igual a y?
x == y	x igual a y?
x != y	x diferente de y?
!x	Negativa de x
x | y	x ou y são verdadeiros?
x & y	x e y são verdadeiros?
x %in% y	x percente a y?
xor(x, y)	x ou y são verdadeiros (apenas um deles)?

Por fim, veja algumas diferenças entre comparações lógicas no SQL e no R:

- **Igualdade**: no SQL é só um sinal de igual: `2 = 1`. No R são dois: `2 == 1`.
- **Diferença**: no SQL, usamos `<>`. No R usamos `!=`.
- **Negação**: em vez de usar a palavra `NOT` igual ao SQL, usamos `!` no R. Por exemplo, `id not in ('1', '2', '3')` fica `!(id %in% c(1, 2, 3))`.

Exercícios

1 O código abaixo vai guardar no objeto `segredo` um número inteiro entre 0 e 10. Sem olhar qual número foi guardado no objeto, resolva os itens a seguir:

```
segredo <- round(runif(1, min = 0, max = 10))
```

- **a.** Teste se o segredo é maior ou igual a 0.
- **b.** Teste se o segredo é menor ou igual a 10.
- **c.** Teste se o segredo é maior que 5.
- **d.** Teste se o segredo é par.
- **e.** Teste se `segredo * 5` é maior que a sua idade.
- **f.** Desafio. Escreva um teste para descobrir o valor do segredo.

2. Escreva um código em R que devolva apenas os valores maiores ou iguais a 10 do vetor abaixo:

```
vetor <- c(4, 8, 15, 16, 23, 42)
```

3. Use o vetor numeross abaixo para responder as questoes seguintes.

```
numeros <- -4:2
```

- **a.** Escreva um código que devolva apenas valores positivos do vetor `numeros`.
- **b.** Escreva um código que devolta apenas os valores pares do vetor `numeros`.
- **c.** Filtre o vetor para que retorne apenas aqueles valores que, quando elevados a 2, são menores do que 4.

3.8 Valores especiais

Vimos anteriormente que se você tentar acessar uma posição que não existe dentro de um vetor, ele retorna um valor estranho.

```
vetor <- c(1, 2, 3)

vetor[4]
## [1] NA
```

Esse valor, o `NA`, é tratado de forma especial no R. Ele representa a *ausência de informação*, isto é, a informação existe, mas nós (e o R) não sabemos qual é.

O `NA` para o R nada mais é do que o valor faltante ou omisso da Estatística. O famoso *missing*. Geralmente, quando temos uma base com valores faltando, como a idade para alguns indivíduos da nossa amostra, não significa que a idade deles não existe. Significa apenas que não temos essa informação.

Esse conceito é muito importante para entender o resultado da expressão abaixo.

```
5 == NA
## [1] NA
```

Em um primeiro momento, poderíamos esperar que o resultado fosse `TRUE`. Mas, sabendo o significado por trás do `NA` — um valor desconhecido —, a verdadeira pergunta que estamos fazendo é: 5 é igual a um valor que existe, mas que não sei qual é? É como se eu perguntasse se eu tenho 5 moedas na mão, mas lhe mostrasse a mão fechada. A resposta para isso é *não sei* ou, dentro do R, `NA`.

Um outro exemplo:

```
idade_ana <- 30
idade_beto <- NA
idade_carla <- NA

idade_ana == idade_beto
## [1] NA

idade_beto == idade_carla
## [1] NA
```

Eu posso saber a idade da Ana, mas se eu não souber a idade do Beto, não sei se os dois tem a mesma idade. Por isso, `NA`. Da mesma forma, se não sei nem a idade do Beto nem da Carla, também não tenho como saber se os dois têm a mesma idade. Outra vez `NA`.

Mas e quando queremos saber se um valor é NA ou não? Para fazer esse teste, temos que rodar `is.na(valor-ou-objeto)`.

```
is.na(NA)
## [1] TRUE

is.na(idade_ana)
## [1] FALSE

is.na(idade_beto)
## [1] TRUE
```

Repare que essa função também é vetorizada.

```
is.na(c(idade_ana, idade_beto, idade_carla))
## [1] FALSE  TRUE  TRUE
```

Assim como o `NA`, existem outros valores especiais muito comuns no R.

O `NaN` (*not a number*) representa indefinições matemáticas.

```
0/0
## [1] NaN

log(-1)
## Warning in log(-1): NaNs produced
## [1] NaN
```

O `Inf` (infinito) representa um número muito grande (que o computador não consegue representar) ou um limite matemático.

```
# O computador não consegue representar um número tão grande.
# O número é então """arredondado""" para infinito.
10^310
## [1] Inf

# Limite matemático.
1 / 0
## [1] Inf

# O "menos infinito" também existe.
-1 / 0
## [1] -Inf
```

O `NULL` (nulo) representa a ausência de um objeto. Ele não tem significado prático para a análise dados. Está mais em sintonia com comportamentos de lógica de programação. Muitas vezes vamos definir um objeto como nulo para dizer ao R que não queremos dar um valor para ele. Muito utilizado em funções.

Da mesma forma que utilizados `is.na()` para testar se um objeto é `NA`, utilizamos `is.nan()`, `is.infinite()` ou `is.null()` para testar se um objeto é `NaN`, infinito ou nulo.

```
nao_sou_um_numero <- NaN
objeto_infinito <- Inf
objeto_nulo <- NULL

is.nan(nao_sou_um_numero)
## [1] TRUE

is.infinite(objeto_infinito)
## [1] TRUE

is.null(objeto_nulo)
## [1] TRUE
```

Exercícios

1. Quais as diferenças entre `NaN`, `NULL`, `NA` e `Inf`? Digite expressões que retornem cada um desses valores.

2. Escreva um código que conte o número de `NAs` do vetor `b`.

```
b <- c(1, 0, NA, NA, NA, NA, 7, NA, NA, NA, NA, NA, 2, NA, NA, 10, 1, 1, NA)
```

3.9 Listas

Chegamos ao último tópico antes de voltarmos aos data frames: as listas.

Listas são objetos muito importantes dentro do R. Primeiro porque **todo data frame é uma lista**. Segundo porque elas são bem parecidas com vetores, mas com uma diferença essencial: você pode misturar diferentes classes de objetos dentro dela.

Para criar uma lista, rodamos `list(valor1, valor2, valor3)`.

```
list(1, "a", TRUE)
## [[1]]
## [1] 1
##
## [[2]]
## [1] "a"
##
## [[3]]
## [1] TRUE
```

Veja que não houve coerção. Ainda temos um valor número, um texto e um valor lógico dentro da lista.

O *subsetting* de listas é um pouco diferente do que o de vetores. Isso porque **cada elemento de uma lista também é uma lista**. Veja o que acontece se tentarmos usar `[]` para pegar um elemento de uma lista.

```
lista <- list(1, "a", TRUE)

lista[1]
## [[1]]
## [1] 1

class(lista[1])
## [1] "list"
```

O R nos retorna uma lista com apenas aquele elemento. Se quisermos o elemento de fato dentro de cada posição, precisamos usar dois colchetes:

```
lista[[1]]
## [1] 1

class(lista[[1]])
## [1] "numeric"
```

Cada elemento de uma lista ser uma lista é importante pois isso nos permite colocar vetores de tamanhos diferentes em cada posição. Isso faz das listas uma estrutura bem flexível para guardar dados.

```
lista <- list(1:3, "a", c(TRUE, TRUE, FALSE, FALSE))

lista
## [[1]]
## [1] 1 2 3
##
## [[2]]
## [1] "a"
##
## [[3]]
## [1]  TRUE  TRUE FALSE FALSE

lista[1]
## [[1]]
## [1] 1 2 3

lista[2]
## [[1]]
## [1] "a"

lista[3]
## [[1]]
## [1]  TRUE  TRUE FALSE FALSE
```

É muito comum darmos nomes para cada posição de uma lista.

```
dados_cliente <- list(cliente = "Ana Silva", idade = 25, estado_civil = NA)

dados_cliente
## $cliente
## [1] "Ana Silva"
##
## $idade
## [1] 25
##
```

```
## $estado_civil
## [1] NA
```

Agora, dentro da lista, o valor `Ana Silva`, por exemplo, está sendo atribuído ao nome `cliente`. Esse nome só existirá dentro da lista.

Um detalhe importante: os iguais utilizados nas atribuições dos nomes dentro da lista **não podem** ser substituídos por `<-`.

Quando as posições de uma lista tem nome, podemos acessar seus valores diretamente utilizando o operador `$`.

```
dados_cliente$cliente
## [1] "Ana Silva"

dados_cliente$idade
## [1] 25

dados_cliente$estado_civil
## [1] NA
```

Repare que o R devolve o valor dentro de cada posição, e não uma lista.

```
dados_cliente[1]
## $cliente
## [1] "Ana Silva"

dados_cliente$cliente
## [1] "Ana Silva"

class(dados_cliente[1])
## [1] "list"

class(dados_cliente$cliente)
## [1] "character"
```

Isto implica que, nesse exemplo, `dados_cliente$cliente` é equivalente a `dados_cliente[[1]]`.

Conforme ficamos mais e mais proficientes na linguagem R, as listas passam a ficar cada vez mais frequentes. Voltaremos a falar delas diversas vezes nos próximos capítulos, em especial no Capítulo 10.

3.10 Mais sobre data frames

Chegou a hora de usarmos tudo o que aprendemos na seção anterior para exploramos ao máximo o nosso objeto favorito: o *data frame*.

Na seção anterior, nós dissemos que *data frames* são listas. Isso é importante pois todas as propriedades de uma lista valem para um *data frame*.

A melhor forma de entender essa equivalência é ver um *data frame* representado como uma lista.

```
as.list(mtcars)
## $mpg
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
##
## $cyl
##  [1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8 4 4 4 8 6 8 4
##
## $disp
##  [1] 160.0 160.0 108.0 258.0 360.0 225.0 360.0 146.7 140.8 167.6 167.6 275.8
## [13] 275.8 275.8 472.0 460.0 440.0  78.7  75.7  71.1 120.1 318.0 304.0 350.0
## [25] 400.0  79.0 120.3  95.1 351.0 145.0 301.0 121.0
##
## $hp
##  [1] 110 110  93 110 175 105 245  62  95 123 123 180 180 180 205 215 230  66  52
## [20]  65  97 150 150 245 175  66  91 113 264 175 335 109
##
## $drat
##  [1] 3.90 3.90 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 3.92 3.07 3.07 3.07 2.93
## [16] 3.00 3.23 4.08 4.93 4.22 3.70 2.76 3.15 3.73 3.08 4.08 4.43 3.77 4.22 3.62
## [31] 3.54 4.11
##
## $wt
##  [1] 2.620 2.875 2.320 3.215 3.440 3.460 3.570 3.190 3.150 3.440 3.440 4.070
## [13] 3.730 3.780 5.250 5.424 5.345 2.200 1.615 1.835 2.465 3.520 3.435 3.840
## [25] 3.845 1.935 2.140 1.513 3.170 2.770 3.570 2.780
##
## $qsec
##  [1] 16.46 17.02 18.61 19.44 17.02 20.22 15.84 20.00 22.90 18.30 18.90 17.40
## [13] 17.60 18.00 17.98 17.82 17.42 19.47 18.52 19.90 20.01 16.87 17.30 15.41
## [25] 17.05 18.90 16.70 16.90 14.50 15.50 14.60 18.60
##
## $vs
##  [1] 0 0 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 0 1 0 0 0 1
```

```
##
## $am
##  [1] 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 1 1 1 1 1 1
##
## $gear
##  [1] 4 4 4 3 3 3 3 4 4 4 4 3 3 3 3 3 3 4 4 4 3 3 3 3 3 4 5 5 5 5 5 4
##
## $carb
##  [1] 4 4 1 1 2 1 4 2 2 4 4 3 3 3 4 4 4 1 2 1 1 2 2 4 2 1 2 2 4 6 8 2
```

O código acima nos permite ver o *data frame* `mtcars` representado como uma lista. Veja que cada coluna da base se transforma em um elemento da lista. E o nome de cada coluna vira o nome de cada posição. Isso é interessante, pois podemos usar nos *data frames* as mesmas operações que aprendemos para listas.

Por exemplo, podemos usar o operador `$` para acessar cada elemento da lista, isto é, cada coluna do *data frame*.

```
mtcars$mpg
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

E assim como cada elemento de uma lista também é uma lista, cada elemento (coluna) de um *data frame* também é um *data frame.

```
mtcars[1]
##                     mpg
## Mazda RX4          21.0
## Mazda RX4 Wag      21.0
## Datsun 710         22.8
## Hornet 4 Drive     21.4
## Hornet Sportabout  18.7
## Valiant            18.1
## Duster 360         14.3
## Merc 240D          24.4
## Merc 230           22.8
## Merc 280           19.2
## Merc 280C          17.8
## Merc 450SE         16.4
## Merc 450SL         17.3
## Merc 450SLC        15.2
## Cadillac Fleetwood 10.4
## Lincoln Continental 10.4
```

```
## Chrysler Imperial   14.7
## Fiat 128            32.4
## Honda Civic         30.4
## Toyota Corolla      33.9
## Toyota Corona       21.5
## Dodge Challenger    15.5
## AMC Javelin         15.2
## Camaro Z28          13.3
## Pontiac Firebird    19.2
## Fiat X1-9           27.3
## Porsche 914-2       26.0
## Lotus Europa        30.4
## Ford Pantera L      15.8
## Ferrari Dino        19.7
## Maserati Bora       15.0
## Volvo 142E          21.4

class(mtcars[1])
## [1] "data.frame"
```

Mas se *data frames* são listas, por que existe a classe *data frame*? Na verdade, *data frames* são um tipo especial de listas, que têm as seguintes propriedades:

1. Todos os seus elementos (colunas) precisam ter o mesmo comprimento (número de linhas).
2. Todos os seus elementos (colunas) precisam ser nomeados.
3. *Data frames* têm 2 dimensões.

As propriedades (1) e (2) se devem ao formato das bases de dados. Elas são retangulares[7] — observamos as mesmas variáveis (colunas) para todas as unidades amostrais (linhas)[8] —, e precisam ter algum nome especificando as colunas.

Da mesma forma que podemos ver um *data frame* como uma lista, também podemos fazer o inverso.

```
dados_cliente <- list(
  cliente = c("Ana Silva", "Beto Pereira", "Carla Souza"),
  idade = c(25, 30, 23),
  estado_civil = c(NA, "Solteiro", "Casada")
```

[7] Também existem bases não retangulares, como dados de imagens por exemplos, mas não trataremos dessas estruturas neste livro.

[8] Mesmo quando uma variável não existe para uma unidade amostral, representamos esse valor como um *missing*

```
)

dados_cliente
## $cliente
## [1] "Ana Silva"    "Beto Pereira" "Carla Souza"
##
## $idade
## [1] 25 30 23
##
## $estado_civil
## [1] NA         "Solteiro" "Casada"

as.data.frame(dados_cliente)
##        cliente idade estado_civil
## 1    Ana Silva    25         <NA>
## 2 Beto Pereira    30     Solteiro
## 3  Carla Souza    23       Casada
```

Quando tivermos valores faltantes na nossa base (células vazias no Excel), eles serão representados por `NA` nos *data frames*.

Observe que não conseguimos transformar uma lista em *data frame* se os elementos da lista não tiverem o mesmo comprimento.

```
dados_cliente <- list(
  cliente = c("Ana Silva", "Beto Pereira", "Carla Souza"),
  idade = c(25, 30),
  estado_civil = c(NA, "Solteiro", "Casada")
)

as.data.frame(dados_cliente)
## Error in (function (..., row.names = NULL, check.rows = FALSE, check.names = TRUE,
```

Transformar uma lista não nomeada em *data frame* é possível, mas o R cria nomes nada agradáveis para as colunas.

```
dados_cliente <- list(
  c("Ana Silva", "Beto Pereira", "Carla Souza"),
  c(25, 30, 23),
  c(NA, "Solteiro", "Casada")
)

as.data.frame(dados_cliente)
##   c..Ana.Silva....Beto.Pereira....Carla.Souza.. c.25..30..23.
## 1                                     Ana Silva            25
```

```
## 2                                  Beto Pereira            30
## 3                                   Carla Souza            23
##   c.NA...Solteiro....Casada..
## 1                         <NA>
## 2                     Solteiro
## 3                       Casada
```

A propriedade (3) é atribuída aos *data frames* para que possamos aproveitar melhor dessa estrutura retangular dentro do R. Na prática, essas duas dimensões representam nada mais que as linhas e as colunas da base. Essa é a maior diferença entre uma lista e um *data frame*.

```
class(mtcars)
## [1] "data.frame"

dim(mtcars)
## [1] 32 11
```

O resultado do código `dim(mtcars)` nos dá as seguintes informações:

- O *data frame* mtcars tem duas dimensões (como todo *data frame*).
- A primeira dimensão tem comprimento 32 e a segunda dimensão tem comprimento 11. Em outras palavras: a base `mtcars` tem 32 linhas e 11 colunas.

Veja a seguir que listas não têm dimensão.

```
mtcars_como_lista <- as.list(mtcars)

class(mtcars_como_lista)
## [1] "list"

dim(mtcars_como_lista)
## NULL
```

Ter duas dimensões significa que devemos usar dois índices para acessar os valores de um *data frame* (fazer *subsetting*). Para isso, ainda usamos o colchete, mas agora com dois argumentos: `[linha, coluna]`.

```
mtcars[2, 3]
## [1] 160
```

O código acima está nos devolvendo o valor presente na segunda linha da terceira coluna da base `mtcars`.

Também podemos pegar todos as linhas de uma coluna ou todas as colunas de uma linha deixando um dos argumentos vazio:

```
# Todas as linhas da coluna 1
mtcars[,1]
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4

# Todas as colunas da linha 1
mtcars[1,]
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4
```

Por fim, lembrando que dento de cada coluna temos um vetor, podemos usar os testes lógicos para filtrar as linhas do nosso *data frame* conforme alguma regra.

```
mtcars$cyl == 4
##  [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE
## [25] FALSE  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE

mtcars[mtcars$cyl == 4, ]
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

O código `mtcars$cyl == 4` nos diz em quais linhas estão os carros com 4 cilindros. Quando usamos o vetor de `TRUE` e `FALSE` resultante dentro do *subsetting* das linhas em `mtcars[mtcars$cyl == 4, ]`, o R nos devolve todos as colunas dos carros com 4 cilindros. A regra é simples: linha com `TRUE` é retornada, linha com `FALSE` não.

Encerramos aqui a nossa primeira conversa sobre `data frames` para falar de outra estrutura muito importante dentro do R: as funções.

Exercícios

1. Quais códigos abaixo retornam **um vetor** com a coluna `mpg` do data frame `mtcars`?

- **a.** `mtcars$mpg`
- **b.** `mtcars[ , 3]`
- **c.** `mtcars("mpg")`
- **d.** `mtcars[ , "mpg"]`
- **e.** `mtcars.mpg`
- **f.** `mtcars[ , 1]`
- **g.** `mtcars[1, 1]`
- **h.** `mpg$mtcars`

2. Para que serve a função `str()`. Dê um exemplo do seu uso.

3. Para que serve a função `nanes()`. Dê um exemplo do seu uso.

4. Use o data frame `airquality` para responder às questões abaixo:

- **a.** Quantas colunas `airquality` tem?
- **b.** Quantas linhas `airquality` tem?
- **c.** O que a função `head()` retorna?
- **d.** Quais são os nomes das colunas?
- **e.** Qual é a classe da coluna `Ozone`?

5. Desafio. Calculando desvio-padrão no R. Use o data frame `airquality` para responder às questões abaixo:

- **a.** Tire a média da coluna Ozone e guarde em um objeto.
- **b.** Guarde em um objeto o vetor correspondente à coluna Ozone subtraída da sua própria média (calculada em no item a).
- **c.** Eleve o vetor calculado em (b) ao quadrado. Salve o resultado em um novo objeto.
- **d.** Tire a média do vetor calculado em (c) e salve o resultado em um objeto chamado `variancia`. Em seguida, calcule a raiz quadrada desse valor e salve em um objeto chamado `desvio_padrao`.

- **e.** Compare o valor de `desvio_padrao` com `sd(airquality$Ozone)` e pesquise por que os valores não são iguais. Dica: veja a documentação da função `sd()`.

6. Use o *data frame* `airquality` para responder às questões abaixo.

- **a.** Conte quantos `NAs` tem na coluna `Solar.R`.
- **b.** Filtre a tabela `airquality`com apenas linhas em que `Solar.R` é `NA`.
- **c.** Filtre a tabela `airquality` com apenas linhas em que `Solar.R` **não** é `NA`.
- **d.** Filtre a tabela `airquality` com apenas linhas em que `Solar.R` **não** é `NA` e `Month` é igual a 5.

3.11 Mais sobre funções

Funções são tão comuns e intuitivas (provavelmente você já usou funções no Excel), que mesmo sem termos abordado o tema com detalhes, nós conseguimos utilizar várias funções nas seções anteriores:

- a função `c()` foi utilizada para criar vetores;
- a função `class()` foi utilizada para descobrir a classe de um objeto;
- a família de funções `is.na()`, `is.nan()`, `is.infinite()` e `is.null` foram utilizadas para testar se um valor é `NA`, `NaN`, infinito ou `NULL`, respectivamente.

Diferentemente dos objetos, as funções podem receber **argumentos**. Argumentos são os valores que colocamos dentro dos parênteses e que as funções precisam para funcionar (calcular algum resultado). Por exemplo, a função `c()` precisa saber quais são os valores que formarão o vetor que ela irá criar.

```
c(1, 3, 5)
## [1] 1 3 5
```

Nesse caso, os valores `1`, `3` e `5` são os argumentos da função `c()`. **Os argumentos de uma função são sempre separados por vírgulas**.

Funções no R têm personalidade. Cada uma pode funcionar de um jeito diferente das demais, mesmo quando fazem tarefas parecidas. Por exemplo, vejamos a função `sum()`.

```
sum(1, 3)
## [1] 4
```

Como você deve ter percebido, essa função retorna a soma de seus argumentos. Também podemos passar um vetor como argumento, e ela retornará a soma dos elementos do vetor.

```
sum(c(1, 3))
## [1] 4
```

Já a função `mean()`, que calcula a média de um conjunto de valores, exige que você passe valores na forma de um vetor:

```
# Só vai considerar o primeiro número na média
mean(1, 3)
## [1] 1

# Considera todos os valores dentro do vetor na média
mean(c(1, 3))
## [1] 2
```

Como cada coluna de um *data frame* é um vetor, podemos calcular a média de uma coluna fazendo:

```
# Podemos passar esse vetor para a função mean()
mean(mtcars$mpg)
## [1] 20.09062
```

Também podemos usar argumentos para modificar o comportamento de uma função. O que acontece se algum elemento do vetor for `NA`?

```
mean(c(1, 3, NA))
## [1] NA
```

Como a função não sabe o valor do terceiro elemento do vetor, ela não sabe qual é a média desses 3 elementos e, então, devolve `NA`. Como é muito comum termos `NA` nas nossas bases de dados, é muito comum tentarmos calcular a média de uma coluna que tem `NA` e recebermos `NA` como resposta.

Na grande maioria dos casos, queremos saber a média de uma coluna apesar dos `NA`s. Isto é, queremos retirar os `NA`s e então calcular a média com os valores que conhecemos. Para isso, podemos utilizar o argumento `na.rm = TRUE` da função `mean()`.

```
mean(c(1, 3, NA), na.rm = TRUE)
## [1] 2
```

Esse argumento diz à função para remover os `NA`s antes de calcular a média. Assim, a média calculada é: `(1 + 3)/2`.

Claro que cada função tem os seus próprios argumentos e nem toda função terá o argumento `na.rm=`. Para saber quais são e como usar os argumentos de uma função, basta acessar a sua documentação:

```
help(mean)
```

Os argumentos das funções também têm nomes, que podemos ou não usar na hora de usar uma função. Veja por exemplo a função `seq()`.

```
seq(from = 4, to = 10, by = 2)
## [1]  4  6  8 10
```

Entre outros argumentos, ela possui os argumentos `from=`, `to=` e `by=`. O que ela faz é criar uma sequência (vetor) de `by` em `by` que começa em `from` e termina em `to`. No exemplo, criamos uma função de 2 em 2 que começa em 4 e termina em 10.

Também poderíamos usar a mesma função sem colocar o nome dos argumentos:

```
seq(4, 10, 2)
## [1]  4  6  8 10
```

Para utilizar a função sem escrever o nome dos argumentos, você precisa colocar os valores na ordem em que os argumentos aparecem. E se você olhar a documentação da função `seq()`, fazendo `help(seq)`, verá que a ordem dos argumentos é justamente `from=`, `to=` e `by=`.

Escrevendo o nome dos argumentos, não há problema em alterar a ordem dos argumentos:

```
seq(by = 2, to = 10, from = 4)
## [1]  4  6  8 10
```

Mas se especificar os argumentos, a ordem importa. Veja que o resultado será diferente.

```
seq(2, 10, 4)
## [1]  2  6 10
```

A seguir, apresentamos algumas funções nativas do R úteis para trabalhar com *data frames* :

- `head()` - Mostra as primeiras 6 linhas.
- `tail()` - Mostra as últimas 6 linhas.
- `dim()` - Número de linhas e de colunas.
- `names()` - Os nomes das colunas (variáveis).
- `str()` - Estrutura do *data frame*. Mostra, entre outras coisas, as classes de cada coluna.
- `cbind()` - Acopla duas tabelas lado a lado.
- `rbind()` - Empilha duas tabelas.

Além de usar funções já prontas, você pode criar a sua própria função. A sintaxe é a seguinte:

```
nome_da_funcao <- function(argumento_1, argumento_2) {

  # Código que a função irá executar

}
```

Repare que `function` é um nome reservado no R, isto é, você não pode criar um objeto com esse nome.

Um exemplo: vamos criar uma função que soma dois números.

```
minha_soma <- function(x, y) {
  soma <- x + y

  soma  # resultado retornado
}
```

Essa função tem os seguintes componentes:

- `minha_soma`: nome da função
- `x` e `y`: argumentos da função
- `soma <- x + y`: operação que a função executa
- `soma`: valor retornado pela função

Após rodarmos o código de criar a função, podemos utilizá-la como qualquer outra função do R.

```
minha_soma(2, 2)
## [1] 4
```

O objeto `soma` só existe *dentro da função*, isto é, além de ele não ser colocado no seu *environment*, ele só existirá na memória (RAM) enquanto o R estiver executando a função. Depois disso, ele será apagado. O mesmo vale para os argumentos `x` e `y`.

O valor retornado pela função representa o resultado que receberemos ao utilizá-la. Por padrão, **a função retornará sempre a última linha de código que existir dentro dela**. No nosso exemplo, a função retorna o valor contido no objeto `soma`, pois é isso que fazemos na última linha de código da função.

Repare que se atribuirmos o resultado a um objeto, ele não será mostrado no console:

```
resultado <- minha_soma(3, 3)

# Para ver o resultado, rodamos o objeto `resultado`
resultado
## [1] 6
```

Agora, o que acontece se a última linha da função não devolver um objeto? Veja:

```
minha_nova_soma <- function(x, y) {
  soma <- x + y
}
```

A função `minha_nova_soma()` apenas cria o objeto `soma`, sem retorná-lo como na função `minha_soma()`. Se utilizarmos essa nova função, nenhum valor é devolvido no console:

```
minha_nova_soma(1, 1)
```

No entanto, a última linha da função agora é a atribuição `soma <- x + y` e esse será o "resultado retornado". Assim, podemos visualizar o resultado da função fazendo:

```
resultado <- minha_nova_soma(1, 1)

resultado
## [1] 2
```

É como se, por trás das cortinas, o R estivesse fazendo `resultado <- soma <- x + y`, mas apenas o objeto `resultado` continua existindo, já que os objetos `soma`, `x`e `y` são descartados após a função ser executada.

Claro que, na prática, é sempre bom criarmos funções que retornem na tela os seus resultados, para evitar esse passo a mais se quisermos apenas ver o resultado no console. Assim, a função `minha_soma()` costuma ser preferível com relação à função `minha_nova_soma()`.

Exercícios

1. Qual dos códigos abaixo devolverá um erro se for avaliado?

- **a.** `3 * 5 + 10`
- **b.** `function <- 10`
- **c.** `mean(1, 10)`
- **d.** `(soma <- sum(1, 1))`

2. Crie uma função que receba um número e retorne o quadrado deste número.

3. Crie uma função que receba 2 números e devolva a raiz quadrada da soma desses números.

4. Crie uma função que receba dois valores (numéricos) e devolva o maior deles.

5. Use a função `runif()` para criar uma função que retorne um número aleatório inteiro entre 0 e 10 (0 e 10 inclusive). Caso você não conheça a função `runif()`, rode `help(runif)` para ler a sua documentação.

6. Rode `help(sample)` para descobrir o que a função `sample()` faz. Em seguida

- **a.** use-a para escrever uma função que devolva uma linha aleatória de um *data frame*;
- **b.** generalize a função para retornar um número qualquer de linhas, escolhido pelo usuário.

3.12 Controle de Fluxo

Como toda boa linguagem de programação, o R possui estruturas de `if`, `else`, `for` e `while`. Esses **controles de fluxo** são muito importantes na hora de

programar, pois nos permitem manipular de modo eficiente as ações do computador.

A seguir, explicaremos para que servem e como utilizar cada uma dessas estruturas.

3.12.1 Condicionamento: if e else

As estruturas `if` e `else` servem para executarmos um código apenas se uma condição (teste lógico) for satisfeita.

No código abaixo, a função `Sys.time()`, que retorna a data/hora no momento da execução, só será avlaiada se o objeto `x` for igual a 1.

```
# Não vai executar a função Sys.time()
x <- 2

if (x == 1) {
  Sys.time()
}

# Vai executar a função Sys.time()
x <- 1

if (x == 1) {
  Sys.time()
}
## [1] "2021-07-14 14:31:30 -03"
```

O R só vai executar o que está na expressão dentro das chaves `{}` se a expressão que estiver dentro dos parênteses `()` retornar `TRUE`. Veja outro exemplo:

```
# Vai fazer  a soma
x <- c(1, 3, 10, 15)

if (class(x) == "numeric") {
  sum(x)
}
## [1] 29

# Não vai fazer a soma
x <- c("a", "b", "c")

if (class(x) == "numeric") {
```

```
  sum(x)
}
```

Nesse exemplo, a soma só é executada se a classe do objeto `x` for numérica, isto é, se `x` for um vetor de números. Essa verificação poderia ser colocada dentro de uma função para evitarmos que ela retorne um erro.

```
minha_soma <- function(x, y) {
  if (class(x) == "numeric" & class(y) == "numeric") {
     x + y
  }
}

# Retorna a soma
minha_soma(1, 2)
## [1] 3

# Não retorna nada
minha_soma("a", "b")
```

Nesses casos, é muito comum o uso das funções `return()` e `stop()` para, respectivamente, retornar um resultado antecipadamente ou para a execução da função e devolver ao usuário uma mensagem de erro personalizada.

Um exemplo usando `return()`.

```
# Devolvendo um resultado antecipadamente
minha_soma_NA <- function(x, y) {
  if (class(x) == "numeric" & class(y) == "numeric") {
     soma <- x + y
     return(soma)
  }

  NA
}

# Retorna a soma
minha_soma_NA(1, 2)
## [1] 3

# Retorna NA
minha_soma_NA("a", "b")
## [1] NA

# Retorna NA
```

```
minha_soma_NA(1, "b")
## [1] NA
```

Na função `minha_soma_NA()`, a soma só é calculada e retornada se `x` e `y` forem numéricos. Caso pelo menos um dos dois não seja, o código dentro do `if` não é executado e o valor retornado é o `NA`.

Agora, usando `stop()`.

```
# Agora, devolvendo um erro
minha_soma_erro <- function(x, y) {
  if (class(x) != "numeric" | class(y) != "numeric") {
    stop("A classe dos objetos x e y deve ser numérica.")
  }

  x + y
}

# Retorna a soma
minha_soma_erro(1, 2)
## [1] 3

# Retorna erro
minha_soma_erro("a", "b")
## Error in minha_soma_erro("a", "b"): A classe dos objetos x e y deve ser numérica.

# Retorna erro
minha_soma_erro(1, "b")
## Error in minha_soma_erro(1, "b"): A classe dos objetos x e y deve ser numérica.
```

Na função `minha_soma_erro()`, testamos no `if` se a classe de `x` **ou** a classe de `y` é diferente de `numeric`, isto é, se pelo menos um dos dois não é um número. Se esse teste retornar `TRUE`, a função para a sua execução e devolve para o usuário a seguinte mensagem de erro: "A classe dos objetos x e y deve ser numérica.". Se o teste retorna `FALSE`, a soma é realizada e seu resultado nos é retornado.

O `else` funciona como uma extensão do `if`, dando uma alternativa caso o teste executado seja falso.

```
# Vai fazer  a soma
x <- c(1, 3, 10, 15)

if (class(x) == "numeric") {
  sum(x)
```

```
} else {
  NA
}
## [1] 29

# Vai retornar NA
x <- c(1, 3, 10, "15")

if (class(x) == "numeric") {
  sum(x)
} else {
  NA
}
## [1] NA
```

Também podemos usar o `else` para encadear vários `if`s. Teste o código abaixo com valores positivos e negativos para `x`.

```
x <- 0

if(x < 0) {

  "negativo"

} else if(x == 0) {

  "neutro"

} else if(x > 0) {

  "positivo"
}
## [1] "neutro"
```

Repare que o `if` no último `else` poderia ser omitido.

```
x <- 0

if(x < 0) {

  "negativo"

} else if(x == 0) {

  "neutro"
```

```
} else {

  "positivo"
}
## [1] "neutro"
```

3.12.2 Iteradores: for e while

O `for` pode ser utilizado para fazer os famosos *loopings* de programação, isto é, repetir uma mesma tarefa para um conjunto de valores diferentes. Cada repetição é chamada de iteração e o objeto que muda de valor em cada **interação** é chamado de **iterador**.

```
numero_de_colunas <- ncol(mtcars)

for (coluna in 1:numero_de_colunas) {
  media <- mean(mtcars[,coluna])

  print(media)
}
## [1] 20.09062
## [1] 6.1875
## [1] 230.7219
## [1] 146.6875
## [1] 3.596563
## [1] 3.21725
## [1] 17.84875
## [1] 0.4375
## [1] 0.40625
## [1] 3.6875
## [1] 2.8125
```

O código acima vai calcular a média de cada coluna do *data frame* `mtcars`. Alguns pontos importantes:

- No exemplo, temos 11 iterações e o objeto `coluna` é o iterador.
- Como `numero_de_colunas` é igual a 11, a expressão `1:numero_de_colunas` cria uma sequência de números de 1 a 11.
- A expressão `coluna in 1:numero_de_colunas` indica que o valor de `coluna` será 1 na primeira iteração, 2 na segunda iteração, 3 na terceira e assim por diante.

- O código dentro do `for` não é retornado para o usuário ao fim de cada iteração. Por isso, para ver os resultados no Console, usamos a função `print()`.

Também podemos salvar as médias em um vetor.

```
numero_de_colunas <- ncol(mtcars)

# Antes, criamos um vetor vazio.
medias <- c()

for (coluna in 1:numero_de_colunas) {
  medias[coluna] <- mean(mtcars[,coluna])
}

medias
##  [1]  20.090625   6.187500 230.721875 146.687500   3.596563   3.217250
##  [7]  17.848750   0.437500   0.406250   3.687500   2.812500
```

Assim como o `for`, o `while` também é um iterador.

O código a seguir irá imprimir na tela o valor de `i` enquanto este objeto for menor que 3. No momento em que a condição dentro das chaves `{}` não for mais respeitada, o processo será interrompido.

```
i <- 1

while (i < 3){
  print(i)
  i <-  i + 1
}
## [1] 1
## [1] 2
```

É importante que o valor de i seja atualizado em cada interação, caso contrário a função entrará em um loop infinito. Por isso fazemos `i <- i + 1` após o `print`.

Exercícios

1. Por que o código abaixo retorna erro? Arrume o código para retornar o valor `TRUE`.

```
x <- 4
if(x = 4) {
  TRUE
}
```

2. Usando `if` e `else`, escreva um código que retorne a string `"número"` caso o valor seja da classe `numeric` ou `integer`; a string `"palavra"` caso o valor seja da classe `character`; e `NA` caso contrário.

3. Usando apenas `for` e a função `length()`, construa uma função que calcule a média de um vetor número qualquer. Construa uma condição para a função retornar `NULL` caso o vetor não seja numérico.

4. Utilize o vetor `a` para resolver as questões a seguir:

```
a <- c(10, 3, 5, -1, 3, -4, 8, 9, -10)
```

- **a.** Utilize o `for` para imprimir as médias acumuladas do vetor `a`, isto é, primeiro vamos imprimir 10, depois a média entre 10 e 3, depois a média entre 10, 3 e 5 e assim por diante.
- **b.** Adapte o laço que você fez no item anterior para ignorar os valores negativos, isto é, em caso de valor negativo, o laço não deve calcular a média e não imprimir nada.

3.13 Outros tópicos

Nesta seção, apresentamos alguns tópicos extras. Alguns deles serão retomanos em capítulos posteriores.

3.13.1 Matrizes

As matrizes no R podem ser tratadas como vetores com duas dimensões. Por serem vetores, elas só podem conter elementos de uma mesma classe. Por possuírem duas dimensões, as operações de *subsetting* devem ser realizadas utilizando a sintaxe `matriz[linha, coluna]`.

Para criar uma matriz, utilizamos a função `matrix()`. Precisamos definir quais elementos formarão a matriz e qual será o número de linhas e colunas.

```
# Uma matriz de 2 linhas e 3 colunas
m <- matrix(1:9, nrow = 3, ncol = 3)

m
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9

dim(m)
## [1] 3 3
```

Repare que os números de 1 a 9 foram dispostos na matriz coluna por coluna (*column-wise*), ou seja, preenchendo de cima para baixo e depois da esquerda para a direita. Esse comportamento pode ser alterado se utilizarmos o argumento `byrow=`.

```
matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

Subsetting de matrizes é muito parecido com o de *data frames*.

```
# Seleciona a terceira linha
m[3, ]
## [1] 3 6 9

# Seleciona a segunda coluna
m[ , 2]
## [1] 4 5 6

# Seleciona o primeiro elemento da segunda coluna
m[1, 2]
## [1] 4
```

A seguir, apresentamos algumas operações úteis para trabalhar com matrizes.

```
# Matriz transposta
t(m)
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9

# Matriz identidade 3 por 3
n <- diag(3)
```

```
n
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1

# Multiplicação por escalar
n * 2
##      [,1] [,2] [,3]
## [1,]    2    0    0
## [2,]    0    2    0
## [3,]    0    0    2

# Multiplicação matricial
m %*% n
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9

# Matriz inversa de m
n2 <- n * 2
solve(n2)
##      [,1] [,2] [,3]
## [1,]  0.5  0.0  0.0
## [2,]  0.0  0.5  0.0
## [3,]  0.0  0.0  0.5
```

3.13.2 Fatores

Fatores são uma classe de objetos no R criada para representar as variáveis categóricas numericamente. Essa classe possui um atributo especial: os `levels`. *Levels* são nada mais do que as categorias possíveis de uma variável categórica.

Como exemplo, imagine que o objeto `sexo` guarde uma coluna que indica o sexo de uma pessoa: `F` para feminino e `M` para masculino. Normalmente, essa coluna seria importada para o R como texto. Podemos transformá-la em fator utilizando a função `as.factor()`.

```
# Variável sexo como texto
sexo <- c("F", "M", "M", "M", "F", "F", "M")
sexo
## [1] "F" "M" "M" "M" "F" "F" "M"
```

```
# Variável sexo, como fator
as.factor(sexo)
## [1] F M M M F F M
## Levels: F M
```

Repare que a saída do objeto `sexo` quando o transformamos em fator tem uma informação a mais. Na última linha, visualizamos os `levels` desse fator, isto é, um conjunto das categorias possíveis do fator `sexo` (no caso, F e M).

Por padrão, **os levels são ordenados por ordem alfabética**. Veremos mais adiante que isso pode fazer diferença na construção de gráficos e na aplicação de modelos.

A diferença entre fatores e texto dentro do R é como eles são representados internamente. Enquanto objetos da classe `character` realmente são representados como texto, fatores são representados como números inteiros.

```
# Em geral, não é possível transforma textos em números
as.numeric(sexo)
## Warning: NAs introduced by coercion
## [1] NA NA NA NA NA NA NA

# Mas podemos transforrma fatores em inteiros
fator <- as.factor(sexo)
as.numeric(fator)
## [1] 1 2 2 2 1 1 2
```

Internamente, cada level de um fator é representado como um inteiro. No exemplo anterior, o level `F` está sendo representado como `1` e o level `M` como `2`. Se um fator tiver 10 levels, teremos os inteiros de `1` a `10` representando esse fator.

```
# letters é um objeto nativo do R
letras <- letters[1:10]

fator <- as.factor(letras)
fator
##  [1] a b c d e f g h i j
## Levels: a b c d e f g h i j

as.numeric(fator)
##  [1]  1  2  3  4  5  6  7  8  9 10
```

O texto que vemos quando avaliamos um fator (`F` e `M` em vez de `1` e `2`, por exemplo) é apenas uma "etiqueta" que o R coloca em cima dos inteiros. As diferentes etiquetas de um fator são justamente os `levels`.

Como fatores são sempre representados interrnamente por inteiros sequencias começando do 1 (1, 2, 3, ...) e esses inteiros são sempre atribuídos conforme a ordem alfabética dos `levels`, um erro muito comum é tentar transformar levels numéricos em números:

```
# Texto
vetor <- c("10", "55", "55", "12", "10", "-5", "-90")
vetor
## [1] "10"  "55"  "55"  "12"  "10"  "-5"  "-90"

# Fator
fator <- as.factor(vetor)
fator
## [1] 10  55  55  12  10  -5  -90
## Levels: -5 -90 10 12 55

# Número
as.numeric(fator)
## [1] 3 5 5 4 3 1 2
```

Quando transformamos o objeto `vetor` em um fator, o R não enxerga os "números" dentro dele. Para o R, é tudo texto. Então, como nos outros exemplos, cada "número" será representado por um inteiro, atribuído pela ordem alfabética.

Uma forma de evitar esse problema é transformar o fator em texto antes de transformá-lo em número.

```
# Voltando para texto
texto <- as.character(fator)
texto
## [1] "10"  "55"  "55"  "12"  "10"  "-5"  "-90"

# Agora sim os números originais
as.numeric(texto)
## [1]  10  55  55  12  10  -5 -90
```

3.13.3 Gráficos (R base)

O R já vem com funções básicas que fazem gráficos estatísticos de todas as naturezas.

- Vantagens: são rápidas e simples.
- Desvantagens: os gráficos são simplórios e geralmente é difícil gerar gráficos mais elaborados.

Nesta seção, mostraremos como construir alguns tipos de gráficos usando as funções base do R, mas o nosso foco em visualização de dados está nas funções do pacote `ggplot2`, apresentadas no Capítulo 8.

Gráfico de dispersão e linhas

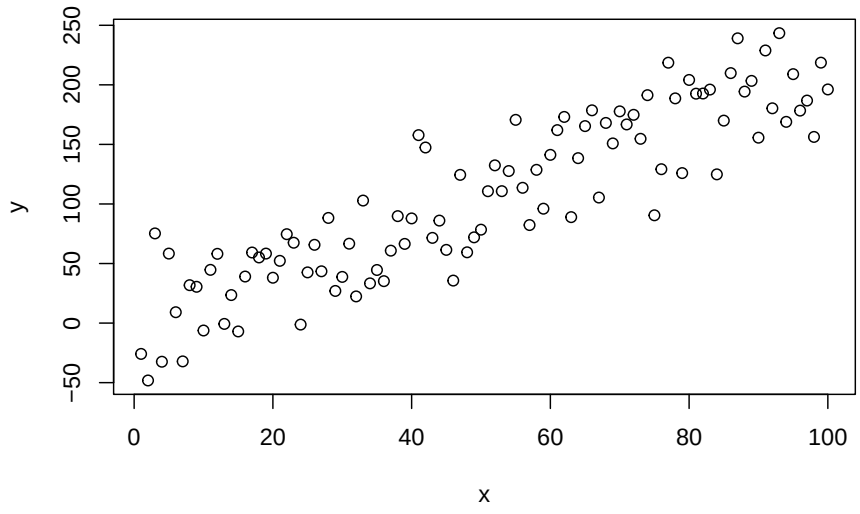
Para construir um gráfico de dispersão, utilizamos a função `plot()`. Seus principais parâmetros são:

- `x`, `y`: vetores para representarem os eixos x e y.
- `type`: tipo de gráfico. Pode ser pontos, linhas, escada, entre outros.

Para mais detalhes sobre os argumentos, ver `help(plot)`.

```
N <- 100
x <- 1:N
y <- 5 + 2 * x + rnorm(N, sd = 30)

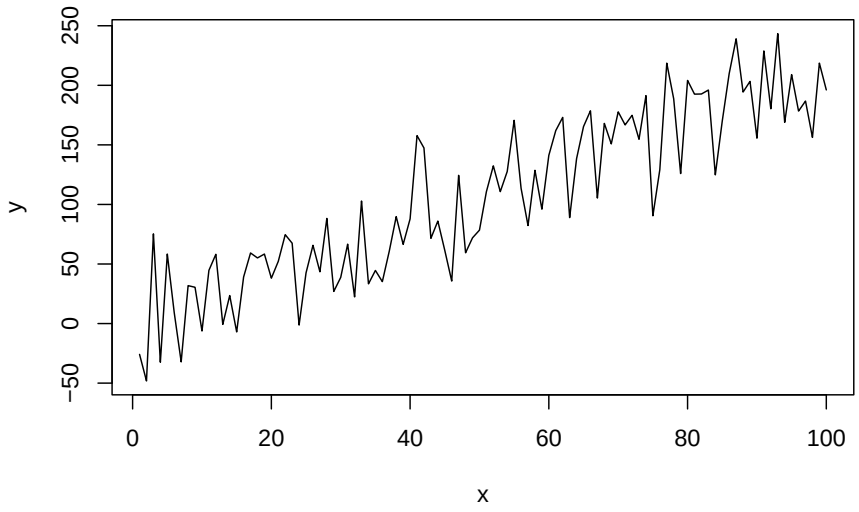
plot(x, y)
```



No código acima, a função `rnorm()` gera uma amostra aleatória da distribuição Normal com média 0 e desvio-padrão 30.

O parâmetro `type = "l"` indica que queremos que os pontos sejam interligados por linhas.

```
plot(x, y, type = "l")
```

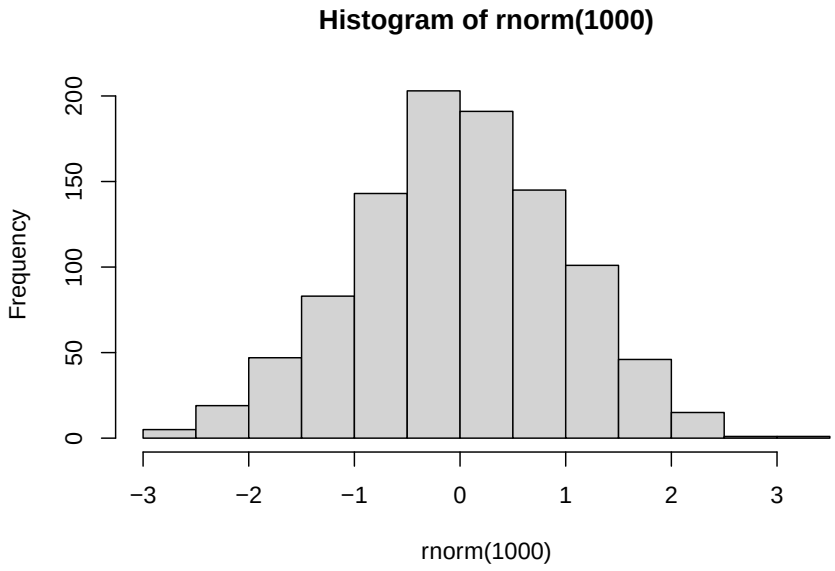


Histograma

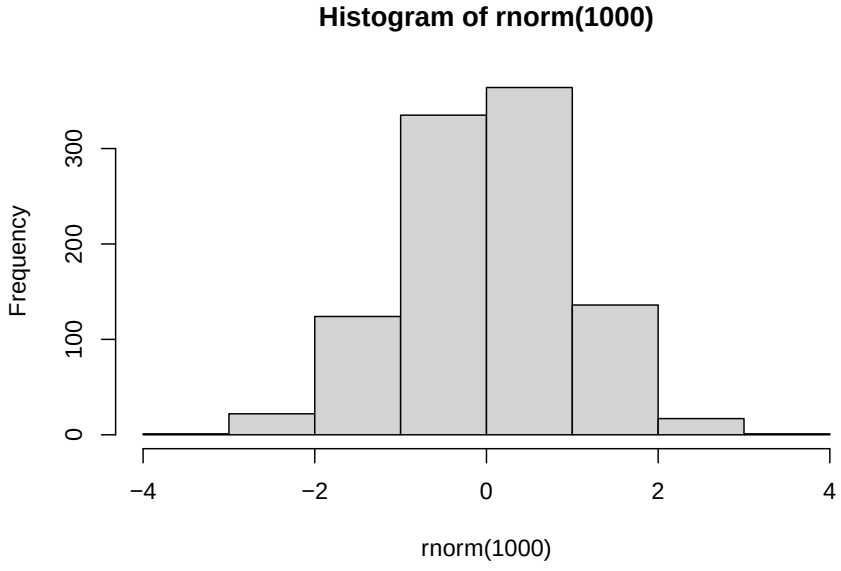
Para construir histogramas, utilizamos a função `hist()`. Os principais parâmetros são:

- `x`: o vetor numérico para o qual o histograma será construído.
- `breaks`: número (aproximado) de retângulos.

```
hist(rnorm(1000))
```




```
hist(rnorm(1000), breaks = 6)
```

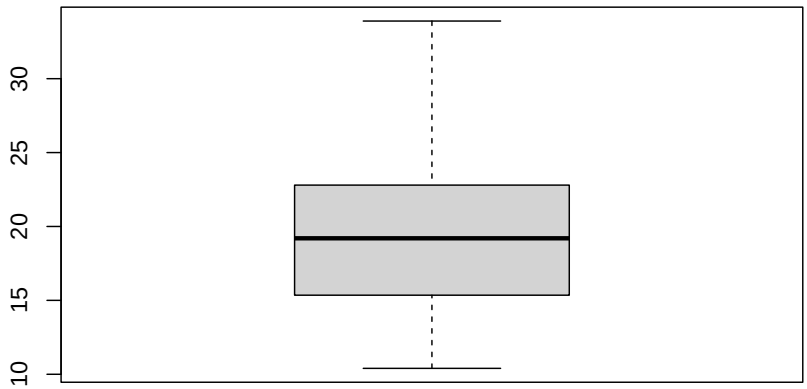


Boxplot

Para construir boxplots, utilizamos a função `boxplot()`. Os principais parâmetros são:

- `x`: o vetor numérico para o qual o boxplot será construído.

```
boxplot(mtcars$mpg, col = "lightgray")
```



Observe que o argumento `col=` muda a cor da caixa do boxplot.

Para mapear duas variáveis ao gráfico, passamos um *data frame* para o argumento `data=` e utilizamos a seguinte sintaxe `var_numerica ~ var_categorica`.

```
boxplot(mpg ~ cyl, data = mtcars, col = "purple")
```

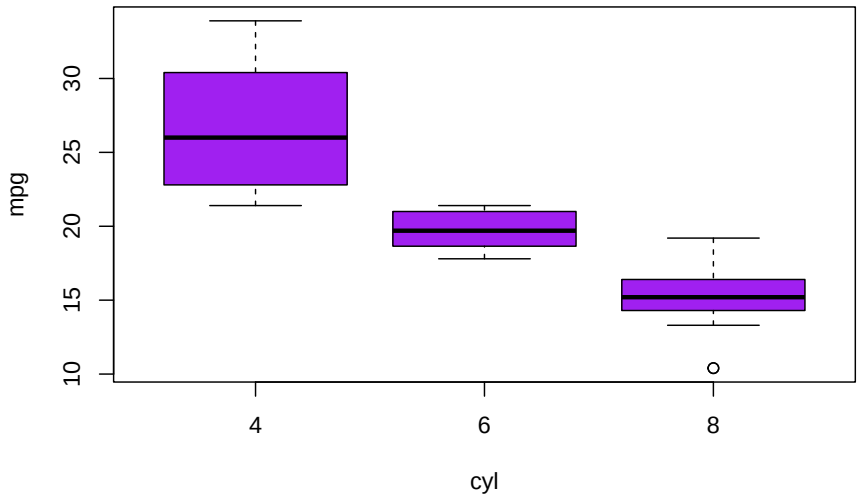


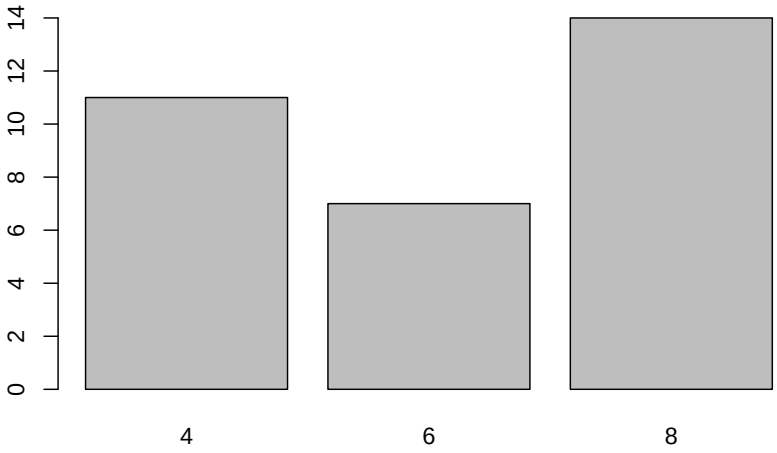
Gráfico de barras

Para construir gráficos de barras, precisamos combinar as funções `table()` e `barplot()`.

No gráfico abaixo, primeiro criamos uma tabela de frequências com a função `table()` e, em seguida, construímos o gráfico com a função `barplot()`.

```
tabela <- table(mtcars$cyl)
tabela
##
##  4  6  8
## 11  7 14

barplot(tabela)
```


0
2
4
6
8
10
12
14
4
6
8

Chapter 4

Pacotes

A primeira aparição da linguagem de progromação R foi em 1993 e, apesar de lá para cá muita coisa ter sido desenvolvida e atualizada, é muito difícil fazer mudanças na base da linguagem sem quebrar os códigos já existentes. Por essa razão, as maiores inovações são realizadas na forma de pacotes.

Um pacote é um conjunto de funções que têm como objetivo resolver um problema específico. São eles que deixam o R poderoso, capaz de enfrentar qualquer tarefa de análise de dados. Assim, fique bastante à vontade para instalar e atualizar muitos e muitos pacotes ao longo da sua experiência com o R.

O legal é que qualquer pessoa pode fazer um novo pacote e disponibilizar para a comunidade, o que acelera bastante o desenvolvemento da ferramenta. Dificilmente você vai fazer uma análise apenas com as funções básicas do R e dificilmente não vai existir um pacote com as funções que você precisa.

4.1 Instalação de pacotes

Existem três principais maneiras de instalar pacotes. Em ordem de frequência, são:

- Via CRAN (Comprehensive R Archive Network): `install.packages("nome-do-pacote")`.
- Via Github: `devtools::install_github("nome-do-repo/nome-do-pacote")`.
- Via arquivo .zip/.tar.gz: `install.packages("C:/caminho/nome-do-pacote.zip", repos = NULL)`.

Para conseguir instalar alguns pacotes no Linux, você pode precisar instalar dependências do sistema manualmente. Por exemplo, se você quer instalar o

pacote devtools no R, será necessário ter as bibliotecas `curl`, `openssl`, `httr` e `git2r`.

Essas dependências geralmente podem ser instaladas no terminal por meio do comando `apt-get install nome-da-biblioteca`. Caso você não consiga instalar um pacote devido a ausência de uma dependência, uma maneira de saber quais bibliotecas você precisa instalar é observar as mensagens que aparecem no console durante a tentativa da instalação do pacote.

4.1.1 Via CRAN

Instale pacotes que não estão na sua biblioteca usando a função `install.packages("nome_do_pacote")`. Por exemplo:

```
install.packages("tidyverse")
```

E, de agora em diante, não precisa mais instalar. Basta carregar o pacote com `library(magrittr)`.

> Escreva nome_do_pacote::nome_da_funcao() se quiser usar apenas uma função de um determinado pacote. O operador :: serve para isso. Essa forma também é útil quando se tem duas funções com o mesmo nome e precisamos garantir que o código vá usar a função do pacote correto.

4.1.2 Via Github

Desenvolvedores costumam disponibilizar a última versão de seus pacotes no Github, e alguns deles sequer estão no CRAN. Mesmo assim ainda é possível utilizá-los instalando diretamente pelo github. O comando é igualmente simples:

```
devtools::install_github("rstudio/shiny")
```

Apenas será necessário o username e o nome do repositório (que geralmente tem o mesmo nome do pacote). No exemplo, o username foi “rstudio” e o repositório foi “shiny”.

Se você não é familiar com o github, não se preocupe! Os pacotes disponibilizados na plataforma geralmente têm um `README` cuja primeira instrução é sobre a instalação. Se não tiver, provavelmente este pacote não te merece! =)

4.1.3 Via arquivo .zip ou .tar.gz

Se você precisar instalar um pacote que está zipado no seu computador (ou em algum servidor), utilize o seguinte comando:

```
install.packages("C:/caminho/para/o/arquivo/zipado/nome-do-pacote.zip", repos = NULL)
```

É semelhante a instalar pacotes via CRAN, com a diferença que agora o nome do pacote é o caminho inteiro até o arquivo. O parâmetro `repos = NULL` informa que estamos instalando a partir da máquina local.

A aba ***Packages*** do RStudio também ajuda a administrar os seus pacotes.

4.2 Tidyverse

Muitas pessoas tentam definir *o que é* ciência de dados no mercado e na academia. O problema é que esse termo pode ser descrito de várias formas distintas, seja pela formação específica da pessoa que define ou do interlocutor ao qual ela se comunica. Por isso, a definição de ciência de dados é, de certa forma, vazia.

No entanto, é possível definir *como se faz* ciência de dados. Ou seja, independentemente da definição do termo, o que temos de fazer na prática em projetos reais é algo bastante conhecido.

O "como faz" é definido através do *Ciclo da Ciência de Dados*, descrito na Figura 4.1. Primeiro, os dados brutos são coletados de fontes públicas, como arquivos Excel, portais de dados abertos ou bases de dados internos da companhia. Em seguida, os dados são arrumados, para mitigar problemas de padronização de nomes, obtenção das variáveis de interesse e exclusão de casos que estão fora do escopo de análise, produzindo o que se define como base de dados analítica. A base analítica é então transformada para produzir as tabelas e gráficos e, quando necessário, são utilizadas como insumo para o ajuste de modelos estatísticos. Finalmente, os resultados obtidos são comunicados através de uma série de ferramentas, como relatórios, dashboards interativos, indicadores ou Application Programming Interfaces (API) para automação.

O `{tidyverse}` é um pacote guarda-chuva que consolida uma série de ferramentas que fazem parte o ciclo da ciência de dados. Fazem parte do `{tidyverse}` os pacotes `{ggplot2}`, `{dplyr}`, `{tidyr}`, `{purrr}`, `{readr}`, entre muitos outros, como é possível observar na Figura 4.2. Veremos as características principais desses pacotes nas próximas Seções.

O `{tidyverse}` traz consigo o *manifesto tidy*. Trata-se de um documento que formaliza uma série de princípios que norteiam o desenvolvimento do tidyverse. Como os pacotes do `{tidyverse}` compartilham os mesmos princípios, podem ser utilizados naturalmente em conjunto.

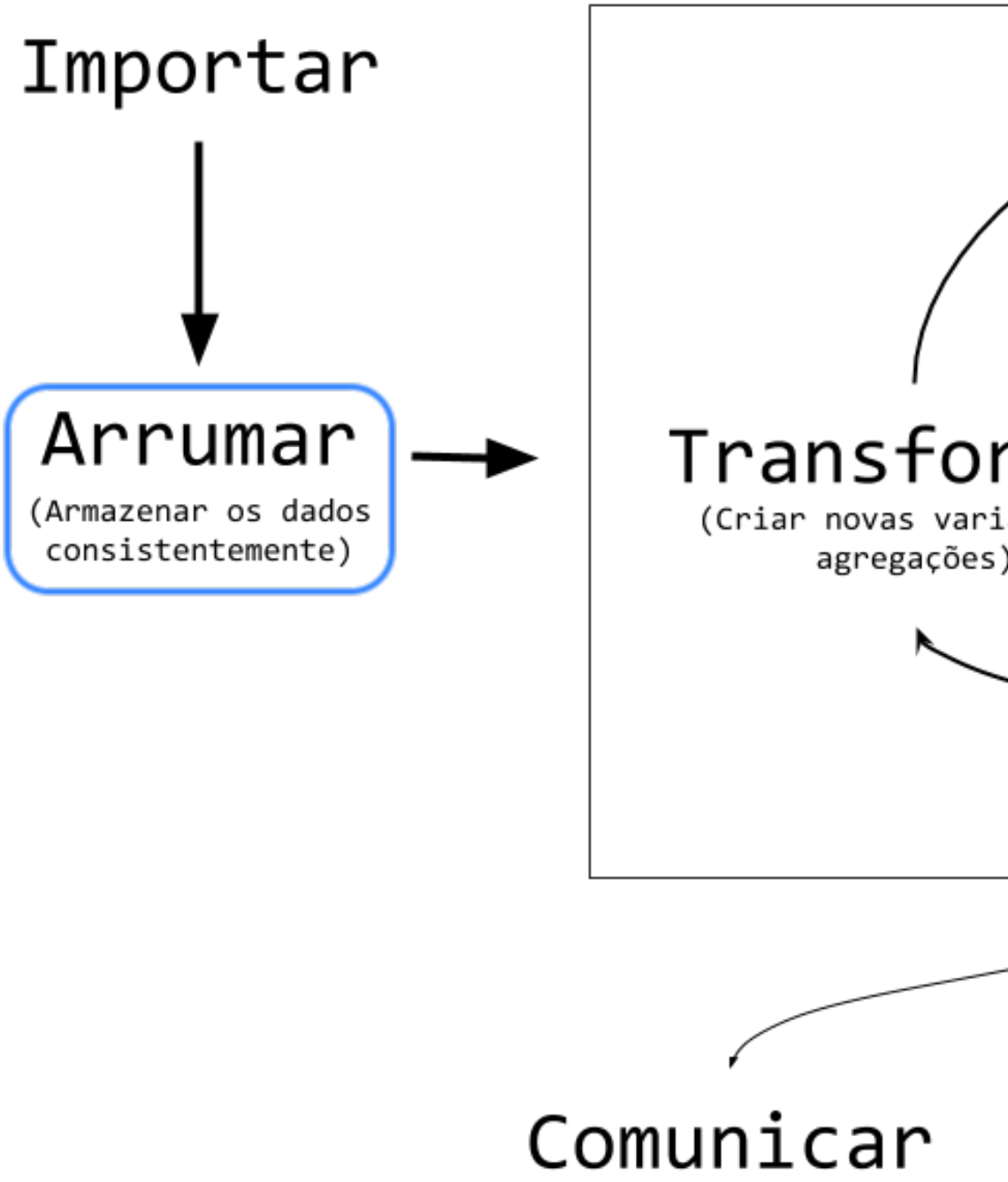


Figure 4.1: O Ciclo da Ciência de Dados.

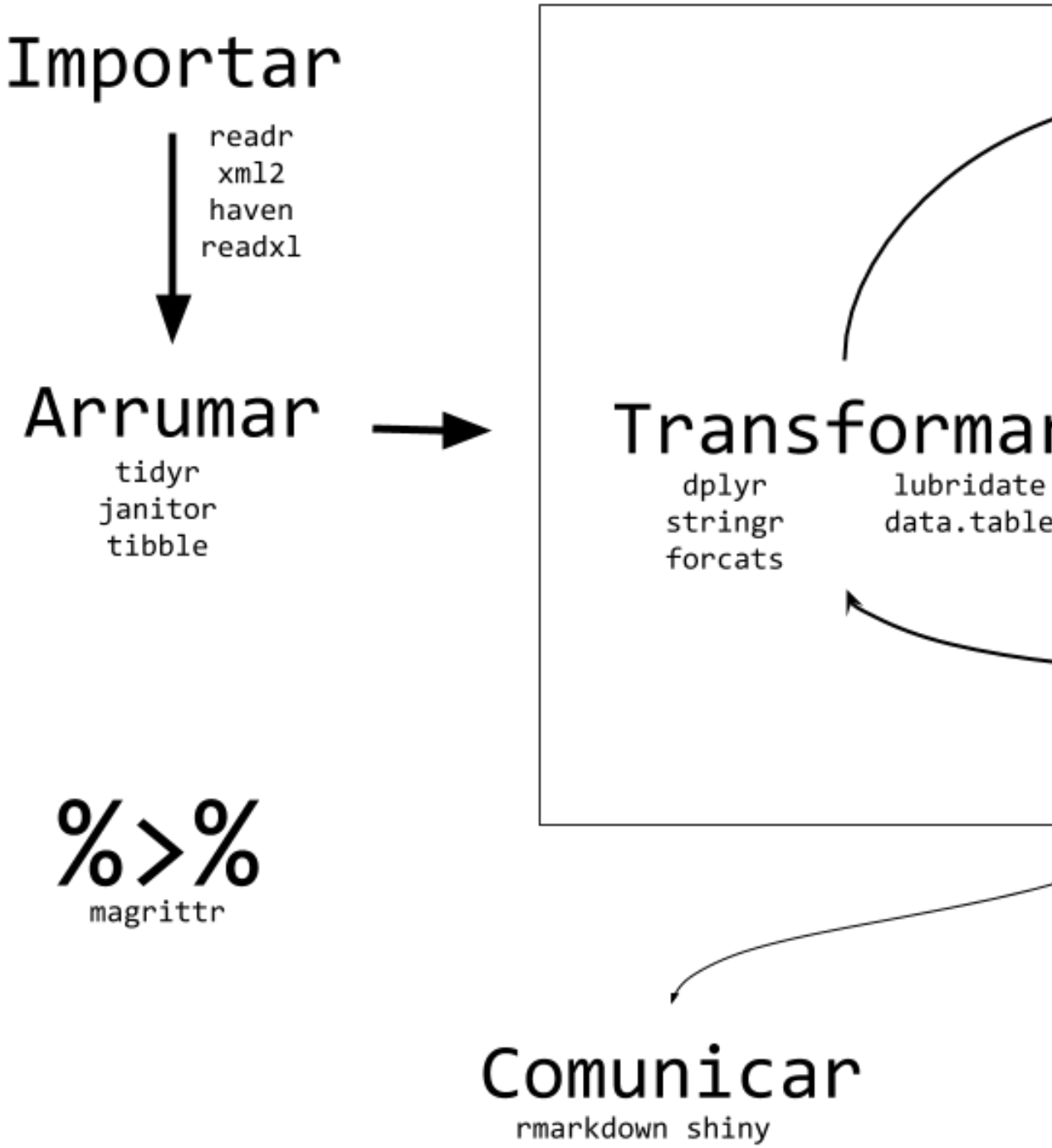


Figure 4.2: A implementação do Ciclo da Ciência de Dados, através do tidyverse. Pela definição estrita do tidyverse, na imagem não fazem parte do tidyverse os pacotes janitor, data.table e os pacotes descritos nas partes de modelagem, comunicação e automatização. No entanto, a maioria desses pacotes também seguem os princípios tidy e podem ser usados em conjunto com o tidyverse sem grandes dificuldades.

Pode-se dizer que existe uma linguagem R antes e outra depois do `{tidyverse}`. A linguagem mudou muito, a comunidade abraçou uso desses princípios e criou centenas de novos pacotes que conversam uns com os outros dessa forma.[1]

Os princípios fundamentais do tidyverse são:

1. Reutilizar estruturas de dados existentes.
2. Organizar funções simples usando o *pipe* (Seção 6).
3. Aderir à programação funcional (Seção 10).
4. Projetado para ser usado por seres humanos.

No texto do manifesto tidy cada um dos lemas é descrito de forma detalhada. No nosso blog, selecionamos os aspectos que achamos mais importante de cada um deles.

[1] Usar a filosofia tidy não é a única forma de fazer pacotes do R. Existem muitos pacotes excelentes que não utilizam essa filosofia. O próprio manifesto diz "O contrário de tidyverse não é o messyverse, e sim muitos outros universos de pacotes interconectados."

Na prática, carregar o (veja o código abaixo) é o mesmo que carregar os seguintes pacotes:

- `{tibble}` para *data frames* repaginados;
- `{readr}` para importarmos bases para o R;
- `{tidyr}` e `{dplyr}` para arrumação e manipulação de dados;
- `{stringr}` para trabalharmos com textos;
- `{forcats}` para trabalharmos com fatores;
- `{ggplot2}` para gráficos;
- `{purrr}` para programação funcional.

Embora o `{tidyverse}` instale diversos outros pacotes, apenas esses são carrregados. Dificilmente fazemos uma análise de dados em que não precisamos usá-los. Falaremos com mais detalhes de todos eles neste livro.

```
library(tidyverse)
```

```
## -- Attaching packages --------------------------------------- tidyverse 1.3.1 --

## v ggplot2 3.3.5     v purrr   0.3.4
## v tibble  3.1.2     v dplyr   1.0.7
## v tidyr   1.1.3     v stringr 1.4.0
## v readr   1.4.0     v forcats 0.5.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

Mensagens de conflito quando carregamos o `{tidyverse}` ou qualquer outro pacote significam que funções anteriormente carregadas foram *mascaradas* por novas funções. No exemplo acima, as funções `filter()` e `lag()` do pacote `stats` foram substituídas na sessão pelas funções `filter()` e `lag()` do pacote `dplyr`. Nesse caso, se quiséssemos usar as funções do pacote `stats` após carregar o `{tidyverse}`, precisaríamos rodar `stats::filter()` e `stats::lag()`.

Se você quiser *descarregar* um pacote, reinicie a sua sessão em `Session > Restart R` ou use a função `detach()` como no exemplo abaixo.

```
detach("package:tidyverse", unload = TRUE)
```

Chapter 5

Importação

Nesta seção, vamos introduzir os principais pacotes para importar dados para o R. Mostraremos como importar dados de arquivos de texto, planilhas do excel e extensões de outros programas estatísticos (SAS e SPSS, por exemplo).

Antes de começarmos, vale a pena destacarmos um ponto importante. As funções de importação do `tidyverse` importam dados em objetos da classe `tibble`, que difere da classe `data.frame` usual em dois pontos importantes:

- imprime os dados na tela (Console) de maneira muito mais organizada, resumida e legível;
- permite a utilização de *list-columns*.

Se você não estiver familiarizado com o conceito de *list-columns*, não se preocupe. Trataremos melhor do assunto no Capítulo 10.

5.1 Caminhos

Um passo importante na tarefa de importação de dados para o R é saber onde está o arquivo que queremos importar. Toda função de importação vai exigir um **caminho**, uma string que representa o endereço do arquivo no computador.

Há duas formas de passarmos o caminho de arquivo: usar o **caminho absoluto** ou usar o **caminho relativo**.

Antes de falarmos sobre a diferença dos dois, precisamos definir o que é o **diretório de trabalho**.

O diretório de trabalho (*working directory*) nada mais é do que a pasta em que o R vai procurar arquivos na hora de ler informações ou gravar arquivos na hora de salvar objetos.

Se você está usando um projeto, o diretório de trabalho da sua sessão será, por padrão, a pasta raiz do seu projeto (é a pasta que contém o arquivo com extensão `.Rproj`). Se você não estiver usando um projeto ou simplesmente não souber qual é o seu diretório de trabalho, você pode descobri-lo usando a seguinte função `getwd()`. Ela vai devolver uma string com o caminho do seu diretório de trabalho.

A função `setwd()` pode ser utilizada para mudar o diretório de trabalho. Como argumento, ela recebe o caminho para o novo diretório.

Caminhos absolutos são aqueles que tem início na pasta raiz do seu computador/usuário. Por exemplo:

```
getwd()
```

```
## [1] "/Users/clente/Documents/livro-material"
```

Esse é o caminho absoluto para a pasta onde esse livro foi produzido.

Na grande maioria dos casos, caminhos absolutos são uma **má prática**, pois deixam o código irreprodutível. Se você trocar de computador ou passar o script para outra pessoa rodar, o código não vai funcionar, pois o caminho absoluto para o arquivo muito provavelmente será diferente.

Caminhos relativos são aqueles que tem início no diretório de trabalho da sua sessão. Assim, se você quiser acessar um arquivo `minha_base.csv` dentro de uma pasta "dados" existente no seu diretório de trabalho, você poderia passar para o R o caminho `"dados/minha_base.csv"`.

Trabalhar com projetos no RStudio ajuda bastante o uso de caminhos relativos, pois nos incentiva a colocar todos os arquivos da análise dentro da pasta do projeto. Assim, se você usar apenas caminhos relativos e compartilhar a pasta do projeto com alguém, todos os caminhos existentes nos códigos continuarão a funcionar em qualquer computador!

5.2 O pacote readr

O pacote `{readr}`do tidyverse é utilizado para importar arquivos de texto, como `.txt` ou `.csv`, para o R. Para carregá-lo, rode o código:

```
library(readr)
```

O `{readr}` transforma arquivos de textos em `tibbles` usando as funções:

- `read_csv()`: para arquivos separados por vírgula.
- `read_tsv()`: para arquivos separados por tabulação.
- `read_delim()`: para arquivos separados por um delimitador genérico. O argumento `delim=` indica qual caracter separa cada coluna no arquivo de texto.
- `read_table()`: para arquivos de texto tabular com colunas separadas por espaço.
- `read_fwf()`: para arquivos compactos que devem ter a largura de cada coluna especificada.
- `read_log()`: para arquivos padrões de log.

Vamos mostrar na próxima seção como importar as extensões mais comuns: `.csv` e `.txt`.

5.2.1 Lendo arquivos de texto

Como exemplo, utilizaremos uma base de filmes do IMDB, gravada em diversos formatos. O download dos arquivos pode ser realizado clicando aqui.

Primeiro, vamos ler a base em formato `.csv`.

```
imdb_csv <- read_csv(file = "imdb.csv")
```

```
##
## -- Column specification --------------------------------------------------------
## cols(
##   titulo = col_character(),
##   ano = col_double(),
##   diretor = col_character(),
##   duracao = col_double(),
##   cor = col_character(),
##   generos = col_character(),
##   pais = col_character(),
##   classificacao = col_character(),
##   orcamento = col_double(),
##   receita = col_double(),
##   nota_imdb = col_double(),
##   likes_facebook = col_double(),
##   ator_1 = col_character(),
##   ator_2 = col_character(),
##   ator_3 = col_character()
## )
```

A mensagem retornada pela função indica qual classe foi atribuída para cada coluna. Repare que o argumento `file=` representa o caminho até o arquivo. Se o arquivo a ser lido não estiver no diretório de trabalho da sua sessão, você precisa especificar o caminho até o arquivo.

```
# Se o arquivo estiver dentro de uma pasta chamada dados.
imdb_csv <- read_csv(file = "dados/imdb.csv")
```

A maioria das funções de leitura do `{readr}` possuem argumentos muito úteis para resolver problemas de importação:

- `col_names=`: indica se a primeira linha da base contém ou não o nome das colunas. Também pode ser utilizado para (re)nomear colunas.
- `col_types=`: caso alguma coluna seja importada com a classe errada (uma coluna de números foi importada como texto, por exemplo), você pode usar esse argumento para especificar a classe das colunas.
- `locale=`: útil para tratar problema de *encoding*.
- `skip=`: pula linhas no começo do arquivo antes de iniciar a importação. Útil para quando o arquivo a ser importado vem com metadados ou qualquer tipo de texto nas primeiras linhas, antes da base.
- `na=`: indica quais *strings* deverão ser considaras `NA` na hora da importação.

Em alguns países, como o Brasil, as vírgulas são utilizadas para separar as casas decimais dos números, inviabilizando o uso de arquivos `.csv`. Nesses casos, quando a vírgula é o separador de decimal, os arquivos `.csv` passam a ser separados por ponto-e-vírgula. Para importar bases de arquivos separados por ponto-e-vírgula no R, basta usar a função `read_csv2()`.

```
imdb_csv2 <- read_csv2("dados/imdb2.csv")
```

Arquivos `.txt` em geral podem ser lidos com a função `read_delim()`. Além do caminho até o arquivo, você também precisa indicar qual é o caractere utilizado para separar as colunas da base. Um arquivo separado por tabulação, por exemplo, pode ser lido utilizando a o código abaixo. O código `\t` é uma forma textual de representar a tecla TAB.

```
imdb_txt <- read_delim("dados/imdb.txt", delim = "\t")
```

Repare que a sintaxe é igual a da função `read_csv()`. Em geral, as funções de importação do `{tidyverse}` terão sintaxe e comportamento muito parecidos.

A seguir, vamos falar das funções `parse_()`, muito úteis para tratar problemas na classe das variáveis na hora da importação.

5.2.2 Locale

Muitas funções de importação e formatação possuem um argumento `locale`. Esse argumento é utilizado para definir opções de formatação próprias de uma certa localidade, como idioma, formato de data e hora, fuso horário, separador de decimal e milhar ou encoding.

O pacote `{readr}` possui uma função chamada `locale()`, que pode ser utilizada para definir todos esses atributos. Para saber quais são os padrões atualmente definidos na sua sessão, basta rodar:

```
locale()
## <locale>
## Numbers:  123,456.78
## Formats:  %AD / %AT
## Timezone: UTC
## Encoding: UTF-8
## <date_names>
## Days:   Sunday (Sun), Monday (Mon), Tuesday (Tue), Wednesday (Wed), Thursday
##         (Thu), Friday (Fri), Saturday (Sat)
## Months: January (Jan), February (Feb), March (Mar), April (Apr), May (May),
##         June (Jun), July (Jul), August (Aug), September (Sep), October
##         (Oct), November (Nov), December (Dec)
## AM/PM:  AM/PM
```

Em geral, teremos padrões norte-americanos. Se quisermos que os nomes de dias e meses fiquem em português, podemos fazer:

```
locale(date_names = "pt")
## <locale>
## Numbers:  123,456.78
## Formats:  %AD / %AT
## Timezone: UTC
## Encoding: UTF-8
## <date_names>
## Days:   domingo (dom), segunda-feira (seg), terça-feira (ter), quarta-feira
##         (qua), quinta-feira (qui), sexta-feira (sex), sábado (sáb)
## Months: janeiro (jan), fevereiro (fev), março (mar), abril (abr), maio (mai),
##         junho (jun), julho (jul), agosto (ago), setembro (set), outubro
##         (out), novembro (nov), dezembro (dez)
## AM/PM:  AM/PM
```

Ou trocar o separador de decimal de ponto para vírgula, caso a base a ser importada esteja nesse formato.

```
locale(decimal_mark = ",")
## <locale>
## Numbers:  123.456,78
## Formats:  %AD / %AT
## Timezone: UTC
## Encoding: UTF-8
## <date_names>
## Days:   Sunday (Sun), Monday (Mon), Tuesday (Tue), Wednesday (Wed), Thursday
##         (Thu), Friday (Fri), Saturday (Sat)
## Months: January (Jan), February (Feb), March (Mar), April (Apr), May (May),
##         June (Jun), July (Jul), August (Aug), September (Sep), October
##         (Oct), November (Nov), December (Dec)
## AM/PM:  AM/PM
```

A função `locale()` deve ser utilizada dentro das funções `read_()`, no argumento `locale`. Uma utilização muito comum é a definição do *encoding* do arquivo. O encoding se refere a como o computador traduz os caracteres que vemos na tela para os valores binários que ele utiliza internamente. Existem vários tipos de encoding e isso é um problema principalmente porque o Windows utiliza um encoding diferente do Linux/Mac. Você saberá que tem um problema de encoding quando letras com acento ou outros caracteres especiais ficarem desconfigurados após importar uma base para o R.

```
# Vamos produzir um problema de encoding

frase_com_acentos <- "Você comerá uma maçã amanhã à tarde"

# Vendo encoding. UTF-8 é o padrão do Linux/Mac
Encoding(frase_com_acentos)
## [1] "UTF-8"

# Forçando um novo encoding.
# latin1 é um dos padrões que funcionam no Windows.
Encoding(frase_com_acentos) <-  "latin1"

# Agora temos um problema de encoding
frase_com_acentos
## [1] "VocÃª comerÃ¡ uma maÃ§Ã£ amanhÃ£ Ã  tarde"
```

Quando estivermos enfrentando esse problema, devemos dizer à função `read_()` qual o encoding deve ser utilizado no arquivo.

```
read_csv("base_que_veio_do_windows.csv", locale = locale(encoding = "latin1"))
```

O `latin1` é apenas um dos encodings que podem funcionar em arquivos do Windows. Outras sugestões são: `windows-1250`, `windows-1252`, `ISO-8859-2` e `ISO-8859-1`. Se você estiver lendo um arquivo cruado no Linux/Mac no Windows, basta usar o encoding `UTF-8`.

> Eu consegui resolver 99% dos meus problemas de encoding quando passei a fingir que o Windows não existe. — Julio Trecenti

Na seção a seguir, mostramos mais alguns exemplos da função `locale()`.

5.2.3 Parseando valores

O pacote `{readr}` possui algumas funções muito úteis para parsear valores. Parsear é um termo muito utilizado em programação e tem o sentido de *arrumar* ou *formatar*. Se estamos parseando um número, por exemplo, estamos pegando um texto que é muito parecido com um número e o transforando em um número de fato.

```
parse_number(c("5", "5.0", "5,0", "R$5.00", "5 a"))
## [1]  5  5 50  5  5
```

\begin{figure}

readr::parse_numb
(just give me the numb
MY_DATA
ID
AMOUNT
1
$12.75 USD
2
SEAT 18 A
TICKETS
crew
characters

{

}

\caption{Arte por Allison Horst (**?**). Veja nas Referências onde encontrá-la.}
\end{figure}

Veja que podemos usar o argumento `locale` para especificar coordenadas para o parseamento

```
parse_number("5,0", locale = locale(decimal_mark = ","))
## [1] 5
```

ou o idioma usado em datas

```
# Inglês
parse_date(
  "01/June/2010",
  format = "%d/%B/%Y"
)
## [1] "2010-06-01"

# Português
parse_date(
  "01/Junho/2010",
  format = "%d/%B/%Y",
  locale = locale(date_names = "pt")
)
## [1] "2010-06-01"
```

Você também pode especificar `NAs` utilizando o argumento `na`.

```
parse_number(c("5", "5.0", "5,0", "R$5.00", "5 a"), na = "5 a")
## [1]  5  5 50  5 NA
```

Outras funções de parseamento úteis são:

- `readr::parse_integer()`, para parsear números inteiros.
- `readr::parse_character()`, para parsear strings
- `readr::parse_date()`, `readr::parse_time()`, `readr::parse_datetime()`para parsear datas, horas e data/horas.

Todas essas funções são utilizadas nas colunas de uma base quando utilizamos as funções de importação do `{readr}`. Assim, se algum valor foi modificado incorretamente durante a importação, você pode testar importar tudo como texto (`character`) e usar essas funções para reproduzir a transformação que o `{readr}` fez.

5.2.4 Escrevendo arquivos de texto

Para a maioria das funções `read_`, existe uma respectiva função `write_`. Essas funções servem para salvar bases em um formato específico de arquivo. Além do caminho/nome do arquivo a ser criado, você também precisa passar o objeto que será escrito. Para o arquivo criado funcionar corretamente, você precisa especificar a extensão correta no nome do arquivo.

```
# Arquivo .csv
write_csv(x = mtcars, path = "data/mtcars.csv")

# Base separada por tabulação
write_delim(x = mtcars, path = "data/mtcars.txt", delim = "\t")
```

5.2.5 Arquivos .rds

A linguagem R tem uma extensão própria de arquivos binários chamada `RDS` ou `.rds`. Essa extensão pode ser utilizada para guarda qualquer tipo de objeto do R, inclusive bases de dados. Temos duas principais vantagens ao utilizarmos essa extensão para salvarvamos as nossas bases:

- ele salva as classes especificadas para as colunas;
- ele pode ser compactado, gerando arquivos muito menores.

A desvantagem é que ele só poderá ser lido dentro do R.

Para criar um arquivo `.rds`, utilize a função `write_rds()`.

```
write_rds(mtcars, path = "mtcars.rds", compress = "gz")
```

O argumento `compress` é opcional e indica qual o tipo de compactação deve ser feito. O padrão é não compactar.

Para ler um arquivo `.rds` de volta para o R, utilizamos a função `read_rds()`. Repare que essa função não possui outros argumentos, pois o objeto importado será exatamente igual ao objeto que foi gravado no arquivo.

```
imdb_rds <- read_rds(path = "imdb.rds")
```

Exercícios

1. Qual a diferença entre as funções `read_csv()` e `read_csv2()`?

2. Leia o arquivo `imdb.csv` utilizando a função `read_delim()`.

3. Escreva a base mtcars em um arquivo `mtcars.csv` que não contenha o nome das colunas.

4. Use a função `write_rds()` para salvar em arquivos

- **a)** Um número.
- **b)** Um vetor de strings.
- **c)** Uma lista com valores númericos, textuais e lógicos.
- **d)** As 3 primeiras colunas da base `mtcars`.

5. Utilize a função `read_rds()` para importar de volta para o R os arquivos criados no exercício 4.

5.3 Os pacotes readxl e writexl

Para ler planilhas do Excel (arquivos `.xlsx` ou `.xls`), basta utilizarmos a função `read_excel()` do pacote `readxl`. Instale o pacote antes caso você ainda não o tenha instalado.

```
install.packages("readxl")
library(readxl)

imdb_xlsx <- read_xls("dados/imdb.xlss")
imdb_xlsx <- read_xlsx("dados/imdb.xlsx")
```

A funçao `read_excel()` auto detecta a extensão do arquivo.

```
read_excel(path = "assets/data/imdb.xls")
read_excel(path = "assets/data/imdb.xlsx")
```

O pacote disponibiliza 5 exemplos de arquivos com formato `.xls` e `.xlsx`.

```
readxl_example()
```

```
##  [1] "clippy.xls"    "clippy.xlsx"   "datasets.xls"  "datasets.xlsx"
##  [5] "deaths.xls"    "deaths.xlsx"   "geometry.xls"  "geometry.xlsx"
##  [9] "type-me.xls"   "type-me.xlsx"
```

Vamos pegar o caminho até o arquivo `datasets.xlsx` usando a função `readxl_example()`.

```
caminho_datasets <- readxl_example("datasets.xlsx")
caminho_datasets
```

```
## [1] "/Library/Frameworks/R.framework/Versions/4.1/Resources/library/readxl/extdata/d
```

No Excel, um arquivo pode ter várias planilhas. Esse é o caso do arquivo `datasets.xlsx`. Você pode ver quais planilhas fazem parte do arquivo utilizando a função `excel_sheets()`.

```
excel_sheets(caminho_datasets)
```

```
## [1] "iris"     "mtcars"   "chickwts" "quakes"
```

Por padrão, as funções de leitura trarão apenas a primeira planilha do arquivo. Para trazer outra planilha, basta utilizarmos o argumento `sheet`.

```
# Pega a primeira planilha
read_excel(caminho_datasets)
```

```
## # A tibble: 150 x 5
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##           <dbl>       <dbl>        <dbl>       <dbl> <chr>
##  1          5.1         3.5          1.4         0.2 setosa
##  2          4.9         3            1.4         0.2 setosa
##  3          4.7         3.2          1.3         0.2 setosa
##  4          4.6         3.1          1.5         0.2 setosa
##  5          5           3.6          1.4         0.2 setosa
##  6          5.4         3.9          1.7         0.4 setosa
##  7          4.6         3.4          1.4         0.3 setosa
##  8          5           3.4          1.5         0.2 setosa
##  9          4.4         2.9          1.4         0.2 setosa
## 10          4.9         3.1          1.5         0.1 setosa
## # ... with 140 more rows
```

```
# Pega a segunda planilha
read_excel(caminho_datasets, sheet = 2)
```

```
## # A tibble: 32 x 11
##      mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
##    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
##  1  21       6  160    110  3.9   2.62  16.5     0     1     4     4
##  2  21       6  160    110  3.9   2.88  17.0     0     1     4     4
```

```
##  3  22.8     4  108     93  3.85  2.32  18.6     1     1     4     1
##  4  21.4     6  258    110  3.08  3.22  19.4     1     0     3     1
##  5  18.7     8  360    175  3.15  3.44  17.0     0     0     3     2
##  6  18.1     6  225    105  2.76  3.46  20.2     1     0     3     1
##  7  14.3     8  360    245  3.21  3.57  15.8     0     0     3     4
##  8  24.4     4  147.    62  3.69  3.19  20       1     0     4     2
##  9  22.8     4  141.    95  3.92  3.15  22.9     1     0     4     2
## 10  19.2     6  168.   123  3.92  3.44  18.3     1     0     4     4
## # ... with 22 more rows
```

```
# Pega a planilha selecionada
read_excel(caminho_datasets, sheet = 'chickwts')
```

```
## # A tibble: 71 x 2
##    weight feed
##     <dbl> <chr>
##  1    179 horsebean
##  2    160 horsebean
##  3    136 horsebean
##  4    227 horsebean
##  5    217 horsebean
##  6    168 horsebean
##  7    108 horsebean
##  8    124 horsebean
##  9    143 horsebean
## 10    140 horsebean
## # ... with 61 more rows
```

A seguir, listamos outros argumentos úteis da função `read_excel()`:

- `col_names` indica se a primeira linha representa o nome das colunas;
- `col_types=` para definir a classe das colunas;
- `skip=` para pular linhas no começo da planilha;
- `na=` indica quais strings devem ser interpretadas como NA.

Também podemos escrever um arquivo Excel (com extensão `.xlsx`) utilizando a função `write_xlsx()` do pacote `writexl`.

```
install.packages("writexl")
library(writexl)

write_excel(mtcars, "imdb.xlsx")
```

5.4 haven

Para ler arquivos gerados por outros softwares, como SPSS, SAS e STATA, você pode usar as funções do pacote `haven`. Este pacote faz parte do `tidyverse` e é um wrapper da biblioteca ReadStat, escrita em C.

```
library(haven)

imdb_sas <- read_sas("assets/data/imdb.sas7bdat")
imdb_spss <- read_spss("assets/data/imdb.sav")
imdb_dta <- read_dta("assets/data/imdb.dta")
```

É possível salvar ou escrever bases em SAS e STATA com as funções `write_sas` e `write_dta`.

```
write_dta(mtcars, 'assets/data/mtcars.dta')
```

Quando importamos arquivos gerados pelo SAS SPSS ou STATA para o R, os rótulos de uma variável podem não ser importados de forma correta. O pacote `haven` tem uma soluçao para este problema.

```
x <- labelled(c(1,1,2,3,2,2,1,2), c(Ruim = 1, Bom = 2, Otimo = 3))
```

`labelled()` adiciona rótulos à valores de uma variável. Para verificar quais são estes rótulos, podemos usar a função `print_labels()`.

```
print_labels(x)
```

```
##
## Labels:
##  value label
##      1  Ruim
##      2   Bom
##      3 Otimo
```

Existe uma função similar a `labelled()`, exclusiva para o SPSS, que além de rotular as variáveis, também defini quais símbolos representam valores faltantes, dado que em SPSS pode haver mais de um tipo de *missing*.

```
x1 <- labelled_spss(c(1,3,0,2,2,1,0,2,4), c(Ruim = 1,Bom = 2, Otimo = 3), na_values = c

is.na(x1)
```

```
## [1] FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE  TRUE
```

Agora que já definimos os *missings* "especiais", podemos transformá-los no *missing* padrão do R, representado pelo símbolo *NA*.

```
x1 <- zap_missing(x1)
x1
```

```
## [1]  1  3 NA  2  2  1 NA  2 NA
## attr(,"labels")
##  Ruim   Bom Otimo
##     1     2     3
## attr(,"class")
## [1] "haven_labelled"
```

Existem outras funções `zap_` interessantes no pacote.

Após rotular os valores do vetor, podemos convertê-los, por exemplo, em fator. Para isso, usamos uma função do pacote `haven`. A função base `as.factor()` também poderia ser usada, mas quando a usamos, os rótulos não são considerados.

```
x_base <- base::as.factor(x)
levels(x_base)
```

```
## [1] "1" "2" "3"
```

```
x_factor <- haven::as_factor(x)
levels(x_factor)
```

```
## [1] "Ruim"  "Bom"   "Otimo"
```

Chapter 6

Pipe

O operador `%>%` (*pipe*) foi uma das grandes revoluções recentes do R, tornando a escrita e leitura de códigos mais intuitiva e compreensível. Ele foi introduzido por Stefan Milton Bache no pacote `magrittr`, cujo nome é uma referência ao famoso quadro do pintor belga René Magritte *La Trahison des images (Ceci n'est pas une pipe)*.

Figure 6.1: Reprodução do quadro La Trahison des images (Ceci n'est pas une pipe). do pintor René Magritte.

Para começar a utilizar o *pipe*, instale e carregue o pacote `magrittr`.

```
install.packages("magrittr")
library(magrittr)
```

6.1 O operador pipe

A ideia do operador `%>%` (*pipe*) é bem simples: usar o valor resultante da expressão do lado esquerdo como primeiro argumento da função do lado direito.

```
# As duas linhas abaixo são equivalentes.

f(x, y)
x %>% f(y)
```

Nos casos mais simples, o *pipe* parece não trazer grandes vantagens. Agora, veja como fica um caso com mais etapas.

```
# Vamos calcular a raiz quadrada da soma dos valores de 1 a 4. Primeiro, sem o pipe.

x <- c(1, 2, 3, 4)
sqrt(sum(x))
```

```
## [1] 3.162278
```

```
# Agora com o pipe.

x %>% sum() %>% sqrt()
```

```
## [1] 3.162278
```

O caminho que o código `x %>% sum %>% sqrt` seguiu foi enviar o objeto `x` como argumento da função `sum()` e, em seguida, enviar a saida da expressão `sum(x)` como argumento da função `sqrt()`. Observe que escrevemos o código na mesma ordem em que as operações são realizadas. A utilização de parênteses após o nome das funções não é necessário, mas recomendável.

Se você ainda não está convencido com o poder do *pipe*, fica que vai ter bolo!

No exemplo abaixo, vamos ilustrar um caso em que temos um grande número de funções aninhadas. Veja como a utilização do *pipe* transforma um código confuso e difícil de ser lido em algo simples e intuitivo.

```
# Receita de bolo sem pipe. Tente entender o que é preciso fazer.

esfrie(
  asse(
    coloque(
      bata(
        acrescente(
          recipiente(
            rep("farinha", 2),
            "água",
            "fermento",
            "leite",
            "óleo"
          ),
          "farinha",
          ate = "macio"
        ),
        duracao = "3min"
      ),
      lugar = "forma",
      tipo = "grande",
      untada = TRUE
    ),
    duracao = "50min"
  ),
  lugar = "geladeira",
  duracao = "20min"
)


# Veja como o código acima pode ser reescrito utilizando-se o pipe. Agora realmente se parece com

recipiente(rep("farinha", 2), "água", "fermento", "leite", "óleo") %>%
  acrescente("farinha", ate = "macio") %>%
  bata(duracao = "3min") %>%
  coloque(lugar = "forma", tipo = "grande", untada = TRUE) %>%
  asse(duracao = "50min") %>%
  esfrie(lugar = "geladeira", duracao = "20min")
```

Às vezes, queremos que o resultado do lado esquerdo vá para outro argumento do lado direito que não o primeiro. Para isso, utilizamos um `.` como marcador.

```
# Queremos que o dataset seja recebido pelo segundo argumento (data=) da função "lm".

airquality %>%
```

```
  na.omit() %>%
  lm(Ozone ~ Wind + Temp + Solar.R, data = .) %>%
  summary()
```

```
##
## Call:
## lm(formula = Ozone ~ Wind + Temp + Solar.R, data = .)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -40.485 -14.219  -3.551  10.097  95.619
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -64.34208   23.05472  -2.791  0.00623 **
## Wind         -3.33359    0.65441  -5.094 1.52e-06 ***
## Temp          1.65209    0.25353   6.516 2.42e-09 ***
## Solar.R       0.05982    0.02319   2.580  0.01124 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 21.18 on 107 degrees of freedom
## Multiple R-squared:  0.6059, Adjusted R-squared:  0.5948
## F-statistic: 54.83 on 3 and 107 DF,  p-value: < 2.2e-16
```

O *pipe* é a força da gravidade dentro do `tidyverse`. Veremos nos próximos capítulos como as funções de diferentes pacotes interagem perfeitamente graças a esse operador.

Exercícios

1. Reescreva a expressão abaixo utilizando o `%>%`.

```
round(mean(sum(1:10)/3), digits = 1)
```

Dica: utilize a função `magrittr::divide_by()`. Veja o help da função para mais informações.

2. Reescreva o código abaixo utilizando o `%>%`.

```
x <- rnorm(100)

x.pos <- x[x>0]
```

```
media <- mean(x.pos)

saida <- round(media, 1)
```

3. Sem rodar, diga qual a saída do código abaixo. Consulte o help das funções caso precise.

```
2 %>%
  add(2) %>%
  c(6, NA) %>%
  mean(na.rm = T) %>%
  equals(5)
```

6.2 Outros operadores

O pacote `{magrittr}` possui outros operadores, que, embora sejam menos utilizados, também são úteis. São eles:

- *Assignment operator* `%<>%`
- *Operador tee* `%T>%`
- *Exposition operator* `%$%`

Imagine que queremos tirar a raiz quadrada de um vetor de números.

```
x <- c(1, 2, 3, 4, 5)

x %>% sqrt()
```

```
## [1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

Se quisermos sobrescrever o objeto `x` com a raiz quadrada dos seus valores, basta utilizarmos o nosso bom e velho operador de atribuição `<-`.

```
x <- x %>% sqrt()
```

Podemos, no entanto, utilizar o operador `%<>%` para reescrever o código acima de uma maneira mais compacta.

```
x <- c(1, 2, 3, 4, 5)
x %<>% sqrt()
```

Além de mandar o objeto `x` para o primeiro argumento da função `sqrt()`, assim como o `%>%` faria, esse operador também salva o resultado da operação de volta no objeto `x`, o sobrescrevendo.

Este operador pode ser usado sempre que desejamos fazer algo da forma

```
objeto <- objeto %>%
  funcao_1() %>%
  funcao_2() %>%
  ...
  funcao_n()
```

O operador `%T>%` retorna o valor do comando anterior a ele, não o resultado do lado direito como o `%>%` faz. O seguinte exemplo vai imprimir na tela os valores de 1 a 10. Se usássemos o pipe, o código retornaria a soma dos dez números.

```
1:10 %T>% sum() %>% cat()
```

```
## 1 2 3 4 5 6 7 8 9 10
```

Neste caso, o operador não parece fazer sentido e apenas deixa o código mais complicado, mas se desejamos usar funções como `cat()` ou `plot()` que não retornam nada, o operador se torna muito útil.

```
# Vamos imprimir na tela os valores de 1 a 10 e depois soma-los.
1:10 %T>%
  cat() %>%
  sum()
```

```
## 1 2 3 4 5 6 7 8 9 10
```

```
## [1] 55
```

O operador `%$%` pode ser utilizado para *expor* as colunas de um *data frame* para a função aplicada no lado direito.

```
# Podemos chamar qualquer coluna da base diretamente.
mtcars %$% mean(mpg)
```

```
## [1] 20.09062
```

Se não ficou claro o que esse operador está fazendo, imagine que ele transforma todas as colunas da base em objetos (assim como a nefasta função `attach()`), mas sem salvar nada no nosso *environment*.

```
mtcars %$%
  mpg %>%
  mean() %>%
  sqrt()
```

```
## [1] 4.482257
```

Ele faz um papel equivalente ao operador `$`.

```
mtcars$mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

```
mtcars %$% mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

Para mais informações sobre o `pipe` e outras funções do pacote `{magrittr}`, visite a página Ceci n'est pas un pipe.

Chapter 7

Manipulação

Esta seção trata do tema *manipulação de dados*. Trata-se de uma tarefa dolorosa e demorada, tomando muitas vezes a maior parte do tempo de uma análise de dados. Apesar de negligenciada pela Estatística, essa etapa é essencial para o sucesso da análise.

> *The fact that data science exists as a field is a colossal failure of statistics. To me, what I do is what statistics is all about. It is gaining insight from data using modelling and visualization. Data munging and manipulation is hard and statistics has just said that's not our domain.* — Hadley Wickham

Usualmente, o cientista de dados parte de uma base "crua" e a transforma até obter uma base de dados analítica, que, a menos de transformações simples, está preparada para gerar tabelas e gráficos e alimentar modelos.

Um conceito importante para obtenção de uma base analítica é o *data tidying*, ou arrumação de dados. Uma base é considerada *tidy* se

- 1. Cada linha da base representa uma observação.
- 2. Cada coluna da base representa uma variável.

A base de dados analítica é estruturada de tal forma que pode ser colocada diretamente em ambientes de modelagem estatística ou de visualização. Nem sempre uma base de dados analítica está no formato *tidy*, mas usualmente são necessários poucos passos para migrar de uma para outra. A filosofia *tidy* é a base do tidyverse.

Os principais pacotes encarregados da tarefa de estruturar os dados são o `dplyr` e o `tidyr`. Eles serão o tema desse capítulo. Instale e carregue os pacotes utilizando:

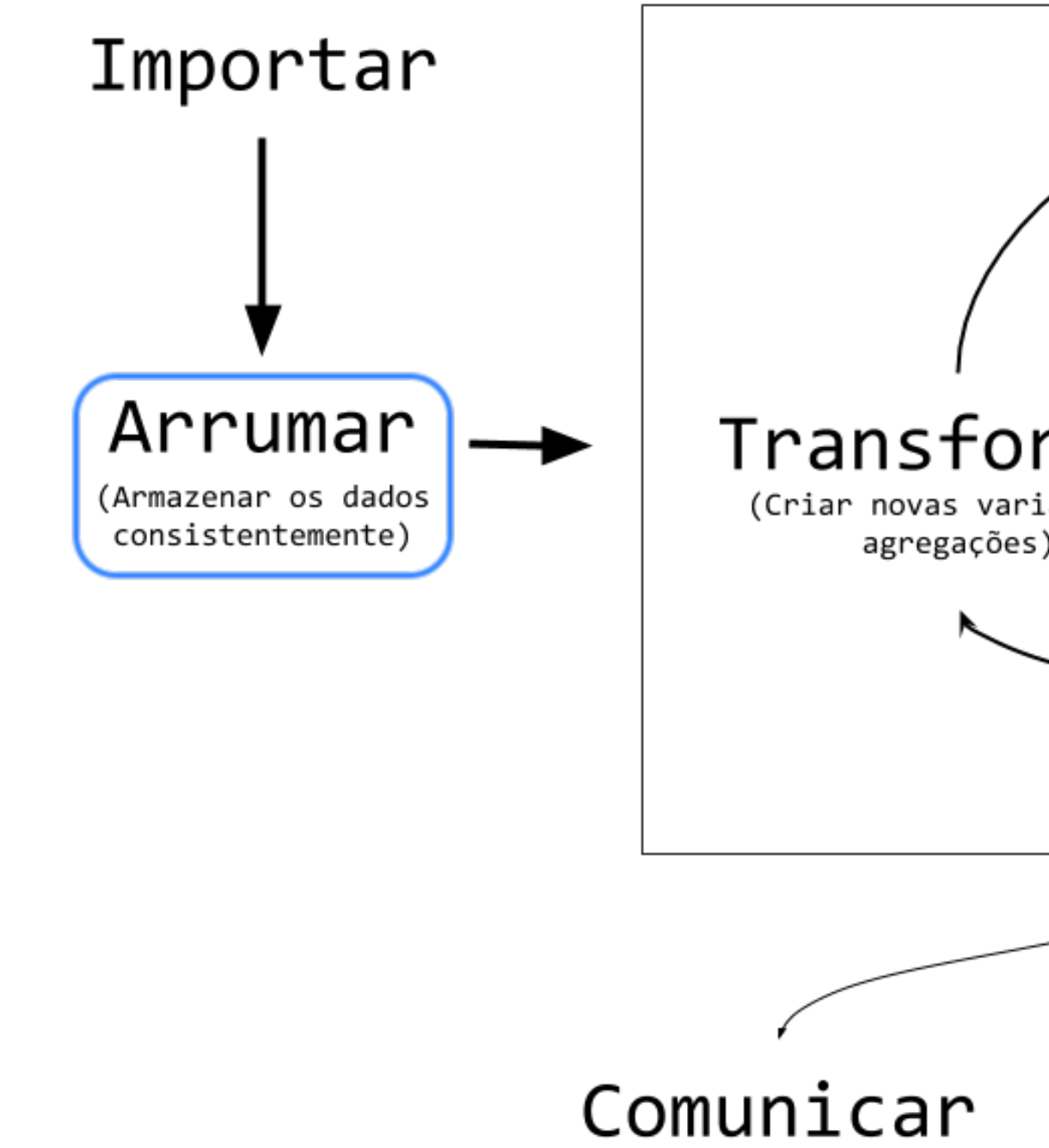


Figure 7.1: Estamos na etapa de arrumação do ciclo da Ciência de Dados.

```
install.packages(c("dplyr", "tidyr"))

library(dplyr)
library(tidyr)
```

Mas antes de apresentar as principais funções do `dplyr` e do `tidyr`, precisamos falar sobre o conceito de `tibbles`.

7.1 Tibbles

Uma *tibble* é um *data frame* com ajustes que as deixam mais amigáveis a nós cientistas de dados. Elas são parte do pacote `{tibble}`. Assim, para começar a usá-las, instale e carregue o pacote.

```
install.packages("tibble")
library(tibble)
```

Há duas formas de criar uma *tibble*. A primeira é transformar um *data frame* em *tibble* utilizando a função `as_tibble()`.

```
as_tibble(mtcars)
```

```
## # A tibble: 32 x 11
##      mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
##    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
##  1  21       6  160    110  3.9   2.62  16.5     0     1     4     4
##  2  21       6  160    110  3.9   2.88  17.0     0     1     4     4
##  3  22.8     4  108     93  3.85  2.32  18.6     1     1     4     1
##  4  21.4     6  258    110  3.08  3.22  19.4     1     0     3     1
##  5  18.7     8  360    175  3.15  3.44  17.0     0     0     3     2
##  6  18.1     6  225    105  2.76  3.46  20.2     1     0     3     1
##  7  14.3     8  360    245  3.21  3.57  15.8     0     0     3     4
##  8  24.4     4  147.    62  3.69  3.19  20       1     0     4     2
##  9  22.8     4  141.    95  3.92  3.15  22.9     1     0     4     2
## 10  19.2     6  168.   123  3.92  3.44  18.3     1     0     4     4
## # ... with 22 more rows
```

Veja que, por padrão, apenas as dez primeiras linhas da base são apresentadas. Além disso, colunas que não couberem na largura da tela serão omitidas. Também são apresentadas a dimensão da tabela e as classes de cada coluna. Compare a diferença de impressão com relação a um *data frame*.

```
mtcars
```

```
##                     mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4          21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag      21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710         22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive     21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout  18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Valiant            18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Duster 360         14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Merc 240D          24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230           22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 280           19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C          17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Merc 450SE         16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 450SL         17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 450SLC        15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## Cadillac Fleetwood 10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4  8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Chrysler Imperial  14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Fiat 128           32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic        30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla     33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona      21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Dodge Challenger   15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## AMC Javelin        15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Camaro Z28         13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Pontiac Firebird   19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Fiat X1-9          27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2      26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa       30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Ford Pantera L     15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Ferrari Dino       19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Maserati Bora      15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## Volvo 142E         21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

Veja também que, ao contrário dos *data frames*, *tibbles* não utilizam nome nas linhas.

A segunda forma de criar uma *tibble* é a partir de vetores individuais, utilizando a função `tibble()`.

```
tibble(
  coluna1 = c("a", "b", "c", "d"),
  coluna2 = 1:4,
```

```
  coluna3 = coluna2 ^ 2,
  coluna4 = 0
)
```

```
## # A tibble: 4 x 4
##   coluna1 coluna2 coluna3 coluna4
##   <chr>     <int>   <dbl>   <dbl>
## 1 a             1       1       0
## 2 b             2       4       0
## 3 c             3       9       0
## 4 d             4      16       0
```

Observamos pelo código acima que:

- podemos criar uma coluna a partir de uma função de colunas anteriores (a `coluna3` é uma função da `coluna2`);
- se passarmos um vetor de tamanho 1 a uma das colunas, esse vetor será reciclado, igualando-se ao número de linha da *tibble* (o `0` na `coluna4` é transformado em `c(0, 0, 0, 0)`).

Uma outra diferença entre *data frames* e *tibbles* está no *subsetting*. *Tibbles* **nunca fazem correspondência parcial**. Tibbles retornam `NULL` e um aviso quando você tenta selecionar uma coluna que não existe.

```
# O nome da coluna é disp, mas você pode acessá-la
# apenas a procurando por "dis", já que não existe
# nenhuma outra coluna que começa com "dis".
mtcars$dis
```

```
##  [1] 160.0 160.0 108.0 258.0 360.0 225.0 360.0 146.7 140.8 167.6 167.6 275.8
## [13] 275.8 275.8 472.0 460.0 440.0  78.7  75.7  71.1 120.1 318.0 304.0 350.0
## [25] 400.0  79.0 120.3  95.1 351.0 145.0 301.0 121.0
```

```
# Em tibbles isso não funciona.
mtcars_tibble <- as_tibble(mtcars)
mtcars_tibble$dis
```

```
## Warning: Unknown or uninitialised column: `dis`.
```

```
## NULL
```

Além de nos possibilitar a criar *tibbles*, o pacote `{tibble}` possui algumas funções úteis para manipularmos esses objetos:

- `add_row()` e `add_column()`: para adicionar linhas e colunas a uma *tibble*;
- `rowid_to_column()`: cria uma coluna com um `id` numérico e sequencial para as linhas, começando de `1`.

```
mtcars %>%
  rowid_to_column() %>%
  head(10)
```

```
##    rowid  mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## 1      1 21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## 2      2 21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## 3      3 22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## 4      4 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## 5      5 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## 6      6 18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## 7      7 14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## 8      8 24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## 9      9 22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## 10    10 19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
```

- `has_rownames()`: verifica se um *data frame* possui linhas nomeadas.
- `remove_rownames()`: remove o nome das linhas de um *data frame*.
- `rownames_to_column()`: transforma o nome das linhas em uma coluna.

```
rownames_to_column(mtcars, var = "modelo_do_carro")
```

```
##         modelo_do_carro  mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## 1             Mazda RX4 21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## 2         Mazda RX4 Wag 21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## 3            Datsun 710 22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## 4        Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## 5     Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## 6               Valiant 18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## 7            Duster 360 14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## 8             Merc 240D 24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## 9              Merc 230 22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## 10             Merc 280 19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## 11            Merc 280C 17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## 12           Merc 450SE 16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## 13           Merc 450SL 17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## 14          Merc 450SLC 15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## 15  Cadillac Fleetwood 10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
```

```
## 16 Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
## 17   Chrysler Imperial 14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## 18            Fiat 128 32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## 19         Honda Civic 30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## 20      Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## 21       Toyota Corona 21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## 22    Dodge Challenger 15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## 23         AMC Javelin 15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## 24          Camaro Z28 13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## 25    Pontiac Firebird 19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## 26           Fiat X1-9 27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## 27       Porsche 914-2 26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## 28        Lotus Europa 30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## 29      Ford Pantera L 15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## 30        Ferrari Dino 19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## 31       Maserati Bora 15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## 32          Volvo 142E 21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

- `column_to_rownames()`: transforma uma coluna em nomes das linhas.

Mais informações sobre `tibbles` podem ser encontradas neste tutorial do RStudio e neste caplítulo do R for Data Science.

7.2 O pacote dplyr

O `dplyr` é o pacote mais útil para realizar transformação de dados, aliando simplicidade e eficiência de uma forma elegante. Os scripts em R que fazem uso inteligente dos verbos `dplyr` e as facilidades do operador *pipe* tendem a ficar mais legíveis e organizados sem perder velocidade de execução.

As principais funções do `dplyr` são:

- `select()` - seleciona colunas
- `arrange()` - ordena a base
- `filter()` - filtra linhas
- `mutate()` - cria/modifica colunas
- `group_by()` - agrupa a base
- `summarise()` - sumariza a base

Todas essas funções seguem as mesmas características:

- O *input* é sempre uma `tibble` e o *output* é sempre um `tibble`.
- Colocamos a `tibble` no primeiro argumento e o que queremos fazer nos outros argumentos.

- A utilização é facilitada com o emprego do operador `%>%`.

As principais vantagens de se usar o `dplyr` em detrimento das funções do R base são:

- Manipular dados se torna uma tarefa muito mais simples.
- O código fica mais intuitivo de ser escrito e mais simples de ser lido.
- O pacote `dplyr` utiliza `C` e `C++` por trás da maioria das funções, o que geralmente torna o código mais rápido.
- É possível trabalhar com diferentes fontes de dados, como bases relacionais (SQL) e `data.table`.

Se você ainda não tiver o `dplyr` instalado, rode o código abaixo.

```
install.packages("dplyr")
library(dplyr)
```

Neste capítulo, vamos trabalhar com uma base de filmes do IMDB. Essa base pode ser baixada clicando aqui.

Assim, utilizaremos o objeto `imdb` para acessar os dados.

```
imdb <- readr::read_rds("imdb.rds")
imdb
```

```
## # A tibble: 3,807 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Agora, vamos avaliar com mais detalhes as principais funções do pacote `dplyr`.

7.2.1 Selecionando colunas

Para selecionar colunas, utilizamos a função `select()`.

O primeiro argumento da função é a base de dados e os demais argumentos são os nomes das colunas que você gostaria de selecionar. Repare que você não precisa colocar o nome da coluna entre aspas.

```
select(imdb, titulo)
```

```
## # A tibble: 3,807 x 1
##    titulo
##    <chr>
##  1 Avatar
##  2 Pirates of the Caribbean: At World's End
##  3 The Dark Knight Rises
##  4 John Carter
##  5 Spider-Man 3
##  6 Tangled
##  7 Avengers: Age of Ultron
##  8 Batman v Superman: Dawn of Justice
##  9 Superman Returns
## 10 Pirates of the Caribbean: Dead Man's Chest
## # ... with 3,797 more rows
```

Você também pode selecionar várias colunas.

```
select(imdb, titulo, ano, orcamento)
```

```
## # A tibble: 3,807 x 3
##    titulo                                     ano orcamento
##    <chr>                                    <int>     <int>
##  1 Avatar                                    2009 237000000
##  2 Pirates of the Caribbean: At World's End  2007 300000000
##  3 The Dark Knight Rises                     2012 250000000
##  4 John Carter                               2012 263700000
##  5 Spider-Man 3                              2007 258000000
##  6 Tangled                                   2010 260000000
##  7 Avengers: Age of Ultron                   2015 250000000
##  8 Batman v Superman: Dawn of Justice        2016 250000000
##  9 Superman Returns                          2006 209000000
## 10 Pirates of the Caribbean: Dead Man's Chest 2006 225000000
## # ... with 3,797 more rows
```

O operador `:` é muito útil para selecionar colunas consecutivas.

```
select(imdb, titulo:cor)
```

```
## # A tibble: 3,807 x 5
##    titulo                                    ano diretor          duracao cor
##    <chr>                                   <int> <chr>              <int> <chr>
##  1 Avatar                                   2009 James Cameron        178 Color
##  2 Pirates of the Caribbean: At World's En~ 2007 Gore Verbinski       169 Color
##  3 The Dark Knight Rises                    2012 Christopher Nol~     164 Color
##  4 John Carter                              2012 Andrew Stanton       132 Color
##  5 Spider-Man 3                             2007 Sam Raimi            156 Color
##  6 Tangled                                  2010 Nathan Greno         100 Color
##  7 Avengers: Age of Ultron                  2015 Joss Whedon          141 Color
##  8 Batman v Superman: Dawn of Justice       2016 Zack Snyder          183 Color
##  9 Superman Returns                         2006 Bryan Singer         169 Color
## 10 Pirates of the Caribbean: Dead Man's Ch~ 2006 Gore Verbinski       151 Color
## # ... with 3,797 more rows
```

O `dplyr` possui o conjunto de funções auxiliares muito úteis para seleção de colunas. As principais são:

- `starts_with()`: para colunas que começam com um texto padrão
- `ends_with()`: para colunas que terminam com um texto padrão
- `contains()`: para colunas que contêm um texto padrão

Selecionamos a seguir todas as colunas que começam com o texto "ator".

```
select(imdb, starts_with("ator"))
```

```
## # A tibble: 3,807 x 3
##    ator_1          ator_2            ator_3
##    <chr>           <chr>             <chr>
##  1 CCH Pounder     Joel David Moore  Wes Studi
##  2 Johnny Depp     Orlando Bloom     Jack Davenport
##  3 Tom Hardy       Christian Bale    Joseph Gordon-Levitt
##  4 Daryl Sabara    Samantha Morton   Polly Walker
##  5 J.K. Simmons    James Franco      Kirsten Dunst
##  6 Brad Garrett    Donna Murphy      M.C. Gainey
##  7 Chris Hemsworth Robert Downey Jr. Scarlett Johansson
##  8 Henry Cavill    Lauren Cohan      Alan D. Purwin
##  9 Kevin Spacey    Marlon Brando     Frank Langella
## 10 Johnny Depp     Orlando Bloom     Jack Davenport
## # ... with 3,797 more rows
```

Para retirar colunas da base, base acrescentar um - antes da seleção.

```
imdb %>%
  select(-ano, - diretor)
```

```
## # A tibble: 3,807 x 13
##    titulo  duracao cor   generos pais  classificacao orcamento receita nota_imdb
##    <chr>     <int> <chr> <chr>   <chr> <chr>             <int>   <int>     <dbl>
##  1 Avatar      178 Color Action~ USA   A partir de ~ 237000000  7.61e8       7.9
##  2 Pirate~     169 Color Action~ USA   A partir de ~ 300000000  3.09e8       7.1
##  3 The Da~     164 Color Action~ USA   A partir de ~ 250000000  4.48e8       8.5
##  4 John C~     132 Color Action~ USA   A partir de ~ 263700000  7.31e7       6.6
##  5 Spider~     156 Color Action~ USA   A partir de ~ 258000000  3.37e8       6.2
##  6 Tangle~     100 Color Advent~ USA   Livre         260000000  2.01e8       7.8
##  7 Avenge~     141 Color Action~ USA   A partir de ~ 250000000  4.59e8       7.5
##  8 Batman~     183 Color Action~ USA   A partir de ~ 250000000  3.30e8       6.9
##  9 Superm~     169 Color Action~ USA   A partir de ~ 209000000  2.00e8       6.1
## 10 Pirate~     151 Color Action~ USA   A partir de ~ 225000000  4.23e8       7.3
## # ... with 3,797 more rows, and 4 more variables: likes_facebook <int>,
## #   ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

```
imdb %>%
  select(-starts_with("ator"))
```

```
## # A tibble: 3,807 x 12
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 3 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>
```

Exercícios

Utilize a base `imdb` nos exercícios a seguir.

1. Teste aplicar a função `glimpse()` do pacote `{dplyr}` à base `imdb`. O que ela faz?

2. Crie uma tabela com apenas as colunas `titulo`, `diretor`, e `orcamento`. Salve em um objeto chamado `imdb_simples`.

3. Selecione apenas as colunas `ator_1`, `ator_2` e `ator_3` usando o ajudante `contains()`.

4. Usando a função `select()` (e seus ajudantes), escreva códigos que retornem a base IMDB sem as colunas `ator_1`, `ator_2` e `ator_3`. Escreva todas as soluções diferentes que você conseguir pensar.

7.2.2 Ordenando a base

Para ordenar linhas, utilizamos a função `arrange()`. O primeiro argumento é a base de dados. Os demais argumentos são as colunas pelas quais queremos ordenar as linhas. No exemplo a seguir, ordenamos as linhas da base por ordem crescente de orçamento.

```
arrange(imdb, orcamento)
```

```
## # A tibble: 3,807 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Tarnati~  2003 Jonatha~      88 Color Biograph~ USA   Outros              218
##  2 My Date~  2004 Jon Gunn      90 Color Document~ USA   Livre              1100
##  3 A Plagu~  2013 Benjami~      76 Color Drama|Ho~ USA   Outros             1400
##  4 The Mon~  2005 Anthony~      84 Color Crime|Dr~ USA   A partir de ~      3250
##  5 Primer    2004 Shane C~      77 Color Drama|Sc~ USA   A partir de ~      7000
##  6 El Mari~  1992 Robert ~      81 Color Action|C~ USA   A partir de ~      7000
##  7 Newlywe~  2011 Edward ~      95 Color Comedy|D~ USA   Outros             9000
##  8 Pink Fl~  1972 John Wa~     108 Color Comedy|C~ USA   A partir de ~     10000
##  9 The Tou~  2007 Jane Cl~       7 Color Romance|~ USA   Outros            13000
## 10 Paranor~  2007 Oren Pe~      84 Color Horror    USA   A partir de ~     15000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Também podemos ordenar de forma decrescente usando a função `desc()`.

```
arrange(imdb, desc(orcamento))
```

```
## # A tibble: 3,807 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
```

```
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>            <int>
##  1 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  2 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  3 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  4 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Pirates~  2011 Rob Mar~     136 Color Action|A~ USA   A partir de ~ 250000000
## 10 Captain~  2016 Anthony~     147 Color Action|A~ USA   A partir de ~ 250000000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

E claro, ordenar segundo duas ou mais colunas.

```
arrange(imdb, desc(ano), desc(orcamento))
```

```
## # A tibble: 3,807 x 15
##    titulo      ano diretor  duracao cor   generos  pais  classificacao orcamento
##    <chr>     <int> <chr>      <int> <chr> <chr>    <chr> <chr>             <int>
##  1 Batman v~  2016 Zack Sn~     183 Color Action|~ USA   A partir de ~ 250000000
##  2 Captain ~  2016 Anthony~     147 Color Action|~ USA   A partir de ~ 250000000
##  3 Star Tre~  2016 Justin ~     122 Color Action|~ USA   A partir de ~ 185000000
##  4 The Lege~  2016 David Y~     110 Color Action|~ USA   A partir de ~ 180000000
##  5 The Lege~  2016 David Y~     110 Color Action|~ USA   A partir de ~ 180000000
##  6 X-Men: A~  2016 Bryan S~     144 Color Action|~ USA   A partir de ~ 178000000
##  7 Suicide ~  2016 David A~     123 Color Action|~ USA   A partir de ~ 175000000
##  8 Alice Th~  2016 James B~     113 Color Adventu~ USA   Livre         170000000
##  9 Independ~  2016 Roland ~     120 Color Action|~ USA   A partir de ~ 165000000
## 10 Warcraft   2016 Duncan ~     123 Color Action|~ USA   A partir de ~ 160000000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Exercícios

Utilize a base `imdb` nos exercícios a seguir.

1. Ordene os filmes em ordem crescente de `ano` e decrescente de `receita` e salve em um objeto chamado `filmes_ordenados`.

2. Selecione apenas as colunas `titulo` e `orcamento` e então ordene de forma decrescente pelo `orcamento`.

7.2.3 O pipe em ação

Na grande maioria dos casos, vamos aplicar mais de uma função de manipulação em uma base para obtermos a tabela que desejamos. Poderíamos, por exemplo, querer uma tabela apenas com o título e ano dos filmes, ordenada de forma crescente de lançamento. Para fazer isso, poderíamos aninhar as funções

```
arrange(select(imdb, titulo, ano), ano)
```

```
## # A tibble: 3,807 x 2
##    titulo                                              ano
##    <chr>                                             <int>
##  1 Intolerance: Love's Struggle Throughout the Ages   1916
##  2 Over the Hill to the Poorhouse                     1920
##  3 The Big Parade                                     1925
##  4 The Broadway Melody                                1929
##  5 Hell's Angels                                      1930
##  6 A Farewell to Arms                                 1932
##  7 42nd Street                                        1933
##  8 She Done Him Wrong                                 1933
##  9 It Happened One Night                              1934
## 10 Top Hat                                            1935
## # ... with 3,797 more rows
```

ou criar um objeto intermediário

```
tab_titulo_ano <- select(imdb, titulo, ano)

arrange(tab_titulo_ano, ano)
```

```
## # A tibble: 3,807 x 2
##    titulo                                              ano
##    <chr>                                             <int>
##  1 Intolerance: Love's Struggle Throughout the Ages   1916
##  2 Over the Hill to the Poorhouse                     1920
##  3 The Big Parade                                     1925
##  4 The Broadway Melody                                1929
##  5 Hell's Angels                                      1930
##  6 A Farewell to Arms                                 1932
##  7 42nd Street                                        1933
##  8 She Done Him Wrong                                 1933
##  9 It Happened One Night                              1934
## 10 Top Hat                                            1935
## # ... with 3,797 more rows
```

Os dois códigos funcionam e levam ao mesmo resultado, mas não são muito boas.

A primeira alternativa é ruim de escrever, já que precisamos escrever primeiro a função que roda por último, e de ler, pois é difícil identificar qual argumento pertence a qual função.

A segunda alternativa é ruim pois exige a criação de objetos auxiliares. Se quiséssimos aplicar 10 operações na base, precisaríamos criar 9 objetos intermediários.

A solução para aplicar diversas operações de manipulação em uma base de dados é aplicar o operador pipe: `%>%`.

```
imdb %>%
  select(titulo, ano) %>%
  arrange(ano)
```

```
## # A tibble: 3,807 x 2
##    titulo                                              ano
##    <chr>                                             <int>
##  1 Intolerance: Love's Struggle Throughout the Ages   1916
##  2 Over the Hill to the Poorhouse                     1920
##  3 The Big Parade                                     1925
##  4 The Broadway Melody                                1929
##  5 Hell's Angels                                      1930
##  6 A Farewell to Arms                                 1932
##  7 42nd Street                                        1933
##  8 She Done Him Wrong                                 1933
##  9 It Happened One Night                              1934
## 10 Top Hat                                            1935
## # ... with 3,797 more rows
```

O que está sendo feito no código com pipe? Da primeira para a segunda linha, estamos aplicando a função `select()` à base imdb. Da segunda para a terceira, estamos aplicando a função `arrange()` à base resultante da função `select()`.

O resultado desse código é identico às tentativas sem pipe, com a vantagem de termos escrito o código na ordem em que as funções são aplicadas, de termos um código muito mais legível e de não precisarmos utilizar objetos intermediários.

7.2.4 Filtrando linhas

Para filtrar valores de uma coluna da base, utilizamos a função `filter()`.

```
imdb %>% filter(nota_imdb > 9)
```

```
## # A tibble: 3 x 15
##   titulo      ano diretor   duracao cor   generos  pais  classificacao orcamento
##   <chr>     <int> <chr>       <int> <chr> <chr>    <chr> <chr>             <int>
## 1 The Shaw~  1994 Frank Da~     142 Color Crime|D~ USA   A partir de ~  25000000
## 2 The Godf~  1972 Francis ~     175 Color Crime|D~ USA   A partir de ~   6000000
## 3 Kickboxe~  2016 John Sto~      90 <NA>  Action   USA   Outros         17000000
## # ... with 6 more variables: receita <int>, nota_imdb <dbl>,
## #   likes_facebook <int>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Podemos selecionar apenas as colunas título e nota para visualizarmos as notas:

```
imdb %>%
  filter(nota_imdb > 9) %>%
  select(titulo, nota_imdb)
```

```
## # A tibble: 3 x 2
##   titulo                    nota_imdb
##   <chr>                         <dbl>
## 1 The Shawshank Redemption        9.3
## 2 The Godfather                   9.2
## 3 Kickboxer: Vengeance            9.1
```

Podemos estender o filtro para duas ou mais colunas. Para isso, separamos cada operação por uma vírgula.

```
imdb %>% filter(ano > 2010, nota_imdb > 8.5)
```

```
## # A tibble: 5 x 15
##   titulo      ano diretor   duracao cor   generos  pais  classificacao orcamento
##   <chr>     <int> <chr>       <int> <chr> <chr>    <chr> <chr>             <int>
## 1 Interste~  2014 Christop~     169 Color Adventu~ USA   A partir de ~ 165000000
## 2 Running ~  2015 Mike May~      88 Color Family   USA   Outros          5000000
## 3 A Beginn~  2016 Mitchell~      87 Color Comedy|~ USA   Outros               NA
## 4 Kickboxe~  2016 John Sto~      90 <NA>  Action   USA   Outros         17000000
## 5 Butterfl~  2014 Cary Bell      78 Color Documen~ USA   Outros           180000
## # ... with 6 more variables: receita <int>, nota_imdb <dbl>,
## #   likes_facebook <int>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Também podemos fazer operações com as colunas da base dentro da função filter. O código abaixo devolve uma tabela apenas com os filmes que lucraram.

```
imdb %>% filter(receita - orcamento > 0)
```

```
## # A tibble: 1,761 x 15
##    titulo      ano diretor  duracao cor   generos  pais  classificacao orcamento
##    <chr>     <int> <chr>      <int> <chr> <chr>    <chr> <chr>             <int>
##  1 Avatar     2009 James C~     178 Color Action|~ USA   A partir de ~ 237000000
##  2 Pirates ~  2007 Gore Ve~     169 Color Action|~ USA   A partir de ~ 300000000
##  3 The Dark~  2012 Christo~     164 Color Action|~ USA   A partir de ~ 250000000
##  4 Spider-M~  2007 Sam Rai~     156 Color Action|~ USA   A partir de ~ 258000000
##  5 Avengers~  2015 Joss Wh~     141 Color Action|~ USA   A partir de ~ 250000000
##  6 Batman v~  2016 Zack Sn~     183 Color Action|~ USA   A partir de ~ 250000000
##  7 Pirates ~  2006 Gore Ve~     151 Color Action|~ USA   A partir de ~ 225000000
##  8 Man of S~  2013 Zack Sn~     143 Color Action|~ USA   A partir de ~ 225000000
##  9 The Aven~  2012 Joss Wh~     173 Color Action|~ USA   A partir de ~ 220000000
## 10 The Amaz~  2012 Marc We~     153 Color Action|~ USA   A partir de ~ 230000000
## # ... with 1,751 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Naturalmente, podemos filtrar colunas categóricas. O exemplo abaixo retorna uma tabela apenas com os filmes com a Angelina Jolie Pitt ou o Brad Pitt no papel principal.

```
imdb %>%
  filter(ator_1 %in% c('Angelina Jolie Pitt', "Brad Pitt"))
```

```
## # A tibble: 29 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Malefic~  2014 Robert ~      97 Color Action|A~ USA   Livre          180000000
##  2 The Cur~  2008 David F~     166 Color Drama|Fa~ USA   A partir de ~ 150000000
##  3 Kung Fu~  2011 Jennife~      90 Color Action|A~ USA   Livre          150000000
##  4 Troy      2004 Wolfgan~     196 Color Adventure USA   A partir de ~ 175000000
##  5 Kung Fu~  2008 Mark Os~      92 Color Action|A~ USA   Livre          130000000
##  6 Salt      2010 Phillip~     101 Color Action|C~ USA   A partir de ~ 110000000
##  7 Ocean's~  2004 Steven ~     125 Color Crime|Th~ USA   A partir de ~ 110000000
##  8 Mr. & M~  2005 Doug Li~     126 Color Action|C~ USA   A partir de ~ 120000000
##  9 Lara Cr~  2001 Simon W~     100 Color Action|A~ USA   A partir de ~ 115000000
## 10 Ocean's~  2001 Steven ~     116 Color Crime|Th~ USA   A partir de ~  85000000
## # ... with 19 more rows, and 6 more variables: receita <int>, nota_imdb <dbl>,
## #   likes_facebook <int>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Para filtrar textos sem correspondência exata, podemos utilizar a função auxiliar `str_detect()` do pacote `{stringr}`. Ela serve para verificar se cada string de um vetor contém um determinado padrão de texto.

```
library(stringr)

str_detect(
  string = c("a", "aa","abc", "bc", "A", NA),
  pattern = "a"
)
```

```
## [1]  TRUE  TRUE  TRUE FALSE FALSE    NA
```

Podemos utilizá-la para filtrar apenas os filmes que contêm o gênero ação.

```
# A coluna gêneros apresenta todos os gêneros dos filmes concatenados
imdb$generos[1:6]
```

```
## [1] "Action|Adventure|Fantasy|Sci-Fi"
## [2] "Action|Adventure|Fantasy"
## [3] "Action|Thriller"
## [4] "Action|Adventure|Sci-Fi"
## [5] "Action|Adventure|Romance"
## [6] "Adventure|Animation|Comedy|Family|Fantasy|Musical|Romance"
```

```
# Podemos detectar se o gênero Action aparece na string
str_detect(
  string = imdb$generos[1:6],
  pattern = "Action"
)
```

```
## [1]  TRUE  TRUE  TRUE  TRUE  TRUE FALSE
```

```
# Aplicamos essa lógica dentro da função filter, para a coluna completa
imdb %>% filter(str_detect(generos, "Action"))
```

```
## # A tibble: 861 x 15
##    titulo      ano diretor  duracao cor   generos  pais  classificacao orcamento
##    <chr>     <int> <chr>      <int> <chr> <chr>    <chr> <chr>             <int>
##  1 Avatar     2009 James C~     178 Color Action|~ USA   A partir de ~ 237000000
##  2 Pirates ~  2007 Gore Ve~     169 Color Action|~ USA   A partir de ~ 300000000
##  3 The Dark~  2012 Christo~     164 Color Action|~ USA   A partir de ~ 250000000
##  4 John Car~  2012 Andrew ~     132 Color Action|~ USA   A partir de ~ 263700000
##  5 Spider-M~  2007 Sam Rai~     156 Color Action|~ USA   A partir de ~ 258000000
##  6 Avengers~  2015 Joss Wh~     141 Color Action|~ USA   A partir de ~ 250000000
##  7 Batman v~  2016 Zack Sn~     183 Color Action|~ USA   A partir de ~ 250000000
```

```
##  8 Superman~  2006 Bryan S~     169 Color Action|~ USA   A partir de ~ 209000000
##  9 Pirates ~  2006 Gore Ve~     151 Color Action|~ USA   A partir de ~ 225000000
## 10 The Lone~  2013 Gore Ve~     150 Color Action|~ USA   A partir de ~ 215000000
## # ... with 851 more rows, and 6 more variables: receita <int>, nota_imdb <dbl>,
## #   likes_facebook <int>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Exercícios

Utilize a base `imdb` nos exercícios a seguir.

1. Crie um objeto chamado `filmes_pb` apenas com filmes preto e branco.

2. Crie um objeto chamado `curtos_legais` com filmes de 90 minutos ou menos de duração e nota no imdb maior do que 8.5.

3. Retorne tabelas (`tibbles`) apenas com:

- **a.** filmes coloridos anteriores a 1950;
- **b.** filmes do "Woody Allen" ou do "Wes Anderson";
- **c.** filmes do "Steven Spielberg" ordenados de forma decrescente por ano, mostrando apenas as colunas `titulo` e `ano`;
- **d.** filmes que tenham "Action" **ou** "Comedy" entre os seus gêneros;
- **e.** filmes que tenham "Action" **e** "Comedy" entre os seus gêneros e tenha `nota_imdb` maior que 8;
- **f.** filmes que não possuem informação tanto de receita quanto de orçamento (isto é, possuem `NA` em ambas as colunas).

7.2.5 Modificando e criando novas colunas

Para modificar uma coluna existente ou criar uma nova coluna, utilizamos a função `mutate()`. O código abaixo divide os valores da coluna duração por 60, mudando a unidade de medida dessa variável de minutos para horas.

```
imdb %>% mutate(duracao = duracao/60)
```

```
## # A tibble: 3,807 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <int>
##  1 Avatar    2009 James C~    2.97 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~    2.82 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~    2.73 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~    2.2  Color Action|A~ USA   A partir de ~ 263700000
```

```
##  5 Spider-~  2007 Sam Rai~    2.6  Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~    1.67 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~    2.35 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~    3.05 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~    2.82 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~    2.52 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Também poderíamos ter criado essa variável em uma nova coluna. Repare que a nova coluna `duracao_horas` é colocada no final da tabela.

```
imdb %>% mutate(duracao_horas = duracao/60)
```

```
## # A tibble: 3,807 x 16
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 7 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>, duracao_horas <dbl>
```

Podemos fazer qualquer operação com uma ou mais colunas. A única regra é que o resultado da operação retorne um vetor com comprimento igual ao número de linhas da base (ou com comprimento 1 para distribuir um mesmo valor em todas as linhas). Você também pode criar/modificar quantas colunas quiser dentro de um mesmo `mutate`.

```
imdb %>%
  mutate(lucro = receita - orcamento, pais = "Estados Unidos") %>%
  select(titulo, lucro, pais)
```

```
## # A tibble: 3,807 x 3
##    titulo                                        lucro pais
```

```
##    <chr>                                          <int> <chr>
##  1 Avatar                                       523505847 Estados Unidos
##  2 Pirates of the Caribbean: At World's End       9404152 Estados Unidos
##  3 The Dark Knight Rises                        198130642 Estados Unidos
##  4 John Carter                                 -190641321 Estados Unidos
##  5 Spider-Man 3                                 78530303 Estados Unidos
##  6 Tangled                                     -59192738 Estados Unidos
##  7 Avengers: Age of Ultron                     208991599 Estados Unidos
##  8 Batman v Superman: Dawn of Justice           80249062 Estados Unidos
##  9 Superman Returns                             -8930592 Estados Unidos
## 10 Pirates of the Caribbean: Dead Man's Chest   198032628 Estados Unidos
## # ... with 3,797 more rows
```

Exercícios

Utilize a base `imdb` nos exercícios a seguir.

1. Crie uma coluna chamada `prejuizo` (`orcamento - receita`) e salve a nova tabela em um objeto chamado `imdb_prejuizo`. Em seguida, filtre apenas os filmes que deram prejuízo e ordene a tabela por ordem crescente de prejuízo.

2. Fazendo apenas uma chamada da função mutate(), crie as seguintes colunas novas na base `imdb`:

- **a.** `lucro = receita - orcamento`
- **b.** `lucro_medio`
- **c.** `lucro_relativo = (lucro - lucro_medio)/lucro_medio`
- **d.** `houve_lucro = ifelse(lucro > 0, "sim", "não")`

3. Crie uma nova coluna que classifique o filme em `"recente"` (posterior a 2000) e `"antigo"` (de 2000 para trás).

7.2.6 Summarisando a base

Sumarização é a técnica de se resumir um conjunto de dados utilizando alguma métrica de interesse. A média, a mediana, a variância, a frequência, a proporção, por exemplo, são tipos de sumarização que trazem diferentes informações sobre uma variável.

Para sumarizar uma coluna da base, utilizamos a função `summarize()`. O código abaixo resume a coluna orçamento pela sua média.

```
imdb %>% summarize(media_orcamento = mean(orcamento, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   media_orcamento
##             <dbl>
## 1       35755986.
```

Repare que a saída da função continua sendo uma tibble.

Podemos calcular diversas sumarizações diferentes em um mesmo `summarize`. Cada sumarização será uma coluna da nova base.

```
imdb %>% summarise(
  media_orcamento = mean(orcamento, na.rm = TRUE),
  mediana_orcamento = median(orcamento, na.rm = TRUE),
  variancia_orcamento = var(orcamento, na.rm = TRUE)
)
```

```
## # A tibble: 1 x 3
##   media_orcamento mediana_orcamento variancia_orcamento
##             <dbl>             <int>               <dbl>
## 1       35755986.          20000000             1.82e15
```

E também sumarizar diversas colunas.

```
imdb %>% summarize(
  media_orcamento = mean(orcamento, na.rm = TRUE),
  media_receita = mean(receita, na.rm = TRUE),
  media_lucro = mean(receita - orcamento, na.rm = TRUE)
)
```

```
## # A tibble: 1 x 3
##   media_orcamento media_receita media_lucro
##             <dbl>         <dbl>       <dbl>
## 1       35755986.     55214607.   17258230.
```

Muitas vezes queremos sumarizar uma coluna agrupada pelas categorias de uma segunda coluna. Para isso, além do `summarize`, utilizamos também a função `group_by()`.

O código a seguir calcula a receita média dos filmes para cada categoria da coluna "cor".

```
imdb %>%
  group_by(cor) %>%
  summarise(receita_media = mean(receita, na.rm = TRUE))
```

```
## # A tibble: 3 x 2
##   cor             receita_media
##   <chr>                   <dbl>
## 1 Black and White     36938737.
## 2 Color               55765813.
## 3 <NA>                80014842
```

A única alteração que a função `group_by()` faz na base é a marcação de que a base está agrupada.

```
imdb %>% group_by(cor)
```

```
## # A tibble: 3,807 x 15
## # Groups:   cor [3]
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

Exercícios

Utilize a base `imdb` nos exercícios a seguir.

1. Calcule a duração média e mediana dos filmes da base.

2. Calcule o lucro médio dos filmes com duração menor que 60 minutos.

3. Apresente na mesma tabela o lucro médio dos filmes com duracao menor que 60 minutos e o lucro médio dos filmes com duracao maior ou igual a 60 minutos.

4. Retorne tabelas (`tibbles`) apenas com:

- **a.** a nota IMDB média dos filmes por tipo de classificacao;
- **b.** a receita média e mediana dos filmes por ano;
- **c.** apenas o nome dos diretores com mais de 10 filmes.

7.2.7 Juntando duas bases

Podemos juntar duas tabelas a partir de uma (coluna) chave utilizando a função `left_join()`. Como exempo, vamos inicialmente calcular o lucro médio dos filmes de cada diretor e salvar no objeto `tab_lucro_diretor`.

```
tab_lucro_diretor <- imdb %>%
  group_by(diretor) %>%
  summarise(lucro_medio = mean(receita - orcamento, na.rm = TRUE))

tab_lucro_diretor
```

```
## # A tibble: 1,813 x 2
##    diretor          lucro_medio
##    <chr>                  <dbl>
##  1 A. Raven Cruz            NaN
##  2 Aaron Hann               NaN
##  3 Aaron Schneider      1676553
##  4 Aaron Seltzer       28546578
##  5 Abel Ferrara       -11272676
##  6 Adam Carolla        -1394057
##  7 Adam Goldberg       -1647420
##  8 Adam Green               NaN
##  9 Adam Jay Epstein         NaN
## 10 Adam Marcus          13435068
## # ... with 1,803 more rows
```

E se quisermos colocar essa informação na base original? Basta usar a função `left_join()` utilizando a coluna `diretor` como chave. Observe que a coluna `lucro_medio` aparece agora no fim da tabela.

```
imdb_com_lucro_medio <- left_join(imdb, tab_lucro_diretor, by = "diretor")

imdb_com_lucro_medio
```

```
## # A tibble: 3,807 x 16
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
```

```
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 7 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>, lucro_medio <dbl>
```

Na tabela `imdb_com_lucro_medio`, como na tabela `imdb`, cada linha continua a representar um filme diferente, mas agora temos também a informação do lucro médio do diretor de cada filme.

A primeira linha, por exemplo, traz as informações do filme Avatar. O valor do `lucro_medio` nessa linha representa o lucro médio de todos os filmes do James Cameron, que é o diretor de Avatar. Com essa informação, podemos calcular o quanto o lucro do Avatar se afasta do lucro médio do James Cameron.

```
imdb_com_lucro_medio %>%
  mutate(
    lucro = receita - orcamento,
    lucro_relativo = (lucro - lucro_medio)/lucro_medio,
    lucro_relativo = scales::percent(lucro_relativo)
  ) %>%
  select(titulo, diretor, lucro, lucro_medio, lucro_relativo)
```

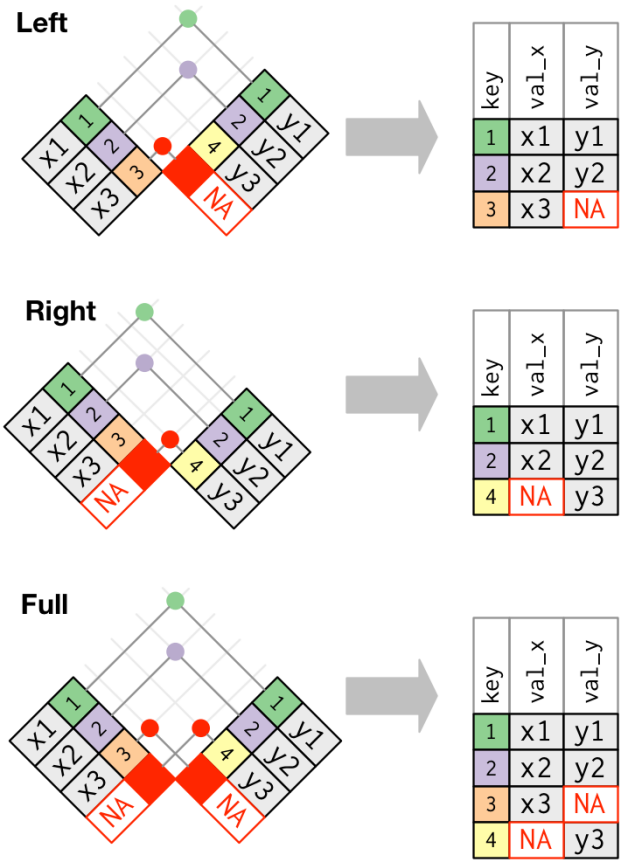
```
## # A tibble: 3,807 x 5
##    titulo                      diretor          lucro lucro_medio lucro_relativo
##    <chr>                       <chr>            <int>       <dbl> <chr>
##  1 Avatar                      James Cameron   5.24e8  194620985  168.98736%
##  2 Pirates of the Caribbean: ~ Gore Verbins~   9.40e6   36942999. -74.54416%
##  3 The Dark Knight Rises       Christopher ~   1.98e8  101028447  96.11372%
##  4 John Carter                 Andrew Stant~  -1.91e8   46668146  -508.50417%
##  5 Spider-Man 3                Sam Raimi       7.85e7   63300090. 24.06033%
##  6 Tangled                     Nathan Greno   -5.92e7  -59192738  0.00000%
##  7 Avengers: Age of Ultron     Joss Whedon     2.09e8  250221657  -16.47741%
##  8 Batman v Superman: Dawn of~ Zack Snyder     8.02e7   33149106. 142.08514%
##  9 Superman Returns            Bryan Singer   -8.93e6   -2887024. 209.33560%
## 10 Pirates of the Caribbean: ~ Gore Verbins~   1.98e8   36942999. 436.04913%
## # ... with 3,797 more rows
```

Observamos então que o Avatar obteve um lucro aproximadamente 169% maior que a média dos filmes do James Cameron.

Além da função `left_join()`, também são muito utilizadas as funções `right_join()` e `full_join()`.

- `right_join()`: retorna todas as linhas da base `y` e todas as colunas das bases `x` e `y`. Linhas de `y` sem correspondentes em `x` receberão `NA` na nova base.
- `full_join()`: retorna todas as linhas e colunas de `x`e `y`. Valores sem correspondência entre as bases receberão `NA` na nova base.

A figura a seguir esquematiza as operações dessas funções:



Exercícios

1. Utilize a base `imdb` para resolver os itens a seguir.

a. Salve em um novo objeto uma tabela com a nota média dos filmes de cada diretor. Essa tabela deve conter duas colunas (`diretor` e `nota_imdb_media`) e cada linha deve ser um diretor diferente.

b. Use o `left_join()` para trazer a coluna `nota_imdb_media` da tabela do item anterior para a tabela `imdb` original.

7.2.8 dplyr 1.0

A versão 1.0 do pacote `dplyr` foi oficialmente lançada em junho de 2020 e contou com diversas novidades. Vamos falar das principais mudanças:

- A nova função `across()`, que facilita aplicar uma mesma operação em várias colunas.
- A repaginada função `rowwise()`, para fazer operações por linha.
- Novas funcionalidades das funções `select()` e `rename()` e a nova função `relocate()`.

Para trabalhar essas funções, vamos utilizar a base `casas` do pacote `dados`. Para instalar esse pacote, rode os códigos abaixo:

```
install.packages("remotes")
remotes::install_github("cienciadedatos/dados")
```

Para trazer os dados para o nosso *environment*, podemos rodar:

```
casas <- dados::casas
```

A base `casas` possui dados de venda de casas na cidade de Ames, nos Estados Unidos. São 2930 linhas e 77 colunas, sendo que cada linha corresponde a uma casa vendida e cada coluna a uma característica da casa ou da venda. Essa versão é uma tradução da base original, que pode ser encontrada no pacote `AmesHousing`:

```
install.packages("AmesHousing")
data(ames_raw, package = "AmesHousing")
```

A função `across()`

A função `across()` substitui a família de verbos `_all()`, `_if` e `_at()`. A ideia é facilitar a aplicação de uma operação a diversas colunas da base. Para sumarizar a base para mais de uma variável, antigamente poderíamos fazer:

```
casas %>%
  group_by(geral_qualidade) %>%
  summarise(
    lote_area_media = mean(lote_area, na.rm = TRUE),
    venda_valor_medio = mean(venda_valor, na.rm = TRUE)
  )
```

```
## # A tibble: 10 x 3
##    geral_qualidade lote_area_media venda_valor_medio
##    <chr>                     <dbl>             <dbl>
##  1 abaixo da média           8464.           106485.
##  2 acima da média            9788.           162130.
##  3 boa                      10309.           205026.
##  4 excelente                12777.           368337.
##  5 média                     9996.           134753.
##  6 muito boa                10618.           270914.
##  7 Muito excelente          18071.           450217.
##  8 muito ruim               15214.            48725
##  9 regular                   9439.            83186.
## 10 ruim                      9326.            52325.
```

Ou então utilizar a função `summarise_at()`:

```
casas %>%
  group_by(geral_qualidade) %>%
  summarise_at(
    .vars = vars(lote_area, venda_valor),
    list(mean),
    na.rm = TRUE
  )
```

```
## # A tibble: 10 x 3
##    geral_qualidade lote_area venda_valor
##    <chr>                <dbl>       <dbl>
##  1 abaixo da média      8464.     106485.
##  2 acima da média       9788.     162130.
##  3 boa                 10309.     205026.
##  4 excelente           12777.     368337.
##  5 média                9996.     134753.
##  6 muito boa           10618.     270914.
##  7 Muito excelente     18071.     450217.
##  8 muito ruim          15214.      48725
##  9 regular              9439.      83186.
## 10 ruim                 9326.      52325.
```

Com a nova função `across()`, fazemos:

```
casas %>%
  group_by(geral_qualidade) %>%
  summarise(across(
    .cols = c(lote_area, venda_valor),
    .fns = mean,
    na.rm = TRUE
  ))
```

```
## # A tibble: 10 x 3
##    geral_qualidade lote_area venda_valor
##    <chr>               <dbl>       <dbl>
##  1 abaixo da média     8464.     106485.
##  2 acima da média      9788.     162130.
##  3 boa                10309.     205026.
##  4 excelente          12777.     368337.
##  5 média               9996.     134753.
##  6 muito boa          10618.     270914.
##  7 Muito excelente    18071.     450217.
##  8 muito ruim         15214.      48725
##  9 regular             9439.      83186.
## 10 ruim                9326.      52325.
```

A sintaxe é parecida com a função `summarise_at()`, mas agora não precisamos mais usar a função `vars()` e nem usar `list(nome_da_funcao)`para definir a função aplicada nas colunas.

Usando `across()`, podemos facilmente aplicar uma função em todas as colunas da nossa base. Abaixo, calculamos o número de valores distintos para todas as variáveis da base `casas`. O padrão do parâmetro `.cols` é `everithing()`, que representa "todas as colunas".

```
casas %>%
  summarise(across(.fns = n_distinct))
```

```
## # A tibble: 1 x 82
##   ordem   pid moradia_classe moradia_zoneamento lote_fachada lote_area rua_tipo
##   <int> <int>          <int>              <int>        <int>     <int>    <int>
## 1  2930  2930             16                  7          129      1960        2
## # ... with 75 more variables: beco_tipo <int>, lote_formato <int>,
## #   terreno_contorno <int>, utilidades <int>, lote_config <int>,
## #   terreno_declive <int>, vizinhanca <int>, condicao_1 <int>,
## #   condicao_2 <int>, moradia_tipo <int>, moradia_estilo <int>,
## #   geral_qualidade <int>, geral_condicao <int>, construcao_ano <int>,
```

```
## #   remodelacao_ano <int>, telhado_estilo <int>, telhado_material <int>,
## #   exterior_cobertura_1 <int>, exterior_cobertura_2 <int>,
## #   alvenaria_tipo <int>, alvenaria_area <int>, exterior_qualidade <int>,
## #   exterior_condicao <int>, fundacao_tipo <int>, porao_qualidade <int>,
## #   porao_condicao <int>, porao_exposicao <int>, porao_acabamento_1 <int>,
## #   porao_area_com_acabamento_1 <int>, porao_acabamento_2 <int>,
## #   porao_area_com_acabamento_2 <int>, porao_area_sem_acabamento <int>,
## #   porao_area_total <int>, porao_num_banheiros <int>,
## #   porao_num_banheiros_lavabos <int>, aquecimento_tipo <int>,
## #   aquecimento_qualidade_condicao <int>, ar_condicionado_central <int>,
## #   sistema_eletrico_tipo <int>, primeiro_andar_area <int>,
## #   segundo_andar_area <int>, acabamento_baixa_qualidade_area <int>,
## #   acima_solo_area <int>, acima_solo_num_banheiros <int>,
## #   acima_solo_num_lavabos <int>, acima_solo_num_quartos <int>,
## #   acima_solo_num_cozinhas <int>, cozinha_qualidade <int>,
## #   acima_solo_num_comodos <int>, funcional <int>, total_num_lareiras <int>,
## #   lareira_qualidade <int>, garagem_tipo <int>, garagem_ano_construcao <int>,
## #   garagem_acabamento <int>, garagem_capacidade_carros <int>,
## #   garagem_area <int>, garagem_qualidade <int>, garagem_condicao <int>,
## #   entrada_veiculo_pavimentada <int>, deck_madeira_area <int>,
## #   varanda_aberta_area <int>, varanda_fechada_area <int>,
## #   varanda_3ssn_area <int>, varanda_com_tela_area <int>, piscina_area <int>,
## #   piscina_qualidade <int>, cerca_qualidade <int>,
## #   funcionalidades_diversas <int>, funcionalidades_valor <int>,
## #   venda_mes <int>, venda_ano <int>, venda_tipo <int>, venda_condicao <int>,
## #   venda_valor <int>
```

Para fazer essa mesma operação, antes utilizaríamos a função `summarise_all()`.

```
casas %>%
  summarise_all(.funs = ~n_distinct(.x))
```

```
## # A tibble: 1 x 82
##   ordem   pid moradia_classe moradia_zoneamento lote_fachada lote_area rua_tipo
##   <int> <int>          <int>              <int>        <int>     <int>    <int>
## 1  2930  2930             16                  7          129      1960        2
## # ... with 75 more variables: beco_tipo <int>, lote_formato <int>,
## #   terreno_contorno <int>, utilidades <int>, lote_config <int>,
## #   terreno_declive <int>, vizinhanca <int>, condicao_1 <int>,
## #   condicao_2 <int>, moradia_tipo <int>, moradia_estilo <int>,
## #   geral_qualidade <int>, geral_condicao <int>, construcao_ano <int>,
## #   remodelacao_ano <int>, telhado_estilo <int>, telhado_material <int>,
## #   exterior_cobertura_1 <int>, exterior_cobertura_2 <int>,
## #   alvenaria_tipo <int>, alvenaria_area <int>, exterior_qualidade <int>,
```

```
## #   exterior_condicao <int>, fundacao_tipo <int>, porao_qualidade <int>,
## #   porao_condicao <int>, porao_exposicao <int>, porao_acabamento_1 <int>,
## #   porao_area_com_acabamento_1 <int>, porao_acabamento_2 <int>,
## #   porao_area_com_acabamento_2 <int>, porao_area_sem_acabamento <int>,
## #   porao_area_total <int>, porao_num_banheiros <int>,
## #   porao_num_banheiros_lavabos <int>, aquecimento_tipo <int>,
## #   aquecimento_qualidade_condicao <int>, ar_condicionado_central <int>,
## #   sistema_eletrico_tipo <int>, primeiro_andar_area <int>,
## #   segundo_andar_area <int>, acabamento_baixa_qualidade_area <int>,
## #   acima_solo_area <int>, acima_solo_num_banheiros <int>,
## #   acima_solo_num_lavabos <int>, acima_solo_num_quartos <int>,
## #   acima_solo_num_cozinhas <int>, cozinha_qualidade <int>,
## #   acima_solo_num_comodos <int>, funcional <int>, total_num_lareiras <int>,
## #   lareira_qualidade <int>, garagem_tipo <int>, garagem_ano_construcao <int>,
## #   garagem_acabamento <int>, garagem_capacidade_carros <int>,
## #   garagem_area <int>, garagem_qualidade <int>, garagem_condicao <int>,
## #   entrada_veiculo_pavimentada <int>, deck_madeira_area <int>,
## #   varanda_aberta_area <int>, varanda_fechada_area <int>,
## #   varanda_3ssn_area <int>, varanda_com_tela_area <int>, piscina_area <int>,
## #   piscina_qualidade <int>, cerca_qualidade <int>,
## #   funcionalidades_diversas <int>, funcionalidades_valor <int>,
## #   venda_mes <int>, venda_ano <int>, venda_tipo <int>, venda_condicao <int>,
## #   venda_valor <int>
```

Se quisermos selecionar as colunas a serem modificadas a partir de um teste lógico, utilizamos o ajudante `where()`.

No exemplo abaixo, calculamos o número de valores distintos das colunas de categóricas.

```
casas %>%
  summarise(across(where(is.character), n_distinct))
```

```
## # A tibble: 1 x 47
##     pid moradia_classe moradia_zoneamento rua_tipo beco_tipo lote_formato
##   <int>          <int>              <int>    <int>     <int>        <int>
## 1  2930             16                  7        2         3            4
## # ... with 41 more variables: terreno_contorno <int>, utilidades <int>,
## #   lote_config <int>, terreno_declive <int>, vizinhanca <int>,
## #   condicao_1 <int>, condicao_2 <int>, moradia_tipo <int>,
## #   moradia_estilo <int>, geral_qualidade <int>, geral_condicao <int>,
## #   telhado_estilo <int>, telhado_material <int>, exterior_cobertura_1 <int>,
## #   exterior_cobertura_2 <int>, alvenaria_tipo <int>, exterior_qualidade <int>,
## #   exterior_condicao <int>, fundacao_tipo <int>, porao_qualidade <int>,
## #   porao_condicao <int>, porao_exposicao <int>, porao_acabamento_1 <int>,
## #   porao_acabamento_2 <int>, aquecimento_tipo <int>,
```

```
## #   aquecimento_qualidade_condicao <int>, ar_condicionado_central <int>,
## #   sistema_eletrico_tipo <int>, cozinha_qualidade <int>, funcional <int>,
## #   lareira_qualidade <int>, garagem_tipo <int>, garagem_acabamento <int>,
## #   garagem_qualidade <int>, garagem_condicao <int>,
## #   entrada_veiculo_pavimentada <int>, piscina_qualidade <int>,
## #   cerca_qualidade <int>, funcionalidades_diversas <int>, venda_tipo <int>,
## #   venda_condicao <int>
```

Todas as colunas da base resultante eram colunas com classe `character` na base `casas`.

Anteriormente, utilizávamos a função `summarise_if()`.

```
casas %>%
  summarise_if(is.character, n_distinct)
```

```
## # A tibble: 1 x 47
##     pid moradia_classe moradia_zoneamento rua_tipo beco_tipo lote_formato
##   <int>          <int>              <int>    <int>     <int>        <int>
## 1  2930             16                  7        2         3            4
## # ... with 41 more variables: terreno_contorno <int>, utilidades <int>,
## #   lote_config <int>, terreno_declive <int>, vizinhanca <int>,
## #   condicao_1 <int>, condicao_2 <int>, moradia_tipo <int>,
## #   moradia_estilo <int>, geral_qualidade <int>, geral_condicao <int>,
## #   telhado_estilo <int>, telhado_material <int>, exterior_cobertura_1 <int>,
## #   exterior_cobertura_2 <int>, alvenaria_tipo <int>, exterior_qualidade <int>,
## #   exterior_condicao <int>, fundacao_tipo <int>, porao_qualidade <int>,
## #   porao_condicao <int>, porao_exposicao <int>, porao_acabamento_1 <int>,
## #   porao_acabamento_2 <int>, aquecimento_tipo <int>,
## #   aquecimento_qualidade_condicao <int>, ar_condicionado_central <int>,
## #   sistema_eletrico_tipo <int>, cozinha_qualidade <int>, funcional <int>,
## #   lareira_qualidade <int>, garagem_tipo <int>, garagem_acabamento <int>,
## #   garagem_qualidade <int>, garagem_condicao <int>,
## #   entrada_veiculo_pavimentada <int>, piscina_qualidade <int>,
## #   cerca_qualidade <int>, funcionalidades_diversas <int>, venda_tipo <int>,
## #   venda_condicao <int>
```

Com o `across()`, podemos combinar os efeitos de um `summarise_if()` e um `summarise_at()` em um único `summarise()`. A seguir, calculamos as áreas médias, garantindo que pegamos apenas variáveis numéricas.

```
casas %>%
  summarise(across(where(is.numeric) & contains("_area"), mean, na.rm = TRUE))
```

```
## # A tibble: 1 x 17
```

```
##   lote_area alvenaria_area porao_area_com_ac~ porao_area_com_a~ porao_area_sem_~
##       <dbl>          <dbl>              <dbl>             <dbl>            <dbl>
## 1    10148.           102.               443.              49.7             559.
## # ... with 12 more variables: porao_area_total <dbl>,
## #   primeiro_andar_area <dbl>, segundo_andar_area <dbl>,
## #   acabamento_baixa_qualidade_area <dbl>, acima_solo_area <dbl>,
## #   garagem_area <dbl>, deck_madeira_area <dbl>, varanda_aberta_area <dbl>,
## #   varanda_fechada_area <dbl>, varanda_3ssn_area <dbl>,
## #   varanda_com_tela_area <dbl>, piscina_area <dbl>
```

Além disso, com a função `across()`, podemos fazer sumarizações complexas que não seria possível utilizando apenas as funções `summarise()`, `summarise_if()` e `summarise_at()`. No exemplo a seguir, calculamos a média das áreas, o número de `NAs` de variáveis categóricas e o número de casas para cada tipo de fundação. Tudo em um mesmo `summarise()`!

```
casas %>%
  group_by(fundacao_tipo) %>%
  summarise(
    across(where(is.numeric) & contains("area"), mean, na.rm = TRUE),
    across(where(is.character), ~sum(is.na(.x))),
    n_obs = n(),
  ) %>%
  select(1:4, n_obs)
```

```
## # A tibble: 6 x 5
##   fundacao_tipo       lote_area alvenaria_area porao_area_com_acabamento_1 n_obs
##   <chr>                   <dbl>          <dbl>                       <dbl> <int>
## 1 bloco de concreto      10616.           85.0                       468.   1244
## 2 concreto derrramado    10054.          144.                        506.   1310
## 3 laje                   10250.           35.2                         0      49
## 4 madeira                 9838.           16                         812.      5
## 5 pedra                   8659.            0                          43.9    11
## 6 tijolo e telha          8712.           10.2                       151.    311
```

Embora a nova sintaxe, usando `across()`, não seja muito diferente do que fazíamos antes, realizar sumarizações complexas não é a única vantagem desse novo *framework*.

O `across()` pode ser utilizado em todos os verbos do `{dplyr}` (com exceção do `select()` e `rename()`, já que ele não trás vantagens com relação ao que já podia ser feito) e isso unifica o modo de fazermos essas operações no `dplyr`. Em vez de termos uma família de funções para cada verbo, temos agora apenas o próprio verbo e a função `across()`.

Vamos ver um exemplo para o `mutate()` e para o `filter()`.

O código abaixo transforma todas as variáveis que possuem "area" no nome, passando os valores de pés quadrados para metros quadrados.

```
casas %>%
  mutate(across(
    contains("area"),
    ~ .x / 10.764
  ))
```

Já o código a seguir filtra apenas as casas que possuem varanda aberta, cerca e lareira (o `NA` nessa base significa que a casa não possui tal característica).

```
casas %>%
  filter(across(
    c(varanda_aberta_area, cerca_qualidade, lareira_qualidade),
    ~!is.na(.x)
  ))
```

Não precisamos do `across()` na hora de selecionar colunas. A função `select()` já usa naturalmente o mecanismo de seleção de colunas que o `across()` proporciona.

```
casas %>%
  select(where(is.numeric))
```

```
## # A tibble: 2,930 x 35
##    ordem lote_fachada lote_area construcao_ano remodelacao_ano alvenaria_area
##    <int>        <int>     <int>          <int>           <int>          <int>
##  1     1          141     31770           1960            1960            112
##  2     2           80     11622           1961            1961              0
##  3     3           81     14267           1958            1958            108
##  4     4           93     11160           1968            1968              0
##  5     5           74     13830           1997            1998              0
##  6     6           78      9978           1998            1998             20
##  7     7           41      4920           2001            2001              0
##  8     8           43      5005           1992            1992              0
##  9     9           39      5389           1995            1996              0
## 10    10           60      7500           1999            1999              0
## # ... with 2,920 more rows, and 29 more variables:
## #   porao_area_com_acabamento_1 <int>, porao_area_com_acabamento_2 <int>,
## #   porao_area_sem_acabamento <int>, porao_area_total <int>,
## #   porao_num_banheiros <int>, porao_num_banheiros_lavabos <int>,
## #   primeiro_andar_area <int>, segundo_andar_area <int>,
## #   acabamento_baixa_qualidade_area <int>, acima_solo_area <int>,
```

```
## #   acima_solo_num_banheiros <int>, acima_solo_num_lavabos <int>,
## #   acima_solo_num_quartos <int>, acima_solo_num_cozinhas <int>,
## #   acima_solo_num_comodos <int>, total_num_lareiras <int>,
## #   garagem_ano_construcao <int>, garagem_capacidade_carros <int>,
## #   garagem_area <int>, deck_madeira_area <int>, varanda_aberta_area <int>,
## #   varanda_fechada_area <int>, varanda_3ssn_area <int>,
## #   varanda_com_tela_area <int>, piscina_area <int>,
## #   funcionalidades_valor <int>, venda_mes <int>, venda_ano <int>,
## #   venda_valor <int>
```

O mesmo vale para o `rename()`. Se quisermos renomer várias colunas, a partir de uma função, utilizamos o `rename_with()`.

```
casas %>%
  rename_with(toupper, contains("venda"))
```

```
## # A tibble: 2,930 x 82
##    ordem pid    moradia_classe  moradia_zoneame~ lote_fachada lote_area rua_tipo
##    <int> <chr>  <chr>           <chr>                   <int>     <int> <chr>
##  1     1 05263~ 020             Residencial bai~          141     31770 pavimen~
##  2     2 05263~ 020             Residencial alt~           80     11622 pavimen~
##  3     3 05263~ 020             Residencial bai~           81     14267 pavimen~
##  4     4 05263~ 020             Residencial bai~           93     11160 pavimen~
##  5     5 05271~ 060             Residencial bai~           74     13830 pavimen~
##  6     6 05271~ 060             Residencial bai~           78      9978 pavimen~
##  7     7 05271~ desenvolviment~ Residencial bai~           41      4920 pavimen~
##  8     8 05271~ desenvolviment~ Residencial bai~           43      5005 pavimen~
##  9     9 05271~ desenvolviment~ Residencial bai~           39      5389 pavimen~
## 10    10 05271~ 060             Residencial bai~           60      7500 pavimen~
## # ... with 2,920 more rows, and 75 more variables: beco_tipo <chr>,
## #   lote_formato <chr>, terreno_contorno <chr>, utilidades <chr>,
## #   lote_config <chr>, terreno_declive <chr>, vizinhanca <chr>,
## #   condicao_1 <chr>, condicao_2 <chr>, moradia_tipo <chr>,
## #   moradia_estilo <chr>, geral_qualidade <chr>, geral_condicao <chr>,
## #   construcao_ano <int>, remodelacao_ano <int>, telhado_estilo <chr>,
## #   telhado_material <chr>, exterior_cobertura_1 <chr>,
## #   exterior_cobertura_2 <chr>, alvenaria_tipo <chr>, alvenaria_area <int>,
## #   exterior_qualidade <chr>, exterior_condicao <chr>, fundacao_tipo <chr>,
## #   porao_qualidade <chr>, porao_condicao <chr>, porao_exposicao <chr>,
## #   porao_acabamento_1 <chr>, porao_area_com_acabamento_1 <int>,
## #   porao_acabamento_2 <chr>, porao_area_com_acabamento_2 <int>,
## #   porao_area_sem_acabamento <int>, porao_area_total <int>,
## #   porao_num_banheiros <int>, porao_num_banheiros_lavabos <int>,
## #   aquecimento_tipo <chr>, aquecimento_qualidade_condicao <chr>,
## #   ar_condicionado_central <chr>, sistema_eletrico_tipo <chr>,
```

```
## #   primeiro_andar_area <int>, segundo_andar_area <int>,
## #   acabamento_baixa_qualidade_area <int>, acima_solo_area <int>,
## #   acima_solo_num_banheiros <int>, acima_solo_num_lavabos <int>,
## #   acima_solo_num_quartos <int>, acima_solo_num_cozinhas <int>,
## #   cozinha_qualidade <chr>, acima_solo_num_comodos <int>, funcional <chr>,
## #   total_num_lareiras <int>, lareira_qualidade <chr>, garagem_tipo <chr>,
## #   garagem_ano_construcao <int>, garagem_acabamento <chr>,
## #   garagem_capacidade_carros <int>, garagem_area <int>,
## #   garagem_qualidade <chr>, garagem_condicao <chr>,
## #   entrada_veiculo_pavimentada <chr>, deck_madeira_area <int>,
## #   varanda_aberta_area <int>, varanda_fechada_area <int>,
## #   varanda_3ssn_area <int>, varanda_com_tela_area <int>, piscina_area <int>,
## #   piscina_qualidade <chr>, cerca_qualidade <chr>,
## #   funcionalidades_diversas <chr>, funcionalidades_valor <int>,
## #   VENDA_MES <int>, VENDA_ANO <int>, VENDA_TIPO <chr>, VENDA_CONDICAO <chr>,
## #   VENDA_VALOR <int>
```

A função `relocate()`

O `{dplyr}` possui agora uma função própria para reorganizar colunas: `relocate()`. Por padrão, ela coloca uma ou mais colunas no começo da base.

```
casas %>%
  relocate(venda_valor, venda_tipo)
```

```
## # A tibble: 2,930 x 82
##    venda_valor venda_tipo ordem pid    moradia_classe          moradia_zoneamen~
##          <int> <chr>      <int> <chr>  <chr>                   <chr>
##  1      215000 "WD "          1 05263~ 020                     Residencial baix~
##  2      105000 "WD "          2 05263~ 020                     Residencial alta~
##  3      172000 "WD "          3 05263~ 020                     Residencial baix~
##  4      244000 "WD "          4 05263~ 020                     Residencial baix~
##  5      189900 "WD "          5 05271~ 060                     Residencial baix~
##  6      195500 "WD "          6 05271~ 060                     Residencial baix~
##  7      213500 "WD "          7 05271~ desenvolvimento de uni~ Residencial baix~
##  8      191500 "WD "          8 05271~ desenvolvimento de uni~ Residencial baix~
##  9      236500 "WD "          9 05271~ desenvolvimento de uni~ Residencial baix~
## 10      189000 "WD "         10 05271~ 060                     Residencial baix~
## # ... with 2,920 more rows, and 76 more variables: lote_fachada <int>,
## #   lote_area <int>, rua_tipo <chr>, beco_tipo <chr>, lote_formato <chr>,
## #   terreno_contorno <chr>, utilidades <chr>, lote_config <chr>,
## #   terreno_declive <chr>, vizinhanca <chr>, condicao_1 <chr>,
## #   condicao_2 <chr>, moradia_tipo <chr>, moradia_estilo <chr>,
## #   geral_qualidade <chr>, geral_condicao <chr>, construcao_ano <int>,
```

```
## #   remodelacao_ano <int>, telhado_estilo <chr>, telhado_material <chr>,
## #   exterior_cobertura_1 <chr>, exterior_cobertura_2 <chr>,
## #   alvenaria_tipo <chr>, alvenaria_area <int>, exterior_qualidade <chr>,
## #   exterior_condicao <chr>, fundacao_tipo <chr>, porao_qualidade <chr>,
## #   porao_condicao <chr>, porao_exposicao <chr>, porao_acabamento_1 <chr>,
## #   porao_area_com_acabamento_1 <int>, porao_acabamento_2 <chr>,
## #   porao_area_com_acabamento_2 <int>, porao_area_sem_acabamento <int>,
## #   porao_area_total <int>, porao_num_banheiros <int>,
## #   porao_num_banheiros_lavabos <int>, aquecimento_tipo <chr>,
## #   aquecimento_qualidade_condicao <chr>, ar_condicionado_central <chr>,
## #   sistema_eletrico_tipo <chr>, primeiro_andar_area <int>,
## #   segundo_andar_area <int>, acabamento_baixa_qualidade_area <int>,
## #   acima_solo_area <int>, acima_solo_num_banheiros <int>,
## #   acima_solo_num_lavabos <int>, acima_solo_num_quartos <int>,
## #   acima_solo_num_cozinhas <int>, cozinha_qualidade <chr>,
## #   acima_solo_num_comodos <int>, funcional <chr>, total_num_lareiras <int>,
## #   lareira_qualidade <chr>, garagem_tipo <chr>, garagem_ano_construcao <int>,
## #   garagem_acabamento <chr>, garagem_capacidade_carros <int>,
## #   garagem_area <int>, garagem_qualidade <chr>, garagem_condicao <chr>,
## #   entrada_veiculo_pavimentada <chr>, deck_madeira_area <int>,
## #   varanda_aberta_area <int>, varanda_fechada_area <int>,
## #   varanda_3ssn_area <int>, varanda_com_tela_area <int>, piscina_area <int>,
## #   piscina_qualidade <chr>, cerca_qualidade <chr>,
## #   funcionalidades_diversas <chr>, funcionalidades_valor <int>,
## #   venda_mes <int>, venda_ano <int>, venda_condicao <chr>
```

Podemos usar os argumentos `.after` e `.before` para fazer mudanças mais complexas.

O código baixo coloca a coluna `venda_ano` depois da coluna `construcao_ano`.

```
casas %>%
  relocate(venda_ano, .after = construcao_ano)
```

O código baixo coloca a coluna `venda_ano` antes da coluna `construcao_ano`.

```
casas %>%
  relocate(venda_ano, .before = construcao_ano)
```

A função `rowwise()`

Por fim, vamos discutir operações feitas por linha. Tome como exemplo a tabela abaixo. Ela apresenta as notas de alunos em quatro provas.

```
tab_notas <- tibble(
  student_id = 1:5,
  prova1 = sample(0:10, 5),
  prova2 = sample(0:10, 5),
  prova3 = sample(0:10, 5),
  prova4 = sample(0:10, 5)
)

tab_notas
```

```
## # A tibble: 5 x 5
##   student_id prova1 prova2 prova3 prova4
##        <int>  <int>  <int>  <int>  <int>
## 1          1      5      2      4      8
## 2          2      6     10      6      2
## 3          3      3      5      0      7
## 4          4      2      1      1     10
## 5          5      4      0      9      0
```

Se quisermos gerar uma coluna com a nota média de cada aluno nas quatro provas, não poderíamos usar o `mutate()` diretamente.

```
tab_notas %>% mutate(media = mean(c(prova1, prova2, prova3, prova4)))
```

```
## # A tibble: 5 x 6
##   student_id prova1 prova2 prova3 prova4 media
##        <int>  <int>  <int>  <int>  <int> <dbl>
## 1          1      5      2      4      8  4.25
## 2          2      6     10      6      2  4.25
## 3          3      3      5      0      7  4.25
## 4          4      2      1      1     10  4.25
## 5          5      4      0      9      0  4.25
```

Neste caso, todas as colunas estão sendo empilhadas e gerando uma única média, passada a todas as linhas da coluna `media`.

Para fazermos a conta para cada aluno, podemos agrupar por aluno. Agora sim a média é calculada apenas nas notas de cada estudante.

```
tab_notas %>%
  group_by(student_id) %>%
  mutate(media = mean(c(prova1, prova2, prova3, prova4)))
```

```
## # A tibble: 5 x 6
## # Groups:   student_id [5]
##   student_id prova1 prova2 prova3 prova4 media
##        <int>  <int>  <int>  <int>  <int> <dbl>
## 1          1      5      2      4      8  4.75
## 2          2      6     10      6      2  6
## 3          3      3      5      0      7  3.75
## 4          4      2      1      1     10  3.5
## 5          5      4      0      9      0  3.25
```

Também podemos nos aproveitar da sintaxe do `across()` neste caso. Para isso, precisamos substutir a função `c()` pela função `c_across()`.

```
tab_notas %>%
  group_by(student_id) %>%
  mutate(media = mean(c_across(starts_with("prova"))))
```

```
## # A tibble: 5 x 6
## # Groups:   student_id [5]
##   student_id prova1 prova2 prova3 prova4 media
##        <int>  <int>  <int>  <int>  <int> <dbl>
## 1          1      5      2      4      8  4.75
## 2          2      6     10      6      2  6
## 3          3      3      5      0      7  3.75
## 4          4      2      1      1     10  3.5
## 5          5      4      0      9      0  3.25
```

Equivalentemente ao `group_by()`, neste caso, podemos usar a função `rowwise()`.

```
tab_notas %>%
  rowwise(student_id) %>%
  mutate(media = mean(c_across(starts_with("prova"))))
```

```
## # A tibble: 5 x 6
## # Rowwise:  student_id
##   student_id prova1 prova2 prova3 prova4 media
##        <int>  <int>  <int>  <int>  <int> <dbl>
## 1          1      5      2      4      8  4.75
## 2          2      6     10      6      2  6
## 3          3      3      5      0      7  3.75
## 4          4      2      1      1     10  3.5
## 5          5      4      0      9      0  3.25
```

Ela é muito útil quando queremos fazer operação por linhas, mas não temos uma coluna de identificação. Por padrão, se não indicarmos nenhuma coluna, cada linha será um "grupo".

```
tab_notas %>%
  rowwise() %>%
  mutate(media = mean(c_across(starts_with("prova"))))
```

```
## # A tibble: 5 x 6
## # Rowwise:
##   student_id prova1 prova2 prova3 prova4 media
##        <int>  <int>  <int>  <int>  <int> <dbl>
## 1          1      5      2      4      8  4.75
## 2          2      6     10      6      2  6
## 3          3      3      5      0      7  3.75
## 4          4      2      1      1     10  3.5
## 5          5      4      0      9      0  3.25
```

Veja que `student_id` não é passada para a função `rowwise()`. Não precisaríamos dessa coluna na base para reproduzir a geração da columa `media` neste caso.

Exercícios

A base `casas` abaixo pode ser encontrada a partir do código abaixo:

```
remotes::install_github("cienciadedatos/dados")
dados::casas
```

1. Reescreva os códigos abaixo utilizando as funções `across()` e `where()`.

a.

```
casas %>%
  group_by(geral_qualidade) %>%
  summarise(
    acima_solo_area_media = mean(acima_solo_area, na.rm = TRUE),
    garagem_area_media = mean(garagem_area, na.rm = TRUE),
    valor_venda_medio = mean(venda_valor, na.rm = TRUE)
  )
```

b.

```
casas %>%
  filter_at(
    vars(porao_qualidade, varanda_fechada_area, cerca_qualidade),
    ~!is.na(.x)
  )
```

c.

```
casas %>%
  mutate_if(is.character, ~tidyr::replace_na(.x, replace = "Não possui"))
```

2. Utilizando a base `casas`, resolva os itens a seguir.

- **a.** Usando o `case_when()` crie um código para categorizar a variável venda_valor da seguinte maneira:
  - **barata**: $0 a $129.500
  - **preço mediano**: $129.500 a $180.796
  - **cara**: $ 180.796 a $213.500
  - **muito cara**: maior que $213.500

- **b.** Utilize o código feito na letra (a) para agrupar a base `casas` pela variável venda_valor categorizada e calcular todas as áreas médias para cada uma dessas categorias.

3. Escreva um código que receba a base `casas` e retorne uma tabela com apenas

- **a.** as colunas referentes à garagem da casa.
- **b.** as colunas referentes a variáveis de qualidade.
- **c.** colunas numéricas que representam áreas da casa e do terreno.
- **d.** colunas numéricas.
- **e.** colunas referentes à piscina, porão e o valor de venda.

4. Usando a função `rename_with()`, troque todos os `"_"` dos nomes das colunas por um espaço `" "`.

5. Escreva um código para colocar todas as colunas relativas a venda no começo da base `casas`.

6. 5. Escreva um código para colocar todas as colunas numéricas da base `casas` no começo da tabela e todas as colunas categóricas no final.

7.3 O pacote tidyr

Dentro do `tidyverse`, uma base *tidy* é uma base fácil de se trabalhar, isto é, fácil de se fazer manipulação de dados, fácil de se criar visualizações, fácil de se ajustar modelos e por aí vai.

Na prática, uma base *tidy* é aquela que se encaixa bem no *framework* do `tidyverse`, pois os pacotes como o `dplyr` e o `ggplot2` foram desenvolvidos para funcionar bem com bases *tidy*. E assim como esses pacotes motivaram o uso de bases *tidy*, o conceito *tidy* motiva o surgimento de novos *frameworks*, como o `tidymodels` para modelagem.

As duas propriedades mais importantes de uma base *tidy* são:

- cada coluna é uma variável;
- cada linha é uma observação.

Essa definição proporciona uma maneira consistente de se referir a variáveis (nomes de colunas) e observações (índices das linhas).

O pacote `{tidyr}` possui funções que nos ajudam a deixar uma base bagunçada em uma base *tidy*. Ou então, nos ajudam a bagunçar um pouquinho a nossa base quando isso nos ajudar a produzir o resultados que queremos.

Vamos ver aqui algumas de suas principais funções:

- `separate()` e `unite()`: para separar variáveis concatenadas em uma única coluna ou uni-las.
- `pivot_wider()` e `pivot_longer()`: para pirvotar a base.
- `nest()` e `unnest()`: para criar *list columns*.

Como motivação para utilizar esssas funções, vamos utilizar a nossa boa e velha base `imdb`. Essa base pode ser baixada clicando aqui.

```
library(dplyr)
library(tidyr)
library(ggplot2)

imdb <- readr::read_rds("imdb.rds")
```

7.3.1 `separate()` e `unite()`

A função `separate()` separa duas ou mais variáveis que estão concatenadas em uma mesma coluna. A sintaxe da função está apresentada abaixo.

```
dados %>%
  separate(
    col = coluna_velha,
    into = c("colunas", "novas"),
    sep = "separador"
  )
```

Como exemplo, vamos transformar a coluna `generos` da base IMDB em três colunas, cada uma com um dos gêneros do filme. Lembrando que os valores da coluna `generos` estão no seguinte formato:

```
imdb %>% pull(generos) %>% head()
```

```
## [1] "Action|Adventure|Fantasy|Sci-Fi"
## [2] "Action|Adventure|Fantasy"
## [3] "Action|Thriller"
## [4] "Action|Adventure|Sci-Fi"
## [5] "Action|Adventure|Romance"
## [6] "Adventure|Animation|Comedy|Family|Fantasy|Musical|Romance"
```

Veja que agora, temos 3 colunas de gênero. Filmes com menos de 3 gêneros recebem `NA` na coluna `genero2` e/ou `genero3`. Os gêneros sobressalentes são descartados, assim como a coluna `generos` original.

```
imdb %>%
  separate(
    col = generos,
    into = c("genero1", "genero2", "genero3"),
    sep = "\\|"
  )
```

```
## Warning: Expected 3 pieces. Additional pieces discarded in 1052 rows [1, 6, 12,
## 13, 16, 18, 23, 24, 25, 28, 32, 34, 36, 39, 40, 43, 47, 48, 49, 50, ...].
```

```
## Warning: Expected 3 pieces. Missing pieces filled with `NA` in 1537 rows [3, 19,
## 21, 42, 84, 88, 92, 102, 106, 111, 113, 129, 138, 147, 178, 183, 186, 219, 222,
## 233, ...].
```

```
## # A tibble: 3,807 x 17
##    titulo             ano diretor    duracao cor   genero1 genero2 genero3 pais
##    <chr>            <int> <chr>        <int> <chr> <chr>   <chr>   <chr>   <chr>
##  1 Avatar            2009 James Cam~     178 Color Action  Advent~ Fantasy USA
##  2 Pirates of the ~  2007 Gore Verb~     169 Color Action  Advent~ Fantasy USA
##  3 The Dark Knight~  2012 Christoph~     164 Color Action  Thrill~ <NA>    USA
##  4 John Carter       2012 Andrew St~     132 Color Action  Advent~ Sci-Fi  USA
##  5 Spider-Man 3      2007 Sam Raimi      156 Color Action  Advent~ Romance USA
##  6 Tangled           2010 Nathan Gr~     100 Color Advent~ Animat~ Comedy  USA
##  7 Avengers: Age o~  2015 Joss Whed~     141 Color Action  Advent~ Sci-Fi  USA
##  8 Batman v Superm~  2016 Zack Snyd~     183 Color Action  Advent~ Sci-Fi  USA
##  9 Superman Return~  2006 Bryan Sin~     169 Color Action  Advent~ Sci-Fi  USA
## 10 Pirates of the ~  2006 Gore Verb~     151 Color Action  Advent~ Fantasy USA
## # ... with 3,797 more rows, and 8 more variables: classificacao <chr>,
## #   orcamento <int>, receita <int>, nota_imdb <dbl>, likes_facebook <int>,
## #   ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

A função `unite()` realiza a operação inversa da função `separate()`. Ela concatena os valores de várias variáveis em uma única coluna. A sintaxe é a seguinte:

```
dados %>%
  unite(
    col = coluna_nova,
    colunas_para_juntar,
    sep = "separador"
  )
```

Como exemplo, vamos agora transformar as colunas `ator1`, `ator2` e `ator3` em uma única coluna `atores`. Lembrando que essas colunas estão no formato abaixo.

```
imdb %>% select(starts_with("ator")) %>% head(3)
```

```
## # A tibble: 3 x 3
##   ator_1      ator_2           ator_3
##   <chr>       <chr>            <chr>
## 1 CCH Pounder Joel David Moore Wes Studi
## 2 Johnny Depp Orlando Bloom    Jack Davenport
## 3 Tom Hardy   Christian Bale   Joseph Gordon-Levitt
```

Veja que agora a coluna `atores` possui os 3 atores concatenados. Se a ordem das colunas `ator1`, `ator2` e `ator3` nos trazia a informação de protagonismo, essa informação passa a ficar implícita nesse novo formato. As 3 colunas originais são removidas da base resultante.

```
imdb %>%
  unite(
    col = "elenco",
    starts_with("ator"),
    sep = " - "
  ) %>%
  select(elenco)
```

```
## # A tibble: 3,807 x 1
##    elenco
##    <chr>
##  1 CCH Pounder - Joel David Moore - Wes Studi
##  2 Johnny Depp - Orlando Bloom - Jack Davenport
##  3 Tom Hardy - Christian Bale - Joseph Gordon-Levitt
##  4 Daryl Sabara - Samantha Morton - Polly Walker
##  5 J.K. Simmons - James Franco - Kirsten Dunst
##  6 Brad Garrett - Donna Murphy - M.C. Gainey
##  7 Chris Hemsworth - Robert Downey Jr. - Scarlett Johansson
##  8 Henry Cavill - Lauren Cohan - Alan D. Purwin
##  9 Kevin Spacey - Marlon Brando - Frank Langella
## 10 Johnny Depp - Orlando Bloom - Jack Davenport
## # ... with 3,797 more rows
```

7.3.2 Pivotagem

O conceito de pivotagem no *tidyverse* se refere a mudança da estrutura da base, geralmente para alcançar o formato *tidy*.

Geralmente realizamos pivotagem quando nossas linhas não são unidades observacionais ou nossas colunas não são variáveis. Ela é similiar à pivotagem do Excel, mas um pouco mais complexa.

O ato de pivotar resulta em transformar uma base de dados *long* em *wide* e vice-versa.

Uma base no formato *long* possui mais linhas e pode ter menos colunas, enquanto no formato *wide* poussi menos linhas e pode ter mais colunas

Esses formatos são sempre relativos às colunas que estão sendo pivotadas, sendo que uma base *tidy* pode estar tanto no formato *long* quanto *wide*.

Antigamente, utilizávamos as funções `gather()` e `spread()` para fazer as operações de pivotagem.

Agora, no lugar de `gather()`, utilizamos a função `pivot_longer()`. Abaixo, transformamos as colunas `ator1`, `ator2` e `ator3` em duas colunas: `ator_atriz` e `protagonismo`.

```
imdb %>%
  pivot_longer(
    cols = starts_with("ator"),
    names_to = "protagonismo",
    values_to = "ator_atriz"
  ) %>%
  select(titulo, ator_atriz, protagonismo) %>%
  head(6)
```

```
## # A tibble: 6 x 3
##   titulo                                      ator_atriz       protagonismo
##   <chr>                                       <chr>            <chr>
## 1 Avatar                                      CCH Pounder      ator_1
## 2 Avatar                                      Joel David Moore ator_2
## 3 Avatar                                      Wes Studi        ator_3
## 4 Pirates of the Caribbean: At World's End    Johnny Depp      ator_1
## 5 Pirates of the Caribbean: At World's End    Orlando Bloom    ator_2
## 6 Pirates of the Caribbean: At World's End    Jack Davenport   ator_3
```

Se considerarmos que na análise da base IMDB cada observação deve ser um filme, então essa nova base já não mais *tidy*, pois agora cada filme aparece em três linhas diferentes, uma vez para cada um de seus atores.

Nesse sentido, embora possa parecer que a variável `protagonismo` estava implícita na base original, ela não é uma variável de fato. Todos filmes têm um `ator_1`, um `ator_2` e um `ator_3`. Não existe nenhuma informação sobre o filme que podemos tirar da coluna `protagonismo`, pois ela qualifica apenas os atores, não o filme em si.

A função `pivot_wider()` faz a operação inversa da `pivot_longer()`. Sem aplicarmos as duas funções em sequência, voltamos para a base original.

```
imdb %>%
  pivot_longer(
    cols = starts_with("ator"),
    names_to = "ator_protagonismo",
    values_to = "ator_nome"
  ) %>%
  pivot_wider(
    names_from = "ator_protagonismo",
    values_from = "ator_nome"
  ) %>%
  head(4)
```

```
## Warning: Values are not uniquely identified; output will contain list-cols.
```

```
## * Use `values_fn = list` to suppress this warning.
## * Use `values_fn = length` to identify where the duplicates arise
## * Use `values_fn = {summary_fun}` to summarise duplicates
```

```
## # A tibble: 4 x 15
##   titulo      ano diretor  duracao cor   generos   pais  classificacao orcamento
##   <chr>     <int> <chr>      <int> <chr> <chr>     <chr> <chr>             <int>
## 1 Avatar     2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
## 2 Pirates ~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
## 3 The Dark~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
## 4 John Car~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
## # ... with 6 more variables: receita <int>, nota_imdb <dbl>,
## #   likes_facebook <int>, ator_1 <list>, ator_2 <list>, ator_3 <list>
```

A base `imdb` não possui nenhuma variável que faça sentido aplicarmos diretamente a função `pivot_wider()`. Vamos então considerar a seguinte tabela derivada da base `imdb`:

```
tab_romance_terror <- imdb %>%
  filter(ano >= 2010) %>%
  mutate(
    genero = case_when(
      stringr::str_detect(generos, "Romance") ~ "Romance",
      stringr::str_detect(generos, "Horror") ~ "Horror",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(genero)) %>%
  group_by(ano, genero) %>%
  summarise(receita_media = mean(receita, na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'ano'. You can override using the `.groups` argument.
```

Essa tabela possui a receita média dos filmes de romance e terror nos anos de 2010 a 2016.

Para apresentar essa tabela em uma reunião, por exemplo, pode ficar ser mais agradável ter os anos nas colunas e não nas linhas. Para isso, basta utilizarmos a função `pivot_wider()`.

```
# Tabela original
tab_romance_terror
```

```
## # A tibble: 14 x 3
```

```
## # Groups:   ano [7]
##      ano genero  receita_media
##    <int> <chr>           <dbl>
##  1  2010 Horror      30242147.
##  2  2010 Romance     48834552
##  3  2011 Horror      33186210.
##  4  2011 Romance     40780528.
##  5  2012 Horror      36090815.
##  6  2012 Romance     53134506.
##  7  2013 Horror      56829163
##  8  2013 Romance     25590508
##  9  2014 Horror      37324785.
## 10  2014 Romance     51353872.
## 11  2015 Horror      27965711.
## 12  2015 Romance     60166036.
## 13  2016 Horror      57896396.
## 14  2016 Romance    105439985.
```

```
# Tabela com os anos nas colunas
tab_romance_terror %>%
  pivot_wider(
    names_from = ano,
    values_from = receita_media
  )
```

```
## # A tibble: 2 x 8
##   genero     `2010`    `2011`    `2012`   `2013`    `2014`    `2015`     `2016`
##   <chr>       <dbl>     <dbl>     <dbl>    <dbl>     <dbl>     <dbl>      <dbl>
## 1 Horror  30242147. 33186210. 36090815. 56829163 37324785. 27965711.  57896396.
## 2 Romance 48834552  40780528. 53134506. 25590508 51353872. 60166036. 105439985.
```

Esse é um caso que *bagunçar* um pouquinho a tabela nos trouxe um resultado desejado.

7.3.3 *List columns*

Um terceiro conceito de dados *tidy* é que cada célula da tabela possui um valor. No entanto, quando estamos programando, muitas vezes vale apena abandonar essa definição e guardarmos objetos mais complexos nas células de uma tabela.

Utilizando as chamadas *list columns* podemos guardar virtualmente qualquer objeto em nossas *tibbles*, como gráficos, resultados de modelos ou até mesmo outras tabelas.

Uma forma de trabalhar com *list columns* consiste em utilizarmos as funções

- `nest()`: para criar uma *list column*;
- `unnest()`: para desfazer uma *list column*.

A forma mais simples de utilizarmos uma *list column* é *aninhar* a nossa base com relação a uma variável.

```
imdb_nest <- imdb %>%
  group_by(ano) %>%
  nest() %>%
  arrange(ano)

head(imdb_nest, 8)
```

```
## # A tibble: 8 x 2
## # Groups:   ano [8]
##     ano data
##   <int> <list>
## 1  1916 <tibble [1 x 14]>
## 2  1920 <tibble [1 x 14]>
## 3  1925 <tibble [1 x 14]>
## 4  1929 <tibble [1 x 14]>
## 5  1930 <tibble [1 x 14]>
## 6  1932 <tibble [1 x 14]>
## 7  1933 <tibble [2 x 14]>
## 8  1934 <tibble [1 x 14]>
```

A base `imdb_nest` possui duas colunas `ano` e `data` e uma linha para cada ano. Na coluna `data`, temos o restante da base `imdb`, recortada para cada um dos anos.

Abaixo, acessmos os dados do único filme de 1916 (primeira linha da base `imdb_nest`).

```
imdb_nest$data[[1]]
```

```
## # A tibble: 1 x 14
##   titulo    diretor  duracao cor   generos pais  classificacao orcamento receita
##   <chr>     <chr>      <int> <chr> <chr>   <chr> <chr>             <int>   <int>
## 1 Intolera~ D.W. Gr~     123 Blac~ Drama|~ USA   Outros           385907      NA
## # ... with 5 more variables: nota_imdb <dbl>, likes_facebook <int>,
## #   ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Imagine que queiramos fazer, para cada ano, um gráfico de dispersão da receita contra o orçamento dos filmes lançados no ano.

Com a base no formato de *list columns*, basta criarmos uma função para gerar o gráfico e utilizarmos a função `purrr::map()`.

Abaixo, construímos a função `fazer_grafico_dispersao()`, que será aplicada a cada uma das bases contidas na coluna `data` da base `imdb_nest`. Os gráficos, respectivamos a cada ano, são salvos na coluna `grafico`.

```
fazer_grafico_dispersao <- function(tab) {
  tab %>%
    ggplot(aes(x = orcamento, y = receita)) +
    geom_point()
}

imdb_graficos <- imdb_nest %>%
  mutate(
    grafico = purrr::map(data, fazer_grafico_dispersao)
  )

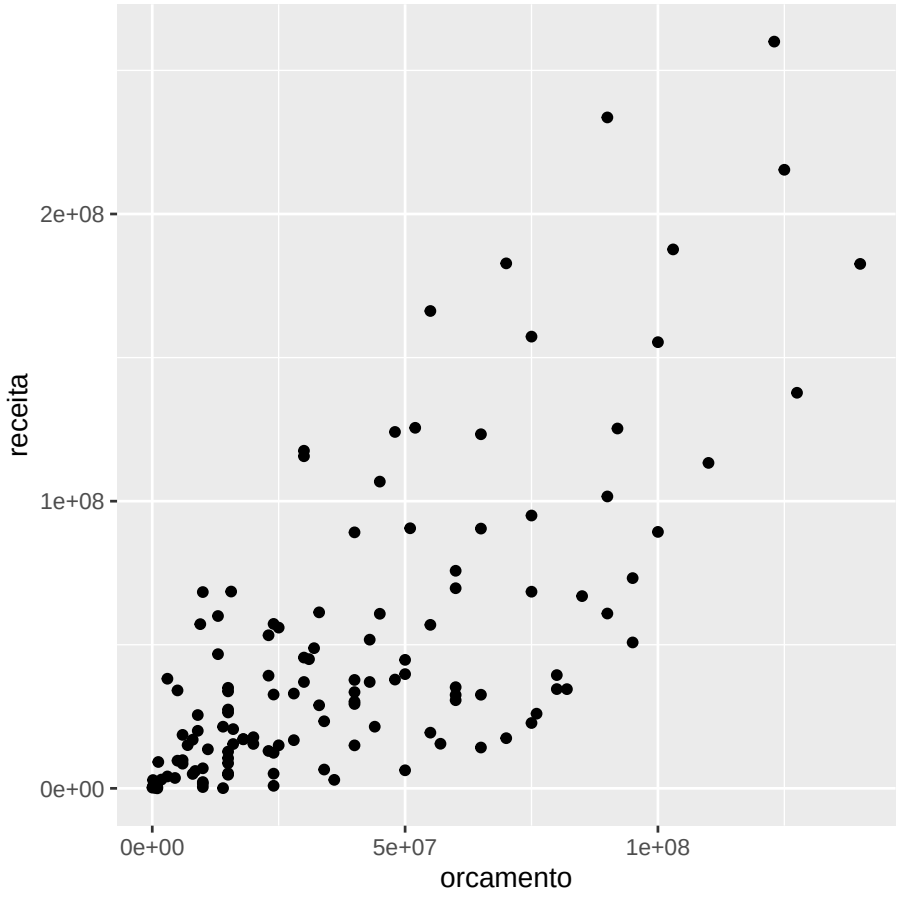
head(imdb_graficos, 6)
```

```
## # A tibble: 6 x 3
## # Groups:   ano [6]
##     ano data              grafico
##   <int> <list>            <list>
## 1  1916 <tibble [1 x 14]> <gg>
## 2  1920 <tibble [1 x 14]> <gg>
## 3  1925 <tibble [1 x 14]> <gg>
## 4  1929 <tibble [1 x 14]> <gg>
## 5  1930 <tibble [1 x 14]> <gg>
## 6  1932 <tibble [1 x 14]> <gg>
```

Para acessar cada um dos gráficos, basta rodar o código abaixo.

```
# Pegando o gráfico referente ao ano de 2000
imdb_graficos$grafico[[74]]
```

```
## Warning: Removed 5 rows containing missing values (geom_point).
```

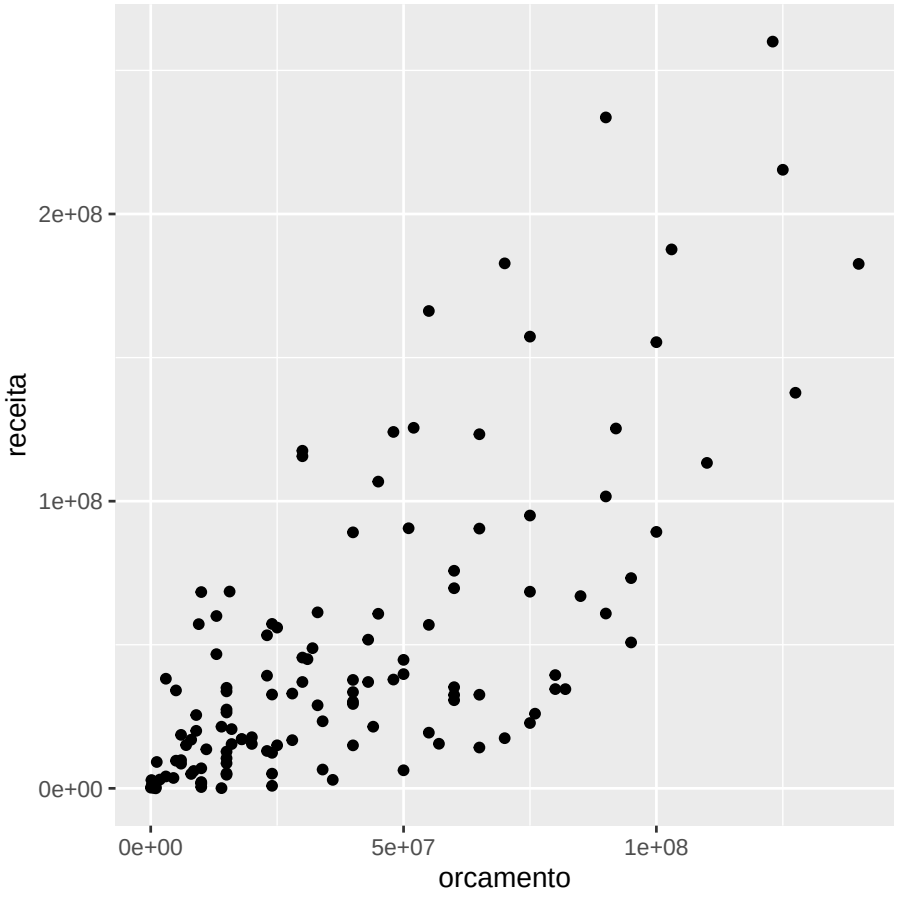


Ou, escolhendo diretamente pelo ano

```
imdb_graficos %>%
  filter(ano == 2000) %>%
  pull(grafico)
```

```
## [[1]]
```

```
## Warning: Removed 5 rows containing missing values (geom_point).
```



A função `unnest()` remove a estrutura de *list column*. Fazendo a operação abaixo, voltamos para a base `imdb` original.

```
imdb_nest %>%
  unnest(cols = "data")
```

```
## # A tibble: 3,807 x 15
## # Groups:   ano [91]
##      ano titulo    diretor  duracao cor   generos  pais  classificacao orcamento
##    <int> <chr>     <chr>      <int> <chr> <chr>    <chr> <chr>             <int>
##  1  1916 Intolera~ D.W. Gr~     123 Blac~ Drama|H~ USA   Outros           385907
##  2  1920 Over the~ Harry F~     110 Blac~ Crime|D~ USA   Outros           100000
##  3  1925 The Big ~ King Vi~     151 Blac~ Drama|R~ USA   Outros           245000
##  4  1929 The Broa~ Harry B~     100 Blac~ Musical~ USA   Outros           379000
##  5  1930 Hell's A~ Howard ~      96 Blac~ Drama|W~ USA   Outros          3950000
```

```
##  6  1932 A Farewe~ Frank B~      79 Blac~ Drama|R~ USA   Outros          800000
##  7  1933 42nd Str~ Lloyd B~      89 Blac~ Comedy|~ USA   Outros          439000
##  8  1933 She Done~ Lowell ~      66 Blac~ Comedy|~ USA   Outros          200000
##  9  1934 It Happe~ Frank C~      65 Blac~ Comedy|~ USA   Outros          325000
## 10  1935 Top Hat   Mark Sa~      81 Blac~ Comedy|~ USA   Outros          609000
## # ... with 3,797 more rows, and 6 more variables: receita <int>,
## #   nota_imdb <dbl>, likes_facebook <int>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

7.3.4 Exercícios

1. Crie 5 novas colunas de gêneros na base imdb, cada uma com um dos gêneros contidos na coluna generos. Para os filmes com menos de 5 gêneros, substitua os valores `NA` pela string "inexistente".

2. Substitua os "????" no código abaixo para criar uma tabela do lucro médio dos filmes ao longo dos anos de 2000 a 2016, com cada ano sendo uma coluna da base.

```
imdb %>%
  mutate(lucro = receita - orcamento) %>%
  filter(ano %in% "????") %>%
  group_by("????") %>%
  summarise(lucro_medio = "????") %>%
  pivot_wider(names_from = "????", values_from = "????")
```

Para os exercícios 3, 4 e 5, vamos utilize a base `pokemon`, disponível no pacote `basesCursoR`.

```
install.packages("remotes")
remotes::install_github("curso-r/basesCursoR")
```

3. Utilize a função `unite()` para juntar as duas colunas de tipo em uma única coluna na base pokemon.

4. Utilize a função `unite()` para juntar as três colunas de cor em uma única coluna na base pokemon. Faça isso sem remover as 3 colunas originais.

5. Utilizando a base `pokemon`, resolva os itens abaixo:

a. Utilize a função `pivot_longer()` para criar uma única coluna de tipo na base pokemon.

b. Utilize a base criada no item (a) e escreva um código para descobrir qual o tipo mais frequente na base, independentemente se ele é primário (tipo_1) ou secundário (tipo_2).

6. Escreva uma função que receba uma base qualquer e o nome de uma coluna numérica dessa base e retorne uma figura com um gráfico de dispersão da coluna escolhida contra cada uma das outras variáveis numéricas da base.

7.4 O pacote stringr

7.4.1 Introdução

Variáveis de texto são muito comuns nos bancos de dados e, geralmente, dão bastante trabalho para serem manipuladas. É muito comum encontrarmos colunas com categorias não padronizadas, como, por exemplo, uma variável `Estado` com "SP", "sp", "Sao Paulo", "São Paulo" etc, todas indicando o mesmo estado.

O R possui várias funções para manipular textos (ou *strings*). No entanto, as funções do `base` não possuem uma interface consistente e cada uma tem a sua forma de passar os parâmetros, dificultando a programação durante a análise.

Pensando nisso, Hadley Wickham deu aquela força para a comunidade R e criou o pacote `stringr`, que possui uma sintaxe consistente, permitindo o usuário manipular textos com muito mais facilidade.

Vantagens do stringr em relação às funções do R base

- Sintaxe unificada, o que auxilia na memorização das funções e leitura do código.
- Todas as funções são vetorizadas.
- Construído sobre a biblioteca ICU, implementada em `C` e `C++`, uma garantia de resultados mais rápidos.

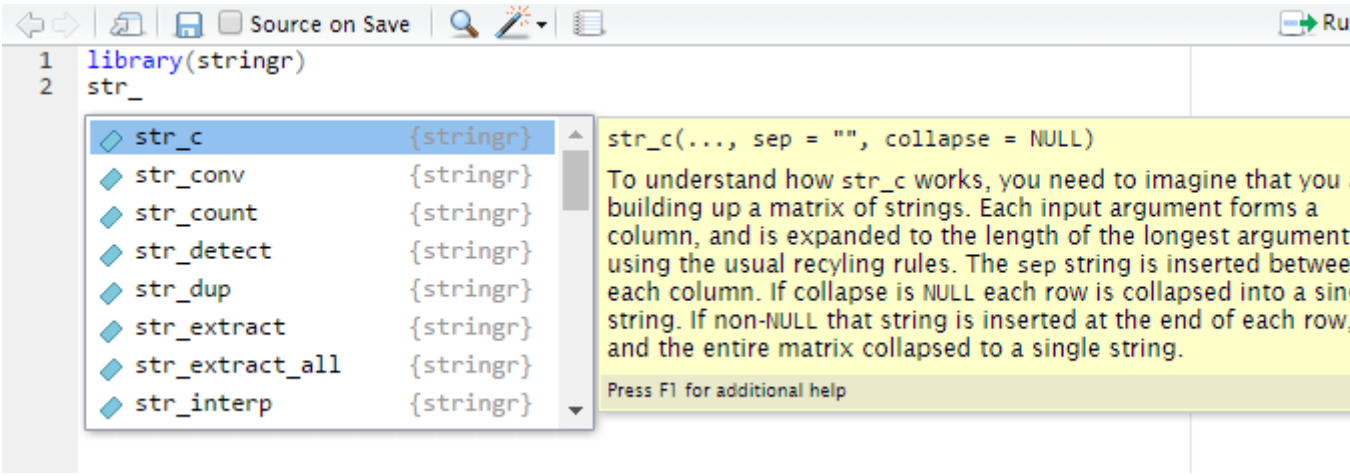
Regras básicas do pacote

- As funções de manipulação de texto começam com `str_`. Caso esqueça o nome de uma função, basta digitar `stringr::str_` e apertar `TAB` para ver quais são as opções.
- O primeiro argumento da função é sempre uma *string* ou um vetor de *strings*.

Curiosidade Inicialmente, o `stringr` era um *wrapper* de funções do `base`. Depois disso, surgiu um novo pacote `stringi`, com sintaxe similar ao `stringr`, mas funcionando como *wrapper* da biblioteca ICU. Wickham gostou tanto do pacote `stringi` que decidiu reescrever o `stringr` como um *wrapper* do `stringi`.

Veja essa página para detalhes.

Começando a usar Todas as funções do `stringr` começam com o prefixo `str_`. Isso ajuda na hora de encontrar a função que você está procurando. No RStudio, digite `str_` e aperte **TAB** para visualizar a lista de funções com esse prefixo. Você pode verificar o que cada função faz até encontrar a que atende às suas necessidades.



Na próxima seção, vamos apresentar as funções mais simples do `{stringr}`. Em seguida, vamos falar um pouco de expressões regulares (regex) e então veremos funções mais avançadas do pacote, que utilizam regex para identificar padrões.

Antes de mais nada, instale e carregue o pacote `stringr`.

```
install.packages("stringr")
```

```
library(stringr)
```

7.4.2 Funções básicas

A seguir, apresentamos as funções mais simples do `{stringr}`, que não utilizam o conceito de expressões regulares.

str_length A função mais simples do `stringr()` é a função `str_length()`. Esta função recebe como argumento um vetor de *strings* e retorna o número de caracteres de cada *string*. Repare que o espaço `" "` é considerado um caracter.

```
str_length("São Paulo")
## [1] 9
str_length(c("São Paulo", "Rio de Janeiro",
             "Rio Grande do Norte", "Acre"))
## [1]  9 14 19  4
```

Note que `str_length()` é diferente de `length()`. O primeiro retorna o número de caracteres e o segundo retorna o comprimento do objeto. Isso fica mais claro no seguinte exemplo:

```
s <- c("São Paulo", "Rio de Janeiro",
       "Rio Grande do Norte", "Acre")

str_length(s)
## [1]  9 14 19  4
length(s)
## [1] 4
```

A função `str_length()` retornou um vetor com o número de caracteres de cada elemento do vetor `s`, enquanto `length()` retornou o comprimento do vetor `s`.

str_to_upper, str_to_lower, str_to_title Essas funções servem para modificar a caixa das letras. Veja alguns exemplos:

```
s <- "Somos a curso-r"
str_to_lower(s)
## [1] "somos a curso-r"
str_to_upper(s)
## [1] "SOMOS A CURSO-R"
str_to_title(s)
## [1] "Somos A Curso-R"
```

str_trim É muito comum encontrar textos que vêm com espaços a mais, principalmente de dados provenientes de formulários em que cada usuário escreve da forma que prefere. Isso é um problema pois cria categorias diferentes para valores que deveriam ser iguais. Espaços antes e após o texto são especialmente chatos, pois pode ser difícil detectá-los.

```
s <- c("M", "F", "F", " M", " F ", "M")
as.factor(s)
## [1] M   F   F    M   F  M
## Levels:  F   M F M
```

A função `str_trim()` ajuda removendo os espaços excedentes antes e depois da string.

```
string_aparada <- str_trim(s)
as.factor(string_aparada)
## [1] M F F M F M
## Levels: F M
```

str_sub Não é raro você precisar obter uma parte fixa de uma *string*, como, por exemplo, manipular textos da forma:

```
s <- c("01-Feminino", "02-Masculino", "03-Indefinido")
```

Você pode querer apenas a parte final da string. Neste caso, pode usar a função `str_sub()`.

```
# pegar do quarto até o último caractere
str_sub(s, start = 4)
## [1] "Feminino"   "Masculino"  "Indefinido"
```

Também é possível obter apenas os números.

```
# pegar apenas os dois primeiros caracteres
str_sub(s, end = 2)
## [1] "01" "02" "03"
```

Em outros casos você precisa obter os últimos 2 caracteres.

```
s <- c("Feminino-01", "Masculino-02", "Indefinido-03")
str_sub(s, end = -4)
## [1] "Feminino"   "Masculino"  "Indefinido"
str_sub(s, start = -2)
## [1] "01" "02" "03"
```

É possível usar os argumentos `start` e `end` conjuntamente.

```
s <- c("__SP__", "__MG__", "__RJ__")
str_sub(s, 3, 4)
## [1] "SP" "MG" "RJ"
```

str_c Análoga à função `paste0()` do R base, concatena *strings* em uma única string.

```
string1 <- "O valor p é: "
string2 <- 0.03

str_c(string1, string2)
## [1] "O valor p é: 0.03"
```

Pode misturar objetos com strings definidas diretamente na função.

```
string1 <- "Brigadeiro"
string2 <- "bom"
string3 <- "melhor"

str_c(string1, " é a prova de que não existe nada tão ", string2,
      " que não pode ficar ", string3, ".")
## [1] "Brigadeiro é a prova de que não existe nada tão bom que não pode ficar melhor.
```

A função é vetorizada.

```
string1 <- c("Brigadeiro", "A política brasileira")
string2 <- c("bom", "ruim")
string3 <- c("melhor", "pior")

str_c(string1, " é a prova de que não existe nada tão ", string2,
      " que não pode ficar ", string3, ".")
## [1] "Brigadeiro é a prova de que não existe nada tão bom que não pode ficar melhor.
## [2] "A política brasileira é a prova de que não existe nada tão ruim que não pode f
```

Essas são as funções mais simples do pacote `stringr` e não exigem nenhum conhecimento de **expressões regulares**. Note que nenhuma delas possui o parâmetro `pattern`. Você verá como especificar esse parâmetro nas próximas seções.

7.4.3 Expressões Regulares

Trabalhar com textos exige um certo conhecimento de expressões regulares. As expressões regulares — ou simplesmente **regex** — permitem identificar

conjuntos de caracteres, palavras e outros padrões por meio de uma sintaxe concisa.

O `stringr` utiliza regex da forma descrita neste documento. A própria definição de regex do R é um ótimo manual.

Vamos estudar expressões regulares por meio de exemplos e da função `str_detect()`. Ela retorna `TRUE` se uma *string* atende a uma expressão regular e `FALSE` caso contrário. Por exemplo:

```
str_detect("sao paulo", pattern = "paulo$")
## [1] TRUE
str_detect("sao paulo sp", pattern = "paulo$")
## [1] FALSE
```

A regex/pattern "paulo$" indica que o texto deve ser terminado em "paulo". Existem diversos caracteres auxiliares que vão auxiliar na manipulação dos textos, assim como o "$".

Importante: o valor passado para o argumento `pattern` de qualquer função do pacote `stringr` será entendido como uma regex.

A tabela abaixo mostra a aplicação de cinco regex em seis *strings* distintas.

- '^ban' reconhece apenas o que começa exatamente com "ban".
- 'b ?an' reconhece tudo que tenha "ban", com ou sem espaço entre o "b" e o "a".
- 'ban' reconhece tudo que tenha "ban", apenas minúsculo.
- BAN' reconhece tudo que tenha "BAN", apenas maiúsculo.
- 'ban$' reconhece apenas o que termina exatamente em "ban"

strings	^ban	b ?an	ban	BAN	ban$
abandonado	FALSE	TRUE	TRUE	FALSE	FALSE
ban	TRUE	TRUE	TRUE	FALSE	TRUE
banana	TRUE	TRUE	TRUE	FALSE	FALSE
BANANA	FALSE	FALSE	FALSE	TRUE	FALSE
ele levou ban	FALSE	TRUE	TRUE	FALSE	TRUE
pranab anderson	FALSE	TRUE	FALSE	FALSE	FALSE

Quantificadores Os caracteres `+`, `*` e `{x,y}` indicam quantas vezes um padrão se repete:

- `ey+` significa `e` e depois `y` "**uma vez** ou mais". Por exemplo, reconhece `hey`, `heyy`, `a eyyy`, mas não reconhece `e`, `y` nem `yy`.
- `ey*` significa "**nenhuma vez** ou mais". Por exemplo, reconhece `hey`, `heyy`, `a eyyy` e `e`, mas não reconhece `y` nem `yy`.
- `ey{3}` significa "exatamente três vezes". Por exemplo, reconhece `eyyy` e `eyyyy`, mas não reconhece `eyy`.
- `ey{1,3}` significa "entre uma e três vezes".

Para aplicar um quantificador a um conjunto de caracteres, use parênteses. Por exemplo, `(ey )+` reconhece `ey ey`.

Conjuntos Colocando caracteres dentro de `[]`, reconhecemos quaisquer caracteres desse conjunto. Alguns exemplos práticos:

- `[Cc]asa` para reconhecer "casa" em maiúsculo ou minúsculo.
- `[0-9]` para reconhecer somente números. O mesmo vale para letras `[a-z]`, `[A-Z]`, `[a-zA-Z]` etc.
- O símbolo `^` dentro do colchete significa negação. Por exemplo, `[^0-9]` significa pegar tudo o que não é número.
- O símbolo `.` fora do colchete indica "qualquer caractere", mas dentro do colchete é apenas ponto.
- Use `[[:space:]]+` para reconhecer espaços e `[[:punct:]]+` para reconhecer pontuações.

Miscelânea

- Use `abjutils::rm_accent()` para retirar os acentos de um texto.
- Use `|` para opções. Por exemplo, `desfavor|desprov` reconhece tanto "desfavorável" quanto "desprovido".
- O código `\n` pula linha, `\f` é final da página, `\t` é tab. Use `\` para transformar caracteres especiais em literais.

A lista de possibilidades com expressões regulares é extensa. Um bom lugar para testar o funcionamento das regex é o regex101.

7.4.4 Funções que utilizam regex

Agora que já vimos as funções básicas do `stringr` e aprendemos um pouco de regex, vamos às funções mais avançadas. Basicamente, essas funções buscarão `patterns` em um vetor de *strings* e farão alguma coisa quando encontrá-lo.

Como já vimos na sessão sobre regex, a função mais simples que possui o argumento `pattern=` é a `str_detect()`.

str_detect() Retorna `TRUE` se a regex é compatível com a string e `FALSE` caso contrário.

```
library(stringr)
str_detect("sao paulo", pattern = "paulo$")
## [1] TRUE
str_detect("sao paulo sp", pattern = "paulo$")
## [1] FALSE
```

str_replace() e str_replace_all() Substituem um padrão (ou todos) encontrado para um outro padrão.

Substituindo apenas a primeira ocorrência.

```
cidades <- c("S. José do Rio Preto", "São Paulo", "S. José dos Campos", "São Roque", "S. S. da Gr

str_replace(cidades, "S[.]", "São")
## [1] "São José do Rio Preto" "São Paulo"            "São José dos Campos"
## [4] "São Roque"             "São S. da Grama"
```

Veja que no exemplo anterior precisamos colocar o `.` entre colchetes. Se não tivéssemos colocado, ele seria interpretado como uma regex e o padrão procurado seria *"S" seguido de qualquer caracter*:

```
cidades <- c("S. José do Rio Preto", "São Paulo", "S. José dos Campos", "São Roque", "S. S. da Gr

str_replace(cidades, "S.", "São")
## [1] "São José do Rio Preto" "Sãoo Paulo"            "São José dos Campos"
## [4] "Sãoo Roque"            "São S. da Grama"
```

Nesses casos, podemos usar a função `fixed()` para indicar que o padrão procurado não é uma regex:

```
cidades <- c("S. José do Rio Preto", "São Paulo", "S. José dos Campos", "São Roque", "S

str_replace(cidades, fixed("S."), "São")
## [1] "São José do Rio Preto" "São Paulo"             "São José dos Campos"
## [4] "São Roque"             "São S. da Grama"
```

A função `str_replace()` substitui apenas a primeira ocorrência encontrada. No exemplo anterior, apenas o primeiro "S." era substituído por "São". Se quisermos substituir todas as ocorrências de um padrão dentro de uma string, utilizamos a função `str_replace_all()`.

No exemplo abaixo, substituímos todas as ocorrências de `.` e `-` por um espaço.

```
cpf <- c("303.030.111-33", "102-177-011-20", "987.220.199.00")

str_replace_all(cpf, "[.-]", " ")
## [1] "303 030 111 33" "102 177 011 20" "987 220 199 00"
```

str_extract() e str_extract_all() As funções `str_extract()` e `str_extract_all()` extraem padrões de uma *string*. No exemplo abaixo, pegamos apenas os sobrenomes de cada integrante do grupo.

```
r_core_group <- c(
  'Douglas Bates', 'John Chambers', 'Peter Dalgaard',
  'Robert Gentleman', 'Kurt Hornik', 'Ross Ihaka', 'Tomas Kalibera',
  'Michael Lawrence', 'Friedrich Leisch', 'Uwe Ligges', '...'
)

sobrenomes <- str_extract(r_core_group, '[:alpha:]+$')
sobrenomes
##  [1] "Bates"     "Chambers"  "Dalgaard"  "Gentleman" "Hornik"    "Ihaka"
##  [7] "Kalibera"  "Lawrence"  "Leisch"    "Ligges"    NA
```

str_split() e str_split_fixed() Essas funções separam uma *string* em várias de acordo com um separador.

```
texto <- 'Durante um longo período de tempo o "R" foi escrito "P" como no alfabeto ciri

str_split(texto, fixed('.'))
## [[1]]
## [1] "Durante um longo período de tempo o \"R\" foi escrito \"P\" como no alfabeto ci
## [2] " O seu nome no alfabeto fenício era \"rech\""
## [3] " Seu significado era o de uma cabeça, representada pela adaptação do hieróglif
```

```
## [4] " Transformou-se no \"rô\" dos gregos"
## [5] " Os romanos modificaram o rô acrescentando um pequeno traço para diferenciá-lo do no noss
## [6] ""
```

O `str_split_fixed` faz o mesmo que `str_split()`, mas separa apenas `n` vezes.

```
str_split_fixed(texto, fixed('.'), 3)
##      [,1]
## [1,] "Durante um longo período de tempo o \"R\" foi escrito \"P\" como no alfabeto cirílico"
##      [,2]
## [1,] " O seu nome no alfabeto fenício era \"rech\""
##      [,3]
## [1,] " Seu significado era o de uma cabeça, representada pela adaptação do hieróglifo egípcio
```

str_subset() A função `str_subset()` retorna somente as strings compatíveis com a regex.

```
frases <- c('a roupa do rei', 'de roma', 'o rato roeu')
str_subset(frases, 'd[eo]')
## [1] "a roupa do rei" "de roma"
```

É o mesmo que fazer subset usando a função `str_detect`.

```
frases[str_detect(frases, "d[eo]")]
## [1] "a roupa do rei" "de roma"
```

7.4.5 Exercícios

1. O CPF é um número de 11 dígitos, por exemplo: 54491651884. No entanto para facilitar a visualização costumamos mostrá-lo com separadores a cada 3 casas: 544.916.518-84. Crie uma função que transforma um número de 11 dígitos em uma string com as separações, como um CPF.

2. Transforme o vetor de *strings* abaixo em `"01 - Alto" "02 - Médio" "03 - Baixo"`.

```
s <- c('Alto', 'Médio', 'Baixo')
```

3. Crie uma regex que capture múltiplas versões da palavra 'casa'. Ela deve funcionar com as palavras 'Casa', 'CASA', 'CaSa', 'CAsa'. Teste-a usando a função `str_detect()`.

```
s <- c('Casa', 'CASA', 'CaSa', 'CAsa')
```

4. Imagine que a seguinte *string* é a parte final de uma URL.

- `/ac/rio-branco/xpto-xyz-1-0-1fds2396-5`

Transforme-a em "AC - Rio Branco" utilizando funções do pacote `{stringr}`.

```
url <- c('/ac/rio-branco/xpto-xyz-1-0-1fds2396-5')
```

5. Crie uma função que retorna `TRUE` quando a string é um palíndromo e `FALSO` caso não seja.

6. De acordo com as regras da língua portuguesa, antes de "p" ou "b" devemos usar a letra "m". Em outras palavras, com outras consoantes, usamos a letra "N". Suponha que você tem o seguinte texto com erros gramaticais:

```
texto <- 'Nós chamamos os bonbeiros quando começou o incêmdio.'
```

Crie uma função para corrigi-lo.

7. Considere o seguinte texto

```
txt <- "A função mais importante para leitura de dados no `lubridate` é a `ymd`. Essa
```

Extraia todas as combinações da função `ymd`, sem repetições.

8. Considere as frases abaixo

```
s <- c(
  'O produto é muito bom.',
  'O produto não é bom.',
  'O produto não é muito bom.',
  'O produto não é ruim.',
  'O produto não é não bom.'
)
```

Crie uma regra para identificar se o texto refere-se a um feedback positivo ou negativo sobre o produto (considere "não bom = ruim" e "não ruim = bom"). Retorne um vetor lógico que vale `TRUE` se o feedback é positivo e `FALSE` caso contrário.

7.5 O pacote lubridate

Trabalhar com datas no R era uma chateação. As funções do R base são, em grande parte, contraintuitivas e podem mudar de acordo com o tipo do objeto que você está usando (data, hora, data/hora). Além disso, características como fusos horários, anos bissextos, horários de verão, entre outras, podem não estar bem especificadas ou nem mesmo sendo levadas em conta.

O pacote `{lubridate}`, criado por Garrett Grolemund e Hadley Wickham, surgiu para lidar com esses problemas, fazendo o trabalho com datas ser muito mais fácil.

Antes de começar a usar, você precisa instalar e carregar o pacote.

```
install.packages("lubridate")
library(lubridate)
```

Neste seção, falaremos sobre:

- transformar e extrair datas;
- algumas funções úteis para trabalhar com datas;
- trabalhar com fusos horários; e
- operações aritméticas com datas.

7.5.1 A classe `date`

Datas no R são tratadas como um tipo especial de objeto, com classe `date`. Há várias formas de criar objetos dessa classe com o pacote `{lubridate}`:

```
data_string <- "21-10-2015"
class(data_string)
## [1] "character"

data_date <- date(data_string)
class(data_date)
## [1] "Date"
data_date
## [1] "0021-10-20"

data_as_date <- as_date(data_string)
## Warning: All formats failed to parse. No formats found.
class(data_as_date)
## [1] "Date"
data_as_date
## [1] NA
```

```
data_mdy <- dmy(data_string)
class(data_mdy)
## [1] "Date"
data_mdy
## [1] "2015-10-21"
```

Veja que as funções `date()` e `as_date()` converteram a *string* para data, mas não devolveram o valor esperado. A função `dmy()` resolve esse problema estabelecendo explicitamente a ordem dia-mês-ano. Existem funções para todas as ordens possíveis: `dmy()`, `mdy()`, `myd()`, `ymd()`, `ydm()` etc.

As funções `date()` e `as_date()` assumem que a ordem segue o padrão da língua inglesa: ano-mês-dia (ymd).

```
date("2015-10-21")
## [1] "2015-10-21"
as_date("2015-10-21")
## [1] "2015-10-21"
```

Uma grande facilidade que essas funções trazem é poder criar objetos com classe `date` a partir de números e *strings* em diversos formatos.

```
dmy(21102015)
## [1] "2015-10-21"
dmy("21102015")
## [1] "2015-10-21"
dmy("21/10/2015")
## [1] "2015-10-21"
dmy("21.10.2015")
## [1] "2015-10-21"
```

Se além da data você precisar especificar o horário, basta usar as funções do tipo `ymd_h()`, `ymd_hm()`, `ymd_hms()`, sendo que também há uma função para cada formato de dia-mês-ano.

```
ymd_hms(20151021165411)
## [1] "2015-10-21 16:54:11 UTC"
```

No R base, utilizamos a função `as.Date()` para transformar uma string em data:

```
data_base <- as.Date("2015-10-21")
class(data_base)
## [1] "Date"
```

```
data_base
## [1] "2015-10-21"
```

Repare que a função `date()` do R base retorna a data e horário no momento da execução:

```
base::date()
## [1] "Wed Jul 14 12:09:46 2021"
```

7.5.2 Funções úteis

O `lubridate` traz diversas funções para extrair os componentes de um objeto da classe `date`.

- `second()` - extrai os segundos.
- `minute()` - extrai os minutos.
- `hour()` - extrai a hora.
- `wday()` - extrai o dia da semana.
- `mday()` - extrai o dia do mês.
- `month()` - extrai o mês.
- `year()` - extrai o ano.

Os nomes são tão óbvios que a explicação do que cada função faz é praticamente desnecessária.

```
bday <- ymd_hms("1989-07-29 030142")
bday
## [1] "1989-07-29 03:01:42 UTC"

second(bday)
## [1] 42
day(bday)
## [1] 29
month(bday)
## [1] 7
year(bday)
## [1] 1989

wday(bday)
## [1] 7
wday(bday, label = TRUE)
## [1] Sat
## Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

Você também pode usar essas funções para atribuir esses componentes a uma data:

```
data <- dmy("01/04/1991")
data
## [1] "1991-04-01"

hour(data) <- 20
data
## [1] "1991-04-01 20:00:00 UTC"
```

Também existem funções para extrair a data no instante da execução.

```
# Data e horário do dia em que essa página foi editada pela última vez.
today()
## [1] "2021-07-14"
now()
## [1] "2021-07-14 12:09:46 -03"
```

7.5.3 Fusos horários

Uma característica inerente das datas é o fuso horário. Se você estiver trabalhando com datas de eventos no Brasil e na Escócia, por exemplo, é preciso verificar se os valores seguem o mesmo fuso horário. Além disso, quando a data exata de um evento for relevante, você pode precisar converter horários para outros fusos para comunicar seus resultados em outros países.

Para fazer essas coisas, o `lubridate` fornece as funções `with_tz()` e `force_tz()`. Veja um exemplo de como usá-las.

```
estreia_GoT <- ymd_hms("2017-07-16 22:00:00", tz = "America/Sao_Paulo")
estreia_GoT
## [1] "2017-07-16 22:00:00 -03"

# Devolve qual seria a data em outro fuso

with_tz(estreia_GoT, tzone = "GMT")
## [1] "2017-07-17 01:00:00 GMT"
with_tz(estreia_GoT, tzone = "US/Alaska")
## [1] "2017-07-16 17:00:00 AKDT"

# Altera o fuso sem mudar a hora

force_tz(estreia_GoT, tzone = "GMT")
## [1] "2017-07-16 22:00:00 GMT"
```

7.5.4 Operações com datas

O pacote `lubridate` possui ainda funções para calcular intervalos e fazer operações aritméticas com datas.

Intervalos

Intervalos podem ser salvos em objetos com classe `interval`.

```
inicio <- dmy("01-04-1991")
evento <- dmy("31-10-1993")

sobrev <- interval(inicio, evento)
sobrev
## [1] 1991-04-01 UTC--1993-10-31 UTC
class(sobrev)
## [1] "Interval"
## attr(,"package")
## [1] "lubridate"
```

Você pode verificar se dois intervalos tem intersecção com a função `int_overlaps()`.

```
# Outra forma de definir um intervalo: o operador %--%
intervalo_1 <- dmy("01-02-2003") %--% dmy("02-03-2005")

intervalo_2 <- dmy("04-05-2004") %--% dmy("12-03-2012")

int_overlaps(intervalo_1, intervalo_2)
## [1] TRUE
```

7.5.4.1 Aritmética com datas

Veja alguns exemplos de operações aritméticas que você pode fazer utilizando funções do `lubridate`:

```
# Somando datas

data <- dmy("31/01/2000")

data + ddays(1)
## [1] "2000-02-01"
data + dyears(1)
```

```
## [1] "2001-01-30 06:00:00 UTC"

# A operação abaixo retornaria uma data inválida
# por isso obtemos NA
data + months(1)
## [1] NA

# Criando datas recorrentes

reuniao <- dmy("18-03-2017")
reunioes <- reuniao + weeks(0:10)
reunioes
##  [1] "2017-03-18" "2017-03-25" "2017-04-01" "2017-04-08" "2017-04-15"
##  [6] "2017-04-22" "2017-04-29" "2017-05-06" "2017-05-13" "2017-05-20"
## [11] "2017-05-27"

# Duração de um intervalo

intervalo <-dmy("01-03-2003") %--% dmy("31-03-2003")

intervalo / ddays(1)               # Número de dias
## [1] 30
intervalo / dminutes(1)            # Número de minutos
## [1] 43200

as.period(intervalo)
## [1] "30d 0H 0M 0S"
```

Para mais informações sobre o`lubridate`, visite o vignette do pacote.

7.5.5 Exercícios

Nos exercícios a seguir, vamos utilizar a base `lakers`, que contém estatísticas jogo a jogo do Los Angeles Lakers na temporada 2008-2009.

```
lakers %>% as_tibble()
## # A tibble: 34,624 x 13
##      date opponent game_type time  period etype team  player result points type
##     <int> <chr>    <chr>     <chr>  <int> <chr> <chr> <chr>  <chr>   <int> <chr>
##  1 2.01e7 POR      home      12:00      1 jump~ OFF   ""     ""          0 ""
##  2 2.01e7 POR      home      11:39      1 shot  LAL   "Pau ~ "miss~      0 "hoo~
##  3 2.01e7 POR      home      11:37      1 rebo~ LAL   "Vlad~ ""          0 "off"
##  4 2.01e7 POR      home      11:25      1 shot  LAL   "Dere~ "miss~      0 "lay~
##  5 2.01e7 POR      home      11:23      1 rebo~ LAL   "Pau ~ ""          0 "off"
```

```
##  6 2.01e7 POR      home      11:22      1 shot  LAL   "Pau ~ "made"      2 "hoo~
##  7 2.01e7 POR      home      11:22      1 foul  POR   "Greg~ ""          0 "sho~
##  8 2.01e7 POR      home      11:22      1 free~ LAL   "Pau ~ "made"      1 ""
##  9 2.01e7 POR      home      11:00      1 foul  LAL   "Vlad~ ""          0 "per~
## 10 2.01e7 POR      home      10:53      1 shot  POR   "LaMa~ "made"      2 "jum~
## # ... with 34,614 more rows, and 2 more variables: x <int>, y <int>
```

1. Repare que a coluna `date` no data.frame é um vetor de inteiros. Transforme essa coluna em um vetor de valores com classe `date`.

2. Crie uma coluna que junte as informações de data e tempo de jogo (colunas `date` e `time`) em objetos da classe `date`.

3. Crie as colunas `dia`, `mês` e `ano` com as respectivas informações sobre a data do jogo.

4. Em média, quanto tempo o Lakers demora para arremessar a primeira bola no primeiro período?

Dica: arremessos são representados pela categoria `shot` da coluna `etype`.

5. Em média, quanto tempo demora para sair a primeira cesta de três pontos? Considere toda a base, e cestas de ambos os times.

6. Construa boxplots do tempo entre pontos consecultivos para cada períodos. Considere toda a base de dados e apenas pontos do Lakers.

7. Qual foi o dia e mês do jogo que o Lakers demorou mais tempo para fazer uma cesta? Quanto tempo foi?

7.6 O pacote forcats

Se você utiliza R antes do surgimento do `tidyverse`, provavelmente já escreveu a expressão `stringsAsFactors = F` quando importava dados usando as funções `read.csv` ou `read.table`. É muito frustrante quando uma coluna de `strings` é lida como um fator, pois fatores não podem ser manipulados do mesmo jeito que manipulamos vetores de `strings`.

Felizmente, depois de aprender a usar o `readr` você não terá mais esse problema, pois as funções do pacote não lê colunas de caracteres como fatores por padrão. Mas isso não significa que fatores são inúteis. Eles representam uma forma muito prática de lidar com variáveis categorizadas, tanto para fins de modelagem quanto para fins de visualização.

Grande parte da frustração associada ao uso de fatores no R existe por conta da falta de algumas ferramentas úteis no pacote `base`. Para resolver esse o problema, Hadley Wickham ajudou a comunidade R (de novo) desenvolvendo

o pacote `forcats`(**for cat**egorial variable**s**), que implementa algumas dessas ferramentas.

As principais funções do `forcats` servem para alterar a **ordem** e modificar os **níveis** de um fator. Para exemplificar a utilidade dessas funções, nesta seção vamos utilizá-las em situações corriqueiras.

Se você não tem o pacote `{forcats}`, instalado rode o código abaixo antes de utilizá-lo:

```
install.packages("forcats")
```

Nos exemplos a seguir, utilizarmos os seguintes pacotes:

```
library(forcats)
library(ggplot2)
library(dplyr)
```

7.6.1 O que são fatores?

Como vimos na Seção 3.13.2, fatores são uma classe de objetos no R criada para representar as variáveis categóricas numericamente.

Eles são necessários pois muitas vezes precisamos representar variáveis categóricas como números. Quando estamos fazendo um gráfico, por exemplo, só podemos mapear variáveis numéricas em seus eixos, pois o plano cartesiano é formado por duas retas de números reais.

O que fazemos então quando plotamos uma variável categórica? Nós a transformamos em fatores.

Mas como a manipulação de fatores é diferente da manipulação de números e strings (graças aos famosos *levels*), tarefas que parecem simples, como ordenar as barras de um gráfico de barras, acabam se tornando grandes desafios quando não sabemos lidar com essa classe de valores.

Nos exemplos a seguir, vamos utilizar a base `starwars` (do pacote `{dplyr}`) para aprendermos a fazer as principais operações com fatores utilizando o pacote `{forcats}`.

7.6.2 Modificando níveis de um fator

Vamos trabalhar primeiro com a coluna `sex`, que diz qual é o sexo de cada personagem. As possibilidades são:

```
starwars %>%
  pull(sex) %>%
  unique()
## [1] "male"           "none"           "female"         "hermaphroditic"
## [5] NA
```

Veja que se transformarmos a coluna em fator, esses serão os *levels* da variável, não importa se o sub-conjunto que estivermos observando possua ou não todas as categorias.

```
starwars %>%
  mutate(sex = as.factor(sex)) %>%
  pull(sex) %>%
  head()
## [1] male   none   none   male   female male
## Levels: female hermaphroditic male none
```

Vamos criar um objeto com os 16 primeiros valores da coluna `sex` já transformados em fator.

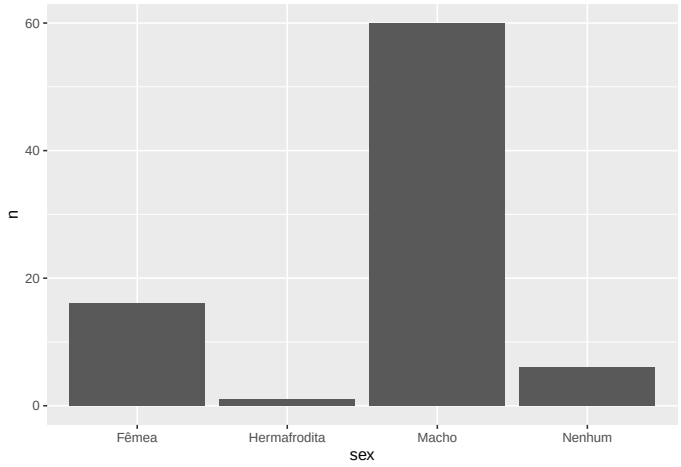
```
fator_sex <- starwars %>%
  pull(sex) %>%
  as.factor() %>%
  head(16)
fator_sex
##  [1] male           none           none           male           female
##  [6] male           female         none           male           male
## [11] male           male           male           male           male
## [16] hermaphroditic
## Levels: female hermaphroditic male none
```

Para mudar os níveis de um fator, podemos utilizar a função `lvls_revalue()`. Veja que, ao mudarmos os níveis de um fator, o *label* de cada valor também muda. Os novos valores precisam ser passados conforme a ordem dos níveis antigos.

```
lvls_revalue(
  fator_sex,
  new_levels = c("Fêmea", "Hermafrodita", "Macho", "Nenhum")
)
##  [1] Macho        Nenhum       Nenhum       Macho        Fêmea
##  [6] Macho        Fêmea        Nenhum       Macho        Macho
## [11] Macho        Macho        Macho        Macho        Macho
## [16] Hermafrodita
## Levels: Fêmea Hermafrodita Macho Nenhum
```

Essa função é uma boa alternativa para mudar o nome das categorias de uma variável antes de construir um gráfico.

```
starwars %>%
  filter(!is.na(sex)) %>%
  count(sex) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum"))
  ) %>%
  ggplot() +
  geom_col(aes(x = sex, y = n))
```



Como as colunas no gráfico respeitam a ordem dos níveis do fator, não importa a ordem que as linhas aparecem na tabela, o gráfico sempre será gerado com as colunas na mesma ordem. Assim, se quiséssemos alterar a ordem das barras do gráfico anterior, precisamos mudar a ordem dos níveis do fator `sex`.

7.6.3 Mudando a ordem dos níveis de um fator

Para mudar a ordem dos níveis de um fator, podemos utilizar a função `lvls_reorder()`. Basta passarmos qual a nova ordem dos fatores, com relação à ordem anterior. No exemplo abaixo definimos que, na nova ordem,

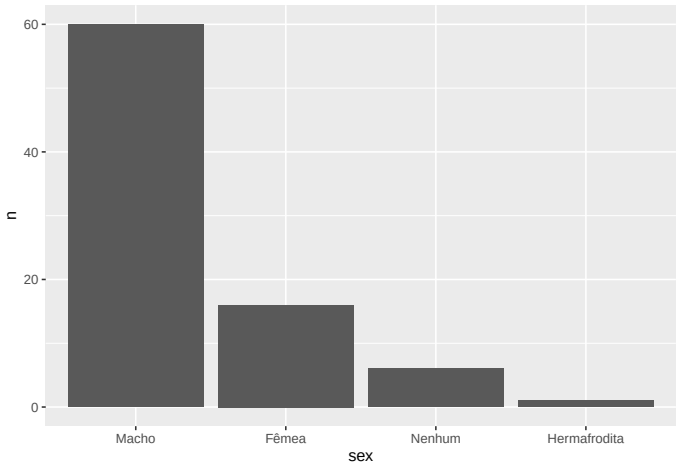
- a primeira posição terá o nível que estava na terceira posição na ordem antiga;
- a segunda posição terá o nível que estava na primeira posição na ordem antiga;

- a terceira posição terá o nível que estava na quarta posição na ordem antiga;
- a quarta posição terá o nível que estava na segunda posição na ordem antiga.

```
lvls_reorder(fator_sex, c(3, 1, 4, 2))
##  [1] male           none           none           male           female
##  [6] male           female         none           male           male
## [11] male           male           male           male           male
## [16] hermaphroditic
## Levels: male female none hermaphroditic
```

Assim, poderíamos usar essa nova ordem para ordenar as colunas do nosso gráfico.

```
starwars %>%
  filter(!is.na(sex)) %>%
  count(sex) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum")),
    sex = lvls_reorder(sex, c(3, 1, 4, 2))
  ) %>%
  ggplot() +
  geom_col(aes(x = sex, y = n))
```



Repare que precisamos passar a nova ordem na mão, o que pode deixar de funcionar se a nossa base mudar (recebemos mais linhas ou fizermos um filtro anteriormente).

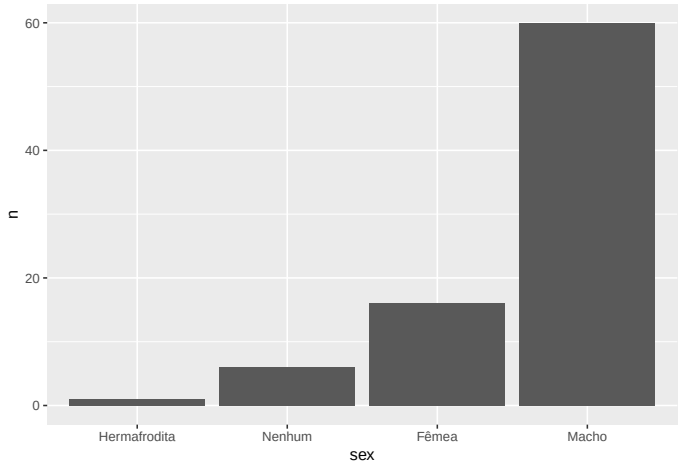
Anterior ao `mutate()` temos a seguinte tabela:

```
starwars %>%
  filter(!is.na(sex)) %>%
  count(sex)
## # A tibble: 4 x 2
##   sex                n
##   <chr>          <int>
## 1 female            16
## 2 hermaphroditic     1
## 3 male              60
## 4 none               6
```

O que queremos é que os níveis da coluna `sex` sejam ordenados segundo os valores da coluna `n`, isto é, quem tiver o maior valor de `n` deve ser o primeiro nível, o segundo maior valor de `n` seja o segundo nível e assim por diante.

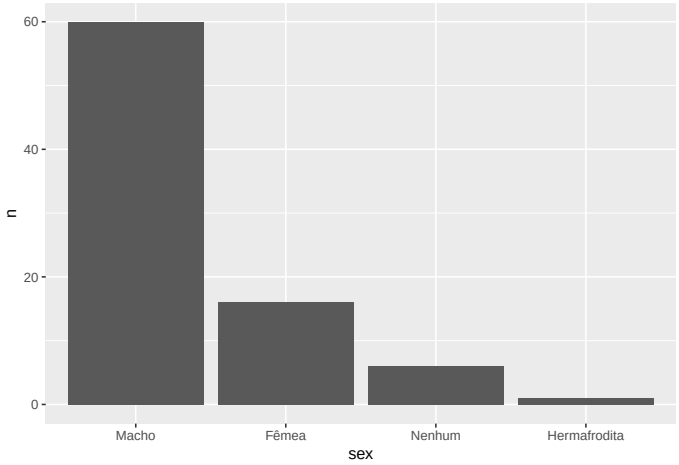
Podemos melhorar esse código utilizando a função `fct_reorder()`. Com ela, em vez de definirmos na mão a ordem dos níveis do fator, podemos ordená-lo segundo valores de uma segunda variável.

```
starwars %>%
  filter(!is.na(sex)) %>%
  count(sex) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum")),
    sex = fct_reorder(sex, n)
  ) %>%
  ggplot() +
  geom_col(aes(x = sex, y = n))
```



É quase o que queríamos! O problema é que os níveis estão sendo ordenados de forma crescente e gostaríamos de ordenar na ordem decrescente. Para isso, basta utilizarmos o parâmetro `.desc`.

```
starwars %>%
  filter(!is.na(sex)) %>%
  count(sex) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum")),
    sex = fct_reorder(sex, n, .desc = TRUE)
  ) %>%
  ggplot() +
  geom_col(aes(x = sex, y = n))
```



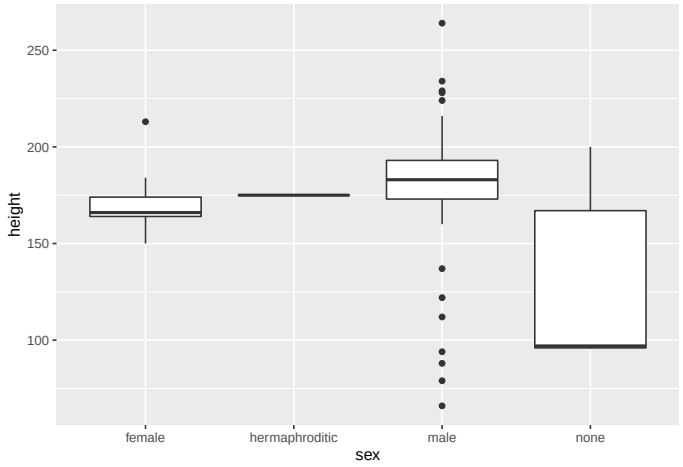
Agora sim! Com esse código, as colunas estarão sendo ordenadas pela frequência, independentemente dos valores de `n` e `sex` que chegarem no `mutate()`.

Se olharmos a documentação da função `fct_reorder()` vamos descobrir que esse exemplo é apenas um caso particular de como podemos utilizá-la. No contexto geral, ela ordena os níveis de um fator segundo **uma função** dos valores de uma segunda variável.

Se em vez de construirmos um gráfico de barras da frequência da variável `sex`, construíssemos boxplots da altura para cada sexo diferente, teríamos o gráfico a seguir.

```
starwars %>%
  filter(!is.na(sex)) %>%
  ggplot() +
```

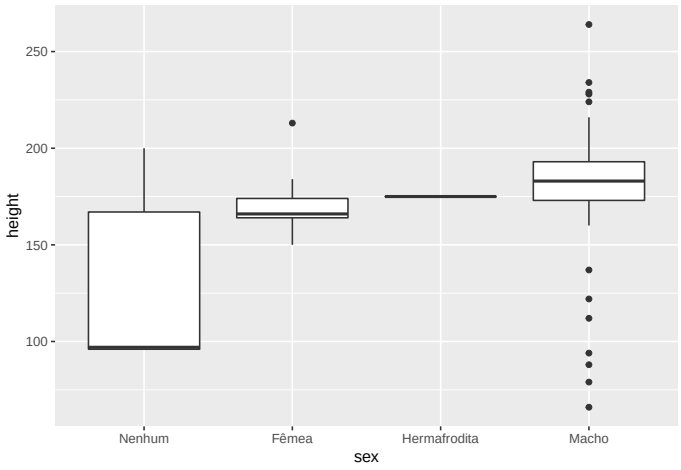
```
  geom_boxplot(aes(x = sex, y = height))
## Warning: Removed 5 rows containing non-finite values (stat_boxplot).
```



Se quiséssemos ordenar cada boxplot (pela mediana, por exemplo), continuamos podendo usar a função `fct_reorder()`. Veja que ela possui o argumento `.fun`, que indica qual função será utilizada na variável secundária para determinar a ordem dos níveis.

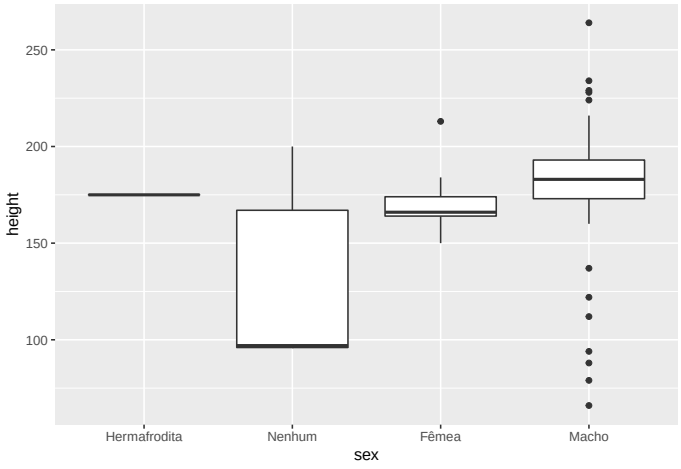
No exemplo abaixo, utilizamos `.fun = median`, o que significa que, para cada nível da variável `sex`, vamos calcular a mediana da variável `height` e ordenaremos os níveis de `sex` conforme a ordem dessas medianas. Assim, o primeiro nível será o sexo com menor altura mediana, o segundo nível será o sexo com a segunda menor algura media e assim por diante. Se quiséssemos ordenar de forma decrescente, bastaria utilizar o argumento `.desc = TRUE`.

```
starwars %>%
  filter(!is.na(sex)) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum")),
    sex = fct_reorder(sex, height, .fun = median, na.rm = TRUE)
  ) %>%
ggplot() +
  geom_boxplot(aes(x = sex, y = height))
## Warning: Removed 5 rows containing non-finite values (stat_boxplot).
```



Também poderíamos ordenar pelo máximo, utilizando `.fun = max`. Neste argumento, podemos usar qualquer função sumarizadora: `min()`, `mean()`, `median()`, `max()`, `sd()`, `var()` etc.

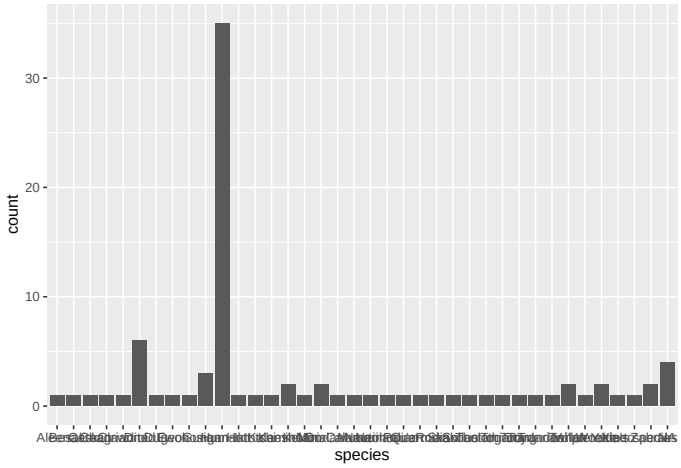
```
starwars %>%
  filter(!is.na(sex)) %>%
  mutate(
    sex = lvls_revalue(sex, c("Fêmea", "Hermafrodita", "Macho", "Nenhum")),
    sex = fct_reorder(sex, height, .fun = max, na.rm = TRUE)
  ) %>%
ggplot() +
  geom_boxplot(aes(x = sex, y = height))
## Warning: Removed 5 rows containing non-finite values (stat_boxplot).
```



7.6.4 Colapsando níveis de um fator

Imagine que quermos fazer um gráfico de barras com a frequência de personagens por espécie.

```
starwars %>%
  ggplot(aes(x = species)) +
  geom_bar()
```



O gráfico resultante é horrível, pois temos muitas espécies diferentes. Uma solução seria agrupar as espécies menos frequentes, criando uma nova categoria (*outras*, por exemplo).

Para isso, podemos usar a função `fct_lump()`. Vamos fazer isso primeiro com o vetor de espécies.

```
fator_especies <- as.factor(starwars$species)
fator_especies
##  [1] Human          Droid          Droid          Human          Human
##  [6] Human          Human          Droid          Human          Human
## [11] Human          Human          Wookiee        Human          Rodian
## [16] Hutt           Human          Human          Yoda's species Human
## [21] Human          Droid          Trandoshan     Human          Human
## [26] Mon Calamari   Human          Human          Ewok           Sullustan
## [31] Human          Neimodian      Human          Gungan         Gungan
## [36] Gungan         <NA>           Toydarian      Dug            <NA>
## [41] Human          Zabrak         Twi'lek        Twi'lek        Vulptereen
## [46] Xexto          Toong          Human          Cerean         Nautolan
## [51] Zabrak         Tholothian     Iktotchi       Quermian       Kel Dor
```

```
## [56] Chagrian       Human          Human          Human          Geonosian
## [61] Mirialan       Mirialan       Human          Human          Human
## [66] Human          Clawdite       Besalisk       Kaminoan       Kaminoan
## [71] Human          Aleena         Droid          Skakoan        Muun
## [76] Togruta        Kaleesh        Wookiee        Human          <NA>
## [81] Pau'an         Human          Human          Human          Droid
## [86] <NA>           Human
## 37 Levels: Aleena Besalisk Cerean Chagrian Clawdite Droid Dug ... Zabrak
```

Temos 37 espécies diferentes na base. Podemos deixar apenas as 3 mais frequentes da seguinte forma:

```
fct_lump(fator_especies, n = 3)
##  [1] Human  Droid  Droid  Human  Human  Human  Human  Droid  Human  Human
## [11] Human  Human  Other  Human  Other  Other  Human  Human  Other  Human
## [21] Human  Droid  Other  Human  Human  Other  Human  Human  Other  Other
## [31] Human  Other  Human  Gungan Gungan Gungan <NA>   Other  Other  <NA>
## [41] Human  Other  Other  Other  Other  Other  Other  Human  Other  Other
## [51] Other  Other  Other  Other  Other  Other  Human  Human  Human  Other
## [61] Other  Other  Human  Human  Human  Human  Other  Other  Other  Other
## [71] Human  Other  Droid  Other  Other  Other  Other  Other  Human  <NA>
## [81] Other  Human  Human  Human  Droid  <NA>   Human
## Levels: Droid Gungan Human Other
```

O fator resultante possui 4 níveis: `Droid`, `Gungan`, `Human` e `Other`. Os 3 primeiros níveis são os mais frequentes, enquanto o nível `Other` foi atribuído a todos os outros 34 níveis que existiam anteiormente.

Poderíamos definir o nome do nível `Others` usando o argumento `other_level`.

```
fct_lump(fator_especies, n = 3, other_level = "Outras espécies")
##  [1] Human           Droid           Droid           Human
##  [5] Human           Human           Human           Droid
##  [9] Human           Human           Human           Human
## [13] Outras espécies Human           Outras espécies Outras espécies
## [17] Human           Human           Outras espécies Human
## [21] Human           Droid           Outras espécies Human
## [25] Human           Outras espécies Human           Human
## [29] Outras espécies Outras espécies Human           Outras espécies
## [33] Human           Gungan          Gungan          Gungan
## [37] <NA>            Outras espécies Outras espécies <NA>
## [41] Human           Outras espécies Outras espécies Outras espécies
## [45] Outras espécies Outras espécies Outras espécies Human
## [49] Outras espécies Outras espécies Outras espécies Outras espécies
## [53] Outras espécies Outras espécies Outras espécies Outras espécies
```

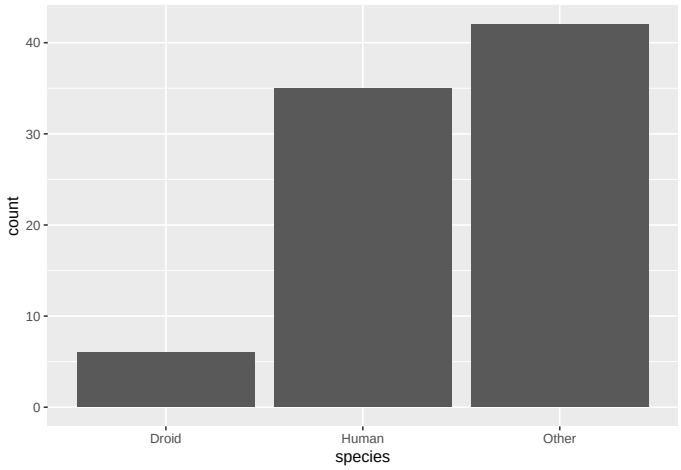
```
## [57] Human           Human           Human           Outras espécies
## [61] Outras espécies Outras espécies Human           Human
## [65] Human           Human           Outras espécies Outras espécies
## [69] Outras espécies Outras espécies Human           Outras espécies
## [73] Droid           Outras espécies Outras espécies Outras espécies
## [77] Outras espécies Outras espécies Human           <NA>
## [81] Outras espécies Human           Human           Human
## [85] Droid           <NA>            Human
## Levels: Droid Gungan Human Outras espécies
```

Também podemos transformar em `Outras` os níveis cuja frequência relativa é menor que um determinado limite, por exemplo, 2%. No exemplo abaixo, apenas espécies que representam mais de 2% dos personagem na base são mantidas. As demais foram transformadas em `Outras`.

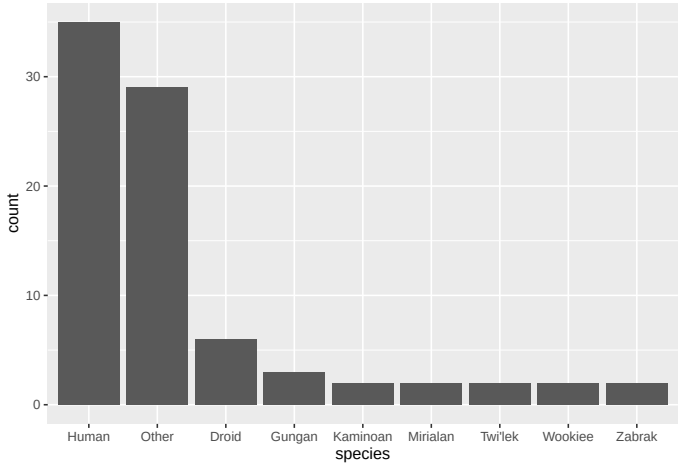
```
fct_lump(fator_especies, p = 0.02, other_level = "Outras")
##  [1] Human    Droid    Droid    Human    Human    Human    Human    Droid
##  [9] Human    Human    Human    Human    Wookiee  Human    Outras   Outras
## [17] Human    Human    Outras   Human    Human    Droid    Outras   Human
## [25] Human    Outras   Human    Human    Outras   Outras   Human    Outras
## [33] Human    Gungan   Gungan   Gungan   <NA>     Outras   Outras   <NA>
## [41] Human    Zabrak   Twi'lek  Twi'lek  Outras   Outras   Outras   Human
## [49] Outras   Outras   Zabrak   Outras   Outras   Outras   Outras   Outras
## [57] Human    Human    Human    Outras   Mirialan Mirialan Human    Human
## [65] Human    Human    Outras   Outras   Kaminoan Kaminoan Human    Outras
## [73] Droid    Outras   Outras   Outras   Outras   Wookiee  Human    <NA>
## [81] Outras   Human    Human    Human    Droid    <NA>     Human
## 9 Levels: Droid Gungan Human Kaminoan Mirialan Twi'lek Wookiee ... Outras
```

Com isso, já conseguimos fazer um gráfico de barras mais agradável.

```
starwars %>%
  filter(!is.na(species)) %>%
  mutate(species = fct_lump(species, n = 2)) %>%
  ggplot(aes(x = species)) +
  geom_bar()
```




```
starwars %>%
  filter(!is.na(species)) %>%
  mutate(
    species = fct_lump(species, p = 0.02),
    species = fct_infreq(species) # Ordena pela frequência de cada nível
  ) %>%
  ggplot(aes(x = species)) +
  geom_bar()
```



Também é possível colapsar níveis criando manualmente os grupos. Para isso, utilizamos a função `fct_collapse()`. No exemplo a seguir, reclassificamos os

níveis da variável `eye_color`. Os níveis não listados são reclassificados como `"outros"`.

```
fator_cor_olhos <- as.factor(starwars$eye_color)

fct_collapse(
  fator_cor_olhos,
  preto = "black",
  castanho = c("brown", "hazel"),
  azul_verde = c("blue", "green"),
  exotico = c("pink", "red", "white"),
  colorido = c("red, blue", "green, yellow"),
  other_level = "outros"
)
## Warning: Unknown levels in `f`: green
##  [1] azul_verde outros     exotico    outros     castanho   azul_verde
##  [7] azul_verde exotico    castanho   outros     azul_verde azul_verde
## [13] azul_verde castanho   preto      outros     castanho   azul_verde
## [19] castanho   outros     castanho   exotico    exotico    castanho
## [25] azul_verde outros     azul_verde castanho   castanho   preto
## [31] azul_verde exotico    azul_verde outros     outros     outros
## [37] azul_verde outros     outros     castanho   castanho   outros
## [43] exotico    castanho   outros     preto      outros     castanho
## [49] outros     preto      castanho   azul_verde outros     outros
## [55] preto      azul_verde castanho   castanho   azul_verde outros
## [61] azul_verde azul_verde castanho   castanho   castanho   castanho
## [67] outros     outros     preto      preto      azul_verde outros
## [73] colorido   outros     outros     preto      colorido   azul_verde
## [79] castanho   exotico    preto      outros     castanho   castanho
## [85] preto      outros     castanho
## Levels: preto azul_verde castanho colorido exotico outros
```

7.6.5 Outras funções úteis

A seguir, listamos outras funções úteis do pacote `{forcats}`, apresentando exemplos simples de como usá-las.

```
fator <- factor(c("a", "b", "a", "c", "c", "a"))
fator
## [1] a b a c c a
## Levels: a b c
```

`fct_c()`

Junta dois fatores (e seus níveis).

```
fator2 <- factor(c("d", "e"))
fct_c(fator, fator2)
## [1] a b a c c a d e
## Levels: a b c d e
```

`fct_count()`

Devolve a frequência dos níveis de um vetor.

```
fct_count(fator)
## # A tibble: 3 x 2
##   f         n
##   <fct> <int>
## 1 a         3
## 2 b         1
## 3 c         2
```

`fct_expand()`

Acrescenta níveis a um fator

```
fct_expand(fator, c("d", "e"))
## [1] a b a c c a
## Levels: a b c d e
```

`fct_rev()`

Inverte os níveis de um fator.

```
fct_rev(fator)
## [1] a b a c c a
## Levels: c b a
```

7.6.6 Exercícios

A base `casas` nos exercícios abaixo está no pacote `dados`:

```
remotes::install_github("cienciadedatos/dados")
```

1. Qual a diferença nos fatores criados com os códigos abaixo?

```
fator1 <- as.factor(c("c", "a", "z", "B"))
fator2 <- forcats::as_factor(c("c", "a", "z", "B"))
```

2. Ordene os níveis do fator `frutas` conforme a sua preferência, isto é, as que você mais gosta primeiro e as que você menos gosta por último.

```
frutas <- as.factor(c("maçã", "banana", "mamão", "laranja", "melancia"))
```

3. Com base no vetor `series`, resolva os itens a seguir.

```
series <- as.factor(c("Game of Thrones", "How I Met your Mother", "Friends", "Lost", "
```

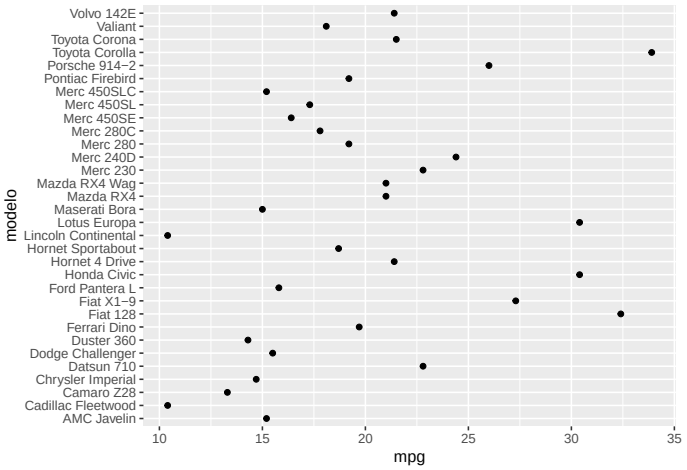
- **a.** Ordene os níveis do vetor `series` conforme a sua preferência, isto é, as que você mais gosta primeiro e as que você menos gosta por último.
- **b.** Junte ao vetor `series` o vetor `novas_series` a seguir, reordenando os níveis para manter a sua ordem de preferência.

```
novas_series <- as.factor(c("The Boys", "Stranger Things", "Queen's Gambit"))
```

- **c.** Renomeie o níveis do vetor criado no item (b) para os nomes em Português das séries. Mantenha o mesmo nome caso não haja tradução.

4. Ordene as categorias do eixo y do gráfico abaixo para que os pontos no eixo x fique em ordem crescente.

```
library(dplyr)
library(ggplot2)
mtcars %>%
  tibble::rownames_to_column("modelo") %>%
  ggplot(aes(x = mpg, y = modelo)) +
  geom_point()
```



5. Utilize a base `dados::casas` para fazer um gráfico de barras mostrando as vizinhanças (coluna `vizinhanca`) com casas mais caras (segundo a coluna `venda_valor`). O gráfico deve conter as 9 vizinhanças mais frequentes e as demais devem ser agrupadas em uma categoria chamada `Outras vizinhanças`.

Chapter 8

Visualização

Agora que já temos em mãos as ferramentas para importar, arrumar, transformar e sumarizar os nossos dados, podemos dar um passo adiante no ciclo da Ciência de Dados: a construção de *visualizações*.

Visualizar os dados é uma etapa importantíssima da análise estatística, pois é a partir dela que criamos boa parte da intuição necessária para escolher o teste ou modelo mais adequado para o nosso problema. Muitas vezes, um problema de análise de dados pode ser resolvido apenas com visualizações. Além disso, elas são o principal combustível da etapa de Comunicação da análise.

Visualizações podem ser uma simples medida resumo (frequência, média, variância, mínimo, máximo etc), um conjunto dessas medidas organizadas em uma tabela ou a representação (de uma parte) dos dados em um gráfico. Neste capítulo, mostraremos como construir gráficos e tabelas bem formatadas dentro do R. Começaremos com a construção de gráficos e, em seguida, falaremos sobre a formatação de tabelas[1].

Mas, antes de mais nada, o que é um gráfico estatístico?

8.1 O pacote ggplot2

A construção de gráficos no R foi revolucionada com a criação do pacote `ggplot2`, fruto da tese de doutorado do Hadley Wickham. Essa revolução teve base na filosofia que Hadley adotou para responder a pergunta "O que é um gráfico estatístico?".

Em 2005, o estatístico norte-americano Leland Wilkinson publicou o livro *The Grammar of graphics* (*A gramática dos gráficos*, em português), uma fonte de

[1] Em geral, boa parte do trabalho necessário para a construção de uma tabela descritiva pode ser feito com as funções dos pacotes `{dplyr}` e `{tidyr}`, vistas no capítulo anterior.

princípios fundamentais para a construção de gráficos estatísticos. No livro, ele defende que um gráfico é o mapeamento dos dados em atributos estéticos (posição, cor, forma, tamanho) de formas geométricas (pontos, linhas, barras, caixas).

A partir dessa definição, Hadley escreveu A Layered Grammar of Graphics (*Uma gramática em camada dos gráficos*), acrescentando que os elementos de um gráfico (dados, sistema de coordenadas, rótulos, anotações, entre outros) são as suas camadas e que a construção de um gráfico se dá pela sobreposição dessas camadas.

Essa é a essência do `ggplot2`: construir um gráfico camada por camada.

Além de uma filosofia bem fundamentada, o `ggplot2` ainda traz outras vantagens em relação aos gráficos do R base:

- gráficos naturalmente mais bonitos;
- fácil personalização (mais simples deixar o gráfico do jeito que você quer);
- a estrutura padronizada das funções deixa o aprendizado muito mais intuitivo;
- a diferença no código entre tipos diferentes de gráficos é muito pequena.

Para discutir os principais aspectos da construção de gráficos com o `ggplot2`, vamos continuar utilizando a base de filmes do IMDB. Você pode baixá-la clicando aqui.

Na próxima seção, vamos conhecer as principais funções do `ggplot2` e começar a construir nossos primeiros gráficos. Não se esqueça de instalar e carregar o pacote antes de rodar os exemplos.

```
install.packages("ggplot2")
library(ggplot2)
```

8.1.1 Gráficos de pontos (dispersão)

No `ggplot2`, os gráficos são construídos camada por camada, sendo a primeira delas dada pela função `ggplot()` (repare que não tem o "2"). Essa função recebe um *data frame* e cria a camada base do gráfico, o nosso *canvas*, onde acrescentaremos todos os outros elementos (camadas).

Se rodarmos apenas a função `ggplot()`, obteremos um painel em branco.

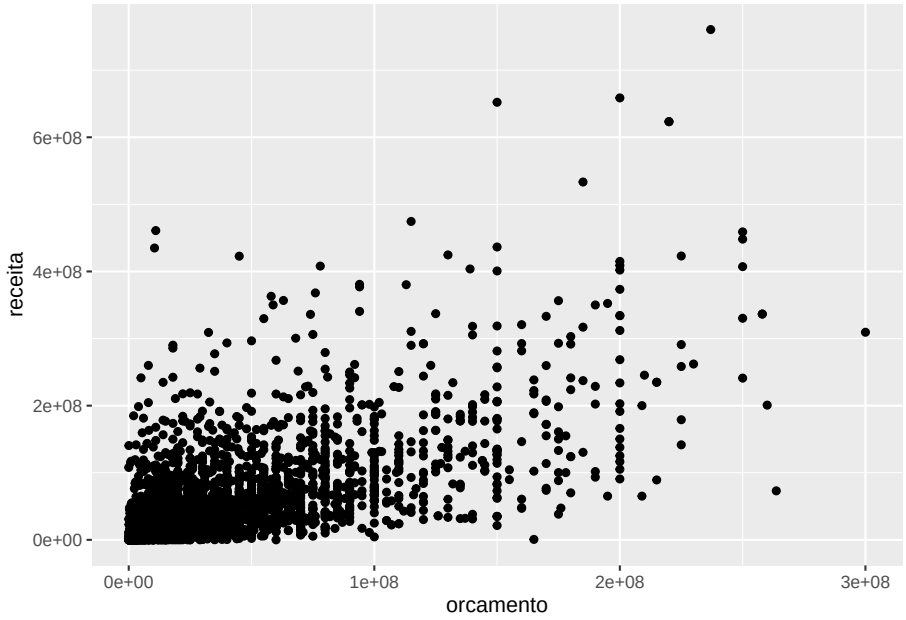
```
imdb <- readr::read_rds("assets/data/imdb.rds")
ggplot(data = imdb)
```

Apesar de termos passado os dados para a função, precisamos especificar como as observações serão mapeadas nos aspectos visuais do gráfico e quais formas geométricas serão utilizadas para isso.

O código abaixo constrói um gráfico de dispersão entre as variáveis orçamento e receita.

```
ggplot(imdb) +
  geom_point(mapping = aes(x = orcamento, y = receita))
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



Observe que:

- a primeira camada é dada pela função `ggplot()` e recebe a nossa base `imdb`;
- a segunda camada é dada pela função `geom_point()`, especificando a forma **geom**étrica utilizada no mapeamento das observações (pontos);
- as camadas são unidas com um `+`;
- o mapeamento na função `geom_point()` recebe a função `aes()`, responsável por descrever como as variáveis serão mapeadas nos aspectos visuais dos pontos (a forma geométrica escolhida);
- neste caso, os aspectos visuais mapeados são a posição do ponto no eixo x e a posição do ponto no eixo y;
- o `Warning` nos avisa sobre a exclusão das observações que possuem `NA` na variável receita e/ou orçamento;
- todas essas funções são do pacote `{ggplot2}`.

A combinação da função `ggplot()` e de uma ou mais funções `geom_()` definirá o tipo de gráfico gerado.

O primeiro argumento de qualquer função `geom` é o `mapping`. Esse argumento serve para mapear os dados nos atributos estéticos da forma geométrica escolhida. Ele sempre receberá a função `aes()`, cujos argumentos vão sempre depender da forma geométrica que estamos utilizando. No caso de um gráfico de dispersão, precisamos definir a posição dos pontos nos eixos x e y. No

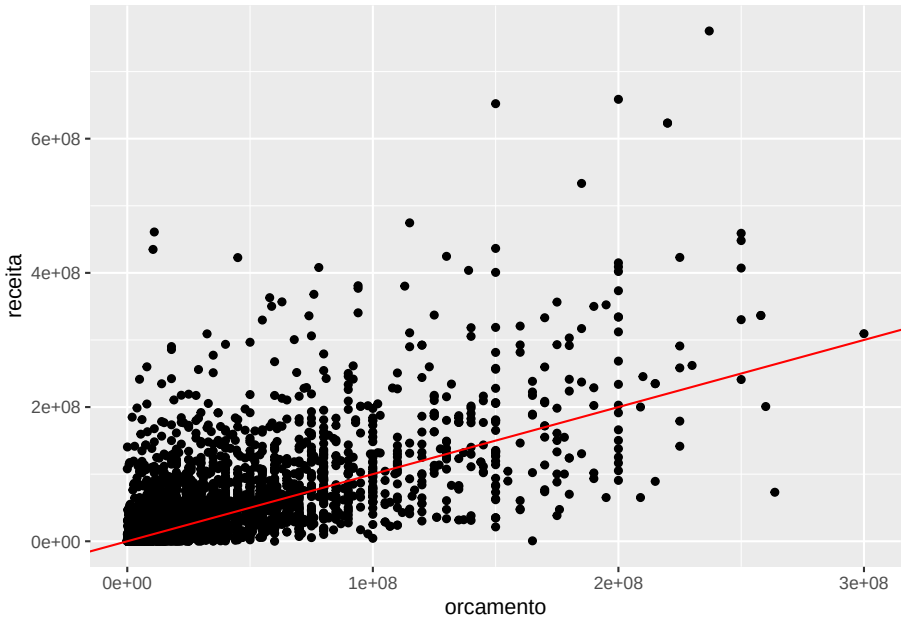
exemplo, a posição do ponto no eixo x foi dada pela coluna `orcamento` e a posição do ponto no eixo y pela coluna `receita`.

> **Atenção!** As camadas dos gráficos são empilhadas utilizando-se o sinal `+`. Como a estrutura é muito parecida com a do *pipe*, é comum trocarmos o `+` por um `%>%` no meio do código, resultando em erro.

Podemos acrescentar uma terceira camada ao gráfico, desenhando a reta `y = x` para visualizarmos os filmes não se pagaram.

```
ggplot(imdb) +
  geom_point(mapping = aes(x = orcamento, y = receita)) +
  geom_abline(intercept = 0, slope = 1, color = "red")
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



Os pontos abaixo da reta representam os filmes com orçamento maior que a receita, isto é, aqueles que deram prejuízo.

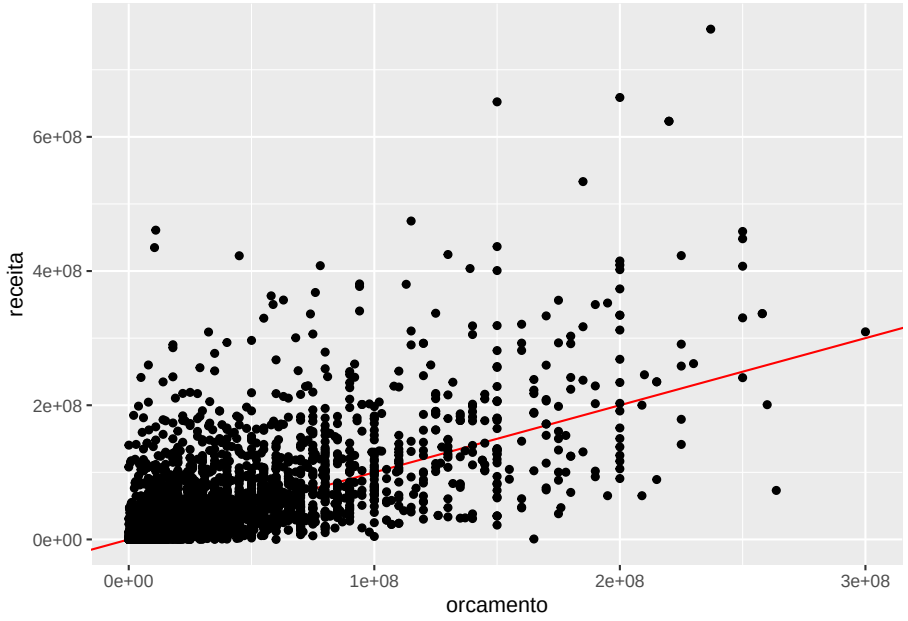
A reta x = y foi acrescentada ao gráfico pela função `geom_abline()`. Esse `geom` pode ser utilizado para desenhar qualquer reta do tipo `y = a + b * x`, sendo `a` o intercepto (*intercept*) da reta e `b` o seu coeficiente angular (*slope*).

Neste caso, como não estamos mapeando colunas da base a essa reta (isto é, essa reta não depende dos dados), não precisamos utilizar o argumento `mapping` da função `geom_abline()`, tampouco a função `aes()`.

Para ver como um ggplot realmente é construído por camadas, veja o que acontece quando colocamos a camada da reta antes da camada dos pontos:

```
ggplot(imdb) +
  geom_abline(intercept = 0, slope = 1, color = "red") +
  geom_point(mapping = aes(x = orcamento, y = receita))
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```

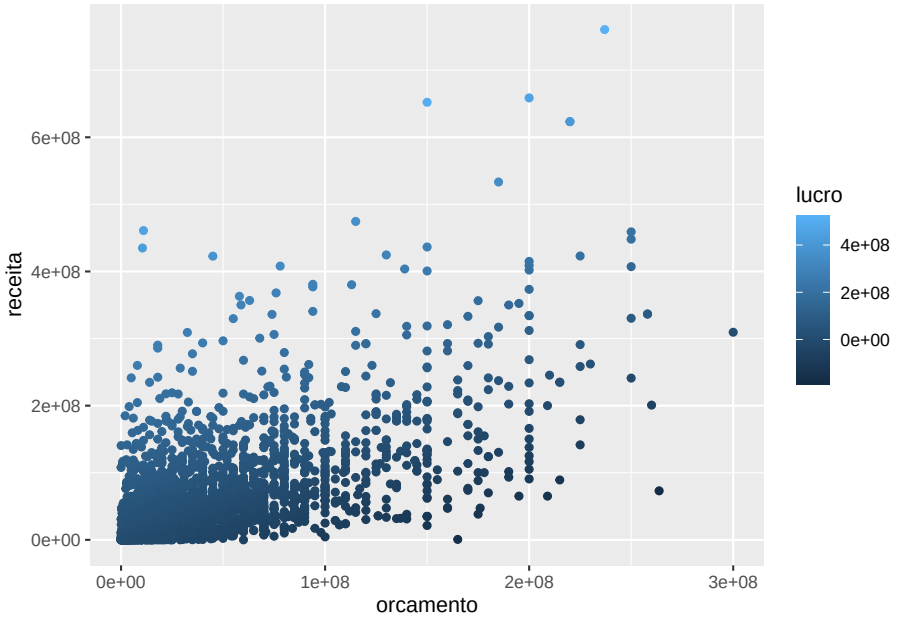


Além da posição nos eixos x e y, podemos mapear a cor dos pontos a uma variável.

```
library(dplyr, warn.conflicts = FALSE)

imdb %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_point(aes(x = orcamento, y = receita, color = lucro))
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



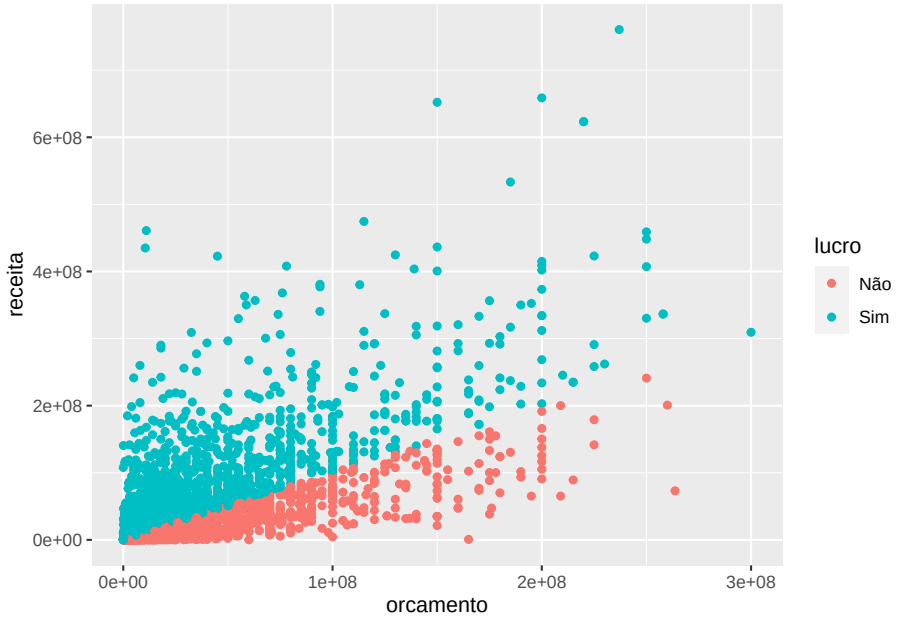
O gráfico acima tem a cor dos pontos definida pelo valor da variável lucro. Como a coluna `lucro` é numérica, um degradê é criado para a cor dos pontos. O azul é a cor padrão nesses casos (veremos mais adiante como escolher a cor).

Veja que criamos a coluna `lucro` utilizando a função `mutate()` antes de iniciarmos a construção do gráfico. O fluxo `base %>% manipulação %>% ggplot` é muito comum no dia-a-dia.

Lembre-se: por trás de um grande gráfico sempre existe uma grande tabela.

Poderíamos também classificar os filmes entre aqueles que lucraram ou não. Neste caso, como a coluna `lucrou` é textual, uma cor é atribuída a cada categoria.

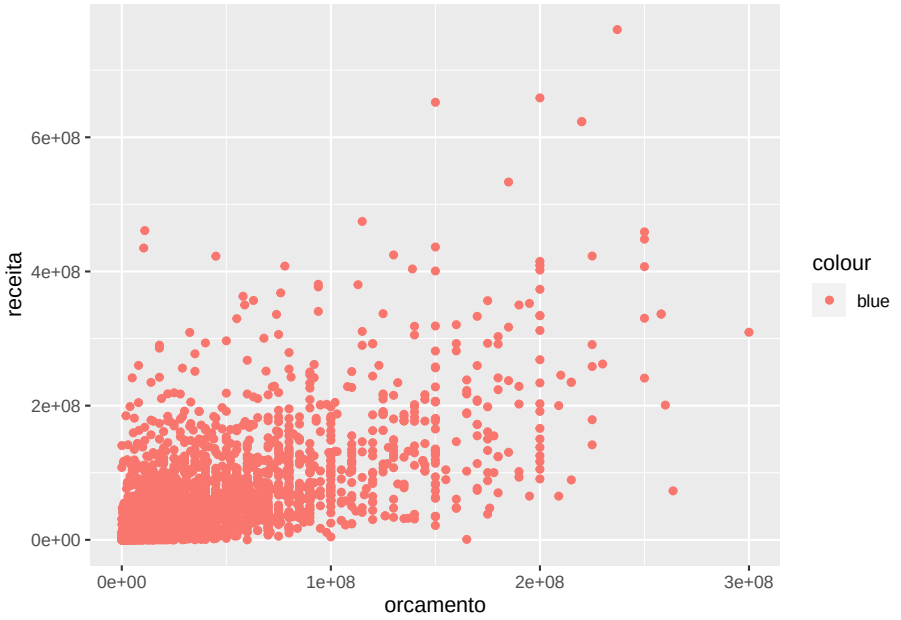
```
imdb %>%
   mutate(
    lucro = receita - orcamento,
    lucro = ifelse(lucro <= 0, "Não", "Sim")
  ) %>%
  filter(!is.na(lucro)) %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita, color = lucro))
```



Um erro comum na hora de definir atributos estéticos de um gráfico é definir valores para atributos estéticos dentro da função `aes()`. Repare o que acontece quando tentamos definir diretamente a cor dos pontos dentro dessa função.

```
ggplot(imdb) +
  geom_point(aes(x = orcamento, y = receita, color = "blue"))
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```

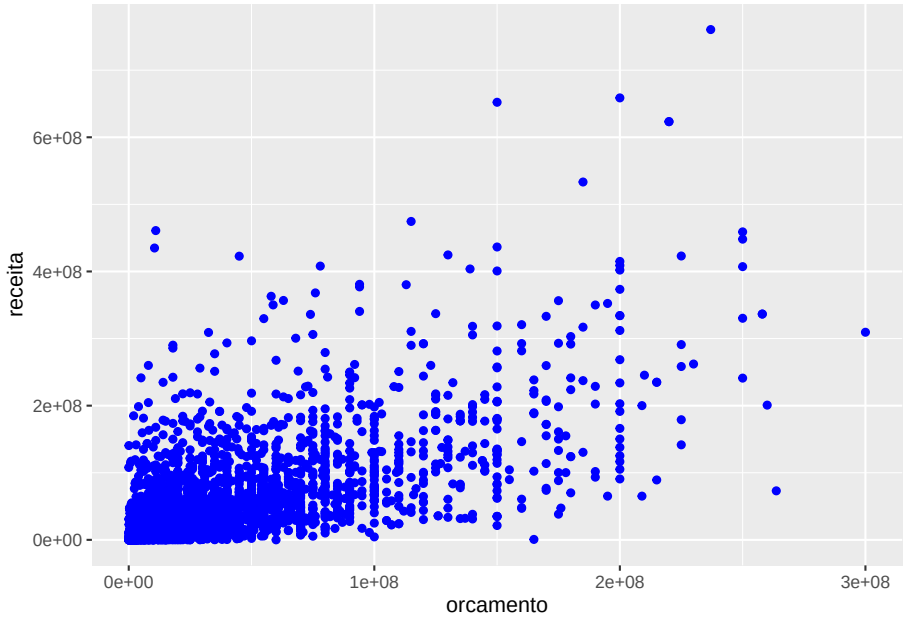


Estranho, não? O que aconteceu foi o seguinte: a função `aes()` espera uma coluna para ser mapeada a cada atributo, então o valor `blue` é tratado como uma nova variável/coluna que tem essa string para todas as observações. Assim, todos pontos têm a mesma cor (vermelha, padrão do ggplot) pois pertencem todos à essa "categoria *blue*".

No caso, o que gostaríamos é de ter pintado todos os pontos de azul. A forma certa de fazer isso é colocando o atributo `color=` fora da função `aes()`:

```
ggplot(imdb) +
  geom_point(aes(x = orcamento, y = receita), color = "blue")
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```

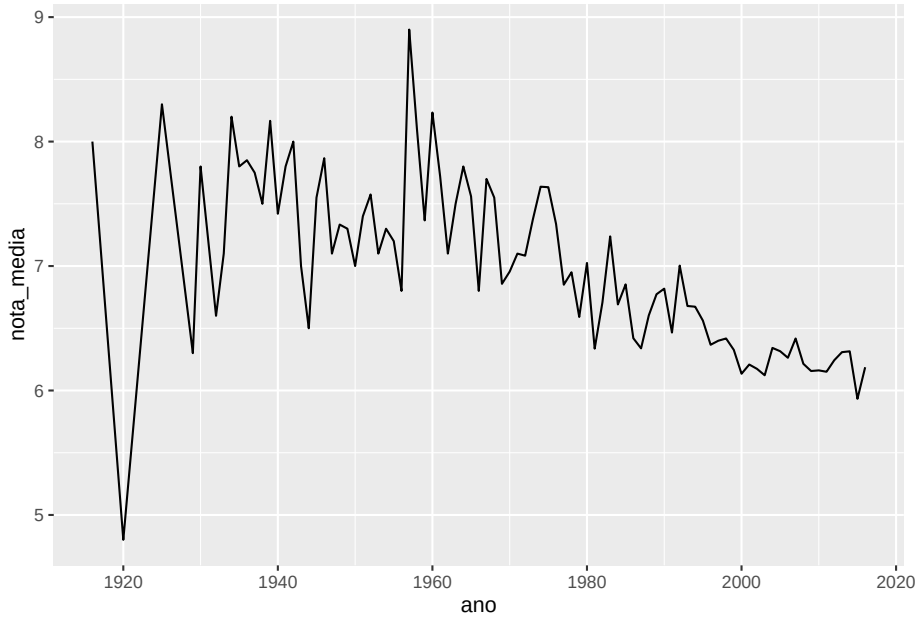


8.1.2 Gráficos de linhas

Utilizamos o `geom_line` para fazer gráficos de linhas. Assim como nos gráficos de pontos, precisamos definir as posições `x` e `y`. O gráfico abaixo representa a evolução da nota média dos filmes ao longo dos anos.

```
imdb %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media))
```

```
## Warning: Removed 1 row(s) containing missing values (geom_path).
```

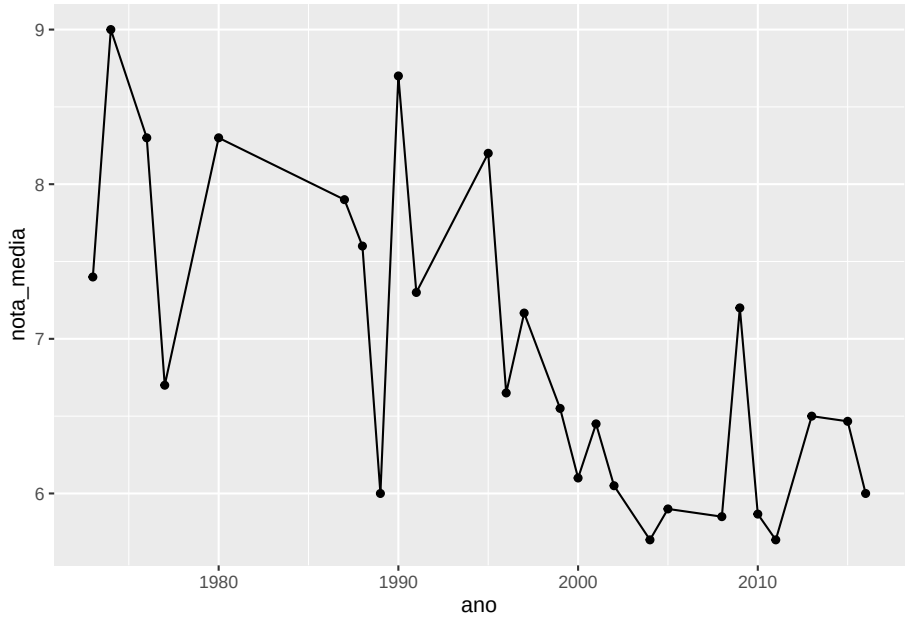


Gráficos de linha são muito utilizados para representar **séries temporais**, isto é, observações medidas repetidamente em intervalos equidistantes[2] de tempo. O gráfico anterior apresenta a série da nota IMDB média ao longo dos anos.

Podemos colocar pontos e retas no mesmo gráfico. Basta acrescentar os dois `geoms`. O gráfico abaixo mostra a nota média anual dos filmes do Robert De Niro.

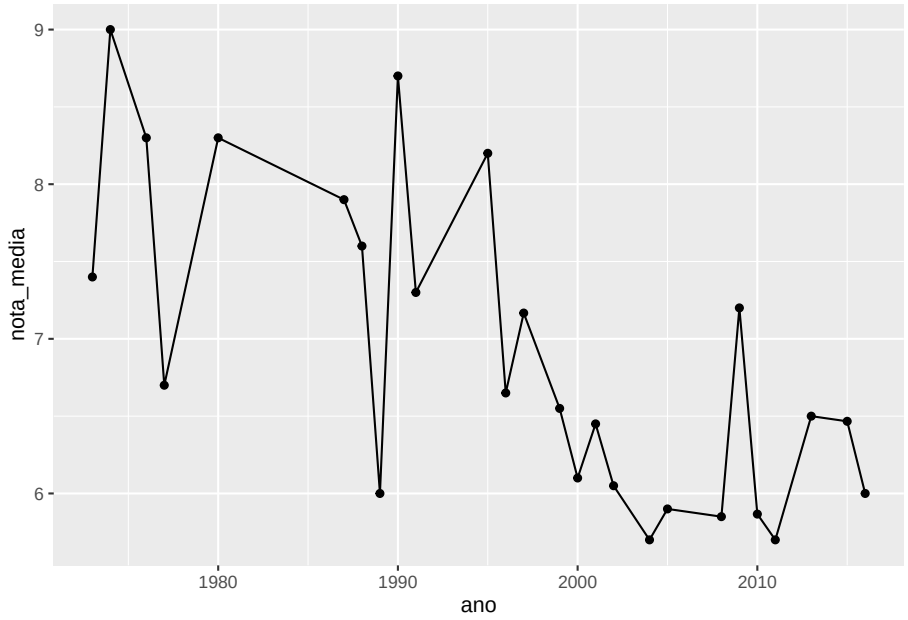
```
imdb %>%
  filter(ator_1 == "Robert De Niro") %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media)) +
  geom_point(aes(x = ano, y = nota_media))
```

[2]Em geral consideramos intervalos equidistantes, mas também existem séries definidas em intervalos não equidistantes de tempo.



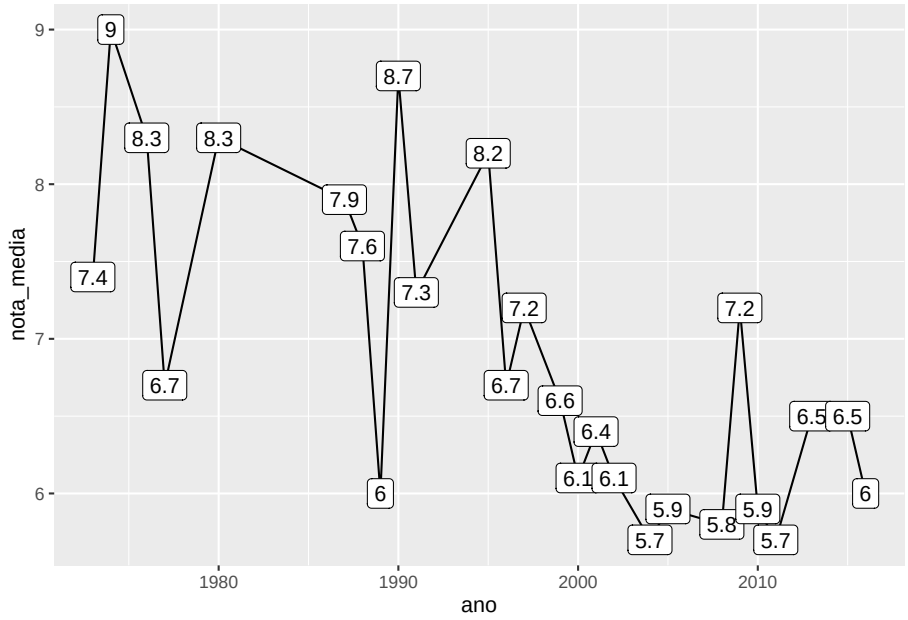
Quando precisamos usar o mesmo `aes()` em vários `geoms`, podemos defini-lo dentro da função `ggplot()`. Esse `aes()` será então distribuído para todo `geom` do gráfico. O código anterior pode ser reescrito da seguinte forma.

```
imdb %>%
  filter(ator_1 == "Robert De Niro") %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = nota_media)) +
  geom_line() +
  geom_point()
```



Se algum `geom` necessitar de um atributo que os outros não precisam, esse atributo pode ser especificado normalmente dentro dele. Abaixo, utilizamos o `geom_label` para colocar as notas médias no gráfico. Além do `x` e `y`, o `geom_label` também precisa do texto que será escrito no gráfico.

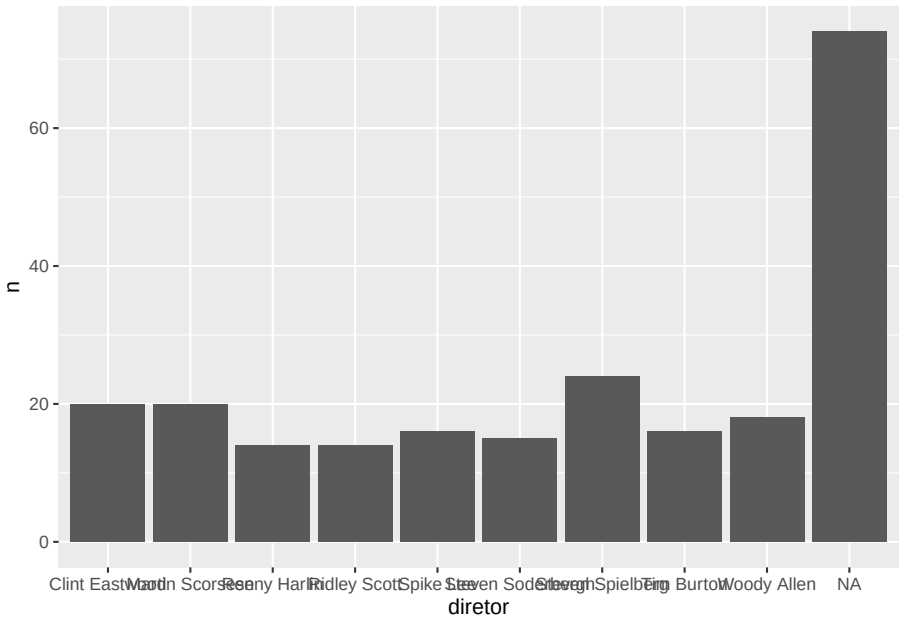
```
imdb %>%
  filter(ator_1 == "Robert De Niro") %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  mutate(nota_media = round(nota_media, 1)) %>%
  ggplot(aes(x = ano, y = nota_media)) +
  geom_line() +
  geom_label(aes(label = nota_media))
```



8.1.3 Gráficos de barras

Para construir gráficos de barras, utilizamos o `geom_col`. A seguir, construímos um gráfico de barras do número de filmes dos 10 diretores que mais aparecem na nossa base do IMDB.

```
imdb %>%
  count(diretor) %>%
  top_n(10, n) %>%
  ggplot() +
  geom_col(aes(x = diretor, y = n))
```

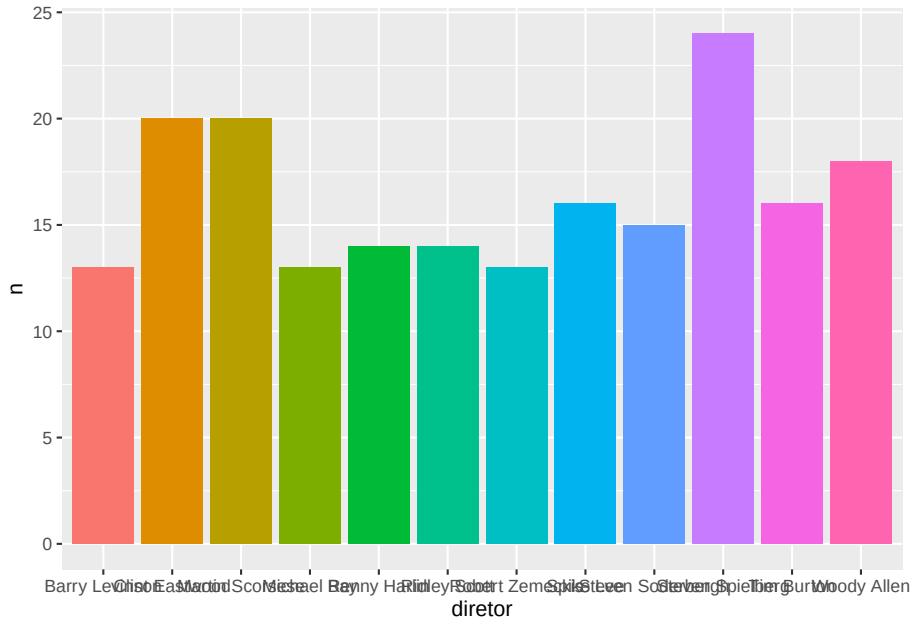


Gráficos de barras também precisam dos atributos `x` e `y`, sendo que o atributo `y` representará a altura de cada barra.

No gráfico anterior, vemos que o `NA` é considerado uma "categoria" de diretor e entra no gráfico. Podemos retirar os `NAs` dessa coluna previamente utilizando a função `filter()`.

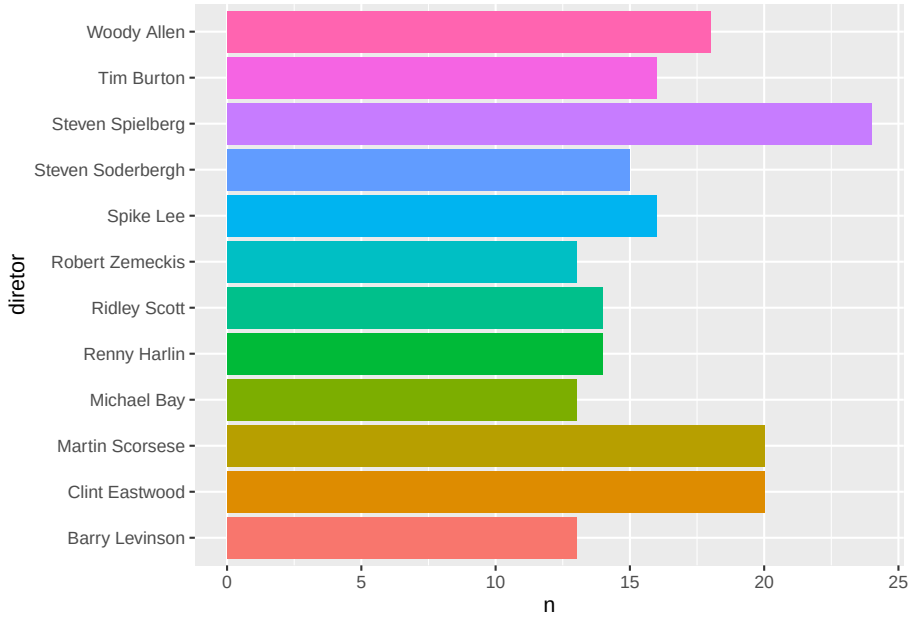
A seguir, além de retirar os `NAs`, atribuímos a coluna `diretor` à cor das colunas. Repare que, nesse caso, não utilizamos o atributo `color` e sim `fill`. A regra é a seguinte: o atributo `color` colore objetos sem área (pontos, linhas, contornos), o atributo `fill` preenche objetos com cor (barras, áreas, polígonos em geral).

```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(10, n) %>%
  ggplot() +
  geom_col(
    aes(x = diretor, y = n, fill = diretor),
    show.legend = FALSE
  )
```



Para consertar as labels do eixo `x`, a melhor prática é invertermos os eixos do gráfico, construindo barras horizontais.

```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(10, n) %>%
  ggplot() +
  geom_col(
    aes(y = diretor, x = n, fill = diretor),
    show.legend = FALSE
  )
```

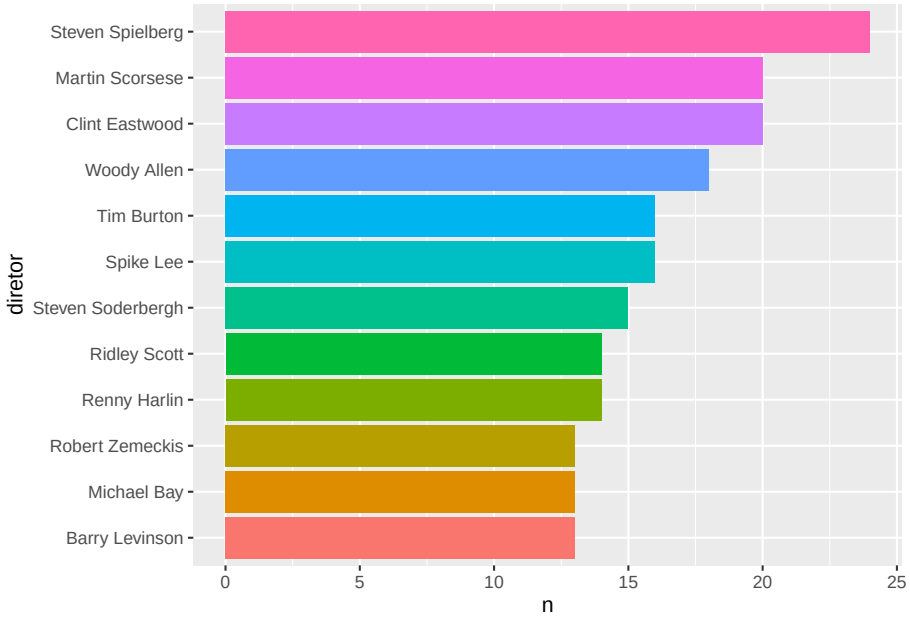


Para ordenar as colunas, precisamos mudar a ordem dos níveis do *fator* `diretor`. Para isso, utilizamos a função `fct_reorder()` do pacote `forcats`. A nova ordem será estabelecida pela coluna `n` (quantidade de filmes).

Fatores dentro do R são números inteiros (1, 2, 3, ...) que possuem uma representação textual (Ver Seção 7.6). Variáveis categóricas são transformadas em fatores pelo ggplot pois todo eixo cartesiano é numérico. Assim, os textos de uma variável categórica são, internamente, números inteiros.

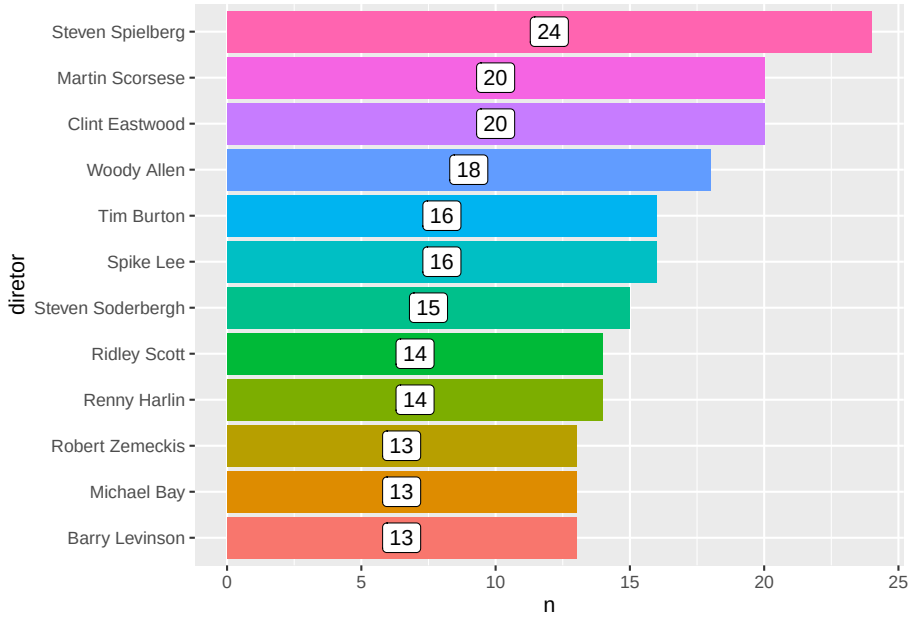
Por padrão, os inteiros são atribuídos a cada categoria de uma variável pela ordem alfabética (repare na ordem dos diretores nos gráficos anteriores). Assim, se transformássemos o vetor `c("banana", "uva", "melancia")` em um fator, a atribuição de inteiros seria: "banana" vira 1, "melancia" vira 2 e "uva" vira 3. Embora sejam inteiros internamente, sempre que chamássemos esse novo vetor, ainda sim veríamos os textos "banana", "uva" e "melancia".

```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(10, n) %>%
  mutate(diretor = forcats::fct_reorder(diretor, n)) %>%
  ggplot() +
  geom_col(
    aes(y = diretor, x = n, fill = diretor),
    show.legend = FALSE
  )
```



Por fim, podemos colocar uma label com o número de filmes de cada diretor dentro de cada barra.

```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(10, n) %>%
  mutate(diretor = forcats::fct_reorder(diretor, n)) %>%
  ggplot() +
  geom_col(aes(x = diretor, y = n, fill = diretor), show.legend = FALSE) +
  geom_label(aes(x = diretor, y = n/2, label = n)) +
  coord_flip()
```



8.1.4 Histogramas e boxplots

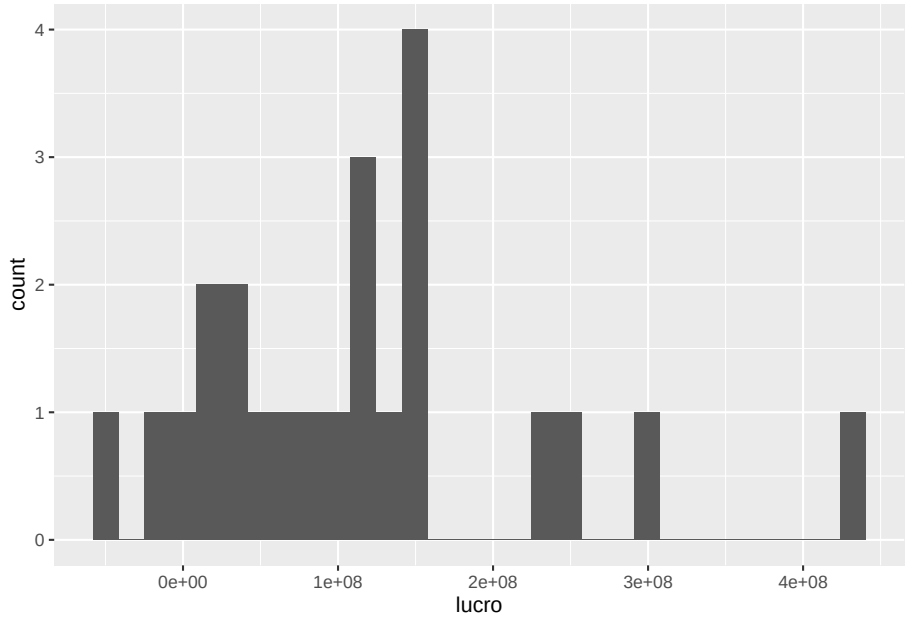
Para construir histogramas, usamos o `geom_histogram`. Esse `geom` só precisa do atributo `x` (o eixo y é construído automaticamente). Histogramas são úteis para avaliarmos a distribuição de uma variável.

A seguir, construímos o histograma do lucro dos filmes do diretor Steven Spielberg. O primeiro *warning* nos diz que o eixo x foi dividido em 30 intervalos para a construção do histograma.

```
imdb %>%
  filter(diretor == "Steven Spielberg") %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_histogram(aes(x = lucro))
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

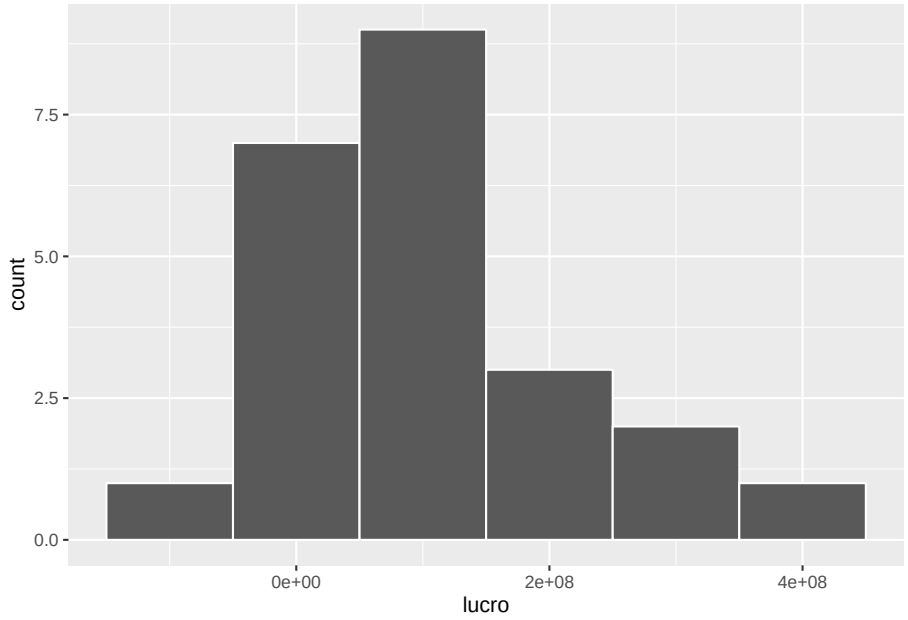
```
## Warning: Removed 1 rows containing non-finite values (stat_bin).
```



Para definir o tamanho de cada intervalo, podemos utilizar o argumento `bindwidth`.

```
imdb %>%
  filter(diretor == "Steven Spielberg") %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_histogram(
    aes(x = lucro),
    binwidth = 100000000,
    color = "white"
  )
```

```
## Warning: Removed 1 rows containing non-finite values (stat_bin).
```



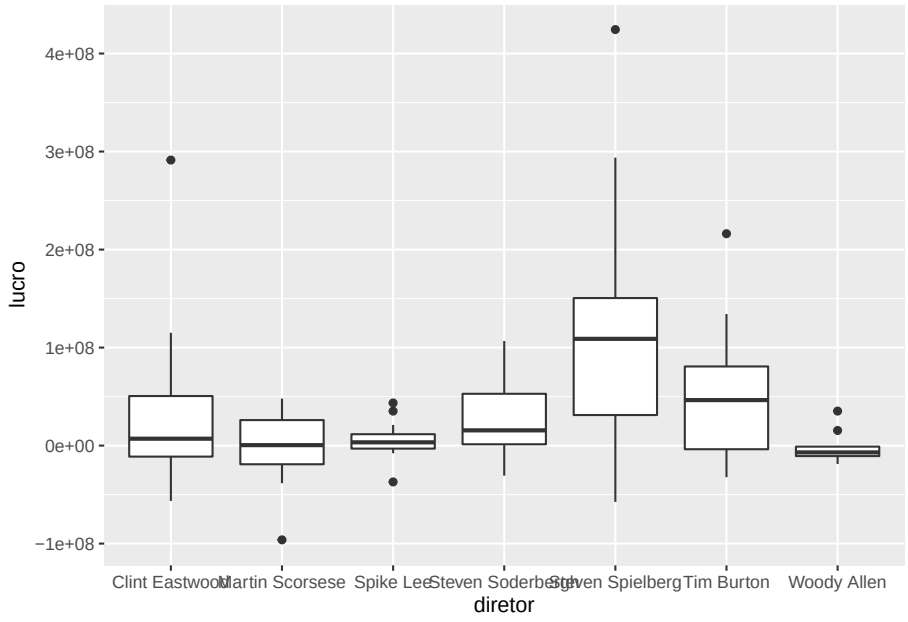
Boxplots também são úteis para estudarmos a distribuição de uma variável, principalmente quando queremos comparar várias distribuições.

Para construir um boxplot no ggplot, utilizamos a função `geom_boxplot`. Ele precisa dos atributos `x` e `y`, sendo que ao atributo `x` devemos mapear uma variável categórica.

A seguir, construímos boxplots do lucro dos filmes dos diretores que fizeram mais de 15 filmes.

```
imdb %>%
  filter(!is.na(diretor)) %>%
  group_by(diretor) %>%
  filter(n() >= 15) %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_boxplot(aes(x = diretor, y = lucro))
```

```
## Warning: Removed 10 rows containing non-finite values (stat_boxplot).
```

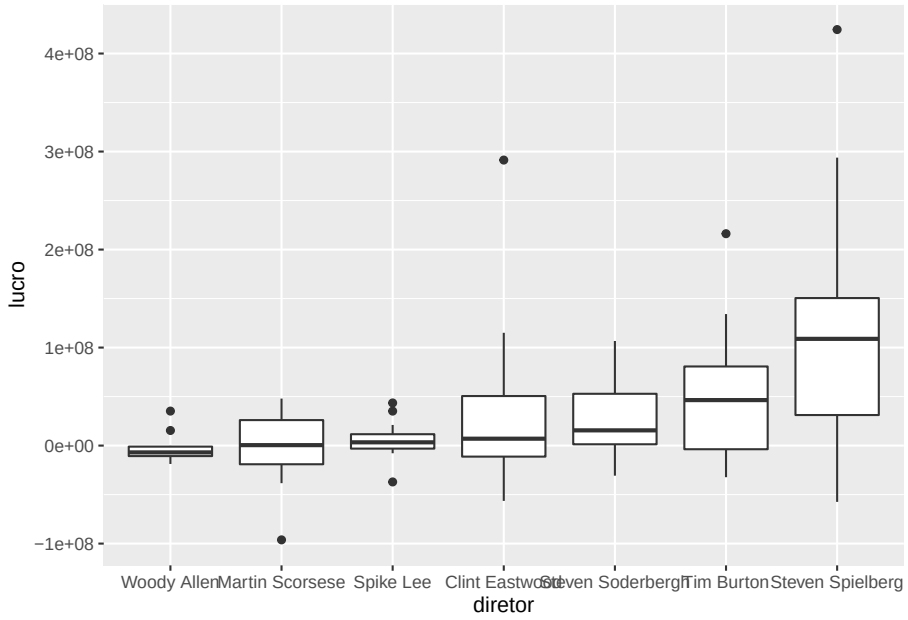


Também podemos reordenar a ordem dos boxplots utilizando a função `forcats::fct_reorder`. Neste caso, os diretores são ordenados pela mediana do lucro[3].

```
imdb %>%
  filter(!is.na(diretor)) %>%
  group_by(diretor) %>%
  filter(n() >= 15) %>%
  ungroup() %>%
  mutate(
    lucro = receita - orcamento,
    diretor = forcats::fct_reorder(diretor, lucro, na.rm = TRUE)
  ) %>%
  ggplot() +
  geom_boxplot(aes(x = diretor, y = lucro))
```

```
## Warning: Removed 10 rows containing non-finite values (stat_boxplot).
```

[3] A mediana é usada por padrão pela função `fct_reorder()`. É possível trocar a função utilizada por meio do argumento `.fun`.

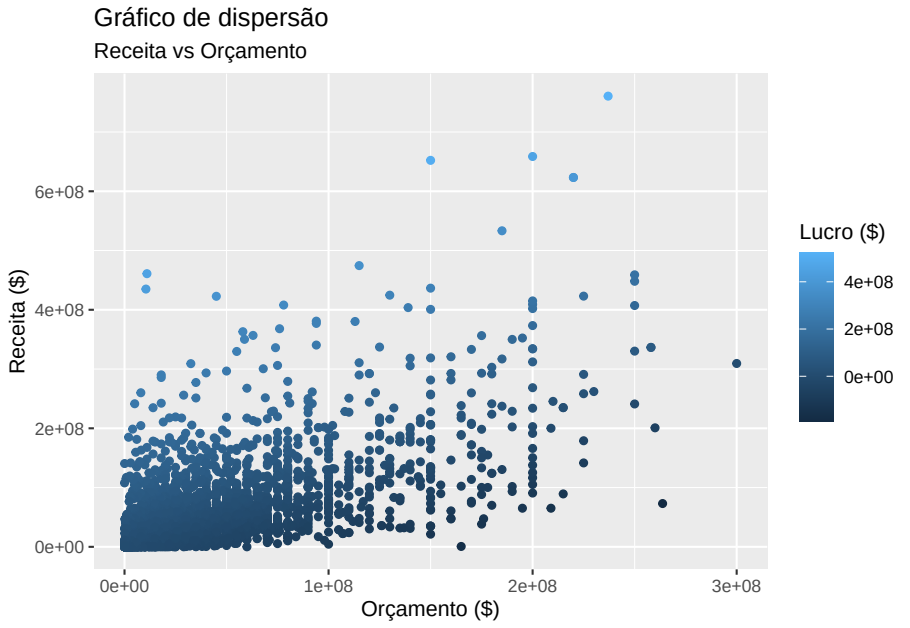


8.1.5 Títulos e *labels*

Os títulos e *labels* do gráfico também são considerados camadas e são criados ou modificados pela função `labs()`. O exemplo a seguir coloca um título e um subtítulo no gráfico, além de modificar os *labels* do eixo x e y e da legenda.

```
imdb %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita, color = lucro)) +
  labs(
    x = "Orçamento ($)",
    y = "Receita ($)",
    color = "Lucro ($)",
    title = "Gráfico de dispersão",
    subtitle = "Receita vs Orçamento"
  )
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



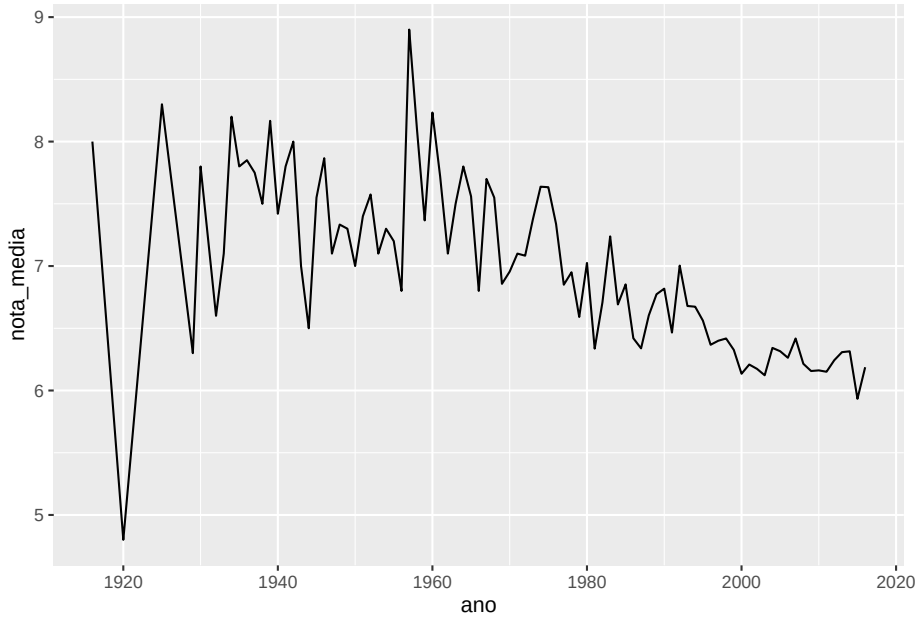
8.1.6 Escalas

O pacote {ggplot2} possui uma família de funções `scale_` para modificarmos propriedades referentes às escalas do gráfico. Como podemos ter escalas de números, categorias, cores, datas, entre outras, temos uma função específica para cada tipo de escala.

Considere o gráfico a seguir.

```
imdb %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media))
```

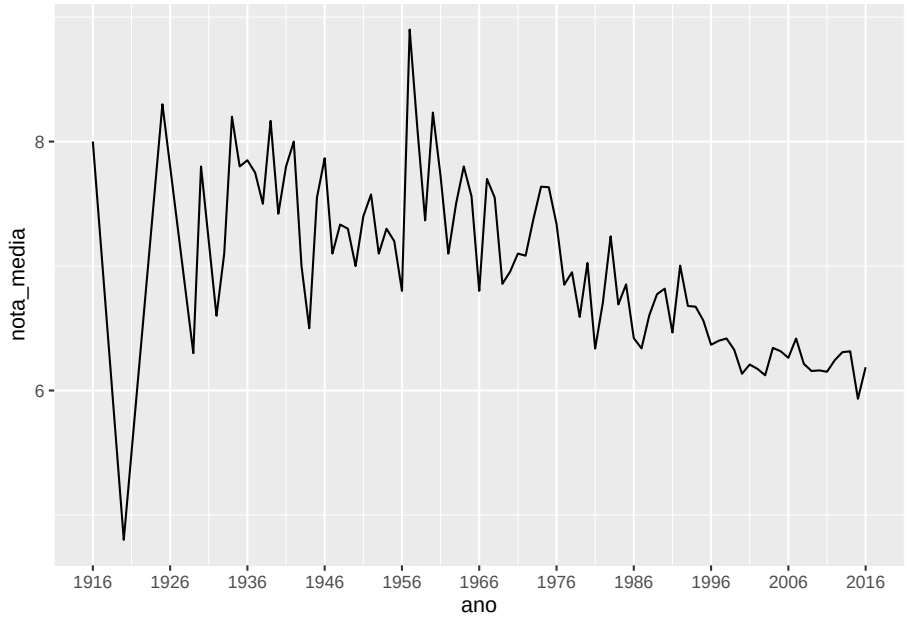
```
## Warning: Removed 1 row(s) containing missing values (geom_path).
```



Vamos redefinir as quebras dos eixos `x` e `y`.

```
imdb %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media)) +
  scale_x_continuous(breaks = seq(1916, 2016, 10)) +
  scale_y_continuous(breaks = seq(0, 10, 2))
```

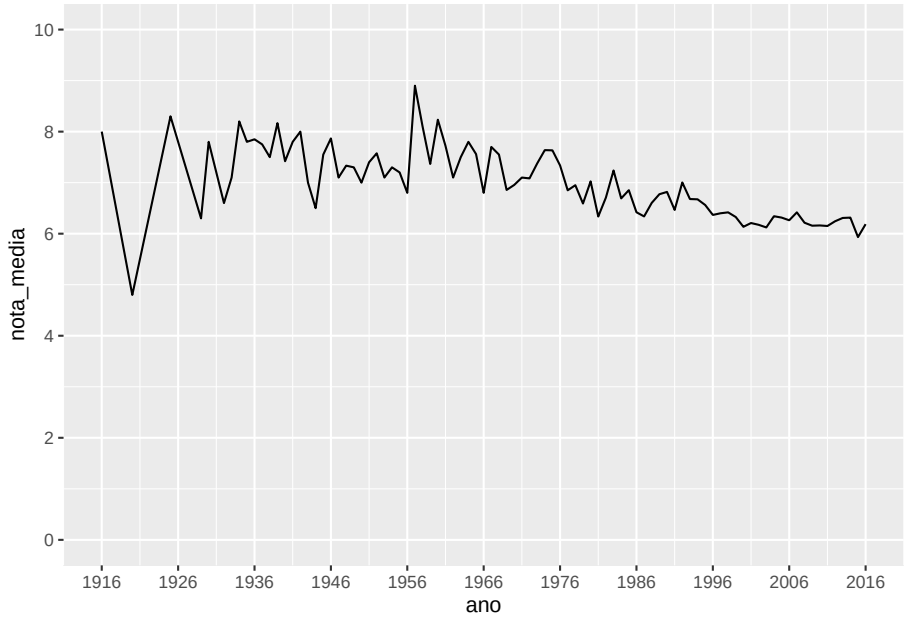
```
## Warning: Removed 1 row(s) containing missing values (geom_path).
```



Como as escalas dos eixos x e y são numéricas, utilizamos nesse caso as funções `scale_x_continuous()` e `scale_y_continuous()`. Veja que, mesmo definindo as quebras entre 0 e 10, o limite do eixo y não é alterado. Para alterá-lo, usamos a função `coord_cartesian()`.

```
imdb %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media)) +
  scale_x_continuous(breaks = seq(1916, 2016, 10)) +
  scale_y_continuous(breaks = seq(0, 10, 2)) +
  coord_cartesian(ylim = c(0, 10))
```

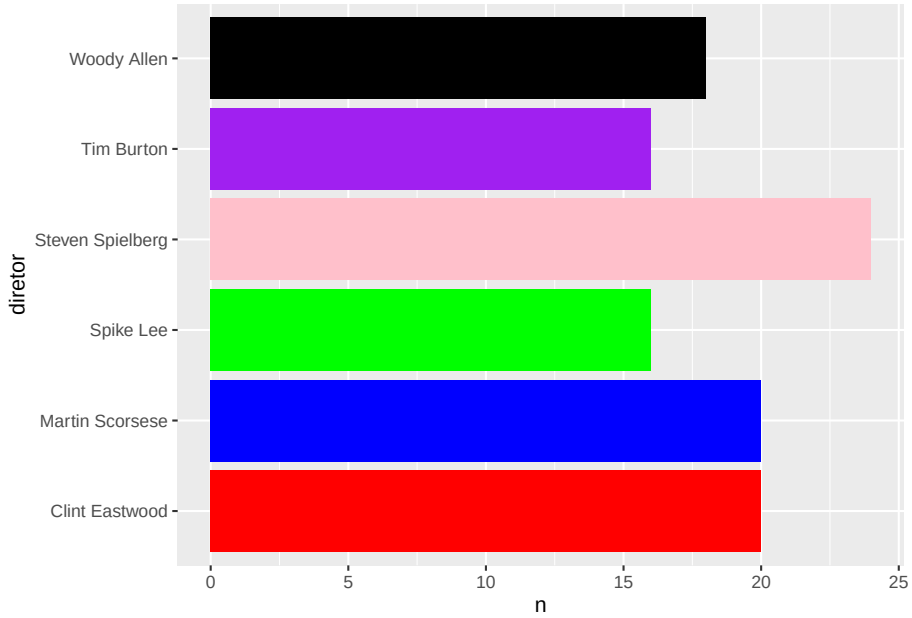
```
## Warning: Removed 1 row(s) containing missing values (geom_path).
```



Para mudarmos as escalas de cores, usamos as funções do tipo: `scale_color_` e `scale_fill_`.

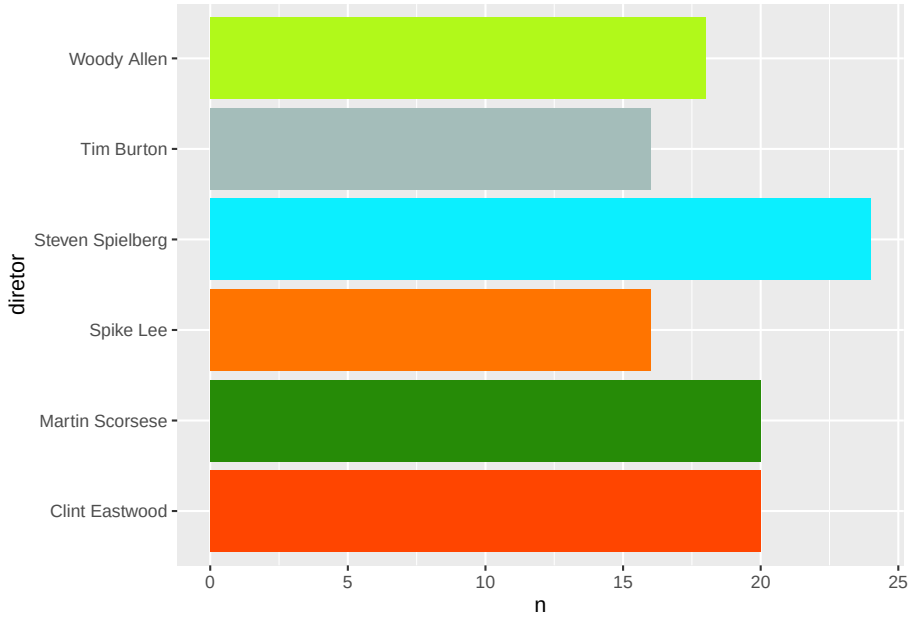
Para escolher manualmente as cores de um gráfico, utilize as funções `scale_color_manual()` e `scale_fill_manual()`. A seguir substituímos as cores padrão do gráfico por um outro conjunto de cores.

```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(5, n) %>%
  ggplot() +
  geom_col(
    aes(x = diretor, y = n, fill = diretor),
    show.legend = FALSE
  ) +
  coord_flip() +
  scale_fill_manual(values = c("red", "blue", "green", "pink", "purple", "black"))
```



Também podemos usar códigos hexadecimais.

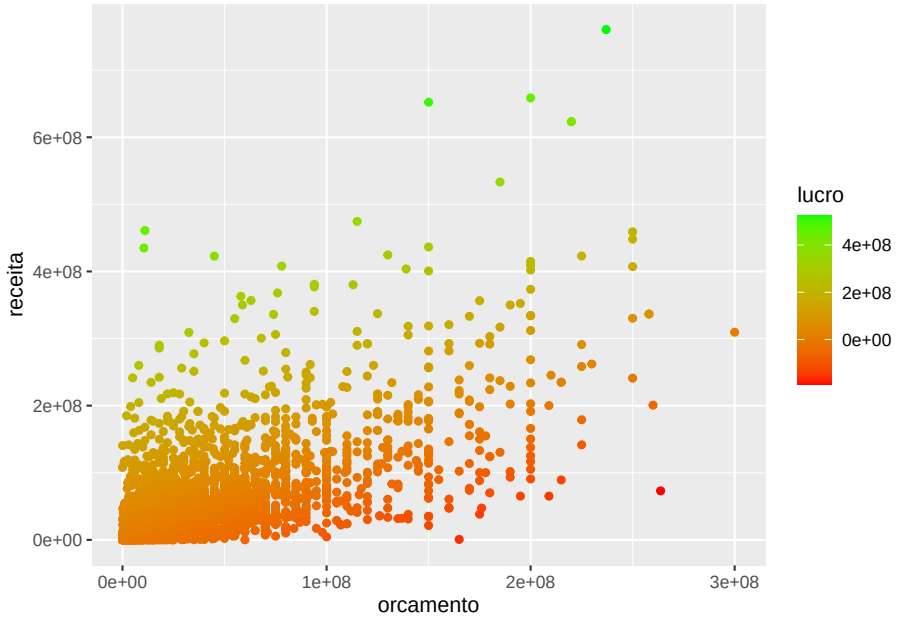
```
imdb %>%
  count(diretor) %>%
  filter(!is.na(diretor)) %>%
  top_n(5, n) %>%
  ggplot() +
  geom_col(
    aes(x = diretor, y = n, fill = diretor),
    show.legend = FALSE
  ) +
  coord_flip() +
  scale_fill_manual(
    values = c("#ff4500", "#268b07", "#ff7400", "#0befff", "#a4bdba", "#b1f91a")
  )
```



Para trocar as cores de um gradiente, utilize as funções `scale_color_gradient()` e `scale_fill_gradient()`.

```
imdb %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot() +
  geom_point(aes(x = orcamento, y = receita, color = lucro)) +
  scale_color_gradient(low = "red", high = "green")
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```

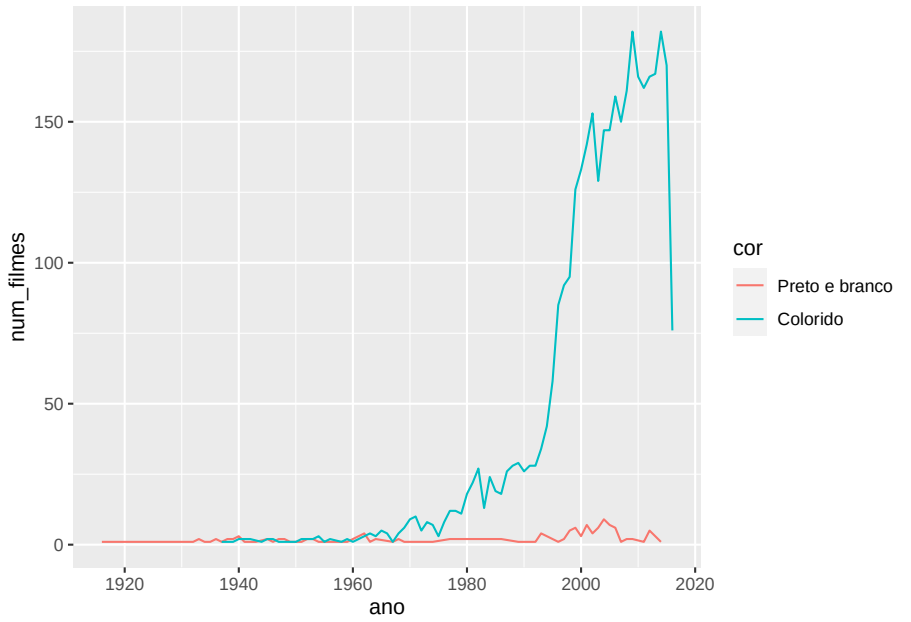


Para trocar o nome das categorias de uma legenda de cores, utilize as funções `scale_color_discrete()` e `scale_fill_discrete()`.

```
imdb %>%
  filter(!is.na(cor)) %>%
  group_by(ano, cor) %>%
  summarise(num_filmes = n()) %>%
  ggplot() +
  geom_line(aes(x = ano, y = num_filmes, color = cor)) +
  scale_color_discrete(labels = c("Preto e branco", "Colorido"))
```

```
## `summarise()` has grouped output by 'ano'. You can override using the `.groups` argu
```

```
## Warning: Removed 2 row(s) containing missing values (geom_path).
```



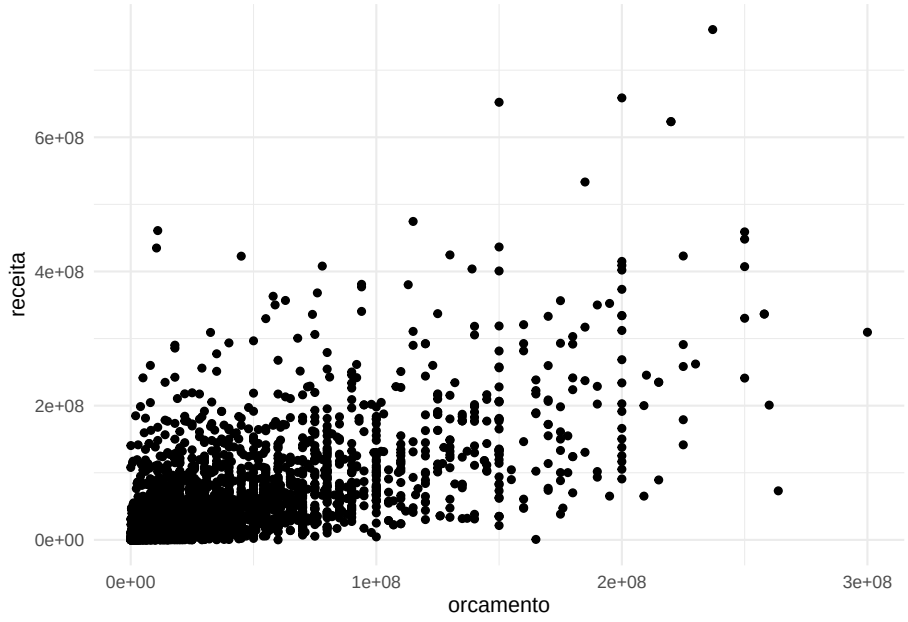
8.1.7 Temas

Os gráficos que vimos até agora usam o tema padrão do ggplot2. Existem outros temas prontos para utilizarmos presentes na família de funções `theme_`.

```
imdb %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita)) +
  theme_minimal()
```

theme_minimal()

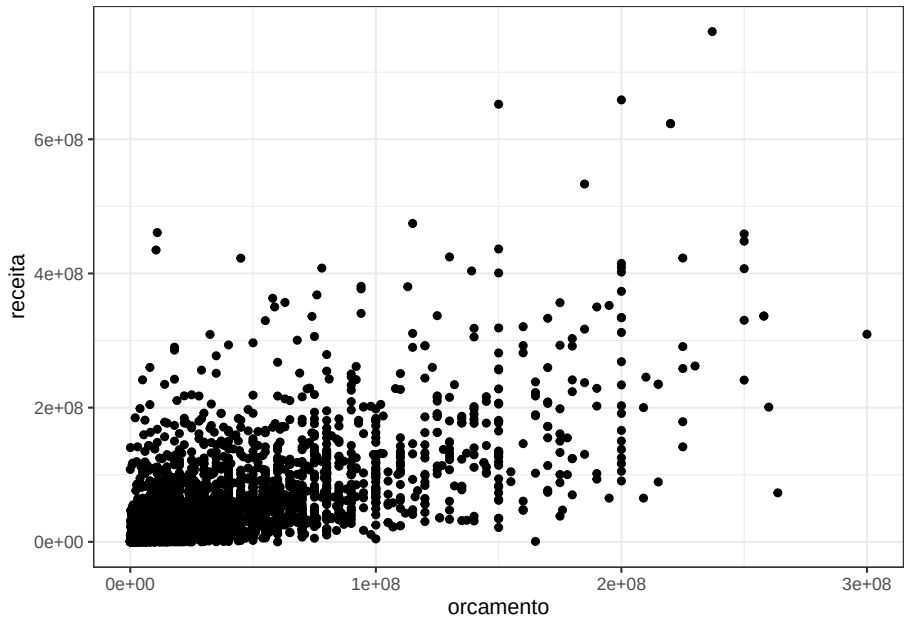
```
## Warning: Removed 733 rows containing missing values (geom_point).
```




```
imdb %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita)) +
  theme_bw()
```

theme_bw()

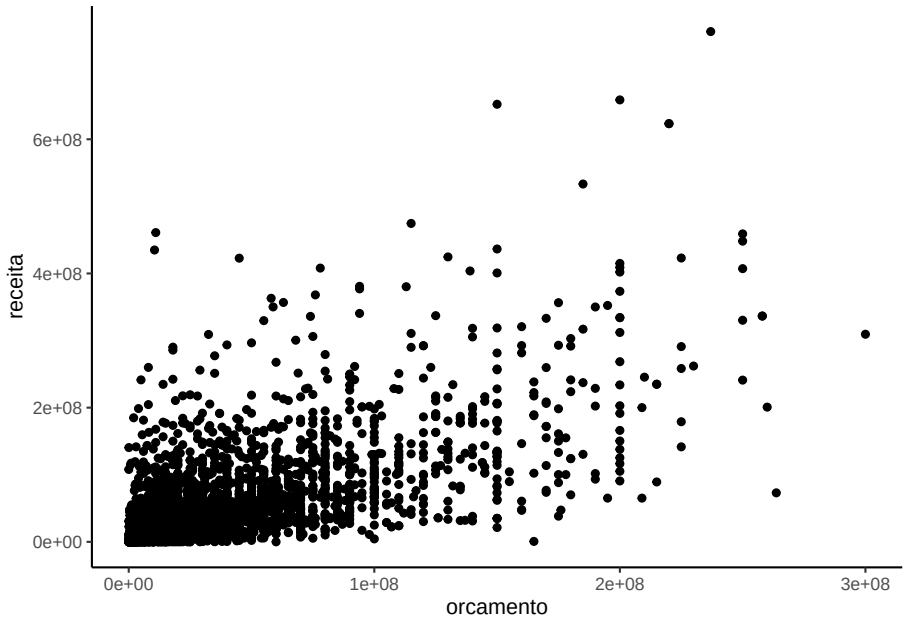
```
## Warning: Removed 733 rows containing missing values (geom_point).
```




```
imdb %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita)) +
  theme_classic()
```

theme_classic()

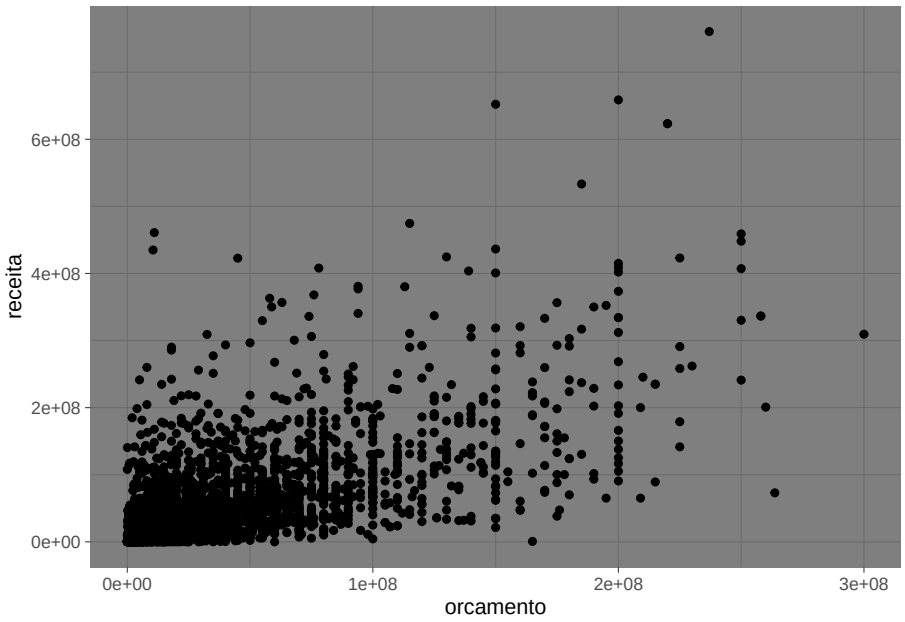
```
## Warning: Removed 733 rows containing missing values (geom_point).
```




```
imdb %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita)) +
  theme_dark()
```

theme_dark()

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



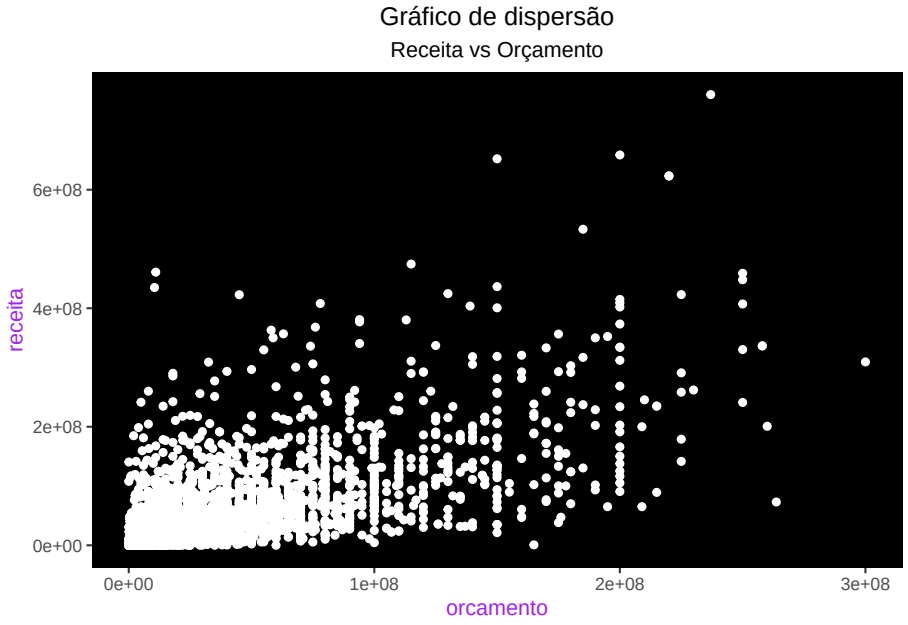
Você também pode criar o seu próprio tema utilizando a função `theme()`. Nesse caso, para trocar os elementos estéticos do gráfico precisamos usar as funções `element_text()` para textos, `element_line()` para linhas, `element_rect()` para áreas e `element_blank()` para remover elementos.

No exemplo a seguir, fizemos as seguintes modificações no tema padrão:

- Alinhamos o título no centro do gráfico.
- Alinhamos o subtítulo no centro do gráfico.
- Pintamos o título dos eixos de roxo.
- Preenchemos o fundo do gráfico de preto.
- Removemos o grid do gráfico.

```
imdb %>%
  ggplot() +
  geom_point(mapping = aes(x = orcamento, y = receita), color = "white") +
  labs(title = "Gráfico de dispersão", subtitle = "Receita vs Orçamento") +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    axis.title = element_text(color = "purple"),
    panel.background = element_rect(fill = "black"),
    panel.grid = element_blank()
  )
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```



8.1.8 Juntando gráficos

No ggplot, temos várias formas de juntar gráficos. Vamos apresentar a seguir as principais.

Vários geoms no mesmo gráfico

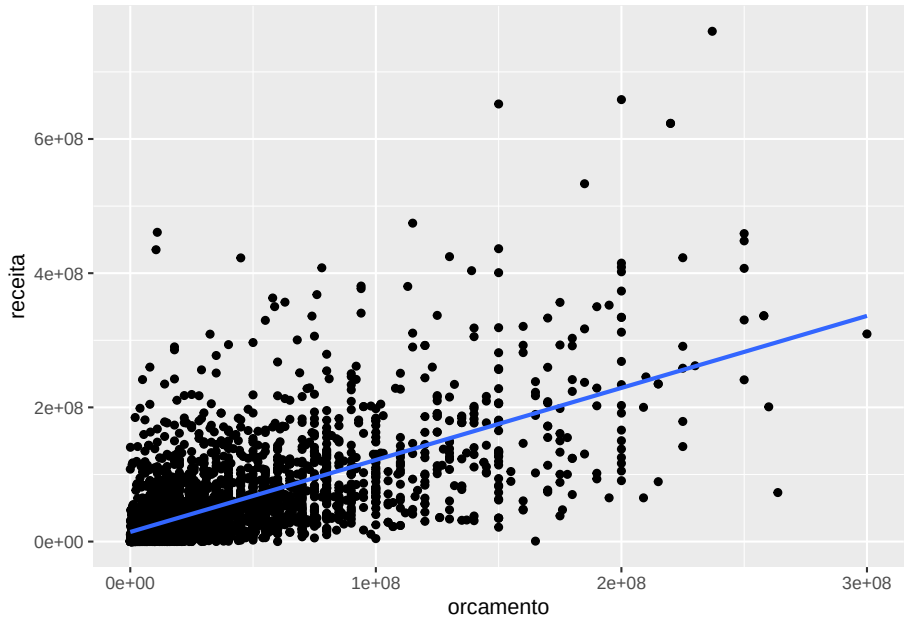
Como vimos anteriormente, podemos acrescentar vários `geoms` em um mesmo gráfico, apenas adicionando novas camadas. No código a seguir, construímos o gráfico de dispersão da receita pelo orçamento dos filmes, acrescentando também uma reta de tendência linear aos pontos.

```
ggplot(imdb, aes(x = orcamento, y = receita)) +
  geom_point() +
  geom_smooth(se = FALSE, method = "lm")
```

```
## `geom_smooth()` using formula 'y ~ x'
```

```
## Warning: Removed 733 rows containing non-finite values (stat_smooth).
```

```
## Warning: Removed 733 rows containing missing values (geom_point).
```

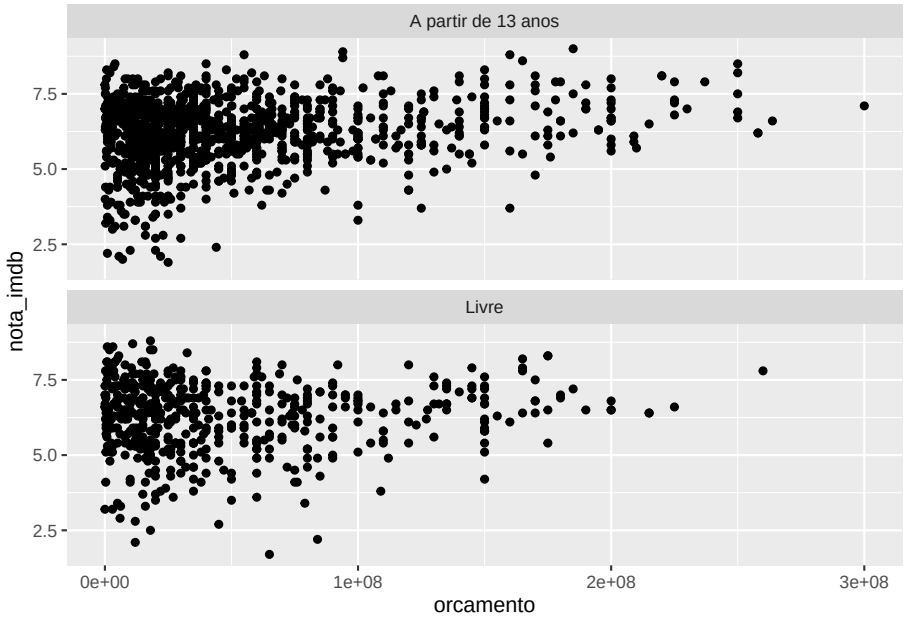


Replicando um gráfico para cada categoria de uma variável

Uma funcionalidade muito útil do `ggplot2` é a possibilidade de usar `facets` para replicar um gráfico para cada categoria de uma variável.

```
imdb %>%
  filter(classificacao %in% c("Livre", "A partir de 13 anos")) %>%
  ggplot() +
  geom_point(aes(x = orcamento, y = nota_imdb)) +
  facet_wrap(~classificacao, nrow = 2)
```

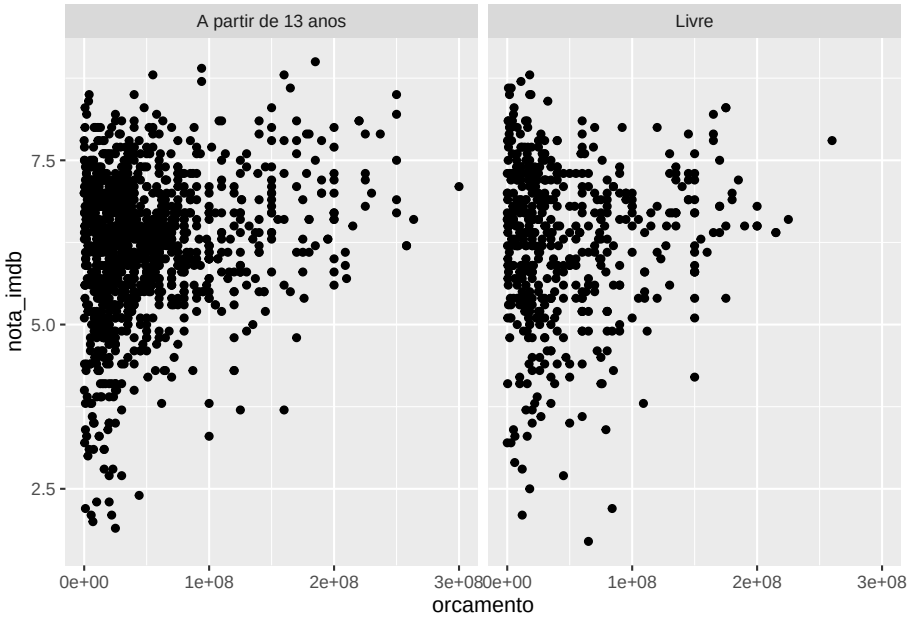
```
## Warning: Removed 85 rows containing missing values (geom_point).
```



Repare que usamos uma fórmula para dizer à função qual variável vamos utilizar para quebrar o gráfico. Podemos especificar se queremos os gráficos lado a lado ou um embaixo do outro pelos argumentos `nrow=` e `ncol=`.

```
imdb %>%
  filter(classificacao %in% c("Livre", "A partir de 13 anos")) %>%
  ggplot() +
  geom_point(aes(x = orcamento, y = nota_imdb)) +
  facet_wrap(~classificacao, ncol = 2)
```

```
## Warning: Removed 85 rows containing missing values (geom_point).
```



Juntando gráficos diferentes

Diversos outros pacotes trazem ferramentas super úteis para trabalharmos com o `ggplot2`. Um deles é o pacote `{patchwork}`. Após carregá-lo, podemos juntar dois gráficos em uma mesma figura com um simples `+`.

```
# Instale antes de carregar
# install.packages("patchwork")

library(patchwork)

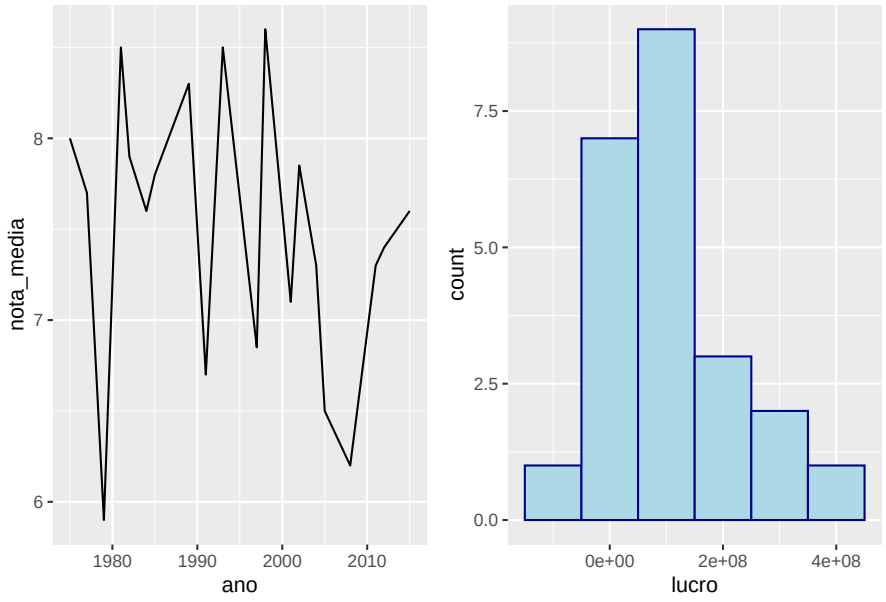
p1 <- imdb %>%
  filter(diretor == "Steven Spielberg") %>%
  group_by(ano) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot() +
  geom_line(aes(x = ano, y = nota_media))

p2 <- imdb %>%
  mutate(lucro = receita - orcamento) %>%
  filter(diretor == "Steven Spielberg") %>%
  ggplot() +
  geom_histogram(
    aes(x = lucro),
```

```
    fill = "lightblue",
    color = "darkblue",
    binwidth = 100000000
  )

p1 + p2
```

```
## Warning: Removed 1 rows containing non-finite values (stat_bin).
```

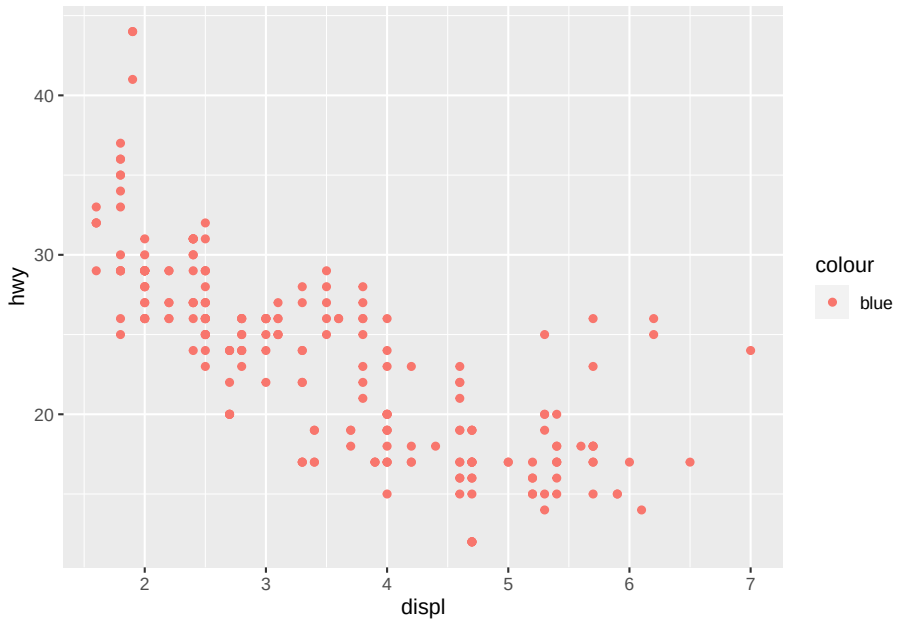


8.1.9 Exercícios

1. O que acontece quando rodamos o código `ggplot(data = mtcars)`?

2. O que tem de errado no código abaixo? Por que os pontos não ficaram azuis?

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = "blue"))
```



3. Mapeie uma variável contínua para a cor, o tamanho e a forma de um gráfico de pontos. Como esses atributos estéticos se comportam diferente para variáveis categóricas vs contínuas?

4. Utilizando o `mtcars`, faça um gráfico de dispersão de `mpg` por `qsec`.

5. Utilizando o `mtcars`, o que acontece se você fizer um gráfico de dispersão de `vs` por `mpg`? Por que o gráfico não é útil?

Para resolver os exercícios a seguir, utilize a base `imdb`.

6. Crie um gráfico de dispersão da nota do imdb pelo orçamento.

7. Faça um gráfico de linhas do orçamento médio dos filmes ao longo dos anos.

8. Transforme o gráfico do exercício anterior em um gráfico de barras.

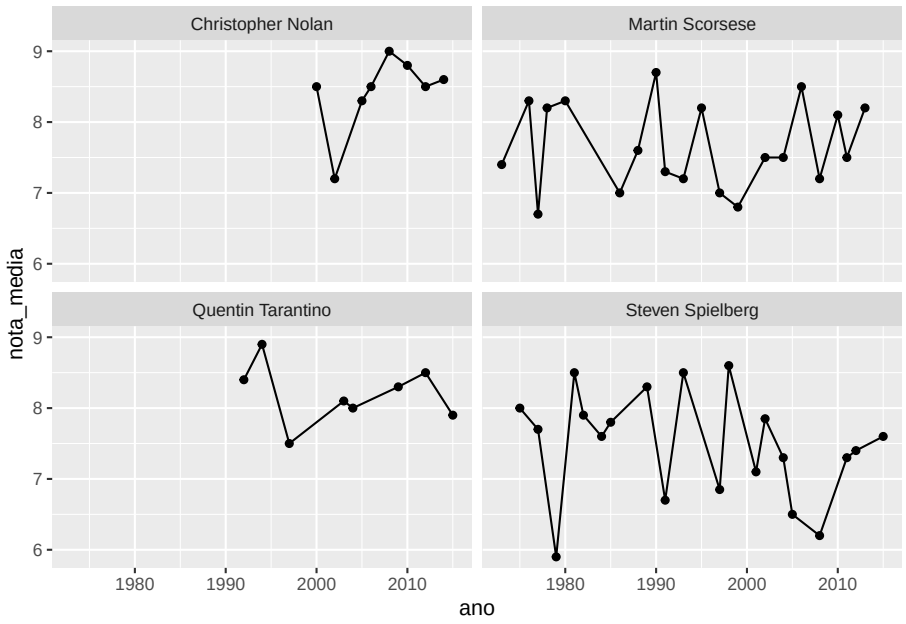
9. Descubra quais são os 5 atores que mais aparecem na coluna ator_1 e faça um boxplot do lucro dos filmes desses atores.

10. Com base no código abaixo, resolva os itens a seguir.

```
diretores <- c(
  "Steven Spielberg",
  "Quentin Tarantino",
  "Christopher Nolan",
  "Martin Scorsese"
)
```

```
imdb %>%
  filter(diretor %in% diretores) %>%
  group_by(ano, diretor) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = nota_media)) +
  geom_point() +
  geom_line() +
  facet_wrap(vars(diretor))
```

```
## `summarise()` has grouped output by 'ano'. You can override using the `.groups` argu
```



- **a.** Analisando o gráfico gerado, descreva o que a função `facet_wrap()` faz.
- **.b** Utilize os argumentos nrow e `ncol` da função `facet_wrap()` para colocar os quatro gráficos em uma única coluna.

11. Resovla os itens a seguir para fazer um gráfico de barras da frequência de filmes com nota maior que 8 ao longo dos anos.

- **a.** Crie uma nova coluna na base IMDB indicando se a nota de um filme é maior que 8 ou não.

- **b.** b. Utilizando a coluna criada em (a) crie uma tabela com o número anual de filmes com nota maior 8.

- **c.** Utilize a tabela criada em (b) para fazer um gráfico de barras do número de filmes com nota maior que 8 ao longo dos anos.

8.2 Extensões do pacote ggplot2

O site ggplot2 extensions tem uma galeria com diversos pacotes que deixam o ggplot2 bem mais poderoso, indo de novos geoms até animações.

A seguir, listamos algumas das mais famosas.

8.2.1 gghighlight

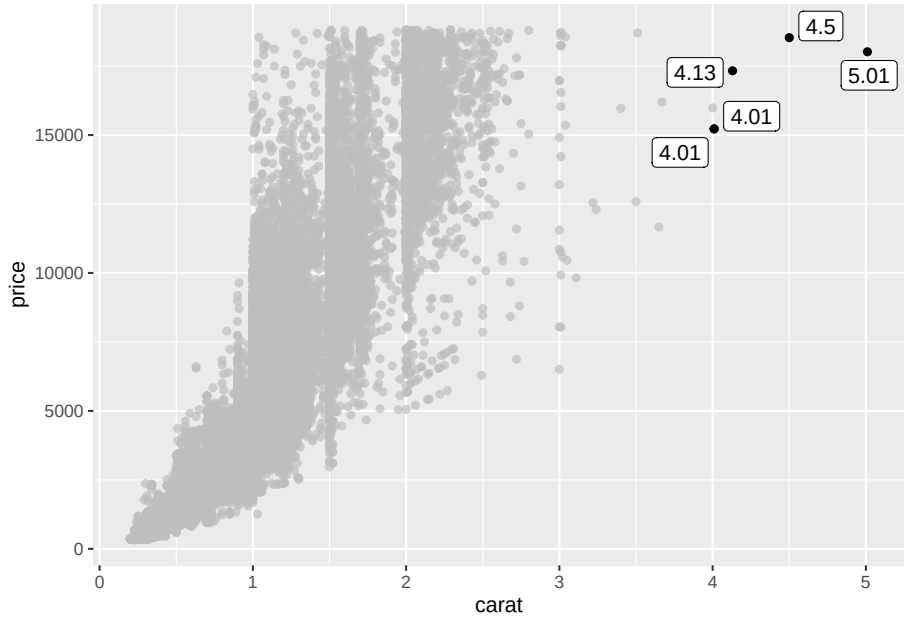
O `gghighlight` é uma extensões do `{ggplot2}` que serve para realçar partes de um gráfico feito com `ggplot`.

A seguir, mostramos como utilizar essa extensão pare realçar gráficos de pontos e linhas.

Realçando pontos

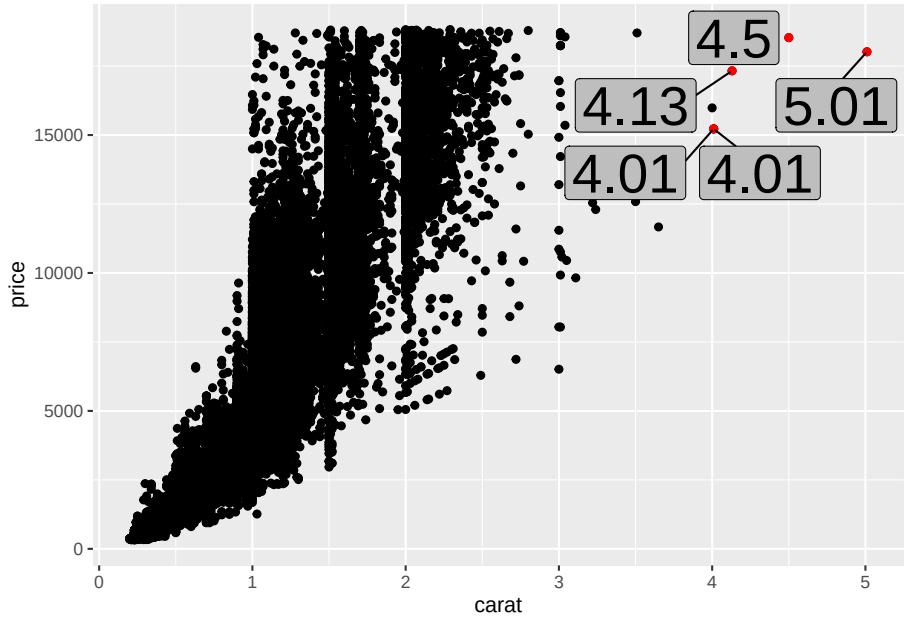
Muitas vezes temos um gráfico de dispersão em que queremos realçar alguns pontos de acordo com alguma característica. Por exemplo, abaixo estamos realçando os pontos que possuem `carat > 4`, além disso colocamos uma label em cada um.

```
diamonds %>%
  ggplot(aes(x = carat, y = price)) +
  geom_point() +
  gghighlight::gghighlight(carat > 4, label_key = carat)
```



Também é possível configurar a cor dos pontos que serão realçados e dos que não serão, bem como o estilo das labels.

```
diamonds %>%
  ggplot(aes(x = carat, y = price)) +
  geom_point(color = "red") +
  gghighlight::gghighlight(
    carat > 4,
    label_key = carat,
    unhighlighted_params = list(colour = "black"),
    label_params = list(size = 10, fill = "grey")
  )
```



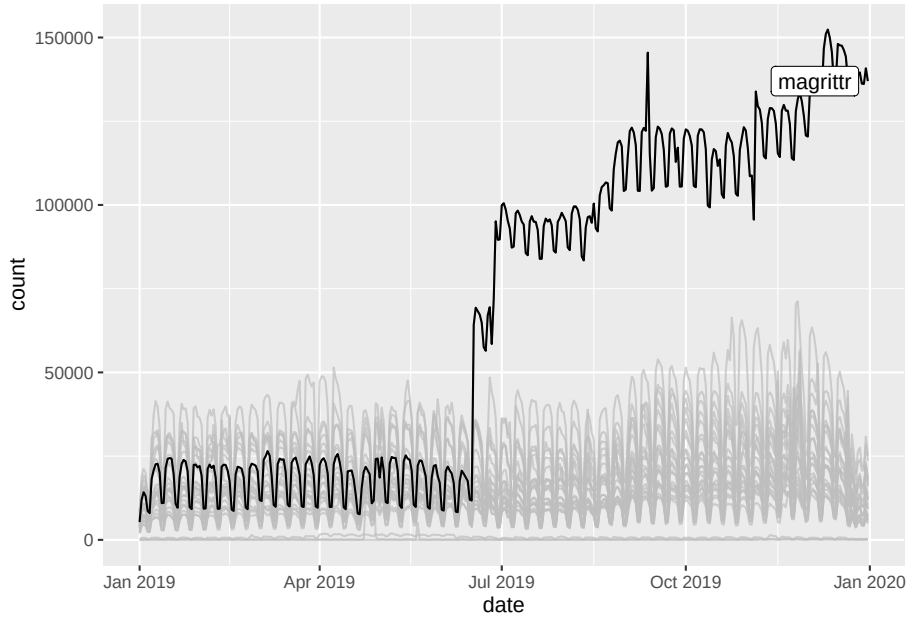
Realçando linhas

Com o `{gghighlight}` também é possível realçar linhas em um gráfico que possui varias linhas. Isso é interessante quando você quer ver como uma série temporal se compara com relação à um conjunto de outras séries.

No gráfico a seguir mostramos o número de downloads de cada um dos pacotes do tidyverse no ano de 2019. Uma das séries se destaca por mudar de padrão no meio do ano. Usamos o `{gghighlight}` para destacá-la no gráfico.

```
tab <- cranlogs::cran_downloads(
  packages = tidyverse::tidyverse_deps()$package,
  from = "2019-01-01", to = "2019-12-31"
)

tab %>%
  ggplot(aes(x = date, y = count, group = package)) +
  geom_line() +
  gghighlight::gghighlight(max(count) > 100000, label_key = package)
```



Para mais informações sobre o `{gghighlight}`, recomendamos a leitura da excelente documentação oficial.

8.2.2 ggridges

A extensão `{ggridges}` é uma ótima alternativa para histogramas e boxplots, quando queremos comparar a distribuição de uma variável A em vários níveis de uma variável B.

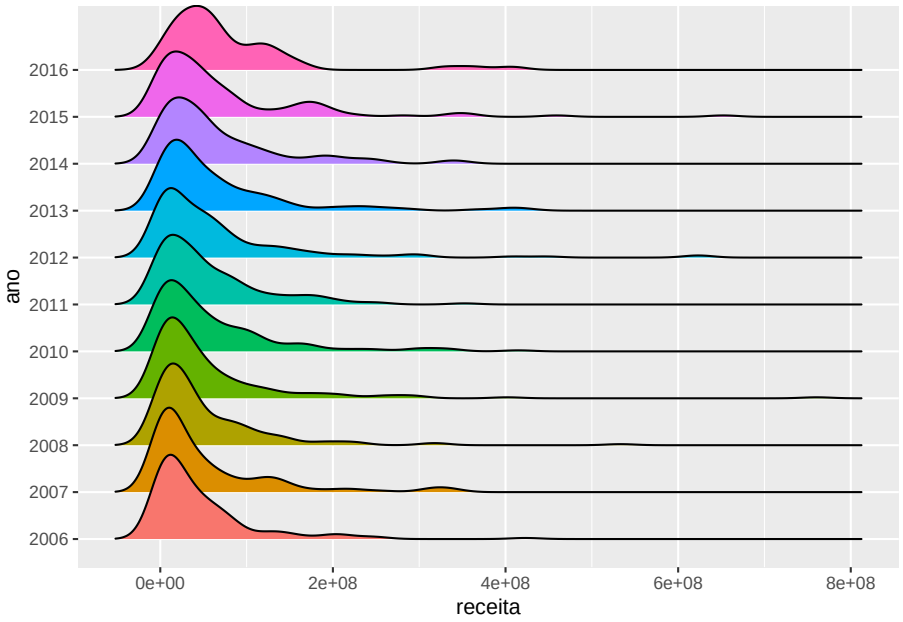
Primeiro, instale o pacote.

```
install.packages("ggridges")
```

No gráfico abaixo, comparamos a distribuição da receita dos filmes em cada um dos anos de 2005 a 2016. Para isso, utilizamos o novo `geom_density_ridges()`, disponibilizado pelo pacote `{ggridges}`.

```
library(ggridges)
imdb %>%
  filter(ano > 2005) %>%
  mutate(ano = as.factor(ano)) %>%
  ggplot(aes(y = ano, x = receita, fill = ano)) +
  geom_density_ridges(na.rm = TRUE, show.legend = FALSE)
```

```
## Picking joint bandwidth of 17300000
```



Você pode baixar a base IMDB utilizada clicando aqui.

Chapter 9

Relatórios

Depois de fazer toda a nossa análise dentro do R, não é justamente na hora de mostrá-la ao mundo que vamos abandonar as vantagens de estarmos dentro de uma linguagem de programação.

Com a ajuda do R Markdown, uma interface de anotações que combina texto livre com códigos em R, podemos construir relatórios não só reprodutíveis, mas elegantemente bem formatados, sem sair do R!

Figure 9.1: Ilustração por Allison Horst.

Além do próprio `{rmarkdown}`, existem diversos pacotes que podem ser utilizados para ampliar os produtos que elaboramos com R Markdown. Podemos criar/escrever:

- blogs e sites — blogdown e distill;
- livros (como este!) — bookdown;
- apresentações — xaringan;
- currículos, cartas e trabalhos acadêmicos — pagedown;
- tutoriais interativos — learnr;
- artigos científicos — rticles.

Antes de mais nada, vamos instalar os pacotes `{rmarkdown}` e `{knitr}`. O primeiro reune todas as funcionalidades para juntar nossos textos narrativos e códigos de R. O segundo vai fazer a *magia* de transformar nossos simples arquivos de texto em arquivos HTML, PDF e Word (`.doc`) bonitões.

```
install.packages(c("rmarkdown", "knitr"))
```

Nas seções a seguir, vamos mostrar como começar a utilizar o R Markdown para criar esses documentos. Não se esqueça de conferir (e deixar por perto) as folhas de cola sobre R Markdown:

- `rmarkdown` cheatsheet
- `rmarkdown` cheatsheet 2.0

9.1 Markdown

Se você é uma pessoa que utiliza R, sabe das possibilidades de utilizar R Markdown (e os pacotes que expandem mais ainda as possibilidades) e gostaria de começar a utilizá-lo, é importante conhecer o Markdown. Por quê? O R Markdown tem como base a linguagem de marcação Markdown.

Os arquivos markdown podem ser abertos por qualquer software que suporte este formato aberto. Além disso, independente da plataforma de trabalho, pode-se migrar para um arquivo de texto sem perder a formatação. O Markdown também é usado em outros lugares, como no GitHub e no Fórum da Curso-R.

Uma referência interessante para ter sempre em mãos é a Folha de cola (*Cheatsheet*) do Markdown.

Nas seções seguintes, descreveremos como podemos marcar os nossos textos e códigos usando `markdown`, e você poderá usar isso tanto em arquivos `.Rmd`, quanto em outros lugares que também utilizam essa marcação (como no Fórum da Curso-R!).

9.1.1 Ênfase

Negrito

Para destacar um texto em negrito, coloque `**` ou `__` ao redor do texto.

Por exemplo:

Como é escrito no código	Como aparece no relatório
`Esse é um texto com uma palavra destacada em **negrito**.`	Esse é um texto com uma palavra destacada em **negrito**.
`Esse é um texto com uma palavra destacada em __negrito__.`	Esse é um texto com uma palavra destacada em **negrito**.

Itálico

Para destacar um texto em itálico, coloque `*` ou `_` ao redor do texto.

Por exemplo:

Como é escrito no código	Como aparece no relatório
`Esse é um texto com uma palavra destacada em *itálico*.`	Esse é um texto com uma palavra destacada em *itálico*.
`Esse é um texto com uma palavra destacada em _itálico_.`	Esse é um texto com uma palavra destacada em *itálico*.

Riscado (ou tachado)

Para riscar/tachar um texto, coloque `~~` ao redor do texto.

Por exemplo:

Esse é um texto com uma palavra ~~riscada/tachada~~.

Como é escrito no código	Como aparece no relatório
`Esse é um texto com uma palavra ~~riscada/tachada~~.`	Esse é um texto com uma palavra ~~riscada/tachada~~.

9.1.2 Títulos

Os títulos funcionam como uma hierarquia, e para criar um título é necessário colocar um `#` no início da linha. Então um `#` marca um título, `##` marca um sub-título, e assim sucessivamente. Veja os exemplos:

Como é escrito no código	Como aparece no relat
`# Título 1 ## Título 2 ### Título 3`	

9.1.3 Listas

Listas ordenadas

Você pode fazer uma lista ordenada usando somente números. Você pode repetir o número quantas vezes quiser:

Como é escrito no código:

```
1. Maçã
1. Banana
1. Uva
```

Como aparece no relatório:

1. Maçã
2. Banana
3. Uva

Listas não ordenadas

Você pode fazer uma lista não ordenada escrevendo com hífens ou asteriscos, como a seguir:

```
* Maçã
* Banana
* Uva

- Maçã
- Banana
- Uva
```

O resultado será:

- Maçã
- Banana
- Uva

Você também pode adicionar sub-itens na lista indicando a hierarquia através da identação no Markdown (dica: utilize a tecla `tab` do teclado):

- Frutas
  - Maçã
  - Banana
  - Uva

9.1.4 Equações

Você pode adicionar equações utilizando LaTeX. Você pode saber mais na página do Overleaf sobre expressões matemáticas. Além disso, existem geradores de equações online que ajudam a escrevâ-las em LaTeX, HTML, entre outras linguagens de marcação.

É possível centralizar a equação envolvendo o código com `$$`. Veja o exemplo abaixo:

Como é escrito no código	Como aparece no relatório
`$$y = \mu + \sum_{i=1}^p \beta_i x_i + \epsilon$$`	$y = \mu + \sum_{i=1}^{p} \beta_i x_i + \epsilon$

Também é possível adicionar a equação na mesma linha que o texto, envolvendo o código com `$`. Veja o exemplo abaixo:

Como é escrito no código	Como aparece no relatório
`Ou também na linha $y = \mu + \sum_{i=1}^p \beta_i x_i + \epsilon$ , junto ao texto!`	Ou também na linha $y = \mu + \sum_{i=1}^{p} \beta_i x_i + \epsilon$, junto ao texto!

9.1.5 Código

É possível **marcar textos para que fiquem formatados como códigos**, usando a crase: ‘

Mas atenção: o texto será **formatado como código**, porém não será executado!

Como é escrito no código	Como aparece no relatório
`` `mean(pinguins$massa_corporal, na.rm = TRUE)` ``	`mean(pinguins$massa_corporal, na.rm = TRUE)`

Também é possível delimitar um trecho maior de código, utilizando três crases.

Exemplo:

Como é escrito no código:

````
```
library(dados)
mean(pinguins$massa_corporal, na.rm = TRUE)
```
````

Como aparece no relatório:

```
library(dados)
mean(pinguins$massa_corporal, na.rm = TRUE)
```

9.1.6 Links

- Você pode criar um link utilizando esta estrutura:

`[texto](http://url-da-pagina.com)`.

Por exemplo:

Como é escrito no código	Como aparece no relatório
`Você pode consultar mais [materiais sobre R nesta página](https://curso-r.com/material/).`	Você pode consultar mais materiais sobre R nesta página.

9.1.7 Imagens

Você pode incluir uma imagem utilizando esta estrutura:

`![descricao da imagem](http://url-da-imagem.com).`

Não se esqueça: a descrição da imagem é importante para acessibilidade do conteúdo através de leitores de tela!

Como é escrito no código	Como aparece no relatório
`![Logo da Curso-R](assets/img/relatorios/logo_cursor_full.png)`	

9.1.8 Tabelas

As tabelas em Markdown têm uma estrutura definida, como mostra o exemplo abaixo.

É possível usar ferramentas online para gerar tabelas em Markdown, como por exemplo o Tables Generator.

Em Markdown:

```
|  a|  b|  c|
|--:|--:|--:|
|  1|  2|  3|
|  2|  3|  4|
```

Resultado:

a	b	c
1	2	3

a	b	c
2	3	4

9.1.9 Recapitulando

Nesta seção, aprendemos que o Markdown é uma linguagem de marcação, usada para marcar textos. Podemos usar o Markdown para formatar textos e adicionar elementos no nossos relatórios.

Porém existem elementos (como código, imagens, tabelas, entre outros) que podemos também adiconar nos relatórios utilizando R. É isso que veremos na seção seguinte!

Exercícios

1. Leia a primeira página o guia de referência do RMarkdown

2. Liste abaixo outros lugares que utilizam o Markdown como base (Dica: outras linguagens de programação, fóruns, aplicativos de mensagem, etc)

3. Qual é o output se você criar uma lista ordenada começando no 2? Por exemplo:

```
2. primeiro item

2. segundo item

2. terceiro item
```

4. O que são os *sabores* de Markdown (Markdown flavor)?

5. Experimente o pandoc no site de demonstração da ferramenta.

9.2 R Markdown

O R Markdown é a junção da linguagem Markdown com o poder de códigos em R. A mágica acontece em arquivos do tipo `.Rmd`, onde é possível adicionar textos, códigos, resultados de códigos e muito mais!

9.2.1 Criando um arquivo `.Rmd`

Falamos que mágica acontece em arquivos do tipo `.Rmd`, porém, como podemos criar esses arquivos? Primeiramente, no menu superior, clique em **File** > **New File** > **R Markdown**, ou utilize os botões da interface gráfica, como mostrado na imagem a seguir:

Uma janela será aberta, chamada **New R Markdown**. Neste momento, iremos focar na seção inicial (**Document**). Nessa janela, é possível informar o título do relatório, o nome das pessoas autoras, e o formato em que deseja criar o relatório (sendo HTML, PDF ou Word). Caso queira gerar um PDF, será necessário instalar o LaTeX (saiba como no capítulo sobre instalações adicionais).

Para facilitar o aprendizado, recomendamos que escolha a opção HTML, sendo que é possível alterar todos esses campos posteriormente. Para que o arquivo seja criado, é necessário clicar no botão **OK**.

9.2.2 Estrutura do arquivo R Markdown

Quando criamos um novo arquivo `.Rmd` (como descrito na seção anterior), o arquivo vem com alguns conteúdos para nos ajudar a organizar corretamente.

O arquivo `.Rmd` possui a seguinte estrutura, ilustrada pela figura a seguir:

- 1) **YAML** - Uma seção de metadados, que apresenta códigos `YAML`. Essa seção apresenta informações que serão usadas para gerar o arquivo final, como por exemplo: título do relatório, autoria, data, formato gerado, etc. É necessário cuidado ao editar essa seção, pois coisas simples como indentação incorreta, fechamento incorreto de textos, etc, podem resultar em um erro ao gerar seu relatório. Essa seção deve estar sempre ao início do documento, e é delimitado por três traços no começo e no final: `---`.
- 2) **Chunks** - Nos campos de código (também chamados de *chunks*) podemos adicionar códigos em R (ou em algumas outras linguagens). Os chunks são delimitados por três crases, e a linguagem deve ser especificada entre chaves. Caso queira adicionar chunks com código em Python, é necessário ter o pacote {reticulate} instalado. Exemplo de um chunk que apresenta código em R:

````
```{r}
1 + 1
```
````

- 3) **Markdown** - Textos marcados com Markdown podem ser adicionados ao longo do relatório, fora das demarcações do YAMl e dos

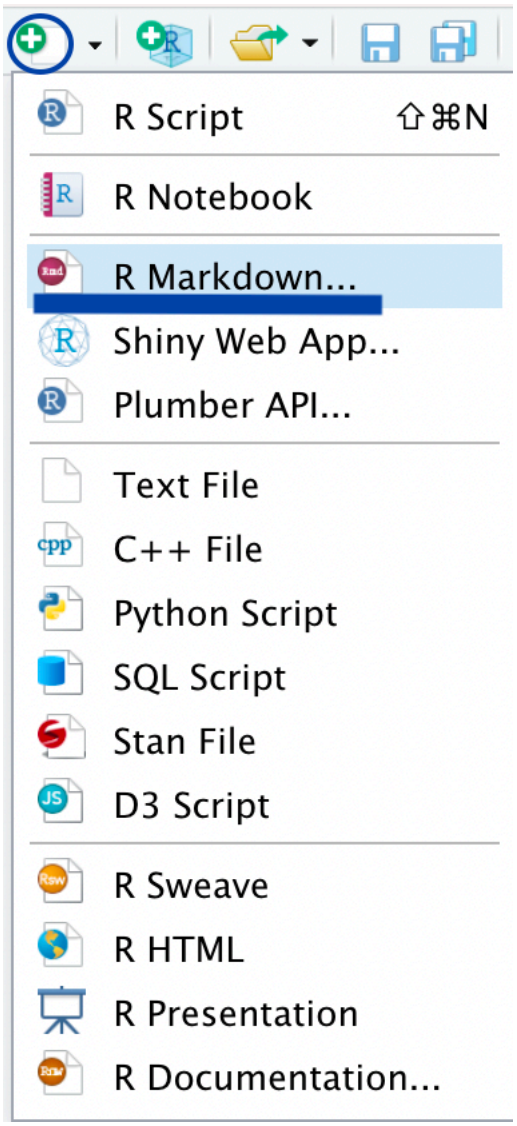


Figure 9.2: Captura de tela da IDE RStudio

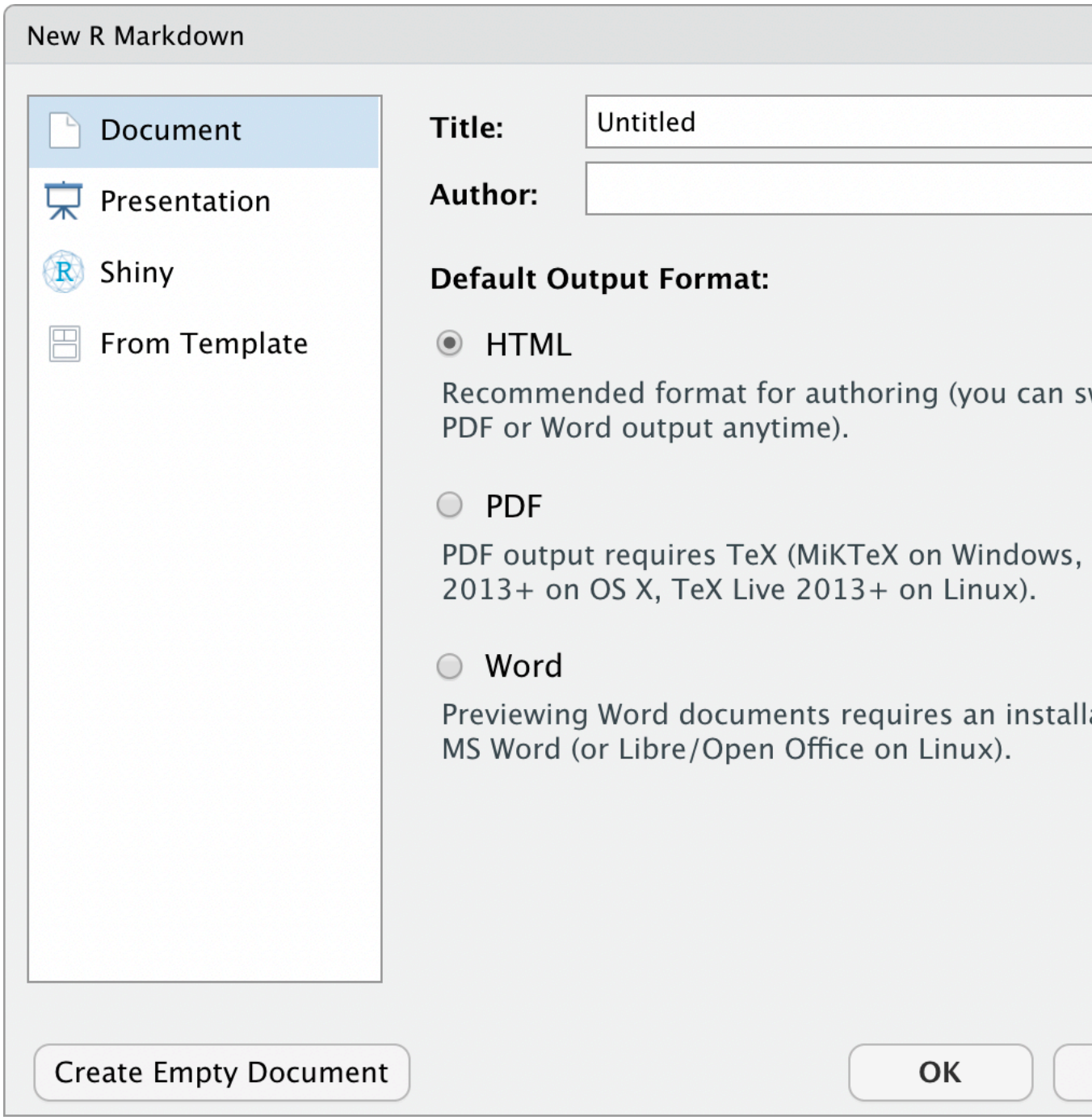


Figure 9.3: Captura de tela da IDE RStudio

Chunks. Caso queira saber mais sobre como marcar textos com Markdown, é possível ler mais na seção anterior deste capítulo.

9.2.3 Adicionando conteúdo

Nesta seção, utilizaremos como exemplos alguns códigos envolvendo a base de dados de pinguins. A versão original deste dataset (em Inglês) está disponível no pacote palmerpenguins, e a versão traduzida para Português está disponível no pacote dados.

Caso não tenha o pacote dados instalado, é possível instalar com o seguinte código:

```
install.packages("remotes")
remotes::install_github("cienciadedatos/dados")
```

Agora carregue os pacotes necessários:

```
library(dados)  # Pacote com bases de dados em Português
library(magrittr)  # Pacote para utilizar o Pipe %>%
```

9.2.3.1 Códigos em R

Ao analisar dados, escrevemos códigos que calculam estatísticas descritivas, geram gráficos e tabelas, entre outros. Para adicionar esses resultados nos relatórios, não é muito prático adicionar manualmente o resultado final (copiando e colando, por exemplo), pois em caso de alguma mudança na base de dados, será necessário gerar novamente estes resultados e adicioná-los novamente!

Com RMarkdown, podemos adicionar códigos nos nossos relatórios que geram os resultados, assim a cada vez que compilarmos o relatório os códigos que geram estes resultados serão executados, e o relatório terá sempre os resultados atualizados.

Caso você queira que o código seja executado como código R, existem duas possibilidades:

- **chunks de código** ou **campos de código** (*chunks* em Inglês), quando o código é executado em um campo de código em R.
- **código em linha** (ou '*inline code*' em Inglês), quando o resultado aparece junto ao texto;

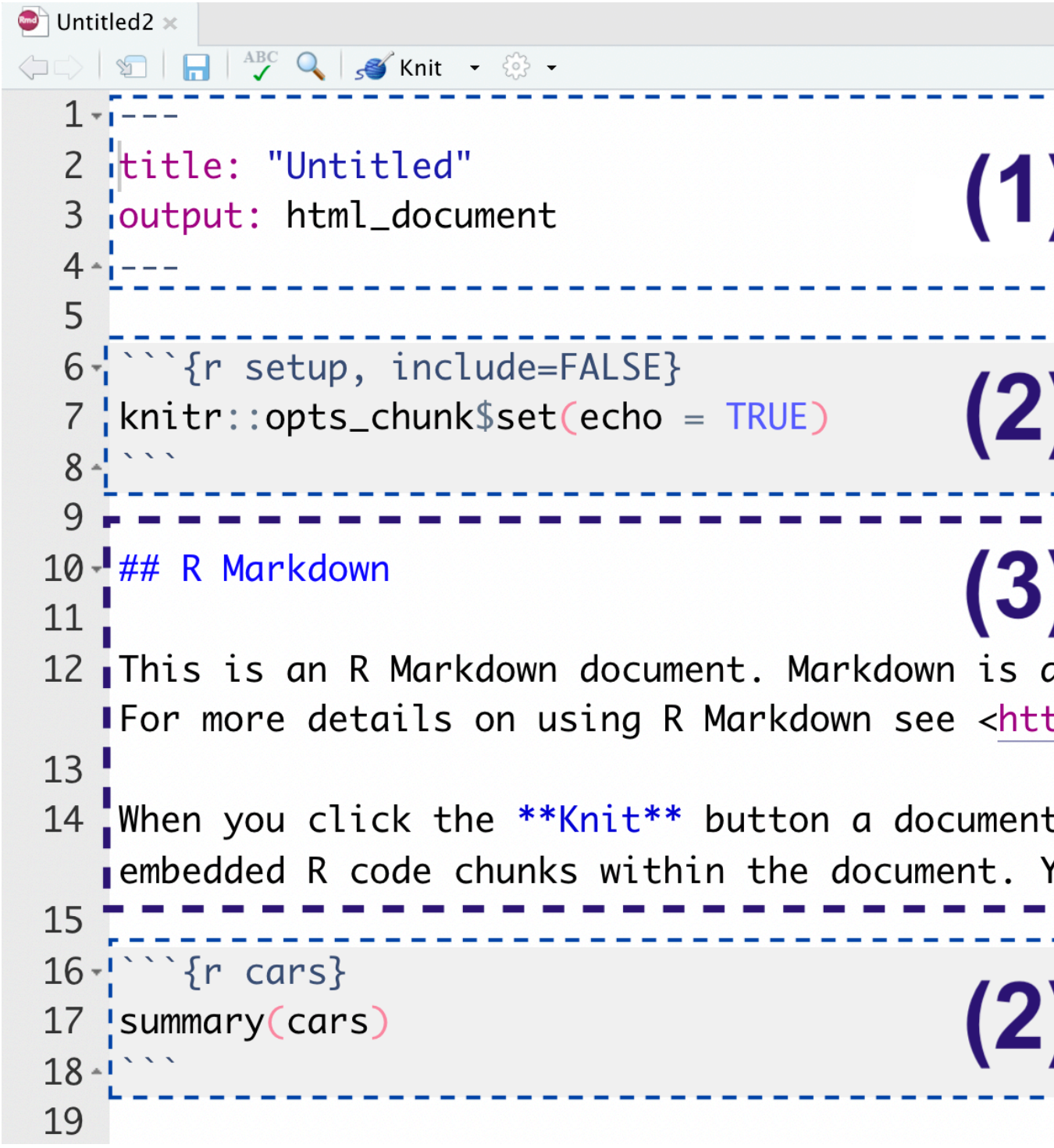


Figure 9.4: Captura de tela da IDE RStudio

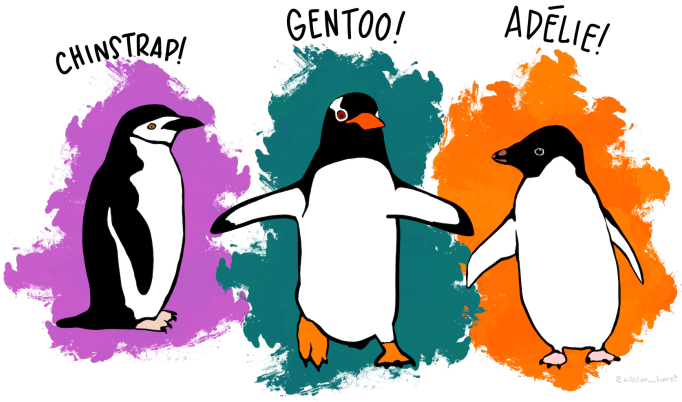


Figure 9.5: Conheça os Pinguins do dataset PalmerPenguins! Ilustração por Allison Horst.

Campos de código (chunks) É possível adicionar campos de código utilizando a seguinte sintaxe:

```
```{r}
codigo em R aqui
```
```

Um exemplo de campo de código é o campo abaixo, onde é possível executar o código e visualizar o resultado.

Exemplo:

```
```{r}
 # calcula a média
media_massa_g <-
  mean(pinguins$massa_corporal, na.rm = TRUE)

 # converte de gramas para kilogramas
media_massa_kg_completo <-
  media_massa_g / 1000

 # arredonda o valor para ter uma casa decimal
media_massa_kg <- round(media_massa_kg_completo, 1)

 # exibe o resultado
media_massa_kg
```
```

Resultado:

```
# calcula a média
media_massa_g <-
  mean(pinguins$massa_corporal, na.rm = TRUE)

# converte de gramas para kilogramas
media_massa_kg_completo <-
  media_massa_g / 1000

# arredonda o valor para ter uma casa decimal
media_massa_kg <- round(media_massa_kg_completo, 1)

# exibe o resultado
media_massa_kg
```

```
## [1] 4.2
```

Veja que no resultado obtivemos o código em R e também o resultado deste código. Dependendo do público alvo do relatório, não queremos mostrar códigos. É possível escondê-los (e fazer muito mais!) alterando as opções de chunks (ou *chunk options* em Inglês). Veremos mais sobre isso posteriormente!

Inline code Para que um resultado de código apareça ao longo do texto, é possivel usar a seguinte estrutura: `` `r ` ``.

Exemplo:

```
A média da massa corporal dos pinguins é `r media_massa_kg` kg.
```

Resultado:

A média da massa corporal dos pinguins é 4.2 kg.

9.2.3.2 Imagens

Na seção anterior (sobre Markdown), mostramos como é possóvel adicionar uma imagem em um relatório utilizando Markdown.

Porém, quando utilizamos arquivos RMarkdown também podemos adicionar imagens usando código em R, com a função `include_graphics` do pacote `knitr`.

Usando o `knitr` é possível personalizar de forma mais intuitiva como a imagem aparecerá no arquivo final: você pode adicionar uma imagem

utilizando como primeiro argumento a url da imagem ou o caminho onde ela está no projeto. Além disso, com as opções de chunks (ou *chunk options* em Inglês), poderemos personalizar diversos atributos da imagem, como: legenda, tamanho, alinhamento, entre outros.

Veja o exemplo abaixo:

```
knitr::include_graphics("assets/img/relatorios/dados-hex.png")
```

9.2.3.3 Tabelas

Na seção anterior (sobre Markdown), foi apresentado como adicionar uma tabela utilizando Markdown.

Porém quando utilizamos arquivos R Markdown também é possível adicionar tabelas que resultam de códigos em R.

Existem algumas funções diferentes para fazer isso. Para apresentar alguns exemplos, primeiramente vamos salvar uma versão da base pinguins:

```
media_massa <- pinguins %>% # usamos a base de pinguins
  # agrupamos por especie e ilhas
  dplyr::group_by(especie, ilha) %>%
  # calculamos a média da massa corporal
  dplyr::summarise(media_massa = mean(massa_corporal, na.rm = TRUE)) %>%
  # criamos uma nova coluna, com a massa em kg, e arredondada com 2 casas decimais
  dplyr::mutate(media_massa_kg = round(media_massa / 1000, 2)) %>%
  # removemos a coluna da massa em gramas
  dplyr::select(-media_massa)

# Apresentamos o resultado da tabela:
media_massa
```

```
## # A tibble: 5 x 3
```

```
## # Groups:   especie [3]
##   especie             ilha      media_massa_kg
##   <fct>               <fct>              <dbl>
## 1 Pinguim-de-adélia   Biscoe              3.71
## 2 Pinguim-de-adélia   Dream               3.69
## 3 Pinguim-de-adélia   Torgersen           3.71
## 4 Pinguim-de-barbicha Dream               3.73
## 5 Pinguim-gentoo      Biscoe              5.08
```

Acima, a tabela é apresentada como ela é escrita no console do R. Porém essa forma não é elegante de ser apresentada em um relatório! Podemos utilizar funções de diferentes pacotes para que as tabelas fiquem formatadas de uma forma mais atraente:

- Utilizando o pacote `knitr`: função `kable()`

```
media_massa %>%
  knitr::kable()
```

especie	ilha	media_massa_kg
Pinguim-de-adélia	Biscoe	3.71
Pinguim-de-adélia	Dream	3.69
Pinguim-de-adélia	Torgersen	3.71
Pinguim-de-barbicha	Dream	3.73
Pinguim-gentoo	Biscoe	5.08

- Utilizando o pacote `DT`: função `datatable()`

```
media_massa %>%
  DT::datatable()
```

Show 10 entries Search:

	especie	ilha	media_massa_kg
1	Pinguim-de-adélia	Biscoe	3.71
2	Pinguim-de-adélia	Dream	3.69
3	Pinguim-de-adélia	Torgersen	3.71
4	Pinguim-de-barbicha	Dream	3.73
5	Pinguim-gentoo	Biscoe	5.08

Showing 1 to 5 of 5 entries Previous 1 Next

Os exemplos anteriores são as formas mais simples de apresentar as tabelas, porém cada pacote possibilita diferentes personalizações (por exemplo, alterar o nome das colunas, etc). Existem outros pacotes que permitem adicionar tabela nos relatórios em RMarkdown, como por exemplo:

- gt
- reactable
- flextable
- kableExtra

9.2.4 Dicas para o RStudio

A versão mais recente do RStudio possui uma ferramenta de edição de Markdown visual, que disponibiliza botões para editar o texto com Markdown.

Veja mais neste post no blog da RStudio, e também a animação disponível neste post:

Exercícios

1. Leia o guia de referência do RMarkdown até o final.

2. Qual o propósito do YAML do documento?

3. Quais são as opções e resultados do parâmetro `results=`?

````
```{r results='markup'}
knitr::kable(head(cars))
```
````

Chapter 10

Programação funcional (purrr)

Programação funcional (PF) é um paradigma de programação com o qual a maior parte dos estatísticos não está familiarizada. Essa técnica costuma ser ignorada na maioria dos tutoriais de R por não estar diretamente envolvida com manipulação e visualização de dados, mas isso não quer dizer que ela não tenha suas vantagens.

Usando programação funcional podemos criar códigos mais concisos e *pipeáveis*, características que por tabela também tornam mais simples o processo de encontrar erros. Além disso, códigos funcionais geralmente são paralelizáveis, permitindo que tratemos problemas muito grandes com poucas modificações.

Apesar de o R `base` já ter funções que podem ser consideradas elementos de PF, a implementação destas não é tão elegante e, portanto, este tutorial abordará somente a implementação de PF realizada pelo pacote `purrr`.

Para instalar e carregar o `purrr`, rode o código a seguir. Nas próximas seções deste tutorial, assumiremos que você têm esse pacote instalado e carregado.

```
install.packages("purrr")
library(purrr)
```

10.1 Iterações básicas

A primeira família de funções do `purrr` que veremos também é a mais útil e extensível. As funções `map()` são quase como substitutas para laços `for`, elas

abstraem a iteração em apenas uma linha. Veja esse exemplo de laço usando `for`:

```
soma_um <- function(x) { x + 1 }
obj <- 10:15
for (i in seq_along(obj)) {
  obj[i] <- soma_um(obj[i])
}
obj
```

```
## [1] 11 12 13 14 15 16
```

O que de fato estamos tentando fazer com o laço acima? Temos um vetor (`obj`) e queremos aplicar uma função (`soma_um()`) em cada elemento dele. A função `map()` remove a necessidade de declaramos um objeto iterador auxiliar (`i`) e simplesmente aplica a função desejada em cada elemento do objeto dado.

```
soma_um <- function(x) { x + 1 }
obj <- 10:15
obj <- map(obj, soma_um)
obj
```

```
## [[1]]
## [1] 11
##
## [[2]]
## [1] 12
##
## [[3]]
## [1] 13
##
## [[4]]
## [1] 14
##
## [[5]]
## [1] 15
##
## [[6]]
## [1] 16
```

Como você deve ter percebido, o resultado da execução acima não é exatamente igual ao que tivemos com o laço. Isso acontece porque a `map()` tenta ser extremamente genérica, retornando por padrão uma lista com um elemento para cada saída.

Se quisermos "achatar" o resultado, devemos informar qual será o seu tipo. Isso é super simples e pode ser feito com as irmãs da `map()`: `map_chr()` (para strings), `map_dbl()` (para números reais), `map_int()` (para números inteiros) e `map_lgl()` (para booleanos).

```
obj <- 10:15
map_dbl(obj, soma_um)
```

```
## [1] 11 12 13 14 15 16
```

> O `purrr` também nos fornece outra ferramenta interessante para achatar listas: a família `flatten()`. No fundo, `map_chr()` é quase um atalho para `map() %>% flatten_chr()`!

Algo bastante útil da família `map()` é a possibilidade de passar argumentos fixos para a função que será aplicada. A primeira forma de fazer isso envolve fórmulas:

```
soma_n <- function(x, n = 1) { x + n }
obj <- 10:15
map_dbl(obj, ~soma_n(.x, 2))
```

```
## [1] 12 13 14 15 16 17
```

Como vemos no exemplo acima, para utilizar fórmulas precisamos colocar um til (`~`) antes da função que será chamada. Feito isso, podemos utilizar o placeholder `.x` para indicar onde deve ser colocado cada elemento de `obj`.

A outra forma de passar argumentos para a função é através das reticências da `map()`. Desta maneira precisamos apenas dar o nome do argumento e seu valor logo após a função `soma_n()`.

```
soma_n <- function(x, n = 1) { x + n }
obj <- 10:15
map_dbl(obj, soma_n, n = 2)
```

```
## [1] 12 13 14 15 16 17
```

Usando fórmulas temos uma maior flexibilidade (podemos, por exemplo, declarar funções anônimas como `~.x+2`), enquanto com as reticências temos maior legibilidade.

10.2 Iterações intermediárias

Agora que já exploramos os básicos da família `map()` podemos partir para iterações um pouco mais complexas. Observe o laço a seguir:

```
soma_ambos <- function(x, y) { x + y }
obj_1 <- 10:15
obj_2 <- 20:25
for (i in seq_along(obj_1)) {
  obj_1[i] <- soma_ambos(obj_1[i], obj_2[i])
}
obj_1
```

```
## [1] 30 32 34 36 38 40
```

Com a função `map2()` podemos reproduzir o laço acima em apenas uma linha. Ela abstrai a iteração em paralelo, aplica a função em cada par de elementos das entradas e, assim como sua prima `map()`, pode achatar o objeto retornado com os sufixos `_chr`, `_dbl`, `_int` e `_lgl`.

> O termo "paralelo" neste capítulos se refere a laços em mais de uma estrutura e não a paralelização de computações em mais de uma unidade de processamento.

```
soma_ambos <- function(x, y) { x + y }
obj_1 <- 10:15
obj_2 <- 20:25
obj_1 <- map2_dbl(obj_1, obj_2, soma_ambos)
obj_1
```

```
## [1] 30 32 34 36 38 40
```

Como o pacote `purrr` é extremamente consistente, a `map2()` também funciona com reticências e fórmulas. Poderíamos, por exemplo, transformar `soma_ambos()` em uma função anônima:

```
obj_1 <- 10:15
obj_2 <- 20:25
map2_dbl(obj_1, obj_2, ~.x+.y)
```

```
## [1] 30 32 34 36 38 40
```

Desta vez também temos acesso ao placeholder `.y` para indicar onde os elementos de do segundo vetor devem ir.

Para não precisar oferecer uma função para cada número de argumentos, o pacote `purrr` fornece a `pmap()`. Para essa função devemos passar uma lista em que cada elemento é um dos objetos a ser iterado:

```
soma_varios <- function(x, y, z) { x + y + z }
obj_1 <- 10:15
obj_2 <- 20:25
obj_3 <- 30:35
obj_1 <- pmap_dbl(list(obj_1, obj_2, obj_3), soma_varios)
obj_1
```

```
## [1] 60 63 66 69 72 75
```

Com a `pmap()` infelizmente não podemos usar fórmulas. Se quisermos usar uma função anônima com ela, precisamos declará-la a função no seu corpo:

```
obj_1 <- 10:15
obj_2 <- 20:25
obj_3 <- 30:35
pmap_dbl(list(obj_1, obj_2, obj_3), function(x, y, z) { x + y + z })
```

```
## [1] 60 63 66 69 72 75
```

A última função que veremos nessa seção é a `imap()`. No fundo ela é um atalho para `map2(x, names(x), ...)` quando `x` tem nomes e para `map2(x, seq_along(x), ...)` caso contrário:

```
obj <- 10:15
imap_dbl(obj, ~.x+.y)
```

```
## [1] 11 13 15 17 19 21
```

Como podemos observar, agora `.y` é o placeholder para o índice atual (equivalente ao `i` no laço com `for`). Naturalmente, assim como toda a família `map()`, a `imap()` também funciona com os sufixos de achatamento.

10.3 Iterações avançadas

Agora que já vimos como substituir iterações de nível básico e de nível intermediário com a família `map()`, podemos passar para os tipos mais obscuros de laços. Cada item desta seção será mais denso do que os das passadas, por isso encorajamos todos os leitores para que também leiam a documentação de cada função aqui abordada.

10.3.1 Iterações com condicionais

Imagine que precisamos aplicar uma função somente em alguns elementos de um vetor. Com um laço isso é uma tarefa fácil, mas com as funções da família `map()` apresentadas até agora isso seria extremamente difícil. Veja o trecho de código a seguir por exemplo:

```
dobra <- function(x) { x*2 }
obj <- 10:15
for (i in seq_along(obj)) {
  if (obj[i] %% 2 == 1) { obj[i] <- dobra(obj[i]) }
  else                  { obj[i] <- obj[i] }
}
obj
```

```
## [1] 10 22 12 26 14 30
```

No exemplo acima, aplicamos a função `dobra()` apenas nos elementos ímpares do vetor `obj`. Com o pacote `purrr` temos duas maneiras de fazer isso: com `map_if()` ou `map_at()`.

A primeira dessas funções aplica a função dada apenas quando um predicado é `TRUE`. Esse predicado pode ser uma função ou uma fórmula (que serão aplicadas em cada elemento da entrada e devem retornar `TRUE` ou `FALSE`). Infelizmente a `map_if()` não funciona com sufixos, então devemos achatar o resultado:

```
eh_impar <- function(x) { x%%2 == 1 }
dobra <- function(x) { x*2 }
obj <- 10:15
map_if(obj, eh_impar, dobra) %>% flatten_dbl()
```

```
## [1] 10 22 12 26 14 30
```

Com fórmulas poderíamos eliminar completamente a necessidade de funções declaradas:

```
obj <- 10:15
map_if(obj, ~.x%%2 == 1, ~.x*2) %>% flatten_dbl()
```

```
## [1] 10 22 12 26 14 30
```

A segunda dessas funções é a irmã gêmea de `map_if()` e funciona de forma muito semelhante. Para `map_at()` devemos passar um vetor de nomes ou índices onde a função deve ser aplicada:

```
obj <- 10:15
map_at(obj, c(2, 4, 6), ~.x*2) %>% flatten_dbl()
```

```
## [1] 10 22 12 26 14 30
```

10.3.2 Iterações com tabelas e funções

Duas funções menos utilizadas da família `map()` são `map_dfc()` e `map_dfr()`, que equivalem a um `map()` seguido de um `dplyr::bind_cols()` ou de um `dplyr::bind_rows()` respectivamente.

> A maior utilidade dessas funções é quando temos uma tabela espalhada em muitos arquivos. Se elas estiverem divididas por grupos de colunas, podemos usar algo como `map_dfc(arquivos, readr::read_csv)` e se elas estiverem divididas por grupos de linhas, `map_dfr(arquivos, readr::read_csv)`

Outro membro obscuro da família `map()` é a `invoke_map()`. Na verdade essa função pode ser considerada um membro da família `invoke()`, mas vamos ver que as semelhanças são muitas. Primeiramente, vamos demonstrar o que faz a `invoke()` sozinha:

```
soma_ambos <- function(x, y) { x + y }
invoke(soma_ambos, list(x = 10, y = 15))
```

```
## [1] 25
```

É fácil de ver que essa função recebe uma função e uma lista de argumentos para usar em uma chamada desta. Agora generalizando esta lógica temos `invoke_map()`, que chama uma mesma função com uma lista de listas de argumentos ou uma lista de funções com uma lista de argumentos. A família `invoke()` também aceita os sufixos como veremos a seguir:

```
soma_ambos <- function(x, y) { x + y }
soma_um <- function(x) { x + 1 }
soma_dois <- function(x) { x + 2 }
invoke_map_dbl(soma_ambos, list(list(x = 10, y = 15), list(x = 20, y = 25)))
```

```
## [1] 25 45
```

```
invoke_map_dbl(list(soma_um, soma_dois), list(x = 10))
```

```
## [1] 11 12
```

10.3.3 Redução e acúmulo

Outras funções simbólicas de programação funcional além da `map()` são `reduce` e `accumulate`, que aplicam transformações em valores acumulados. Observe o laço a seguir:

```
soma_ambos <- function(x, y) { x + y }
obj <- 10:15
for (i in 2:length(obj)) {
  obj[i] <- soma_ambos(obj[i-1], obj[i])
}
obj
```

```
## [1] 10 21 33 46 60 75
```

Essa soma cumulativa é bastante simples, mas não é difícil imaginar uma situação em que um programador desavisado confunde um índice com o outro e o *bug* acaba passando desapercebido. Para evitar esse tipo de situação, podemos utilizar `accumulate()` (tanto com uma função quanto com uma fórmula):

```
soma_ambos <- function(x, y) { x + y }
obj <- 10:15
accumulate(obj, soma_ambos)
```

```
## [1] 10 21 33 46 60 75
```

```
accumulate(obj, ~.x+.y)
```

```
## [1] 10 21 33 46 60 75
```

Obs.: Nesse caso, os placeholders têm significados ligeiramente diferentes. Aqui, `.x` é o valor acumulado e `.y` é o valor "atual" do objeto sendo iterado.

Se não quisermos o valor acumulado em cada passo da iteração, podemos usar `reduce()`:

```
obj <- 10:15
reduce(obj, ~.x+.y)
```

```
## [1] 75
```

Para a nossa comodidade, essas duas funções também têm variedades paralelas (`accumulate2()` e `reduce2()`), assim como variedades invertidas `accumulate_right()` e `reduce_right()`).

10.4 Miscelânea

Por fim, veremos algumas funções do `purrr` que nêo têm exatamente a ver com laços, mas que acabam sendo bastante úteis quando usando as funções que vimos até agora. Elas não serão apresentadas em nenhuma ordem específica, este é apenas um apanhado de funções sortidas que achamos úteis enquanto programando com o `purrr`.

10.4.1 Manter e descartar

Se quisermos filtrar elementos de um vetor ou lista, podemos usar as funções `keep()` e `discard()`. Elas funcionam com fórmulas e podem ser extremamente úteis em situações que `dplyr::select()` e `magrittr::extract()` não conseguem cobrir:

```
obj <- list(10:15, 20:25, c(30:34, NA))
keep(obj, ~any(is.na(.x)))
```

```
## [[1]]
## [1] 30 31 32 33 34 NA
```

```
discard(obj, ~!any(is.na(.x)))
```

```
## [[1]]
## [1] 30 31 32 33 34 NA
```

No exemplo acima descartamos todos os vetores da lista que não têm pelo menos um elemento omisso (`NA`).

10.4.2 A família `is`

Uma outra família do pacote `purrr` é a `is()`. Com essa série de funções podemos fazer verificações extremamente estritas em objetos dos mais variados tipos. Seguem alguns poucos exemplos:

```
is_scalar_integer(10:15)
```

```
## [1] FALSE
```

```
is_bare_integer(10:15)
```

```
## [1] TRUE
```

```
is_atomic(10:15)
```

```
## [1] TRUE
```

```
is_vector(10:15)
```

```
## [1] TRUE
```

10.4.3 Andar e modificar

`walk()` e `modify()` são pequenas alterações da família `map()` que vêm a calhar em diversas situações. A primeira destas funciona exatamente igual à `map()` mas não devolve resultado, apenas efeitos colaterais; a segunda, não muda a estrutura do objeto sendo iterado, ela substitui os próprios elementos da entrada.

> A maior utilidade de `walk` é quando precisamos salvar múltiplas tabelas. Para fazer isso, podemos usar algo como `walk(tabelas, readr::write_csv)`.

Um caso de uso interessante da `modify()` é ao lado do sufixo `_if()`, combinação que nos permite iterar nas colunas de uma tabela e aplicar transformações de tipo apenas quando um predicado for verdade (geralmente de queremos transformar as colunas de fator para caractere).

10.4.4 Transposição e indexação profunda

Quando precisarmos lidar com listas complexas e profundas, o `purrr` nos fornece duas funções extremamente úteis: `transpose()` e `pluck()`. A primeira transpõe uma lista, enquanto a segunda é capaz de acessar elementos profundos de uma lista sem a necessidade de colchetes.

```
obj <- list(list(a = 1, b = 2, c = 3), list(a = 4, b = 5, c = 6))
str(obj)
```

```
## List of 2
##  $ :List of 3
##   ..$ a: num 1
##   ..$ b: num 2
##   ..$ c: num 3
##  $ :List of 3
##   ..$ a: num 4
##   ..$ b: num 5
##   ..$ c: num 6
```

```
pluck(obj, 2, "b")
```

```
## [1] 5
```

```
str(transpose(obj))
```

```
## List of 3
##  $ a:List of 2
##   ..$ : num 1
##   ..$ : num 4
##  $ b:List of 2
##   ..$ : num 2
##   ..$ : num 5
##  $ c:List of 2
##   ..$ : num 3
##   ..$ : num 6
```

Obs.: Se você estiver com muitos problemas com listas profundas, dê uma olhada nas funções relacionadas a `depth()` pois elas podem ser muito úteis.

10.4.5 Aplicação parcial

Se quisermos pré-preencher os argumentos de uma função (seja para usá-la em uma pipeline ou com alguma função do próprio `purrr`), temos `partial()`. Ela

funciona nos moldes da família `invoke()` e pode ser bastante útil para tornar suas pipelines mais enxutas:

```
soma_varios <- function(x, y, z) { x + y + z }
nova_soma <- partial(soma_varios, x = 1, y = 2)
nova_soma(3)
```

```
## [1] 6
```

10.4.6 Execução segura

Não é incomum executarmos uma função e recebermos um erro de volta. Isso pode ser lidado com facilidade em um laço com um condicional, mas essa tarefa já é mais complexa quando se trata de programação funcional. Para isso, no `purrr` temos algumas funções que embrulham uma função e, quando esta retornar um erro, o silenciam e retornam um valor padrão em seu lugar.

`quietly()` retorna uma lista com resultado, saída, mensagem e alertas, `safely()` retorna uma lista com resultado e erro (um destes sempre é `NULL`), e `possibly()` silencia o erro e retorna um valor dado pelo usuário.

```
soma_um <- function(x) { x + 1 }
s_soma_um <- safely(soma_um, 0)
obj <- c(10, 11, "a", 13, 14, 15)
s_soma_um(obj)
```

```
## $result
## [1] 0
##
## $error
## <simpleError in x + 1: non-numeric argument to binary operator>
```

10.5 Exercícios

A base `imdb` nos exercícios abaixo pode ser baixada clicando aqui.

1. Utilize a função `map()` para calcular a média de cada coluna da base `mtcars`.

2. Use a função `map()` para testar se cada elemento do vetor `letters` é uma vogal ou não. Dica: você precisará criar uma função para testar se é uma letra é vogal. Faça o resultado ser (a) uma lista de `TRUE/FALSE` e (b) um vetor de `TRUE/FALSE`.

3 Faça uma função que divida um número por 2 se ele for par ou multiplique ele por 2 caso seja ímpar. Utilize uma função `map` para aplicar essa função ao vetor `1:100`. O resultado do código deve ser um vetor numérico.

4. Use a função `map()` para criar gráficos de dispersão da receita vs orçamento para os filmes da base `imdb`. Os filmes de cada ano deverão compor um gráfico diferente. Faça o resultado ser (a) uma lista de gráficos e (b) uma nova coluna na base `imdb` (utilizando a função `tidyr::nest()`).

5. Utilize a função `walk` para salvar cada ano da base `imdb` em um arquivo `.rds` diferente, isto é, o arquivo `imdb_2001.rds`, por exemplo, deve conter apenas filmes do ano de 2001.

Chapter 11

R avançado

Neste capítulo, abordaremos alguns assuntos avançados, porém muito úteis no dia-a-dia de quem faz Ciência de Dados no R.

11.1 NSE

Se o `{tidyverse}` é o conjunto mais incrível de pacotes, faz sentido querermos criar nossos pacotes e fazer nossas análises usando seus princípios. Funções cujo primeiro argumento é um *data frame* e operações "pipeáveis" são apenas o começo.

Uma das características mais marcantes do `{tidyverse}` é a possibilidade de trabalhar com colunas como se elas fossem objetos comuns, criados fora da função. Basta pensar na `mutate()`: como ela sabe que `mpg` é uma coluna da tabela e não um objeto externo com o mesmo nome?

Existe uma ferramenta especial (e exclusiva ao R!) que nos permite fazer esse tipo de mágica: a *non-standard evaluation* (NSE) ou, em português, a avaliação não-padrão. Usamos NSE desde a primeira vez que escrevemos o comando `library()`, mas o seu funcionamento ainda é um mistério para a maioria.

Para criar funções flexíveis e exutas como as do `{tidyverse}`, é necessário entender o básico da NSE e de como trabalhar com alguns operadores desse novo mundo.

11.1.1 O que é?

Non-standard evaluation é uma propriedade do R que permite capturar o seu código sem avaliá-lo. Isso mesmo: o R deixa de ser uma caixa para a qual nós

enviamos comandos a serem executados e passa a ter a capacidade de **interpretar e trabalhar** os próprios comandos. Genericamente isso se chama **metaprogramação**.

```
# O objeto 'dplyr' simplesmente não existe...
dplyr
```

```
## Error in eval(expr, envir, enclos): object 'dplyr' not found
```

```
# Por que então esse comando funciona?
library(dplyr)
```

Na chamada `library()` acima, a palavra "dplyr" não se refere a nenhum objeto! O R consegue ler o código que nós escrevemos e agir em cima dessa informação. Apesar de ser comum para nós, outras linguagens são incapazes de fazer isso por causa da forma com que elas tratam seus argumentos.

O R tem o que chamamos de avaliação tardia (*delayed evaluation*), ou seja, uma expressão só é avaliada quando ela é necessária e não logo que ela é criada. No R, `print(1 + 2)` é diferente de `print(3)`, mas nas outras linguagens isso não é o caso!

11.1.2 Tidy evaluation

A faceta da NSE que nos interessa no momento é a chamada *tidy evaluation*, ou **tidy eval**, a avaliação não-padrão utilizada pelas funções do `{tidyverse}` e outros pacotes feitos para trabalhar com o mesmo paradigma.

- O mundo sem tidy eval é extremamente verborrágico, pois a tabela precisa ser especificada toda vez que nos referirmos a uma coluna:

```
starwars[starwars$homeworld == "Naboo" & starwars$species == "Human", ,]
```

- Para não precisar do `$`, a nossa única saída é criar objetos com as colunas:

```
homeworld <- starwars$homeworld; species <- starwars$species
starwars[homeworld == "Naboo" & species == "Human", ,]
```

- O `{tidyverse}` permite criar um "mini ambiente" em que as colunas da tabela estão disponíveis como se fossem objetos declarados explicitamente:

```
filter(starwars, homeworld == "Naboo", species == "Human")
```

O problema de capturar o código sem avaliá-lo é que fica difícil avaliar algo antes que ele seja capturado.

```
# O código 'birth_year' é capturado
starwars %>% filter(is.na(birth_year)) %>% nrow()
```

```
## [1] 44
```

```
# O objetivo é filtrar uma coluna especificada pelo usuário
filter_na <- function(df, col) {
  filter(df, is.na(col))
}

# A função captura o código 'col'
starwars %>% filter_na(col = birth_year) %>% nrow()
```

```
## Error: Problem with `filter()` input `..1`.
## i Input `..1` is `is.na(col)`.
## x object 'birth_year' not found
```

11.1.3 Curly-curly

O operador que vai resolver nosso problema é o `{{ }}` (lê-se *curly curly*), que permite interpolar o código, ou seja, avaliá-lo antes da captura.

```
# O objetivo é filtrar uma coluna especificada pelo usuário
filter_na <- function(df, col) {
  filter(df, is.na( {{col}} ))
}

# Agora a função captura o código 'birth_year'
starwars %>% filter_na(col = birth_year) %>% nrow()
```

```
## [1] 44
```

Essa sintaxe vem da interpolação de strings:

```
col <- "birth_year"
stringr::str_glue("Interpolando '{col}'!")
```

```
## Interpolando 'birth_year'!
```

Para passar múltiplos argumentos nem é necessário usar o curly-curly já que a reticência já possui as capacidades de tidy eval:

```
# O objetivo é permitir vários cálculos
summarise_by <- function(df, ..., by) {
  df %>%
    group_by( {{by}} ) %>%
    summarise(...)
}

starwars %>%
  summarise_by(
    media = mean(height, na.rm = TRUE),
    maximo = max(height, na.rm = TRUE),
    by = gender
  )
```

Note como não houve necessidade de interpolar `media` e `maximo`: a `summarise()` não tentou criar uma coluna chamada `...`.

11.1.4 Strings como argumentos

E se quisermos passar strings para as funções do `{tidyverse}`? Se pedirmos o nome de uma coluna para um usuário, a resposta virá como string.

```
# O objetivo é dar um nome para a média
summarise_mean <- function(df, nome, col) {
  summarise(df, nome = mean(col, na.rm = TRUE))
}

# É criada uma coluna 'nome' sem valor ('col' não existe)
summarise_mean(starwars, "media", "height")
```

```
## Warning in mean.default(col, na.rm = TRUE): argument is not numeric or logical:
## returning NA
```

```
## # A tibble: 1 x 1
##    nome
##   <dbl>
## 1    NA
```

Quando o "lado esquerdo" (**antes de um igual**) de uma expressão com tidy eval é uma string (ou se tornará uma quando avaliado), precisamos apenas usar o operador `:=` (lê-se *walrus*, "morsa"):

```
# O objetivo é dar um nome para a média
summarise_mean <- function(df, nome, col) {
  summarise(df, {{nome}} := mean(col, na.rm = TRUE))
}

# É criada uma coluna 'media' sem valor ('col' não existe)
summarise_mean(starwars, "media", "height")
```

```
## Warning in mean.default(col, na.rm = TRUE): argument is not numeric or logical:
## returning NA
```

```
## # A tibble: 1 x 1
##   media
##   <dbl>
## 1    NA
```

Quando uma string (ou algo que se tornará uma quando avaliado) está no "lado direito" (**depois de um igual ou quando não há igual**) de uma expressão com tidy eval, precisamos apenas usar o pronome `.data`:

```
# O objetivo é dar um nome para a média
summarise_mean <- function(df, nome, col) {
  summarise(df, {{nome}} := mean(.data[[col]], na.rm = TRUE))
}

# É criada uma coluna 'media' com a média de 'height'
summarise_mean(starwars, "media", "height")
```

```
## # A tibble: 1 x 1
##   media
##   <dbl>
## 1  174.
```

É como se estivéssemos chamando `df[["height"]]`, mas, como `df` não faria sentido dentro da expressão, usamos `.data`.

11.1.5 Exercícios

1. Escreva uma função que recebe uma *tibble* e uma coluna qualquer e devolve uma *tibble* sem as linhas contendo `NA` na coluna especificada.

2. Escreva uma função que recebe uma *tibble* e uma coluna e faz um gráfico de barras da frequência dessa coluna.

3. Escreva uma função que recebe uma *tibble* e o nome de duas colunas numéricas e faz um gráfico de dispersão dessas colunas.

4. Escreva uma função que recebe uma *tibble*, o nome de uma coluna categótica e o nome de uma coluna numérica e devolva uma tabela com a média da coluna numérica para cada categoria da coluna categórica.

Chapter 12

Miscelânea

12.1 Arrumando banco de dados: o pacote janitor

O pacote `janitor` disponibiliza algumas funções para limpar bases de dados.

Primeiramente, instale e carregue o pacote:

```
install.packages("janitor") # Instale a versão do CRAN, OU:

devtools::install_github("sfirke/janitor") # Versão de desenvolvimento
```

```
library(tidyverse)
library(janitor)
```

12.1.1 Arrumando o nome das variáveis

Utilizaremos a base com informações de pacientes com arritmia cardíaca. O código para obter essa base de exemplo está descrito abaixo:

```
# URL da base para baixar
url <- "https://github.com/curso-r/livro-material/raw/master/assets/data/dados_nomes_variaveis_li

# Arquivo de destino onde a base deverá ser salva
destfile <- "dados_nomes_variaveis_livro.xlsx"

# Fazer o download da base
curl::curl_download(url, destfile)
```

```
# Importar a base
dados_brutos <- readxl::read_excel(destfile)
```

As variáveis presentes na base são:

```
dados_brutos %>% names() # Ver o nome das variáveis
```

```
##  [1] "ID"                   "Sexo"                 "Nascimento"
##  [4] "Idade"                "Inclusão"             "Cor"
##  [7] "Peso"                 "Altura"               "cintura"
## [10] "IMC"                  "Superfície corporal"  "Tabagismo"
## [13] "cg.tabag (cig/dia)"   "Alcool (dose/semana)" "Drogas ilícitas"
## [16] "Cafeína/dia"          "Refrig/dia"           "Sedentario"
## [19] "ativ. Fisica"
```

Os nomes das variáveis contém letras maiúsculas, acentos, parênteses, pontos e barras, o que atrapalha na hora da programação. Para resolver esse problema, usamos a função `clean_names()`.

```
dados_brutos %>%
  janitor::clean_names() %>% # Limpar os nomes das variáveis
  names() # Ver o nome das variáveis
```

```
##  [1] "id"                 "sexo"                "nascimento"
##  [4] "idade"              "inclusao"            "cor"
##  [7] "peso"               "altura"              "cintura"
## [10] "imc"                "superficie_corporal" "tabagismo"
## [13] "cg_tabag_cig_dia"   "alcool_dose_semana"  "drogas_ilicitas"
## [16] "cafeina_dia"        "refrig_dia"          "sedentario"
## [19] "ativ_fisica"
```

Veja que a função removeu os parênteses, pontos e barras e substituiu os espaços por `_`.

E para substituir na base, precisamos atribuir o resultado em um novo objeto:

```
dados <- dados_brutos %>%
  janitor::clean_names() # Limpar os nomes das variáveis
```

12.1.2 Removendo linhas e colunas vazias

Esse banco de dados também contém outro problema: linhas vazias. Na verdade, elas não eram completamente vazias, pois havia algumas informações

de identificação da(o) paciente, mas nenhuma outra variável tinha sido computada.

```
dados %>%
  dplyr::slice(3) %>% # Apresentar apenas a linha 3
  knitr::kable()
```

id	sexo	nascimento	idade	inclusao	cor	peso	altura	cintura	imc	superficie_corporal	tabagism
3	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

Para resolver o problema, é possível utilizar a função `remove_empty()`.

```
dados_sem_linhas_vazias <- dados %>%
  as.data.frame() %>%
  dplyr::select(-id) %>%
  janitor::remove_empty() %>%
  tibble::rowid_to_column("id") %>%
  dplyr::select(id, everything()) %>%
  tibble::as_tibble()

dados_sem_linhas_vazias %>% knitr::kable()
```

id	sexo	nascimento	idade	inclusao	cor	peso	altura	cintura	imc	superficie_corporal
1	F	1964-01-31	41	2006-02-17	branca	75	1.63	98	28.22839	1.81
2	M	1959-01-28	45	2005-11-29	negra	71	1.70	NT	24.57000	1.83
3	M	1957-09-13	50	2008-02-13	NT	80	1.64	NT	29.74420	1.87
4	F	1938-02-06	71	2009-06-25	parda	56	1.51	96	24.56033	1,51

Foi necessário converter para `data.frame` primeiro porque não é possível definir os nomes das linhas de uma `tibble`. Se a linha estivesse completamente vazia, bastaria usar diretamente a função `remove_empty_rows()`.

Equivalentemente para colunas, existe a função `remove_empty_cols()`.

Outra forma de realizar este mesmo procedimento é utilizando a função `drop_na()` do pacote `tidyr`:

```
dados_sem_linhas_vazias <- dados %>%
  tidyr::drop_na(-id) %>%
  dplyr::select(-id) %>%
  tibble::rowid_to_column("id") %>%
  dplyr::relocate(id, .before = sexo)

dados_sem_linhas_vazias %>% knitr::kable()
```

id	sexo	nascimento	idade	inclusao	cor	peso	altura	cintura	imc	superfic
1	F	1964-01-31	41	2006-02-17	branca	75	1.63	98	28.22839	1.81
2	M	1959-01-28	45	2005-11-29	negra	71	1.70	NT	24.57000	1.83
3	M	1957-09-13	50	2008-02-13	NT	80	1.64	NT	29.74420	1.87
4	F	1938-02-06	71	2009-06-25	parda	56	1.51	96	24.56033	1,51

12.1.3 Identificando linhas duplicadas

O pacote `janitor` possui uma função para identificar entradas duplicadas numa base de dados: `get_dupes()`. Vamos criar uma base genérica para testá-la.

```
# Criar a base de exemplo
p_nome <- c("Athos", "Daniel", "Fernando", "Julio", "William")
sobrenome <- c("Damiani", "Falbel", "Corrêa", "Trecenti", "Amorim")

base_exemplo <- tibble::tibble(
  nome = sample(p_nome, 25, replace = TRUE),
  sobrenome = sample(sobrenome, 25, replace = TRUE),
  variavel_importante = rnorm(25)
)
# Dar uma espiada na base de exemplo
dplyr::glimpse(base_exemplo)
```

```
## Rows: 25
## Columns: 3
## $ nome                <chr> "Julio", "Athos", "William", "Athos", "Fernando", ~
## $ sobrenome           <chr> "Amorim", "Falbel", "Trecenti", "Corrêa", "Damiani~
## $ variavel_importante <dbl> 1.03090446, 2.05103806, -1.15686279, 0.77013281, 0~
```

```
# Ver as duplicatas com a função get_dupes()
janitor::get_dupes(base_exemplo, nome, sobrenome)
```

```
## # A tibble: 13 x 4
##    nome     sobrenome dupe_count variavel_importante
##    <chr>    <chr>          <int>               <dbl>
##  1 Athos    Falbel             2              2.05
##  2 Athos    Falbel             2             -1.09
##  3 Athos    Trecenti           2             -0.306
##  4 Athos    Trecenti           2             -0.698
##  5 Daniel   Damiani            3              0.651
##  6 Daniel   Damiani            3             -0.322
##  7 Daniel   Damiani            3              0.148
```

```
##  8 Fernando Damiani            2             0.253
##  9 Fernando Damiani            2             0.451
## 10 Julio    Amorim             2             1.03
## 11 Julio    Amorim             2            -0.0404
## 12 William  Corrêa             2             1.23
## 13 William  Corrêa             2            -0.636
```

Todas as linhas na `tibble` resultante representam uma combinação de nome-sobrenome repetida.

12.1.4 Outras funções

Por fim, o `janitor` também tem funções equivalentes à `table()` para produzir tabelas de frequência:

- `tabyl()` - similar a `table()`, mas pipe-ável e com mais recursos.
- `adorn_totals()` - acrescenta o total das linhas ou colunas.

```
# Fazer uma tabela de frequência
mtcars %>% janitor::tabyl(cyl)
```

```
##  cyl  n percent
##    4 11 0.34375
##    6  7 0.21875
##    8 14 0.43750
```

```
# Fazer uma tabela de frequência com valores totais,
# e porcentagem
mtcars %>%
  janitor::tabyl(cyl) %>%
  janitor::adorn_totals()
```

```
##    cyl  n percent
##      4 11 0.34375
##      6  7 0.21875
##      8 14 0.43750
##  Total 32 1.00000
```

```
# Fazer uma tabela de frequência com duas variáveis,
# e valores totais
mtcars %>%
  janitor::tabyl(cyl, am) %>%
  janitor::adorn_totals(where = "col")
```

```
##  cyl  0 1 Total
##    4  3 8    11
##    6  4 3     7
##    8 12 2    14
```

Esperamos que essas dicas e o pacote `janitor` ajudem a agilizar as suas análises!

Chapter 13

Respostas

Não há apenas uma maneira de resolver os exercícios. Você pode encontrar soluções diferentes das nossas, algumas vezes mais práticas e eficientes, outras vezes menos. Quando estiver fazendo suas análises, tente buscar o equilíbrio entre eficiência e pragmatismo. Economizar 1 hora com a execução do código pode não valer a pena se você demorou 2 horas a mais para programá-lo.

13.1 R básico

R como calculadora

1. Qual a diferença entre o R e o RStudio?

O R é uma linguagem de programação e um programa que instalamos no nosso computador para interpretar código em linguagem R. O RStudio é um ambiente de desenvolvimento para programação em R, isto é, é um programa que nos ajuda a trabalhar com a linguagem R.

---

2. Podemos usar o RStudio sem o R? E o R sem o RStudio?

Não podemos usar o RStudio sem o R, pois o programa R é que interpreta os nossos códigos e devolve os resultados. Por outro lado, podemos usar o R sem o RStudio (ou qualquer outro ambiente de desenvolvimento), mas a nossa vida de pessoa programadora se torna muito mais difícil.

---

3. Precisamos compilar nossos códigos de R?

Não. Um código em R é interpretado linha a linha, isto é, podemos ir rodando nosso código parcialmente, conforme vamos escrevendo.

---

4. Calcule o número de ouro no R. Dica: o número de ouro é dado pela expressão $\frac{1+\sqrt{5}}{2}$.

```
(1 + 5^(1 / 2))/2
```

```
## [1] 1.618034
```

Também poderia ser resolvido usando a função `sqrt()` para calcular a raiz quadrada.

```
(1 + sqrt(5))/2
```

```
## [1] 1.618034
```

---

5. Por que é preferível escrevermos sempre o nosso código no *script* e não no *Console*?

Porque o código é a documentação da nossa análise. Se parte da análise for feita diretamente no Console, parte da documentação será perdida.

Objetos e funções

1. Qual a diferença entre os códigos abaixo?

```
# Código 1
33 / 11

# Código 2
divisao <- 33 / 11
```

O primeiro código calcula a divisão de 33 por 11 e devolve o resultado na tela. O segundo código faz o mesmo cálculo, mas guardo o resultado no objeto `divisao`, isto é, o resultado não é retornado na tela.

---

2. Multiplique a sua idade por meses e salve o resultado em um objeto chamado `idade_em_meses`. Em seguida, multiplique esse objeto por 30 e salve o resultado em um objeto chamado `idade_em_dias`.

```
idade_em_meses <- 31 * 12

idade_em_dias <- idade_em_meses * 30
```

---

3. Por que o nome `meu-objeto` não pode ser utilizado para criar um objeto? O que significa a mensagem de erro resultante?

```
meu-objeto <- 1
```

```
## Error in meu - objeto <- 1: object 'meu' not found
```

Porque o sinal `-` é utilizado pelo R como o operador de subtração. O código acima está sendo interpretado como a subtração dos objetos `meu` e `objeto`. Como nenhum dos objetos existe na sessão, é devolvido um erro dizendo que o objeto `meu` (o primeiro deles) não foi encontrado.

No entanto, mesmo que os objetos existissem, seria retornado um erro. Neste caso, seria um erro de sintaxe, pois o R não sabe como interpretar a atribuição do valor `3` à operação `meu - objeto`. Veja que a mensagem de erro não é muito intuitiva, pois o R entende que `-<-` é uma função que esquecemos de definir.

```
meu <- 1
objeto <- 2
meu-objeto <- 1
```

```
## Error in meu - objeto <- 1: could not find function "-<-"
```

Classes

1. Guarde em um objeto chamado `nome` uma *string* contendo o seu nome completo.

```
nome <- "William Nilson de Amorim"
```

---

2. Guarde em um objeto chamado `cidade` o nome da cidade onde você mora. Em seguida, guarde em um objeto chamado `estado` o nome do estado onde você mora. Usando esses objetos, resolva os itens abaixo:

a. Utilize a função `nchar()` para contar o número de caracteres em cada cada string.

```
cidade <- "São Paulo"
estado <- "SP"

nchar(cidade)
```

```
## [1] 9
```

```
nchar(estado)
```

```
## [1] 2
```

b. Interprete o resultado do seguinte código:

```
paste(cidade, estado)
```

```
## [1] "São Paulo SP"
```

A função `paste` juntou (colou) as duas strings, uma do lado da outra, separando-as por um espaço. O resultado gerado foi uma única string.

c. Interprete o resultado do seguinte código:

```
paste(cidade, estado, sep = " - ")
```

```
## [1] "São Paulo - SP"
```

O argumento `sep` da função `paste()` nos permite escolher qual caracter será utilizado para separar as duas strings. Por padrão será utilizado um espaço em branco `" "`. No exemplo, foi utilizado um espaço, seguido de um traço, seguido de outro espaço. Sem os espaços o resultado seria:

```
paste(cidade, estado, sep = "-")
```

```
## [1] "São Paulo-SP"
```

Veja que qualquer caracter ou string pode ser utilizada como separador:

```
paste(cidade, estado, sep = " #tricolor ")
```

```
## [1] "São Paulo #tricolor SP"
```

d. Desafio. Como você reproduziria o mesmo resultado do item (c) sem utilizar o argumento `sep`?

```
paste(cidade, "-", estado)
```

```
## [1] "São Paulo - SP"
```

e. Qual a diferença entre as funções `paste()` e `paste0()`?

A função `paste0()` é equivalente à função `paste()` com o argumento `sep = ""`.

```
paste(cidade, "-", estado)
```

```
## [1] "São Paulo - SP"
```

```
paste(cidade, "-", estado, sep = "")
```

```
## [1] "São Paulo-SP"
```

```
paste0(cidade, "-", estado)
```

```
## [1] "São Paulo-SP"
```

Vetores

1. Guarde em um objeto a sequência de números de 0 a 5 e resolva os itens abaixos.

a. Use subsetting para fazer o R devolver o primeiro número dessa sequência. Em seguida, faça o R devolver o último número da sequência.

```
vetor <- 0:5

# Primeiro elemento
vetor[1]
```

```
## [1] 0
```

```
# Último elemento
vetor[6]
```

```
## [1] 5
```

b. Multiplique todos os valores do vetor por -1. Guarde o resultado em um novo objeto chamado `vetor_negativo`.

```
vetor_negativo <- vetor * -1
vetor_negativo
```

```
## [1]  0 -1 -2 -3 -4 -5
```

---

2. Crie um vetor com o nome de tres frutas, guarde em um objeto chamado `frutas` e resolva os itens abaixo.

a. Utilize a a função `length()` para verificar o tamanho do vetor.

```
frutas <- c("banana", "maçã", "mamão")

length(frutas)
```

```
## [1] 3
```

b. Inspecione a saída de `paste("eu gosto de", frutas)` e responda se o tamanho do vetor mudou.

```
paste("eu gosto de", frutas)
```

```
## [1] "eu gosto de banana" "eu gosto de maçã"   "eu gosto de mamão"
```

O tamanho do vetor não muda. O que aconteceu foi o acréscimo do prefixo "eu gosto de" a cada valor do vetor ***frutas****. Podemos confirmar essa afirmação calculando o comprimento do novo vetor:*

```
length(paste("eu gosto de", frutas))
```

```
## [1] 3
```

---

3. O que é reciclagem? Escreva um código em R que exemplifique esse comportamento.

Reciclagem é a repetição de valores de um vetor para que uma operação com vetores de tamanhos diferentes possa ser realizada. Embora pareça um comportamento estranho, utilizamos a reciclagem o tempo todo dentro do R, muitas vezes sem perceber. No exemplo a seguir, os valores do `vetor_menor` *serão repetidos (reciclados) até o dois vetores terem o mesmo tamanho. Então, o primeiro valor do vetor menor será somado com o promeiro valor do vetor maior, o segundo do menor somado com o segundo do maior e assim por diante.*

```
# Exemplo
vetor_menor <- c(1, 3)
vetor_maior <- c(1, 2, 3, 4)

vetor_menor + vetor_maior
```

```
## [1] 2 5 4 7
```

---

4. O que é coerção? Escreva um código em R que exemplifique esse comportamento.

Coerção é a homogenização da classe dos valores de um vetor. Se construirmos um vetor com valores de classes diferentes, todos os valores serão reprimidos para a classe mais dominante entre eles. No exemplo abaixo, todos os valores do vetor foram transformados em `character`*, pois* `character` *é mais dominante do que as demais classes presentes (*`numeric` *e* `logical`*).*

Considerando apenas as principais classes, temos a seguinte ordem de domînância: `character` > `numeric` > `integer` > `logical`*.*

```
c(1, -3, "a", TRUE)
```

```
## [1] "1"    "-3"   "a"    "TRUE"
```

---

5. Por que a coerção pode ser um problema na hora de importarmos bases de dados para o R?

Porque as colunas podem acabar sendo importadas com uma classe diferente daquela que estamos esperando e nenhum erro ou warning será devolvido. Por exemplo, se uma coluna da base que deveria ser numérica tiver, por algum descuido, qualquer tipo de caracter não numérico (texto), toda a coluna será importada como texto.

O caso mais comum desse problema é gerado pelo separador de decimais. Se não especificarmos corretamente qual é o separador de decimais que estamos utilizando na nossa base de dados na hora da importação, qualquer coluna com valores decimais será importada como texto.

---

6. Use a função `sum()` para somar os valores de 1 a 100.

```
sum(1:100)
```

```
## [1] 5050
```

---

7. Considere o vetor booleano a seguir:

```
dolar_subiu <- c(TRUE, TRUE, FALSE, FALSE, TRUE, FALSE, TRUE)
```

Este vetor tem informação de uma semana (7 dias, começando no domingo) indicando se o dólar subiu (TRUE) ou não subiu (FALSE) no respectivo dia. Interprete o resultado dos códigos abaixo:

a. `length(dolar_subiu)`

```
length(dolar_subiu)
```

```
## [1] 7
```

Devolve o comprimento do vetor ***`dolar_subiu`****. Podemos interpretar como o número de dias analisados nesse estudo.*

b. `dolar_subiu[2]`

```
dolar_subiu[2]
```

```
## [1] TRUE
```

Verifica se o dólar subiu no segundo dia do estudo (segunda-feira).

c. `sum(dolar_subiu)`

```
sum(dolar_subiu)
```

```
## [1] 4
```

Verifica o número de dias que o dólar subiu (já que na soma, os valores ***TRUE*** *viram 1 e os valores* ***FALSE*** *viram 0), no caso, 4 dias com alta do dólar.*

d. `mean(dolar_subiu)`

```
mean(dolar_subiu)
```

```
## [1] 0.5714286
```

Verifica a proporção de dias em que o dólar subiu (a média de valores 0 e 1 é igual a proporção de valores 1), no caso, cerca de 57% dos dias.

Testes lógicos

1. O código abaixo vai guardar no objeto `segredo` um número inteiro entre 0 e 10. Sem olhar qual número foi guardado no objeto, resolva os itens a seguir:

```
segredo <- round(runif(1, min = 0, max = 10))
```

a. Teste se o segredo é maior ou igual a 0.

```
segredo >= 0
```

```
## [1] TRUE
```

b. Teste se o segredo é menor ou igual a 10.

```
segredo <= 10
```

```
## [1] TRUE
```

c. Teste se o segredo é maior que 5.

```
segredo > 5
```

```
## [1] FALSE
```

d. Teste se o segredo é par.

Precisamos verificar se o resto da divisão por 2 é igual a zero.

```
# Resto da divisão por 2
segredo %% 2
```

```
## [1] 0
```

```
# Guardando o resultado
resto <- segredo %% 2

# Teste
resto == 0
```

```
## [1] TRUE
```

```
# Equivalentemente poderíamos fazer
segredo %% 2 == 0
```

```
## [1] TRUE
```

e. Teste se `segredo * 5` é maior que a sua idade.

```
segredo * 5 > 31
```

```
## [1] FALSE
```

f. Desafio. Escreva um teste para descobrir o valor do segredo.

Vamos descobrir a posição do segredo no vetor `0:10`.

```
numeros_possiveis <- 0:10
numeros_possiveis
```

```
##  [1]  0  1  2  3  4  5  6  7  8  9 10
```

```
segredo == numeros_possiveis
```

```
##  [1]  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

No código acima, para cada valor de ***numero_possiveis****, foi testado se esse valor é igual ao segredo. Em caso afirmativo, um TRUE é devolvido. Veja que há apenas um TRUE no vetor resultante. Essa é posição do nosso segredo dentro do vetor* ***numeros_possiveis****.*

Podemos usar esse vetor de TRUE/FALSE para recuperar o valor do segredo fazendo subsetting.

```
# Eis o valor do segredo
numeros_possiveis[segredo == numeros_possiveis]
```

```
## [1] 0
```

No código acima, foi retornado apenas o número associado ao valor TRUE dado pelo teste ***segredo == numeros_possiveis****.*

---

2. Escreva um código em R que devolva apenas os valores maiores ou iguais a 10 do vetor abaixo:

```
vetor <- c(4, 8, 15, 16, 23, 42)

# Resposta
vetor[vetor >= 10]
```

```
## [1] 15 16 23 42
```

---

3. Use o vetor numeross abaixo para responder as questoes seguintes.

```
numeros <- -4:2
```

a. Escreva um código que devolva apenas valores positivos do vetor `numeros`.

```
numeros[numeros > 0]
```

```
## [1] 1 2
```

b. Escreva um código que devolta apenas os valores pares do vetor `numeros`.

```
# Resto da divisão por 2
numeros %% 2
```

```
## [1] 0 1 0 1 0 1 0
```

```
# Quais números são pares?
numeros %% 2 == 0
```

```
## [1]  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```
# Usando para filtrar o vetor
numeros[numeros %% 2 == 0]
```

```
## [1] -4 -2  0  2
```

c. Filtre o vetor para que retorne apenas aqueles valores que, quando elevados a 2, são menores do que 4.

```
# Quadrado dos valores do vetor
numeros^2
```

```
## [1] 16  9  4  1  0  1  4
```

```
# Aqueles menores que 4
numeros^2 < 4
```

```
## [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE
```

```
# Usando para filtrar
numeros[numeros^2 < 4]
```

```
## [1] -1  0  1
```

Valores especiais

1. Quais as diferenças entre `NaN`, `NULL`, `NA` e `Inf`? Digite expressões que retornem cada um desses valores.

- *O `NaN` representa uma indefinição matemática. Todo `NaN` é um `NA`, mas o contrário não é verdade.*

```
0/0
```

```
## [1] NaN
```

- *O `NA` representa o nosso desconhecimento acerca de um valor. Esse valor existe, mas nós não o conhecemos. Na Estatística, ele é conhecido como valor omisso ou valor faltante.*

```
sum(c(1, NA))
```

```
## [1] NA
```

- *O `NULL` é um valor especial para o R, isto é, ao contrário do NA, ele não tem um significado prático para a análise de dados, mas sim computacional. Ele representa a ausência de um vetor e é útil na construção da lógica de programação de alguns códigos.*

```
mtcars$coluna_que_nao_existe
```

```
## NULL
```

- *O `Inf` representa um número muito grande (que o computador não consegue representar) ou um limite matemático.*

```
2^2000
```

```
## [1] Inf
```

---

2. Escreva um código que conte o número de `NAs` do vetor `b`.

```
b <- c(1, 0, NA, NA, NA, NA, 7, NA, NA, NA, NA, NA, 2, NA, NA, 10, 1, 1, NA)
```

```
sum(is.na(b))
```

```
## [1] 12
```

Data frames

1. Quais códigos abaixo retornam **um vetor** com a coluna `mpg` do data frame `mtcars`?

a. `mtcars$mpg`

```
mtcars$mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

Retorna!

b. `mtcars[ , 3]`

```
mtcars[ , 3]
```

```
##  [1] 160.0 160.0 108.0 258.0 360.0 225.0 360.0 146.7 140.8 167.6 167.6 275.8
## [13] 275.8 275.8 472.0 460.0 440.0  78.7  75.7  71.1 120.1 318.0 304.0 350.0
## [25] 400.0  79.0 120.3  95.1 351.0 145.0 301.0 121.0
```

Não retorna. A coluna `mpg` é a primeira coluna do data frame. A coluna que esse código retornou foi a `disp`.

c. `mtcars("mpg")`

```
mtcars("mpg")
```

```
## Error in mtcars("mpg"): could not find function "mtcars"
```

A sintaxe desse código não é válida pois `()` não são utilizados para fazer subsetting, mas sim para definir os argumentos de uma função (e `mtcars` não é uma função).

d. `mtcars[ , "mpg"]`

```
mtcars[ , "mpg"]
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

"Retorna!"

e. `mtcars.mpg`

```
mtcars.mpg
```

```
## Error in eval(expr, envir, enclos): object 'mtcars.mpg' not found
```

Essa sintaxe não é válida no R. Ele está procurando por um objeto chamado `mtcars.mpg`, que não existe.

f. `mtcars[ , 1]`

```
mtcars[ , 1]
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

Retorna!

g. `mtcars[1, 1]`

```
mtcars[1, 1]
```

```
## [1] 21
```

Esse código retorna apenas o primeiro valor da coluna `mpg`.

h. `mpg$mtcars`

```
mpg$mtcars
```

```
## Error in eval(expr, envir, enclos): object 'mpg' not found
```

Não retorna. Essa sintaxe não é válida para selecionar colunas de um data frame. O correto seria `nome_do_data_frame$nome_da_coluna`.

---

2. Para que serve a função `str()`. Dê um exemplo do seu uso.

Serve para espiarmos a estrutura de um data frame, como número de linhas, colunas, classes das colunas e exemplo de valores em cada uma delas.

```
str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

3. Para que serve a função `nanes()`. Dê um exemplo do seu uso.

Serve para observamos os nomes das colunas de um data frame. O valor retornado é um vetor de strings.

```
names(mtcars)
```

```
##  [1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear"
## [11] "carb"
```

4. Use o data frame `airquality` para responder às questões abaixo:

a. Quantas colunas `airquality` tem?

```
ncol(airquality)
```

```
## [1] 6
```

b. Quantas linhas `airquality` tem?

```
nrow(airquality)
```

```
## [1] 153
```

c. O que a função `head()` retorna?

Retorna as 6 primeiras linhas do data frame.

```
head(airquality)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
```

d. Quais são os nomes das colunas?

```
names(airquality)
```

```
## [1] "Ozone"   "Solar.R" "Wind"    "Temp"    "Month"   "Day"
```

e. Qual é a classe da coluna `Ozone`?

```
str(airquality)
```

```
## 'data.frame':    153 obs. of  6 variables:
##  $ Ozone  : int  41 36 12 18 NA 28 23 19 8 NA ...
##  $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...
##  $ Wind   : num  7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
##  $ Temp   : int  67 72 74 62 56 66 65 59 61 69 ...
##  $ Month  : int  5 5 5 5 5 5 5 5 5 5 ...
##  $ Day    : int  1 2 3 4 5 6 7 8 9 10 ...
```

```
# Também podemos ver apenas a coluna Ozone
class(airquality$Ozone)
```

```
## [1] "integer"
```

5. Desafio. Calculando desvio-padrão no R. Use o data frame `airquality` para responder às questões abaixo:

a. Tire a média da coluna Ozone e guarde em um objeto.

```
media_ozone <- mean(airquality$Ozone, na.rm = TRUE)
```

b. Guarde em um objeto o vetor correspondente à coluna Ozone subtraída da sua própria média (calculada em no item a).

```
ozone_centralizado <- airquality$Ozone - media_ozone
```

c. Eleve o vetor calculado em (b) ao quadrado. Salve o resultado em um novo objeto.

```
ozone_centr_quadrado <- ozone_centralizado ^ 2
```

d. Tire a média do vetor calculado em (c) e salve o resultado em um objeto chamado `variancia`. Em seguida, calcule a raiz quadrada desse valor e salve em um objeto chamado `desvio_padrao`.

```
variancia <- mean(ozone_centr_quadrado, na.rm = TRUE)
desvio_padrao <- sqrt(variancia)
```

e. Compare o valor de `desvio_padrao` com `sd(airquality$Ozone)` e pesquise por que os valores não são iguais. Dica: veja a documentação da função `sd()`.

```
desvio_padrao
```

```
## [1] 32.84539
```

```
sd(airquality$Ozone, na.rm = TRUE)
```

```
## [1] 32.98788
```

Os valores não são iguais, pois na nossa conta, ao calcularmos a média no item (d), dividimos a soma de quadrados por `n` (tamanho da amostra, no caso, 116), enquanto na função `sd()` a divisão é por `n - 1` (115). A divisão por `n - 1`, na Estatística, representa o estimador não viesado para a variância/desvio-padrão de uma quantidade. Na prática, ambas podem ser utilizadas como medidas de variabilidade, mas em contextos inferenciais o valor dividido por `n - 1` é preferível.

Para chegarmos no mesmo resultado, poderíamos fazer:

```
# Número de valores sem NA
n <- length(na.omit(airquality$Ozone))

variancia <- sum(ozone_centr_quadrado, na.rm = TRUE) / (n - 1)
desvio_padrao <- sqrt(variancia)
desvio_padrao
```

```
## [1] 32.98788
```

---

6. Use o *data frame* `airquality` para responder às questões abaixo.

a. Conte quantos `NAs` tem na coluna `Solar.R`.

```
sum(is.na(airquality$Solar.R))
```

```
## [1] 7
```

b. Filtre a tabela `airqualitycom` apenas linhas em que `Solar.R` é `NA`.

```
tab <- airquality[is.na(airquality$Solar.R),]

# Imprimindo apenas as primeiras linhas para facilitar a leitura
head(tab)
```

```
##    Ozone Solar.R Wind Temp Month Day
## 5     NA      NA 14.3   56     5   5
## 6     28      NA 14.9   66     5   6
## 11     7      NA  6.9   74     5  11
## 27    NA      NA  8.0   57     5  27
## 96    78      NA  6.9   86     8   4
## 97    35      NA  7.4   85     8   5
```

c. Filtre a tabela `airquality` com apenas linhas em que `Solar.R` **não** é `NA`.

```
tab <- airquality[!is.na(airquality$Solar.R),]

# Imprimindo apenas as primeiras linhas para facilitar a leitura
head(tab)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 7    23     299  8.6   65     5   7
## 8    19      99 13.8   59     5   8
```

d. Filtre a tabela `airquality` com apenas linhas em que `Solar.R` **não** é `NA` e `Month` é igual a 5.

```
tab <- airquality[!is.na(airquality$Solar.R) & airquality$Month > 5,]

# Imprimindo apenas as primeiras linhas para facilitar a leitura
head(tab)
```

```
##    Ozone Solar.R Wind Temp Month Day
## 32    NA     286  8.6   78     6   1
## 33    NA     287  9.7   74     6   2
## 34    NA     242 16.1   67     6   3
## 35    NA     186  9.2   84     6   4
## 36    NA     220  8.6   85     6   5
## 37    NA     264 14.3   79     6   6
```

Mais sobre funções

1. Qual dos códigos abaixo devolverá um erro se for avaliado?

a. `3 * 5 + 10`

```
3 * 5 + 10
```

```
## [1] 25
```

Sem erro.

b. `function <- 10`

```
function <- 10
```

```
## Error: <text>:1:10: unexpected assignment
## 1: function <-
##              ^
```

Retorna erro pois ***function*** *é um nome reservado no R. Não podemos utilizá-lo para nomear outros objetos.*

c. `mean(1, 10)`

```
mean(1, 10)
```

```
## [1] 1
```

Não retorna erro, mas o 10 não é considerado na média. O código correto seria ***mean(c(1, 10))****.*

d. `(soma <- sum(1, 1))`

```
(soma <- sum(1, 1))
```

```
## [1] 2
```

Não retorna erro. Cercar o código com parênteses força a saída a sempre ser apresentada no Console, além de ser gravada dentro do objeto.

---

2. Crie uma função que receba um número e retorne o quadrado deste número.

```
quadrado <- function(x) {
  x ^ 2
}

quadrado(2)
```

```
## [1] 4
```

```
quadrado(3)
```

```
## [1] 9
```

```
quadrado(0)
```

```
## [1] 0
```

---

3. Crie uma função que receba 2 números e devolva a raiz quadrada da soma desses números.

```
raiz_da_soma <- function(a, b) {
  sqrt(a + b)
}

raiz_da_soma(2, 2)
```

```
## [1] 2
```

```
raiz_da_soma(1, 4)
```

```
## [1] 2.236068
```

---

4. Crie uma função que receba dois valores (numéricos) e devolva o maior deles.

```
retornar_maior <- function(x, y) {
  vetor <- c(x, y)
  filtro <- c(x > y, x <= y)
  vetor[filtro]
}

retornar_maior(10, 20)
```

```
## [1] 20
```

```
retornar_maior(5, -5)
```

```
## [1] 5
```

*Também poderíamos resolver usando a função `max()`.

```
retornar_maior <- function(x, y) {
  max(x, y)
}

retornar_maior(10, 20)
```

```
## [1] 20
```

```
retornar_maior(5, -5)
```

```
## [1] 5
```

---

5. Use a função `runif()` para criar uma função que retorne um número aleatório inteiro entre 0 e 10 (0 e 10 inclusive). Caso você não conheça a função `runif()`, rode `help(runif)` para ler a sua documentação.

```
sortear_numero <- function() {
  numero <- runif(1, min = 0, max = 10)

  numero_inteiro <- round(numero)

  return(numero_inteiro)
}

sortear_numero()
```

```
## [1] 8
```

---

6. Rode `help(sample)` para descobrir o que a função `sample()` faz. Em seguida:

a. Use a função `sample()` para escrever uma função que devolva uma linha aleatória de um *data frame*.

```
sortear_linha <- function(data_frame) {
  num_linhas <- nrow(data_frame)
  linha_sorteada <- sample(1:num_linhas, 1)

  data_frame[linha_sorteada, ]
}

sortear_linha(mtcars)
```

```
##               mpg cyl disp  hp drat   wt qsec vs am gear carb
## Ferrari Dino 19.7   6  145 175 3.62 2.77 15.5  0  1    5    6
```

b. Generalize a função para retornar um número qualquer de linhas, escolhido pelo usuário.

```
sortear_linha <- function(data_frame, n) {
  num_linhas <- nrow(data_frame)
  linha_sorteada <- sample(1:num_linhas, n)

  data_frame[linha_sorteada, ]
}

sortear_linha(mtcars, 10)
```

```
##                      mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Toyota Corolla      33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
```

Controle de fluxo

1. Por que o código abaixo retorna erro? Arrume o código para retornar o valor `TRUE`.

```
x <- 4
if(x = 4) {
  TRUE
}
```

**Porque a expressão dentro do `if` deve retornar um valor lógico (TRUE ou FALSE) e o código `x = 4` é uma atribuição (equivalente a `x <- 4`). O correto seria utilizar `==` em vez de `=`.*

```
x <- 4
if(x == 4) {
  TRUE
}
```

```
## [1] TRUE
```

---

2. Usando `if` e `else`, escreva um código que retorne a string `"número"` caso um valor `x` seja da classe `numeric` ou `integer`; a string `"palavra"` caso seja da classe `character`; e `NA` caso contrário.

```
x <- 10

if (class(x) == "numeric" | class(x) == "integer") {
  "número"
} else if (class(x) == "character") {
  "palavra"
} else {
  NA
}
```

```
## [1] "número"
```

Uma solução mais simples seria utilizar as funções `is.numeric()` e `is.character()`.

```
x <- "qualquer texto"

if (is.numeric(x)) {
  "número"
} else if (is.character(x)) {
  "palavra"
} else {
  NA
}
```

```
## [1] "palavra"
```

---

3. Usando apenas `for` e a função `length()`, construa uma função que calcule a média de um vetor numérico qualquer. Construa uma condição para a função retornar `NULL` caso o vetor não seja numérico.

```
minha_media <- function(vetor) {

  soma <- 0

  for (i in 1:length(vetor)) {
    soma <- soma + vetor[i]
  }
```

```
  media <- soma / length(vetor)

  return(media)

}

minha_media(c(1, 2))
```

```
## [1] 1.5
```

```
minha_media(0:10)
```

```
## [1] 5
```

```
minha_media(c(2, 2))
```

```
## [1] 2
```

---

4. Utilize o vetor `a` para resolver as questões a seguir:

```
a <- c(10, 3, 5, -1, 3, -4, 8, 9, -10)
```

a. Utilize o `for` para imprimir as médias acumuladas do vetor `a`, isto é, primeiro vamos imprimir 10, depois a média entre 10 e 3, depois a média entre 10, 3 e 5 e assim por diante.

```
soma <- 0

for(i in 1:length(a)) {
  soma <- soma + a[i]

  print(soma / i)
}
```

```
## [1] 10
## [1] 6.5
## [1] 6
## [1] 4.25
## [1] 4
```

```
## [1] 2.666667
## [1] 3.428571
## [1] 4.125
## [1] 2.555556
```

b. Adapte o laço que você fez no item anterior para ignorar os valores negativos, isto é, em caso de valor negativo, o laço não deve calcular a média e não imprimir nada.

```
soma <- 0
divisor <- 0

for(i in 1:length(a)) {
  if (a[i] >= 0) {
    soma <- soma + a[i]
    divisor <- divisor + 1
    print(soma / divisor)
  }
}
```

```
## [1] 10
## [1] 6.5
## [1] 6
## [1] 5.25
## [1] 5.8
## [1] 6.333333
```

13.2 Importação

O pacote readr

1. Qual a diferença entre as funções `read_csv()` e `read_csv2()`?

A função ***`read_csv()` lê arquivos CSV separados por vírgula (padrão norte-americano). A função*** *`read_csv2()` lê arquivos CSV separados por ponto-e-vírgula (padrão brasileiro, já que usamos a vírgula como seperador de decimais).*

---

2. Leia o arquivo `imdb.csv` utilizando a função `read_delim()`.

```
library(readr)

read_delim("imdb.csv", delim = ",")
```

---

3. Escreva a base mtcars em um arquivo `mtcars.csv` que não contenha o nome das colunas.

```
write_csv(mtcars, "mtcars.csv", col_names = FALSE)
```

---

4. Use a função `write_rds()` para salvar em arquivos

a) Um número.

```
numero <- 29
write_rds(numero, "numero.rds")
```

b) Um vetor de strings.

```
estados <- c("SP", "RN", "AC", "MG")
write_rds(estados, "estados.rds")
```

c) Uma lista com valores númericos, textuais e lógicos.

```
lista <- list(numero = 1, texto = c("M", "F", "F"), logico = FALSE)
write_rds(lista, "lista.rds")
```

d) As 3 primeiras colunas da base `mtcars`.

```
write_rds(mtcars[,1:3], "mtcars_3_colunas.rds")
```

---

5. Utilize a função `read_rds()` para importar de volta para o R os arquivos criados no exercício 4.

```
read_rds("numero.rds")
read_rds("estados.rds")
read_rds("lista.rds")
read_rds("mtcars_3_colunas.rds")
```

13.3 Pipe

---

1. Reescreva a expressão abaixo utilizando o %>%.

```
round(mean(sum(1:10)/3), digits = 1)
```

```
1:10 %>%
  sum %>%
  divide_by(3) %>%
  round(digits = 1)
```

---

2. Reescreva o código abaixo utilizando o %>%.

```
# Setamos a semente que gera números aleatórios para deixar o resultado reprodutível

set.seed(137)

x <- rnorm(100)
x.pos <- x[x>0]
media <- mean(x.pos)
saida <- round(media, 2)
saida
```

```
set.seed(137)

rnorm(100) %>%
  magrittr::extract(. > 0) %>%
  mean %>%
  round(digits = 2)
```

---

3. Sem rodar, diga qual a saída do código abaixo. Consulte o help das funções caso precise.

```
2 %>%
  add(2) %>%
  c(6, NA) %>%
  mean(na.rm = T) %>%
  equals(5)
```

- *Primeiro, somamos 2 com 2, gerando o valor 4.*
- *Então colocamos esse valor em um vetor com os valores 6 e `NA`.*
- *Em seguida, tiramos a média desse vetor, desconsiderando o `NA`, obtendo o valor 5.*
- *Por fim, testemos se o valor é igual a 5, obtendo o valor `TRUE`.*

13.4 O pacote dplyr

Selecionando colunas

Utilize a base `imdb` nos exercícios a seguir.

1. Teste aplicar a função `glimpse()` do pacote `{dplyr}` à base `imdb`. O que ela faz?

```
glimpse(imdb)
```

```
## Rows: 3,807
## Columns: 15
## $ titulo         <chr> "Avatar ", "Pirates of the Caribbean: At World's End ",~
## $ ano            <dbl> 2009, 2007, 2012, 2012, 2007, 2010, 2015, 2016, 2006, 2~
## $ diretor        <chr> "James Cameron", "Gore Verbinski", "Christopher Nolan",~
## $ duracao        <dbl> 178, 169, 164, 132, 156, 100, 141, 183, 169, 151, 150, ~
## $ cor            <chr> "Color", "Color", "Color", "Color", "Color", "Color", "~
## $ generos        <chr> "Action|Adventure|Fantasy|Sci-Fi", "Action|Adventure|Fa~
## $ pais           <chr> "USA", "USA", "USA", "USA", "USA", "USA", "USA", "USA",~
## $ classificacao  <chr> "A partir de 13 anos", "A partir de 13 anos", "A partir~
## $ orcamento      <dbl> 237000000, 300000000, 250000000, 263700000, 258000000, ~
## $ receita        <dbl> 760505847, 309404152, 448130642, 73058679, 336530303, 2~
## $ nota_imdb      <dbl> 7.9, 7.1, 8.5, 6.6, 6.2, 7.8, 7.5, 6.9, 6.1, 7.3, 6.5, ~
## $ likes_facebook <dbl> 33000, 0, 164000, 24000, 0, 29000, 118000, 197000, 0, 5~
## $ ator_1         <chr> "CCH Pounder", "Johnny Depp", "Tom Hardy", "Daryl Sabar~
## $ ator_2         <chr> "Joel David Moore", "Orlando Bloom", "Christian Bale", ~
## $ ator_3         <chr> "Wes Studi", "Jack Davenport", "Joseph Gordon-Levitt", ~
```

A função `glimpse()` traz um resumo da base, contendo o número de linhas, colunas, o nome e tipo das colunas e as primeiras observações de cada coluna.

---

2. Crie uma tabela com apenas as colunas `titulo`, `diretor`, e `orcamento`. Salve em um objeto chamado `imdb_simples`.

```
imdb_simples <- select(imdb, titulo, diretor, orcamento)
imdb_simples
```

```
## # A tibble: 3,807 x 3
##    titulo                                     diretor           orcamento
##    <chr>                                      <chr>                 <dbl>
##  1 Avatar                                     James Cameron     237000000
##  2 Pirates of the Caribbean: At World's End   Gore Verbinski    300000000
##  3 The Dark Knight Rises                      Christopher Nolan 250000000
##  4 John Carter                                Andrew Stanton    263700000
##  5 Spider-Man 3                               Sam Raimi         258000000
##  6 Tangled                                    Nathan Greno      260000000
##  7 Avengers: Age of Ultron                    Joss Whedon       250000000
##  8 Batman v Superman: Dawn of Justice         Zack Snyder       250000000
##  9 Superman Returns                           Bryan Singer      209000000
## 10 Pirates of the Caribbean: Dead Man's Chest Gore Verbinski    225000000
## # ... with 3,797 more rows
```

---

3. Selecione apenas as colunas `ator_1`, `ator_2` e `ator_3` usando o ajudante `contains()`.

```
select(imdb, contains("ator"))
```

```
## # A tibble: 3,807 x 3
##    ator_1          ator_2            ator_3
##    <chr>           <chr>             <chr>
##  1 CCH Pounder     Joel David Moore  Wes Studi
##  2 Johnny Depp     Orlando Bloom     Jack Davenport
##  3 Tom Hardy       Christian Bale    Joseph Gordon-Levitt
##  4 Daryl Sabara    Samantha Morton   Polly Walker
##  5 J.K. Simmons    James Franco      Kirsten Dunst
##  6 Brad Garrett    Donna Murphy      M.C. Gainey
##  7 Chris Hemsworth Robert Downey Jr. Scarlett Johansson
##  8 Henry Cavill    Lauren Cohan      Alan D. Purwin
##  9 Kevin Spacey    Marlon Brando     Frank Langella
## 10 Johnny Depp     Orlando Bloom     Jack Davenport
## # ... with 3,797 more rows
```

---

4. Usando a função `select()` (e seus ajudantes), escreva códigos que retornem a base IMDB sem as colunas `ator_1`, `ator_2` e `ator_3`. Escreva todas as soluções diferentes que você conseguir pensar.

```
select(imdb, -starts_with("ator"))

select(imdb, -ator_1, -ator_2, -ator_3)

select(imdb, -(ator_1:ator_3))

select(imdb, titulo:likes_facebook)

select(imdb, -contains("ator"))
```

Ordenando a base

1. Ordene os filmes em ordem crescente de `ano` e decrescente de `receita` e salve em um objeto chamado `filmes_ordenados`.

```
filmes_ordenados <- arrange(imdb, ano, desc(receita))
```

---

2. Selecione apenas as colunas `titulo` e `orcamento` e então ordene de forma decrescente pelo `orcamento`.

```
# Aninhando as funções
arrange(select(imdb, titulo, orcamento), desc(orcamento))

# Criando objeto intermediário
imdb_aux <- select(imdb, titulo, orcamento)
arrange(imdb_aux, desc(orcamento))

# Pipe s2
imdb %>%
  select(titulo, orcamento) %>%
  arrange(desc(orcamento))
```

Filtrando linhas

Utilize a base `imdb` nos exercícios a seguir.

1. Crie um objeto chamado `filmes_pb` apenas com filmes preto e branco.

```
# Vendo categorias da variável cor
unique(imdb$cor)
```

```
## [1] "Color"           "Black and White" NA
```

```
# Filtrando
imdb %>% filter(cor == "Black and White")
```

```
## # A tibble: 149 x 15
##    titulo     ano diretor  duracao cor    generos  pais  classificacao orcamento
##    <chr>    <dbl> <chr>      <dbl> <chr>  <chr>    <chr> <chr>             <dbl>
##  1 Pearl H~  2001 Michael~     184 Black~ Action|~ USA   A partir de ~ 140000000
##  2 The Avi~  2004 Martin ~     170 Black~ Biograp~ USA   A partir de ~ 110000000
##  3 Ali       2001 Michael~     165 Black~ Biograp~ USA   A partir de ~ 107000000
##  4 Eragon    2006 Stefen ~     104 Black~ Action|~ USA   Livre         100000000
##  5 Seabisc~  2003 Gary Ro~     140 Black~ Drama|H~ USA   A partir de ~  87000000
##  6 Bewitch~    NA <NA>          25 Black~ Comedy|~ USA   Outros               NA
##  7 Nine      2009 Rob Mar~     118 Black~ Drama|M~ USA   A partir de ~  80000000
##  8 The Lea~  2003 Stephen~     110 Black~ Action|~ USA   A partir de ~  78000000
##  9 War Hor~  2011 Steven ~     146 Black~ Drama|W~ USA   A partir de ~  66000000
## 10 Space C~  2000 Clint E~     130 Black~ Action|~ USA   A partir de ~  65000000
## # ... with 139 more rows, and 6 more variables: receita <dbl>, nota_imdb <dbl>,
## #   likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

---

2. Crie um objeto chamado `curtos_legais` com filmes de 90 minutos ou menos de duração e nota no imdb maior do que 8.5.

```
curtos_legais <- imdb %>%
  filter(duracao <= 90, nota_imdb > 8.5)
```

---

3. Retorne tabelas (`tibbles`) apenas com:

a. filmes coloridos anteriores a 1950;

```
imdb %>%
  filter(cor == "Color", ano < 1950)
```

```
## # A tibble: 14 x 15
##    titulo      ano diretor duracao cor   generos   pais  classificacao orcamento
##    <chr>     <dbl> <chr>     <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 Pinocchi~  1940 Norman~      88 Color Animatio~ USA   Outros           2600000
##  2 Duel in ~  1946 King V~     144 Color Drama|Ro~ USA   Outros           8000000
##  3 Tycoon     1947 Richar~      95 Color Adventur~ USA   Outros           3209000
```

```
##  4 State Fa~  1945 Walter~     100 Color Musical|~ USA   Outros             NA
##  5 Gone wit~  1939 Victor~     226 Color Drama|Hi~ USA   Outros        3977000
##  6 The Pira~  1948 Vincen~     102 Color Adventur~ USA   Outros        3700000
##  7 It's a W~  1946 Frank ~     118 Color Drama|Fa~ USA   Livre         3180000
##  8 Bathing ~  1944 George~     101 Color Comedy|M~ USA   Outros        2361000
##  9 Fantasia   1940 James ~     120 Color Animatio~ USA   Outros        2280000
## 10 She Wore~  1949 John F~     103 Color Western   USA   Outros        1600000
## 11 Spellbou~  1945 Alfred~      95 Color Film-Noi~ USA   Outros        1696377
## 12 Snow Whi~  1937 Willia~      83 Color Animatio~ USA   Outros        2000000
## 13 Casablan~  1942 Michae~      82 Color Drama|Ro~ USA   Livre          950000
## 14 Bambi      1942 James ~      70 Color Animatio~ USA   Outros             NA
## # ... with 6 more variables: receita <dbl>, nota_imdb <dbl>,
## #   likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

b. filmes do "Woody Allen" ou do "Wes Anderson";

```
imdb %>%
  filter(diretor == "Woody Allen" | diretor == "Wes Anderson")
```

```
## # A tibble: 25 x 15
##    titulo      ano diretor duracao cor   generos   pais  classificacao orcamento
##    <chr>     <dbl> <chr>     <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 The Life~  2004 Wes An~     119 Color Adventur~ USA   A partir de ~  50000000
##  2 Fantasti~  2009 Wes An~      87 Color Adventur~ USA   Livre          40000000
##  3 The Gran~  2014 Wes An~      99 Color Adventur~ USA   A partir de ~  25000000
##  4 The Roya~  2001 Wes An~     110 Color Comedy|D~ USA   A partir de ~  21000000
##  5 The Curs~  2001 Woody ~     103 Color Comedy|C~ USA   A partir de ~  26000000
##  6 To Rome ~  2012 Woody ~     112 Color Comedy|R~ USA   A partir de ~  17000000
##  7 Bullets ~  1994 Woody ~      98 Color Comedy|C~ USA   A partir de ~  20000000
##  8 Deconstr~  1997 Woody ~      96 Color Comedy    USA   A partir de ~  20000000
##  9 Everyone~  1996 Woody ~     101 Color Comedy|M~ USA   A partir de ~  20000000
## 10 Blue Jas~  2013 Woody ~      98 Color Drama     USA   A partir de ~  18000000
## # ... with 15 more rows, and 6 more variables: receita <dbl>, nota_imdb <dbl>,
## #   likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

c. filmes do "Steven Spielberg" ordenados de forma decrescente por ano, mostrando apenas as colunas `titulo` e `ano`;

```
imdb %>%
  filter(diretor == "Steven Spielberg") %>%
  arrange(desc(ano)) %>%
  select(titulo, ano)
```

```
## # A tibble: 24 x 2
```

```
##    titulo                                                  ano
##    <chr>                                                 <dbl>
##  1 Bridge of Spies                                        2015
##  2 Lincoln                                                2012
##  3 The Adventures of Tintin                               2011
##  4 War Horse                                              2011
##  5 Indiana Jones and the Kingdom of the Crystal Skull     2008
##  6 War of the Worlds                                      2005
##  7 The Terminal                                           2004
##  8 Minority Report                                        2002
##  9 Catch Me If You Can                                    2002
## 10 A.I. Artificial Intelligence                           2001
## # ... with 14 more rows
```

d. filmes que tenham "Action" **ou** "Comedy" entre os seus gêneros;

```
library(stringr)

# Solução 1
imdb %>%
  filter(str_detect(generos, "Action") | str_detect(generos, "Comedy"))
```

```
## # A tibble: 2,187 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <dbl> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 2,177 more rows, and 6 more variables: receita <dbl>,
## #   nota_imdb <dbl>, likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>
```

e. filmes que tenham "Action" **e** "Comedy" entre os seus gêneros e tenha `nota_imdb` maior que 8;

```
imdb %>%
  filter(str_detect(generos, "Action"), str_detect(generos, "Comedy"))
```

```
## # A tibble: 203 x 15
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <dbl> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 Men in ~  2012 Barry S~     106 Color Action|A~ USA   A partir de ~ 225000000
##  2 Rush Ho~  2007 Brett R~      91 Color Action|C~ USA   A partir de ~ 140000000
##  3 Monster~  2009 Rob Let~      94 Color Action|A~ USA   Livre         175000000
##  4 Wild Wi~  1999 Barry S~     106 Color Action|C~ USA   A partir de ~ 170000000
##  5 Suicide~  2016 David A~     123 Color Action|A~ USA   A partir de ~ 175000000
##  6 Big Her~  2014 Don Hall     102 Color Action|A~ USA   Livre         165000000
##  7 Madagas~  2008 Eric Da~      89 Color Action|A~ USA   Livre         150000000
##  8 G-Force   2009 Hoyt Ye~      88 Color Action|A~ USA   Livre         150000000
##  9 Kung Fu~  2011 Jennife~      90 Color Action|A~ USA   Livre         150000000
## 10 Mars Ne~  2011 Simon W~      88 Color Action|A~ USA   Livre         150000000
## # ... with 193 more rows, and 6 more variables: receita <dbl>, nota_imdb <dbl>,
## #   likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

f. filmes que não possuem informação tanto de receita quanto de orçamento (isto é, possuem `NA` em ambas as colunas).

```
imdb %>%
  filter(is.na(orcamento), is.na(receita))
```

```
## # A tibble: 137 x 15
##    titulo    ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>   <dbl> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 The A-~    NA <NA>          60 Color  Action|A~ USA   Outros               NA
##  2 Ben-Hu~  2016 Timur B~     141 Color  Adventur~ USA   A partir de ~        NA
##  3 Hannib~    NA <NA>          44 Color  Crime|Dr~ USA   Outros               NA
##  4 All Th~  1979 Bob Fos~     123 Color  Comedy|D~ USA   A partir de ~        NA
##  5 Darede~    NA <NA>          54 <NA>   Action|A~ USA   Outros               NA
##  6 Bewitc~    NA <NA>          25 Black~ Comedy|F~ USA   Outros               NA
##  7 Consta~    NA <NA>          43 Color  Drama|Fa~ USA   Outros               NA
##  8 Life       NA <NA>          45 Color  Crime|Dr~ USA   Outros               NA
##  9 Sleepy~    NA <NA>          45 Color  Adventur~ USA   Outros               NA
## 10 Last M~    NA <NA>          30 Color  Comedy    USA   Outros               NA
## # ... with 127 more rows, and 6 more variables: receita <dbl>, nota_imdb <dbl>,
## #   likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

Modificando e criando novas colunas

1. Crie uma coluna chamada `prejuizo` (`orcamento - receita`) e salve a nova tabela em um objeto chamado `imdb_prejuizo`. Em seguida, filtre apenas os filmes que deram prejuízo e ordene a tabela por ordem crescente de prejuízo.

```
imdb_prejuizo <- imdb %>%
  mutate(prejuizo = orcamento - receita)

imdb_prejuizo %>%
  filter(prejuizo > 0) %>%
  arrange(prejuizo)
```

```
## # A tibble: 1,312 x 16
##    titulo     ano diretor   duracao cor   generos  pais  classificacao orcamento
##    <chr>    <dbl> <chr>       <dbl> <chr> <chr>    <chr> <chr>             <dbl>
##  1 The Hor~  2009 Michel O~      93 Color Documen~ USA   Outros           160000
##  2 Newlywe~  2011 Edward B~      95 Color Comedy|~ USA   Outros             9000
##  3 Shortbu~  2006 John Cam~     101 Color Comedy|~ USA   Outros          2000000
##  4 Slam      1998 Marc Lev~     100 Color Drama    USA   A partir de ~   1000000
##  5 Pink Na~  1971 James Bi~      65 Color Drama|F~ USA   Outros            27000
##  6 The FP    2011 Brandon ~      82 Color Comedy   USA   A partir de ~     60000
##  7 Assault~  2005 Jean-Fra~     109 Color Action|~ USA   A partir de ~  20000000
##  8 The Bus~  2002 Sherman ~     103 Color Drama|M~ USA   Outros           200000
##  9 Latter ~  2003 C. Jay C~     107 Color Comedy|~ USA   A partir de ~    850000
## 10 Happy C~  2014 Joe Swan~      82 Color Comedy|~ USA   A partir de ~     70000
## # ... with 1,302 more rows, and 7 more variables: receita <dbl>,
## #   nota_imdb <dbl>, likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>, prejuizo <dbl>
```

---

2. Fazendo apenas uma chamada da função mutate(), crie as seguintes colunas novas na base `imdb`:

a. `lucro = receita - orcamento`

b. `lucro_medio`

c. `lucro_relativo = (lucro - lucro_medio)/lucro_medio`

d. `houve_lucro = ifelse(lucro > 0, "sim", "não")`

```
imdb %>%
  mutate(
    lucro = receita - orcamento,
    lucro_medio = mean(lucro, na.rm = TRUE),
    lucro_relativo = (lucro - lucro_medio) / lucro_medio,
    houve_lucro = ifelse(lucro > 0, "sim", "não")
  )
```

```
## # A tibble: 3,807 x 19
```

```
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <dbl> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 10 more variables: receita <dbl>,
## #   nota_imdb <dbl>, likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>, lucro <dbl>, lucro_medio <dbl>, lucro_relativo <dbl>,
## #   houve_lucro <chr>
```

---

3. Crie uma nova coluna que classifique o filme em `"recente"` (posterior a 2000) e `"antigo"` (de 2000 para trás).

```
imdb %>%
  mutate(antigo_ou_recente = ifelse(ano > 2000, "recente", "antigo"))
```

```
## # A tibble: 3,807 x 16
##    titulo     ano diretor  duracao cor   generos   pais  classificacao orcamento
##    <chr>    <dbl> <chr>      <dbl> <chr> <chr>     <chr> <chr>             <dbl>
##  1 Avatar    2009 James C~     178 Color Action|A~ USA   A partir de ~ 237000000
##  2 Pirates~  2007 Gore Ve~     169 Color Action|A~ USA   A partir de ~ 300000000
##  3 The Dar~  2012 Christo~     164 Color Action|T~ USA   A partir de ~ 250000000
##  4 John Ca~  2012 Andrew ~     132 Color Action|A~ USA   A partir de ~ 263700000
##  5 Spider-~  2007 Sam Rai~     156 Color Action|A~ USA   A partir de ~ 258000000
##  6 Tangled   2010 Nathan ~     100 Color Adventur~ USA   Livre         260000000
##  7 Avenger~  2015 Joss Wh~     141 Color Action|A~ USA   A partir de ~ 250000000
##  8 Batman ~  2016 Zack Sn~     183 Color Action|A~ USA   A partir de ~ 250000000
##  9 Superma~  2006 Bryan S~     169 Color Action|A~ USA   A partir de ~ 209000000
## 10 Pirates~  2006 Gore Ve~     151 Color Action|A~ USA   A partir de ~ 225000000
## # ... with 3,797 more rows, and 7 more variables: receita <dbl>,
## #   nota_imdb <dbl>, likes_facebook <dbl>, ator_1 <chr>, ator_2 <chr>,
## #   ator_3 <chr>, antigo_ou_recente <chr>
```

Summarisando a base

Utilize a base `imdb` nos exercícios a seguir.

1. Calcule a duração média e mediana dos filmes da base.

```
imdb %>%
  summarise(
    duracao_media = mean(duracao, na.rm = TRUE),
    duracao_mediana = median(duracao, na.rm = TRUE)
  )
```

```
## # A tibble: 1 x 2
##   duracao_media duracao_mediana
##           <dbl>           <dbl>
## 1          106.             103
```

---

2. Calcule o lucro médio dos filmes com duração menor que 60 minutos.

```
imdb %>%
  filter(duracao < 60) %>%
  mutate(lucro = receita - orcamento) %>%
  summarise(lucro_medio = mean(lucro, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   lucro_medio
##         <dbl>
## 1     632454.
```

---

3. Apresente na mesma tabela o lucro médio dos filmes com duracao menor que 60 minutos e o lucro médio dos filmes com duracao maior ou igual a 60 minutos.

```
imdb %>%
  mutate(
    lucro = receita - orcamento,
    duracao_cat = ifelse(duracao < 60, "menor 60 min", "60 ou mais min")
  ) %>%
  filter(!is.na(lucro)) %>%
  group_by(duracao_cat) %>%
  summarise(
    lucro_medio = mean(lucro, na.rm = TRUE)
  )
```

```
## # A tibble: 2 x 2
##   duracao_cat    lucro_medio
##   <chr>                <dbl>
## 1 60 ou mais min   17274472.
## 2 menor 60 min       632454.
```

---

4. Retorne tabelas (`tibbles`) apenas com:

a. a nota IMDB média dos filmes por tipo de classificacao;

```
imdb %>%
  group_by(classificacao) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE))
```

```
## # A tibble: 4 x 2
##   classificacao       nota_media
##   <chr>                    <dbl>
## 1 A partir de 13 anos       6.18
## 2 A partir de 18 anos       6.47
## 3 Livre                     6.26
## 4 Outros                    6.65
```

b. a receita média e mediana dos filmes por ano;

```
imdb %>%
  group_by(ano) %>%
  summarise(
    receita_media = mean(receita, na.rm = TRUE),
    receita_mediana = median(receita, na.rm = TRUE)
  )
```

```
## # A tibble: 91 x 3
##      ano receita_media receita_mediana
##    <dbl>         <dbl>           <dbl>
##  1  1916           NaN              NA
##  2  1920       3000000         3000000
##  3  1925           NaN              NA
##  4  1929       2808000         2808000
##  5  1930           NaN              NA
##  6  1932           NaN              NA
##  7  1933       2300000         2300000
##  8  1934           NaN              NA
```

```
##  9  1935       3000000          3000000
## 10  1936        163245           163245
## # ... with 81 more rows
```

c. apenas o nome dos diretores com mais de 10 filmes.

```
imdb %>%
  filter(!is.na(diretor)) %>%
  group_by(diretor) %>%
  summarise(num_filmes = n()) %>%
  filter(num_filmes > 10) %>%
  select(diretor)
```

```
## # A tibble: 22 x 1
##    diretor
##    <chr>
##  1 Barry Levinson
##  2 Clint Eastwood
##  3 Francis Ford Coppola
##  4 John Carpenter
##  5 Kevin Smith
##  6 Martin Scorsese
##  7 Michael Bay
##  8 Renny Harlin
##  9 Ridley Scott
## 10 Rob Cohen
## # ... with 12 more rows
```

Também podemos usar a função `count()`.

```
imdb %>%
  filter(!is.na(diretor)) %>%
  count(diretor, name = "num_filmes") %>%
  filter(num_filmes > 10) %>%
  select(diretor)
```

```
## # A tibble: 22 x 1
##    diretor
##    <chr>
##  1 Barry Levinson
##  2 Clint Eastwood
##  3 Francis Ford Coppola
##  4 John Carpenter
##  5 Kevin Smith
```

```
##  6 Martin Scorsese
##  7 Michael Bay
##  8 Renny Harlin
##  9 Ridley Scott
## 10 Rob Cohen
## # ... with 12 more rows
```

Juntando duas bases

1. Utilize a base `imdb` para resolver os itens a seguir.

a. Salve em um novo objeto uma tabela com a nota média dos filmes de cada diretor. Essa tabela deve conter duas colunas (`diretor` e `nota_imdb_media`) e cada linha deve ser um diretor diferente.

```
nota_diretores <- imdb %>%
  group_by(diretor) %>%
  summarise(nota_imdb_media = mean(nota_imdb))

nota_diretores
```

```
## # A tibble: 1,813 x 2
##    diretor          nota_imdb_media
##    <chr>                      <dbl>
##  1 A. Raven Cruz                1.9
##  2 Aaron Hann                   6
##  3 Aaron Schneider              7.1
##  4 Aaron Seltzer                2.7
##  5 Abel Ferrara                 6.6
##  6 Adam Carolla                 6.1
##  7 Adam Goldberg                5.4
##  8 Adam Green                   5.7
##  9 Adam Jay Epstein             3.8
## 10 Adam Marcus                  4.3
## # ... with 1,803 more rows
```

b. Use o `left_join()` para trazer a coluna `nota_imdb_media` da tabela do item anterior para a tabela `imdb` original.

```
imdb %>%
  left_join(nota_diretores, by = "diretor") %>%
  select(nota_imdb_media, everything())
```

```
## # A tibble: 3,807 x 16
```

```
##    nota_imdb_media titulo        ano diretor   duracao cor   generos       pais
##              <dbl> <chr>       <dbl> <chr>       <dbl> <chr> <chr>         <chr>
##  1            7.88 Avatar       2009 James Ca~     178 Color Action|Adven~ USA
##  2            6.99 Pirates of~  2007 Gore Ver~     169 Color Action|Adven~ USA
##  3            8.43 The Dark K~  2012 Christop~     164 Color Action|Thril~ USA
##  4            7.73 John Carte~  2012 Andrew S~     132 Color Action|Adven~ USA
##  5            6.9  Spider-Man~  2007 Sam Raimi     156 Color Action|Adven~ USA
##  6            7.8  Tangled      2010 Nathan G~     100 Color Adventure|An~ USA
##  7            7.92 Avengers: ~  2015 Joss Whe~     141 Color Action|Adven~ USA
##  8            7.18 Batman v S~  2016 Zack Sny~     183 Color Action|Adven~ USA
##  9            7.26 Superman R~  2006 Bryan Si~     169 Color Action|Adven~ USA
## 10            6.99 Pirates of~  2006 Gore Ver~     151 Color Action|Adven~ USA
## # ... with 3,797 more rows, and 8 more variables: classificacao <chr>,
## #   orcamento <dbl>, receita <dbl>, nota_imdb <dbl>, likes_facebook <dbl>,
## #   ator_1 <chr>, ator_2 <chr>, ator_3 <chr>
```

13.5 O pacote stringr

```
library(stringr)
library(dplyr)
library(glue)
```

1. O CPF é um número de 11 dígitos, por exemplo: 54491651884. No entanto para facilitar a visualização costumamos mostrá-lo com separadores a cada 3 casas: 544.916.518-84. Crie uma função que transforma um número de 11 dígitos em uma string com as separações, como um CPF.

```
# Sem regex
formata_CPF <- function(num) {

  if(str_count(num) != 11) {
    stop("Número inválido!")
  }

  # Concatemos cada parte do número, interpolando
  # com os pontos e o traço.

  s <- str_c(
    str_sub(num, start = 1, end = 3),
    ".",
    str_sub(num, start = 4, end = 6),
    ".",
    str_sub(num, start = 7, end = 9),
```

```
    "-",
    str_sub(num, start = 10, end = 11)
  )

}
# Com regex
formata_CPF <- function(num) {

  if(str_count(num) != 11) {
    stop("Número inválido!")
  }

  str_replace(
    string = num,
    pattern = "([0-9]{3})([0-9]{3})([0-9]{3})",
    replacement = "\\1.\\2.\\3-"
  )
}
```

A função `str_replace()` faz o seguinte:

- O `pattern=` procurará um padrão de 9 números:
  - O primeiro `([0-9]{3})` pega os três primeiros números.
  - O segundo `([0-9]{3})` pega os três próximos números (quarto ao sexto).
  - O terceiro `([0-9]{3})` pega os três números seguintes (sétimo ao nono).
- O `replacement=` substituirá esses 9 números pela *string* formada por:
  - `\\1`, a expressão dada pelo primeiro parêntese do `pattern=`, isto é, os três primeiros números.
  - `.`, o primeiro ponto.
  - `\\2`, a expressão dada pelo segundo parêntese do `pattern=`, isto é, os três próximos números (quarto ao sexto).
  - `.`, o segundo ponto.
  - `\\3`, a expressão dada pelo terceiro parêntese do `pattern=`, isto é, os três números seguintes (sétimo ao nono).
  - `-`, o traço.

---

2. Transforme o vetor de *strings* abaixo em `"01 - Alto" "02 - Médio" "03 - Baixo"`.

```
s <- c('Alto', 'Médio', 'Baixo')
# Usando str_c()
str_c("0", 1:length(s), " - ", s, sep = "")
## [1] "01 - Alto"  "02 - Médio" "03 - Baixo"

# Usando o pacote glue
glue("0{1:length(s)} - {s}")
## 01 - Alto
## 02 - Médio
## 03 - Baixo
```

Para mais informações sobre o pacote `glue`, confira este post.

---

3. Crie uma regex que capture múltiplas versões da palavra 'casa'. Ela deve funcionar com as palavras 'Casa', 'CASA', 'CaSa', 'CAsa'. Teste-a usando a função `str_detect()`.

```
s <- c('Casa', 'CASA', 'CaSa', 'CAsa')
str_detect(s, "[CcAaSsAa]")
## [1] TRUE TRUE TRUE TRUE
str_detect(s, "[Cc][Aa][Ss][Aa]")
## [1] TRUE TRUE TRUE TRUE
```

---

4. Imagine que a seguinte *string* é a parte final de uma URL.

- `/ac/rio-branco/xpto-xyz-1-0-1fds2396-5`

Transforme-a em "AC - Rio Branco" utilizando a função `str_split()`.

```
url <- c('/ac/rio-branco/xpto-xyz-1-0-1fds2396-5')
UF <- url %>%
  str_split("[/]", simplify = TRUE) %>%
  .[1,2] %>%
  str_to_upper()
cidade <- url %>%
  str_split("[/]", simplify = TRUE) %>%
  .[1,3] %>%
  str_replace("-", " ") %>%
  str_to_title()
str_c(UF, " - ", cidade)
## [1] "AC - Rio Branco"
```

---

5. Crie uma função que retorna `TRUE` quando a string é um palíndromo e `FALSO` caso não seja.

```
# Solução 1: usando a função rev() para inverter uma string fragmentada
testa_palindromo <- function(s) {

  s %>%
    str_split("", simplify = T) %>%
    rev %>%
    str_c(collapse = "") %>%
    str_detect(s)

}
testa_palindromo("ana")
## [1] TRUE
testa_palindromo("bananas")
## [1] FALSE
testa_palindromo("socorrammesubinoonibusemmarrocos")
## [1] TRUE

# Solução 2: usando a função str_reverse() do pacote stringi,
# que já inverte a string diretamente.
testa_palindromo <- function(s) {

  s %>%
    stringi::stri_reverse() %>%
    str_detect(s)

}

testa_palindromo("ana")
## [1] TRUE
testa_palindromo("bananas")
## [1] FALSE
testa_palindromo("socorrammesubinoonibusemmarrocos")
## [1] TRUE
```

---

6. De acordo com as regras da língua portuguesa, antes de “p” ou “b” devemos usar a letra “m”. Em outras palavras, com outras consoantes, usamos a letra “N”. Suponha que você tem o seguinte texto com erros gramaticais:

```
s <- 'Nós chamamos os bonbeiros quando começou o incêmdio.'
```

Crie uma função para corrigi-lo.

```
# Função que funciona para o exemplo
corrige_mn <- function(s) {

  s %>%
    str_replace("nb", "mb") %>%
    str_replace("md", "nd")

}
corrige_mn(s)
## [1] "Nós chamamos os bombeiros quando começou o incêndio."

# Função que funciona no caso geral
corrige_mn <- function(s) {

  s %>%
    str_replace_all("m([^aeioubp[[:space:]]+])", "n\\1") %>%
    str_replace_all("n([pb])", "m\\1")

}
corrige_mn(s)
## [1] "Nós chamamos os bombeiros quando começou o incêndio."
```

O padrão `[^aeioubp[[:space:]]+]` significa "tudo menos vogais, b, p ou espaços". O `\\1` devolve o padrão encontrado pelo primeiro parêntese do argumento `patern=`.

---

7. Considere o seguinte texto

```
s <- "A função mais importante para leitura de dados no `lubridate` é a `ymd`. Essa função serve
```

Extraia todas as combinações da função `ymd`, sem repetições.

```
str_extract_all(s, "[ymd]{3}") %>%
  unlist() %>%
  unique
## [1] "ymd" "mdy" "dmy" "dym" "myd" "ydm"
```

---

8. Considere as frases abaixo

```
s <- c(
  'O produto é bom.',
  'O produto não é bom.',
  'O produto não é muito bom.',
  'O produto é muito bom',
  'O produto não é ruim.',
  'O produto não é não ruim.',
  'O produto não é não bom.'
)
```

Crie uma regra para identificar se o texto refere-se a um feedback positivo ou negativo sobre o produto (considere não bom = ruim e não ruim = bom). Retorne um vetor lógico que vale `TRUE` se o feedback for positivo e `FALSE` caso contrário.

```
feedback <- function(s) {

  s %>%
    str_replace("não bom", "ruim") %>%
    str_replace("não ruim", "bom") %>%
    str_replace("muito ", "") %>%
    str_detect("(.*) produto é bom|(.*) não é ruim")

}
feedback(s)
## [1]  TRUE FALSE FALSE  TRUE  TRUE FALSE  TRUE
```

13.6 O pacote lubridate

```
library(dplyr)
library(lubridate)
library(ggplot2)

lakers <- tibble::as_tibble(lakers)
```

1. Repare que a coluna `date` no data.frame é um vetor de inteiros. Transforme essa coluna em um vetor de valores com classe `date`.

```
lakers %>%
  mutate(date = ymd(date))
## # A tibble: 34,624 x 13
##    date       opponent game_type time  period etype  team  player  result points
##    <date>     <chr>    <chr>     <chr>  <int> <chr>  <chr> <chr>   <chr>   <int>
##  1 2008-10-28 POR      home      12:00      1 jump ~ OFF   ""      ""          0
##  2 2008-10-28 POR      home      11:39      1 shot   LAL   "Pau G~ "miss~      0
##  3 2008-10-28 POR      home      11:37      1 rebou~ LAL   "Vladi~ ""          0
##  4 2008-10-28 POR      home      11:25      1 shot   LAL   "Derek~ "miss~      0
##  5 2008-10-28 POR      home      11:23      1 rebou~ LAL   "Pau G~ ""          0
##  6 2008-10-28 POR      home      11:22      1 shot   LAL   "Pau G~ "made"      2
##  7 2008-10-28 POR      home      11:22      1 foul   POR   "Greg ~ ""          0
##  8 2008-10-28 POR      home      11:22      1 free ~ LAL   "Pau G~ "made"      1
##  9 2008-10-28 POR      home      11:00      1 foul   LAL   "Vladi~ ""          0
## 10 2008-10-28 POR      home      10:53      1 shot   POR   "LaMar~ "made"      2
## # ... with 34,614 more rows, and 3 more variables: type <chr>, x <int>, y <int>
```

Repare que `as_date()` não funciona neste caso.

```
lakers %>%
  mutate(date = as_date(date))
## # A tibble: 34,624 x 13
##    date       opponent game_type time  period etype  team  player  result points
##    <date>     <chr>    <chr>     <chr>  <int> <chr>  <chr> <chr>   <chr>   <int>
##  1 56949-12-27 POR      home      12:00      1 jump ~ OFF   ""      ""          0
##  2 56949-12-27 POR      home      11:39      1 shot   LAL   "Pau G~ "miss~      0
##  3 56949-12-27 POR      home      11:37      1 rebou~ LAL   "Vladi~ ""          0
##  4 56949-12-27 POR      home      11:25      1 shot   LAL   "Derek~ "miss~      0
##  5 56949-12-27 POR      home      11:23      1 rebou~ LAL   "Pau G~ ""          0
##  6 56949-12-27 POR      home      11:22      1 shot   LAL   "Pau G~ "made"      2
##  7 56949-12-27 POR      home      11:22      1 foul   POR   "Greg ~ ""          0
##  8 56949-12-27 POR      home      11:22      1 free ~ LAL   "Pau G~ "made"      1
##  9 56949-12-27 POR      home      11:00      1 foul   LAL   "Vladi~ ""          0
## 10 56949-12-27 POR      home      10:53      1 shot   POR   "LaMar~ "made"      2
## # ... with 34,614 more rows, and 3 more variables: type <chr>, x <int>, y <int>
```

Para entender porque a função devolveu um dia do ano 56949, rode os códigos abaixo.

```
as_date(0)
## [1] "1970-01-01"
as_date(-3:3)
## [1] "1969-12-29" "1969-12-30" "1969-12-31" "1970-01-01" "1970-01-02"
## [6] "1970-01-03" "1970-01-04"
```

```
as_date(20081027:20081029)
## [1] "56949-12-26" "56949-12-27" "56949-12-28"
```

---

2. Crie uma coluna que junte as informações de data e hora (colunas `date` e `time`) em objetos da classe `date`.

```
lakers %>%
  mutate(date_time = paste0(date, " 00:", time) %>% ymd_hms) %>%
  select(date_time)
## # A tibble: 34,624 x 1
##    date_time
##    <dttm>
##  1 2008-10-28 00:12:00
##  2 2008-10-28 00:11:39
##  3 2008-10-28 00:11:37
##  4 2008-10-28 00:11:25
##  5 2008-10-28 00:11:23
##  6 2008-10-28 00:11:22
##  7 2008-10-28 00:11:22
##  8 2008-10-28 00:11:22
##  9 2008-10-28 00:11:00
## 10 2008-10-28 00:10:53
## # ... with 34,614 more rows
```

---

3. Crie as colunas `dia`, `mes` e `ano` com as respectivas informações sobre a data do jogo.

```
lakers %>%
  mutate(
    date = ymd(date),
    dia = day(date),
    mes = month(date),
    ano = year(date)
  ) %>%
  select(date, dia, mes, ano)
## # A tibble: 34,624 x 4
##    date         dia   mes   ano
##    <date>     <int> <dbl> <dbl>
##  1 2008-10-28    28    10  2008
```

```
##  2 2008-10-28    28    10  2008
##  3 2008-10-28    28    10  2008
##  4 2008-10-28    28    10  2008
##  5 2008-10-28    28    10  2008
##  6 2008-10-28    28    10  2008
##  7 2008-10-28    28    10  2008
##  8 2008-10-28    28    10  2008
##  9 2008-10-28    28    10  2008
## 10 2008-10-28    28    10  2008
## # ... with 34,614 more rows
```

---

4. Em média, quanto tempo o Lakers demora para arremessar a primeira bola no primeiro período?

Dicas: arremessos são representados pela categoria `shot` da coluna `etype` e cada período tem 12 minutos.

```
lakers %>%
  dplyr::filter(etype == "shot", period == 1, team == "LAL") %>%
  dplyr::mutate(
    time = hms(paste0("00:", time)),
    cronometro = 12*60 - minute(time)*60 - second(time)
  ) %>%
  dplyr::group_by(date) %>%
  dplyr::filter(cronometro == min(cronometro)) %>%
  dplyr::ungroup() %>%
  dplyr::summarise(media = mean(cronometro))
## # A tibble: 1 x 1
##   media
##   <dbl>
## 1  29.6
```

O que foi feito:

- Primeiro filtramos a base para arremessos, `etype == "shot"`, do primeiro período, `period ==`, que fossem do Lakers, `team == "LAL"`.
- Em seguida, mudamos a classe da coluna `time`, de `character` para `period`, e criamos a coluna `cronometro`, que contém o tempo passado (em segundos) até o instante do evento.
- Então agrupamos a base pelo dia e a filtramos apenas para o primeiro evento de cada dia, isto é, o evento que tem o menor valor na coluna `cronometro`. Assim, a coluna `cronometro` da base resultante terá o tempo do primeiro arremesso de cada jogo.

- Por fim, desagrupamos a base e calculamos a média da coluna `cronometro`.

---

5. Em média, quanto tempo demora para sair a primeira cesta de três pontos em cada um dos quatro períodos? Considere toda a base, e cestas de ambos os times.

```
lakers %>%
  dplyr::filter(etype == "shot", period %in% 1:4, points == "3") %>%
  dplyr::mutate(time = hms(paste0("00:", time)),
                cronometro = 12*60 - minute(time)*60 - second(time)) %>%
  dplyr::group_by(date, period) %>%
  dplyr::filter(cronometro == min(cronometro)) %>%
  dplyr::ungroup() %>%
  dplyr::group_by(period) %>%
  dplyr::summarise(media = mean(cronometro))
## # A tibble: 4 x 2
##   period media
##    <int> <dbl>
## 1      1  202.
## 2      2  219.
## 3      3  203.
## 4      4  192.
```

A resolução desse exercício é análoga ao anterior, só mudamos o filtro inicial e o agrupamento. Se você ficou com dúvidas, consulte o passo a passo na resolução do exercício 4.

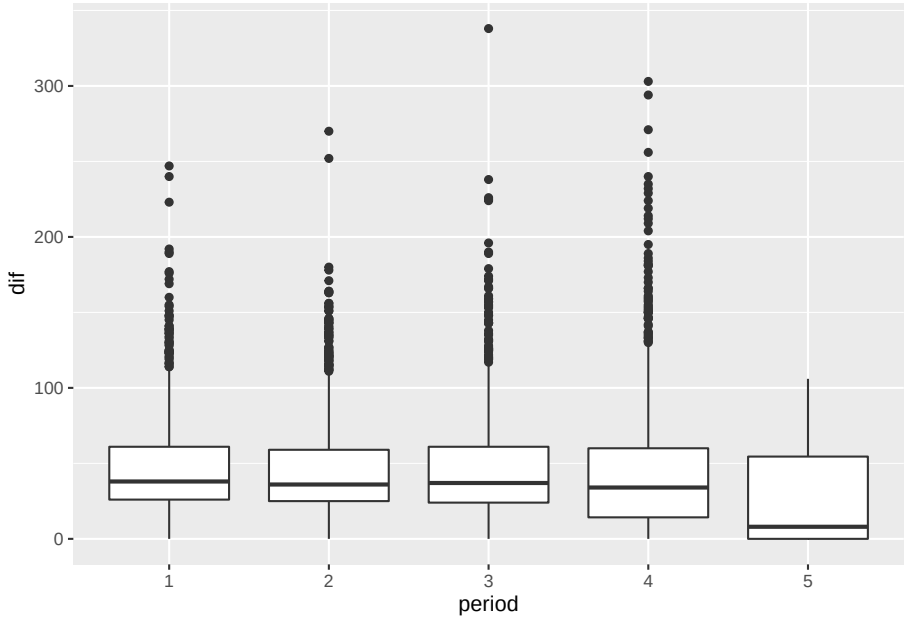
Repare que não precisarímos filtrar por `etype == "shot"`, já que o único evento que gera 3 pontos é a cesta de 3 pontos.

---

6. Construa boxplots do tempo entre pontos consecutivos para cada períodos. Considere toda a base de dados e apenas pontos do Lakers.

```
lakers %>%
  filter(points > 0, team == "LAL") %>%
    dplyr::mutate(
      time = hms(paste0("00:", time)),
      tempo_segundos = 60 * minute(time) + second(time),
      dif = tempo_segundos - lead(tempo_segundos),
      period = as.character(period)
    ) %>%
```

```
  filter(dif >= 0) %>%
  ggplot(aes(x = period, y = dif)) +
  geom_boxplot()
```



---

7. Qual foi o dia e mês do jogo que o Lakers demorou mais tempo para fazer uma cesta? Quanto tempo foi?

```
cesta_demorada <- lakers %>%
  filter(points > 0, team == "LAL", period == 1) %>%
    dplyr::mutate(
      time = hms(paste0("00:", time)),
      time = 60 * minute(time) + second(time),
      date = ymd(date),
      dia = day(date),
      mes = month(date),
    ) %>%
  group_by(date) %>%
  filter(time == max(time)) %>%
  ungroup() %>%
  slice_min(time)
```

```
cesta_demorada %>%
  select(date, opponent, dia, mes)
## # A tibble: 1 x 4
##   date       opponent   dia   mes
##   <date>     <chr>    <int> <dbl>
## 1 2009-04-12 MEM         12     4

tempo_seg <- 12*60 - cesta_demorada$time
seconds_to_period(tempo_seg)
## [1] "2M 37S"
```

13.7 O pacote forcats

```
library(dplyr)
library(forcats)
library(ggplot2)
```

1. Qual a diferença nos fatores criados com os códigos abaixo?

```
fator1 <- as.factor(c("c", "a", "z", "B"))
fator2 <- forcats::as_factor(c("c", "a", "z", "B"))
```

A função do R base `as.factor()` ordena os níveis do fator em ordem alfabética. A função `as_factor()` do pacote `forcats` ordena os níveis do fator na ordem que eles aparecem no vetor.

---

2. Ordene os níveis do fator `frutas` conforme a sua preferência, isto é, as que você mais gosta primeiro e as que você menos gosta por último.

```
frutas <- as.factor(c("maçã", "banana", "mamão", "laranja", "melancia"))
frutas
## [1] maçã     banana   mamão    laranja  melancia
## Levels: banana laranja maçã mamão melancia

# preferência do Willy
lvls_reorder(frutas, c(4, 5, 1, 3, 2))
## [1] maçã     banana   mamão    laranja  melancia
## Levels: mamão melancia banana maçã laranja
```

---

3. Com base no vetor `series`, resolva os itens a seguir.

```
series <- as.factor(c("Game of Thrones", "How I Met your Mother", "Friends", "Lost", "The Office"
```

a. Ordene os níveis do vetor `series` conforme a sua preferência, isto é, as que você mais gosta primeiro e as que você menos gosta por último.

```
levels(series)
## [1] "Breaking Bad"          "Friends"               "Game of Thrones"
## [4] "How I Met your Mother" "Lost"                  "The Office"

# preferência do Willy
series <- lvls_reorder(series, c(6, 5, 1, 3, 2, 4))
```

b. Junte ao vetor `series` o vetor `novas_series` a seguir, reordenando os níveis para manter a sua ordem de preferência.

```
novas_series <- as.factor(c("Stranger Things", "The Boys", "Queen's Gambit"))

series <- series %>%
  fct_c(novas_series)

levels(series)
## [1] "The Office"            "Lost"                  "Breaking Bad"
## [4] "Game of Thrones"       "Friends"               "How I Met your Mother"
## [7] "Queen's Gambit"        "Stranger Things"       "The Boys"

series <- lvls_reorder(series, c(1:7, 9, 8))
levels(series)
## [1] "The Office"            "Lost"                  "Breaking Bad"
## [4] "Game of Thrones"       "Friends"               "How I Met your Mother"
## [7] "Queen's Gambit"        "The Boys"              "Stranger Things"
```

c. Renomeie o níveis do vetor criado no item (b) para os nomes em Português das séries. Mantenha o mesmo nome caso não haja tradução.

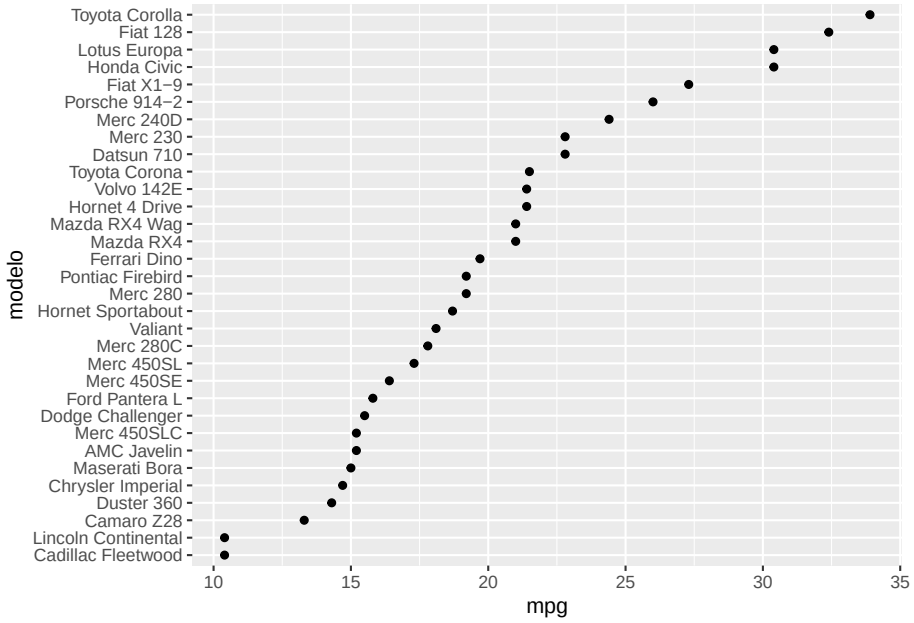
```
series %>%
  fct_recode(
    "A Guerra dos Tronos" = "Game of Thrones",
    "Como Eu Conheci Sua Mãe" = "How I Met your Mother",
    "A Química do Mal" = "Breaking Bad",
    "O Gambito da Rainha" = "Queen's Gambit"
```

```
  )
## [1] A Guerra dos Tronos     Como Eu Conheci Sua Mãe Friends
## [4] Lost                    The Office              A Química do Mal
## [7] Stranger Things         The Boys                O Gambito da Rainha
## 9 Levels: The Office Lost A Química do Mal A Guerra dos Tronos ... Stranger Things
```

---

4. Ordene as categorias do eixo y do gráfico abaixo para que os pontos no eixo x fique em ordem crescente.

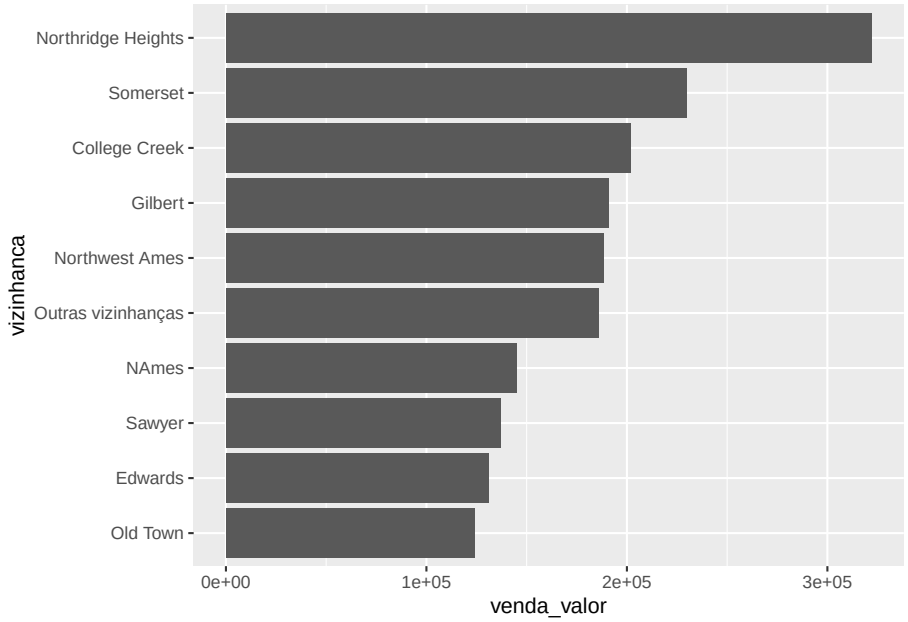
```
mtcars %>%
  tibble::rownames_to_column("modelo") %>%
  mutate(modelo = fct_reorder(modelo, mpg)) %>%
  ggplot(aes(x = mpg, y = modelo)) +
  geom_point()
```



---

5. Utilize a base `dados::casas` para fazer um gráfico de barras mostrando as vizinhanças (coluna `vizinhanca`) com casas mais caras (segundo a coluna `venda_valor`). O gráfico deve conter as 9 vizinhanças mais frequentes e as demais devem ser agrupadas em uma categoria chamada `Outras vizinhanças`.

```
dados::casas %>%
  mutate(vizinhanca = fct_lump_n(vizinhanca, 9, other_level = "Outras vizinhanças")) %>%
  group_by(vizinhanca) %>%
  summarise(venda_valor = mean(venda_valor, na.rm = TRUE)) %>%
  mutate(vizinhanca = fct_reorder(vizinhanca, venda_valor)) %>%
  ggplot(aes(x = venda_valor, y = vizinhanca)) +
  geom_col()
```



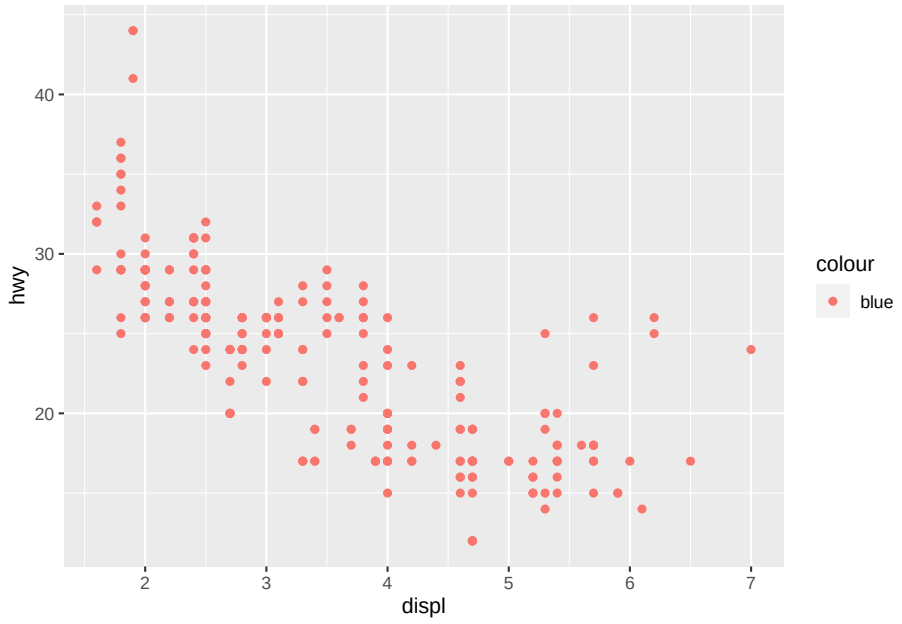
13.8 O pacote ggplot2

1. O que acontece quando rodamos o código `ggplot(data = mtcars)`?

A função `ggplot()` cria o nosso canvas, um quadro em branco onde vamos colocar todas as outras camadas do gráfico. Dentro do R, isso corresponde a uma lista com as informações necessárias para a criação do gráfico. Cada camada adicionada ao ggplot adiciona mais informações nessa lista. No exemplo acima, como passamos a base `mtcars` para a função `ggplot()`, a base mtcars é guardada dentro dessa lista, e será utilizada quando mapearmos variáveis ao gráfico.

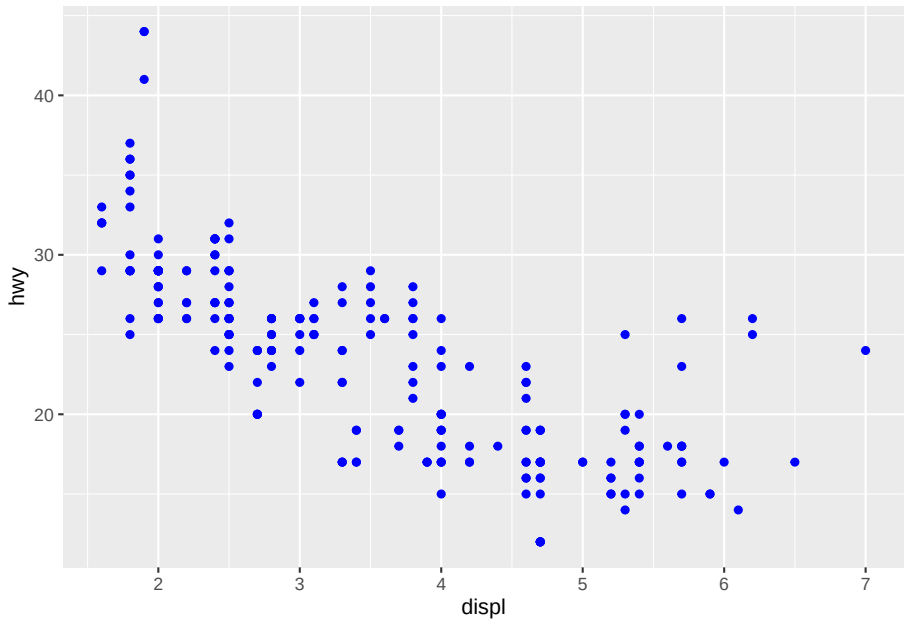
2. O que tem de errado no código abaixo? Por que os pontos não ficaram azuis?

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = "blue"))
```



Quando queremos definir o valor de um atributo estético, fazemos isso fora da função `aes()`*. Usamos a função* `aes()` *apenas para mapear variáveis aos atributos estéticos do gráfico. O correto seria:*

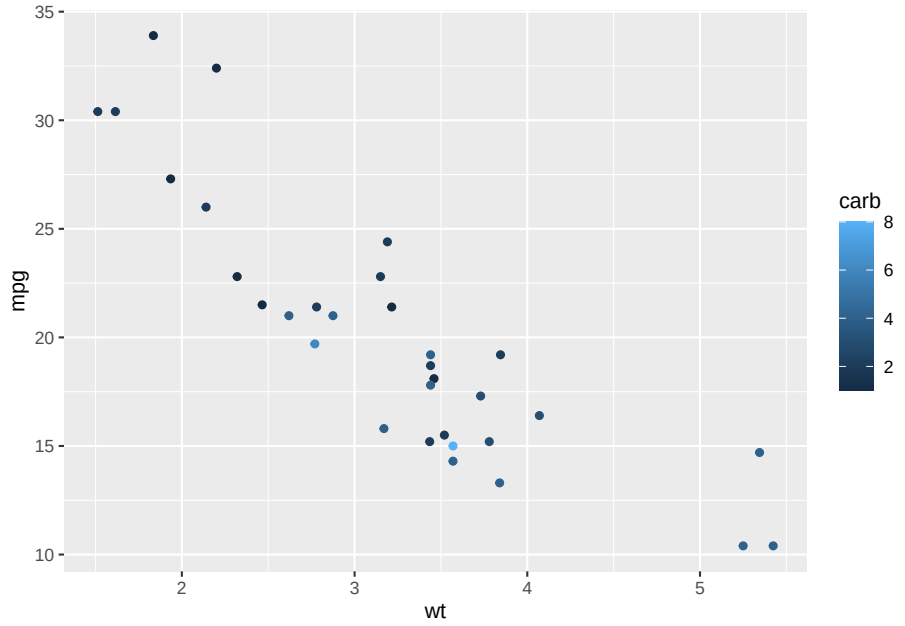
```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```



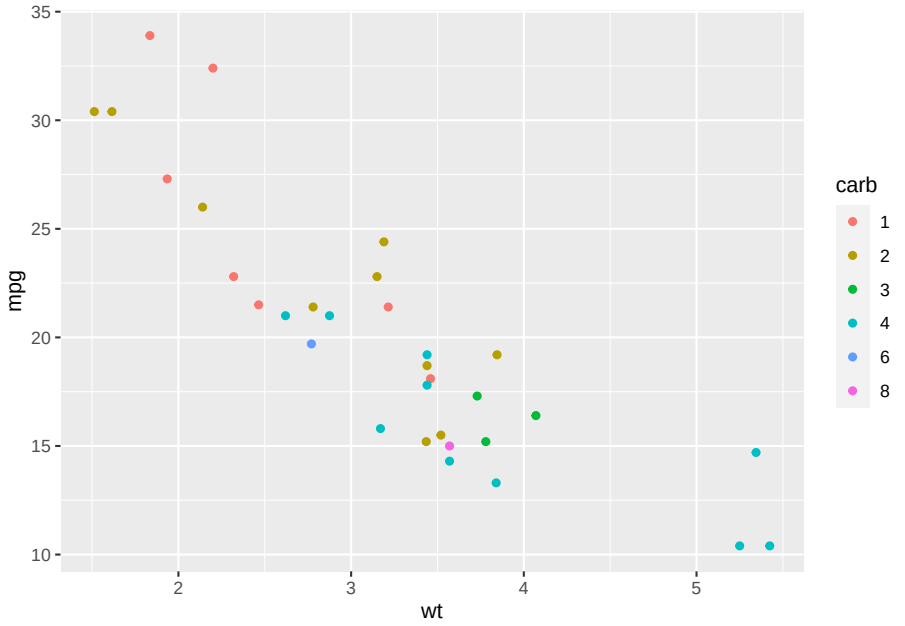
---

3. Mapeie uma variável contínua para a cor, o tamanho e a forma de um gráfico de pontos. Como esses atributos estéticos se comportam diferente para variáveis categóricas vs contínuas?

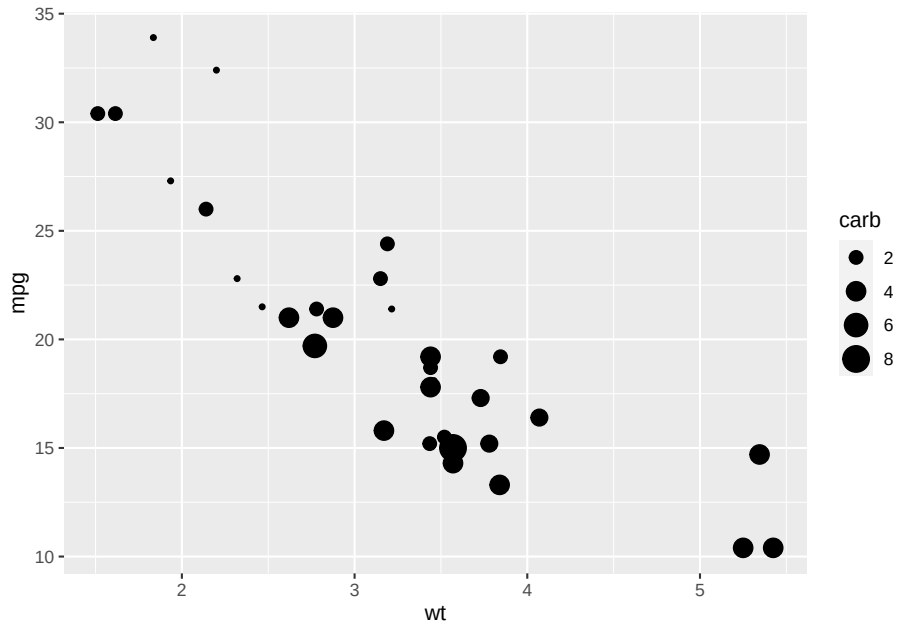
```
# Cor (var continua)
mtcars %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, color = carb))
```




```
# Cor (var discreta)
mtcars %>%
  mutate(carb = as.character(carb)) %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, color = carb))
```

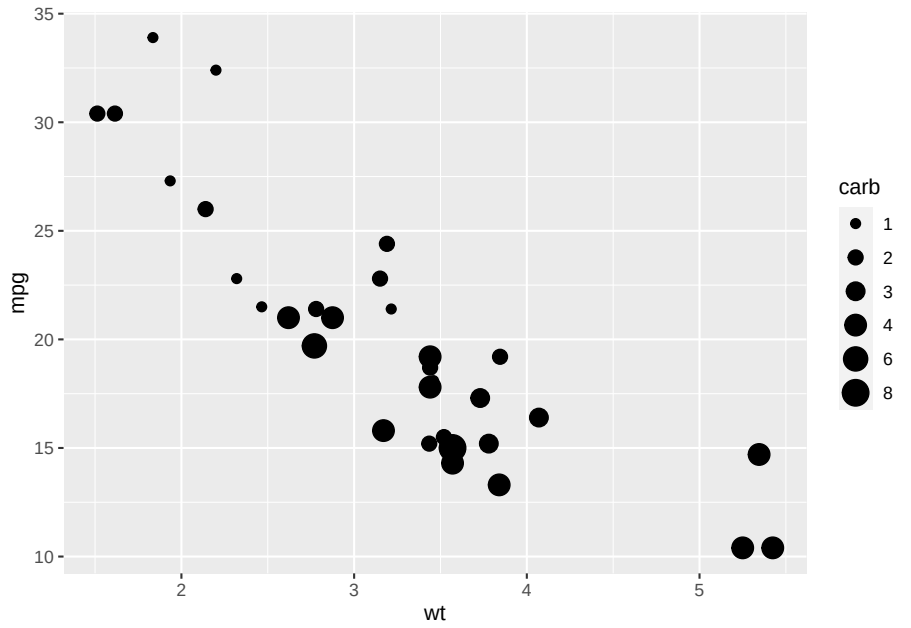



```
# Tamanho (var continua)
mtcars %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, size = carb))
```




```
# Tamanho (var discreta)
mtcars %>%
  mutate(carb = as.character(carb)) %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, size = carb))
```

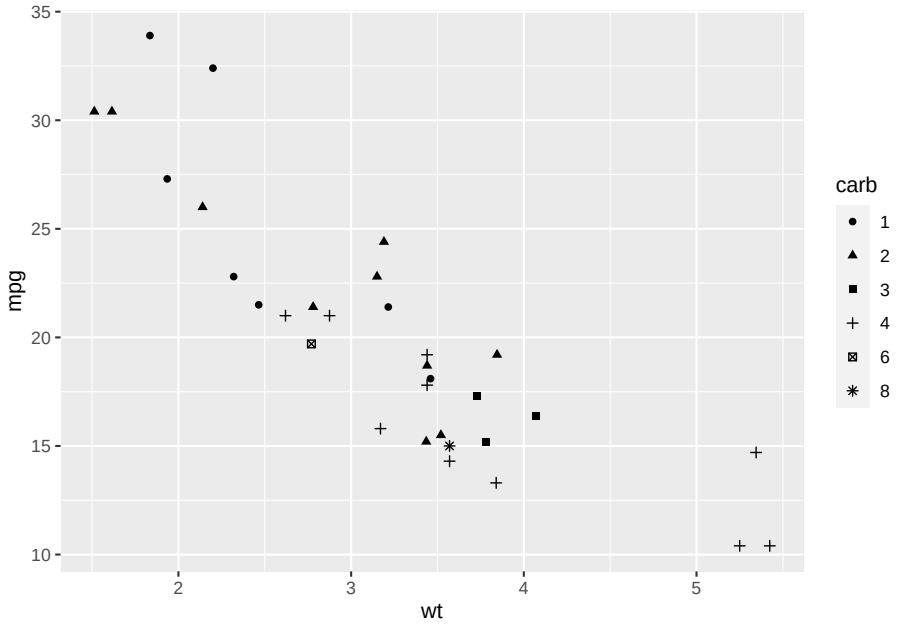
```
## Warning: Using size for a discrete variable is not advised.
```




```
# Forma (var continua)
# Não pode ser utilizada com variável contínua
mtcars %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, shape = carb))
```

```
## Error: A continuous variable can not be mapped to shape
```

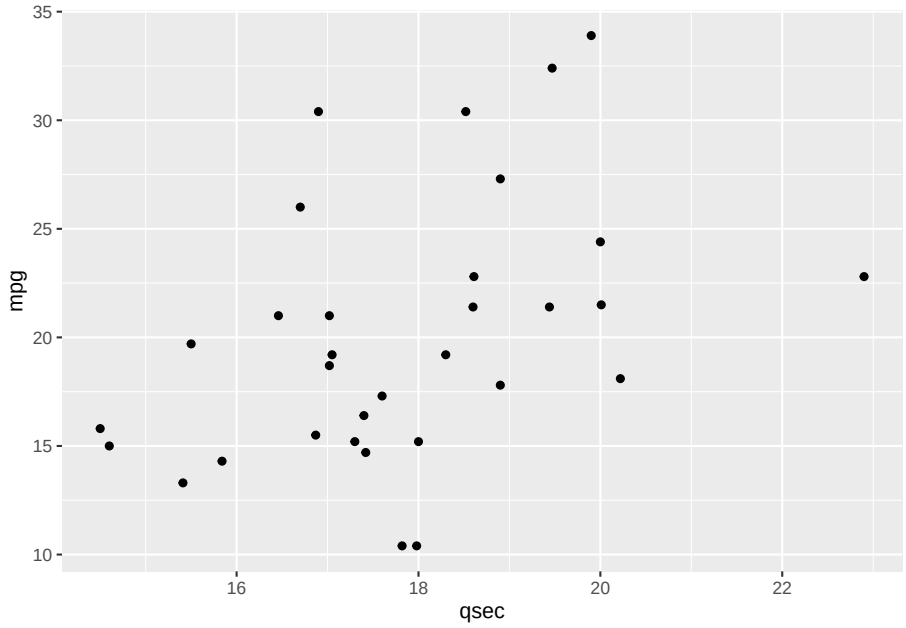
```
# Forma (var discreta)
mtcars %>%
  mutate(carb = as.character(carb)) %>%
  ggplot() +
  geom_point(aes(y = mpg, x = wt, shape = carb))
```



---

4. Utilizando o `mtcars`, faça um gráfico de dispersão de `mpg` por `qsec`.

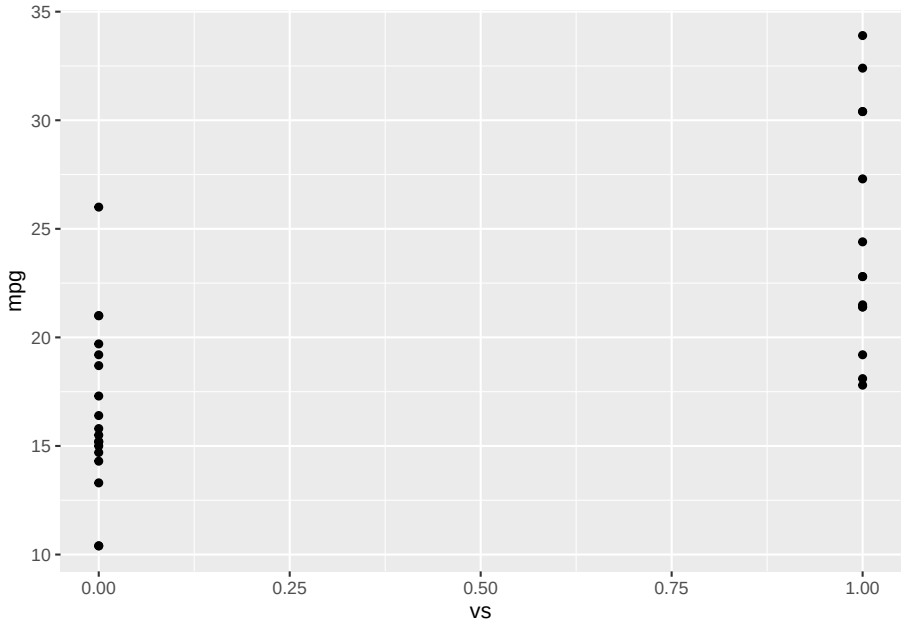
```
mtcars %>%
  ggplot() +
  geom_point(aes(y = mpg, x = qsec))
```



---

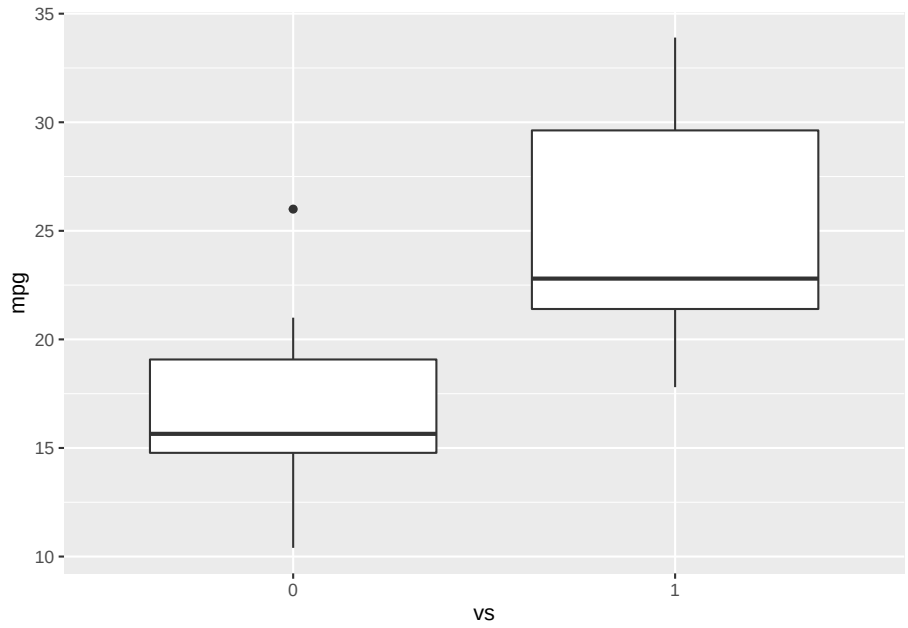
5. Utilizando o `mtcars`, o que acontece se você fizer um gráfico de dispersão de `vs` por `mpg`? Qual gráfico seria melhor para representar esses dados?

```
mtcars %>%
  ggplot(aes(y = mpg, x = vs)) +
  geom_point()
```



Como os pontos ficam sobrepostos, um boxplot seria mais interessante nesse caso pois nos dá uma visão melhor da distribuição da variável ***mpg*** *para cada categoria da variável* ***vs****.*

```
mtcars %>%
  mutate(vs = as.character(vs)) %>%
  ggplot(aes(y = mpg, x = vs)) +
  geom_boxplot()
```

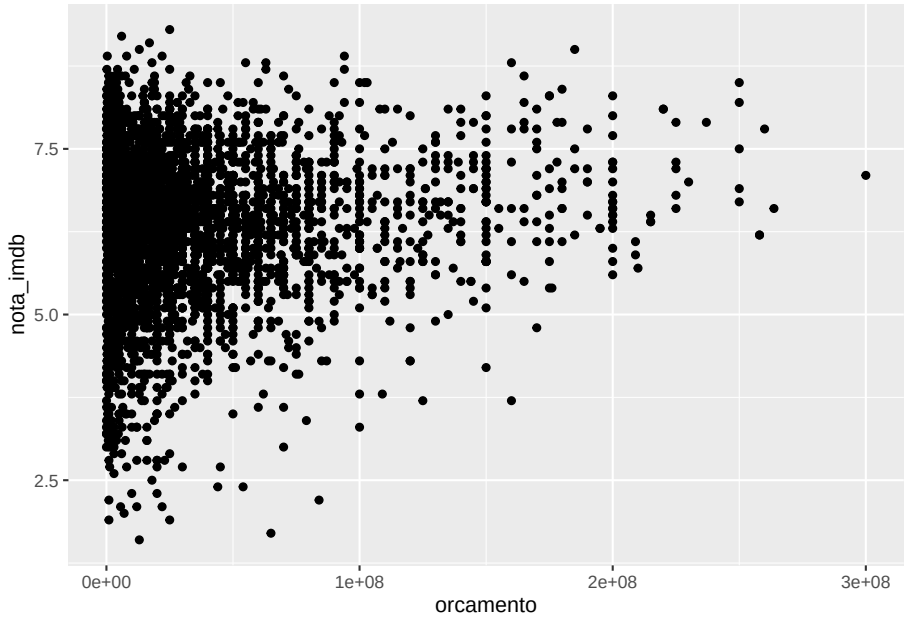


Para resolver os exercícios a seguir, utilize a base `imdb`.

6. Crie um gráfico de dispersão da nota do imdb pelo orçamento.

```
imdb %>%
  ggplot(aes(y = nota_imdb, x = orcamento)) +
  geom_point()
```

```
## Warning: Removed 298 rows containing missing values (geom_point).
```

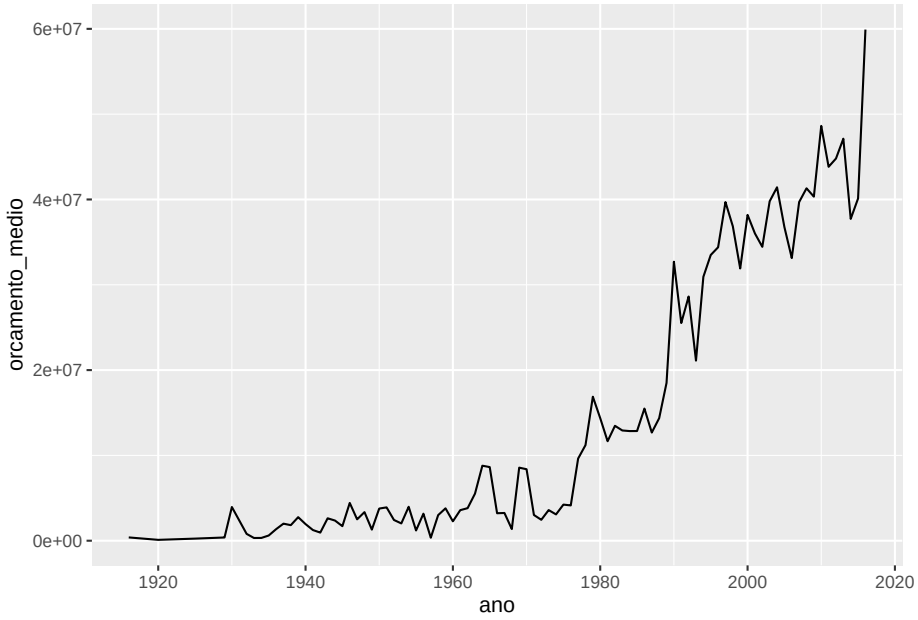


---

7. Faça um gráfico de linhas do orçamento médio dos filmes ao longo dos anos.

```
imdb %>%
  group_by(ano) %>%
  summarise(orcamento_medio = mean(orcamento, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = orcamento_medio)) +
  geom_line()
```

```
## Warning: Removed 1 row(s) containing missing values (geom_path).
```

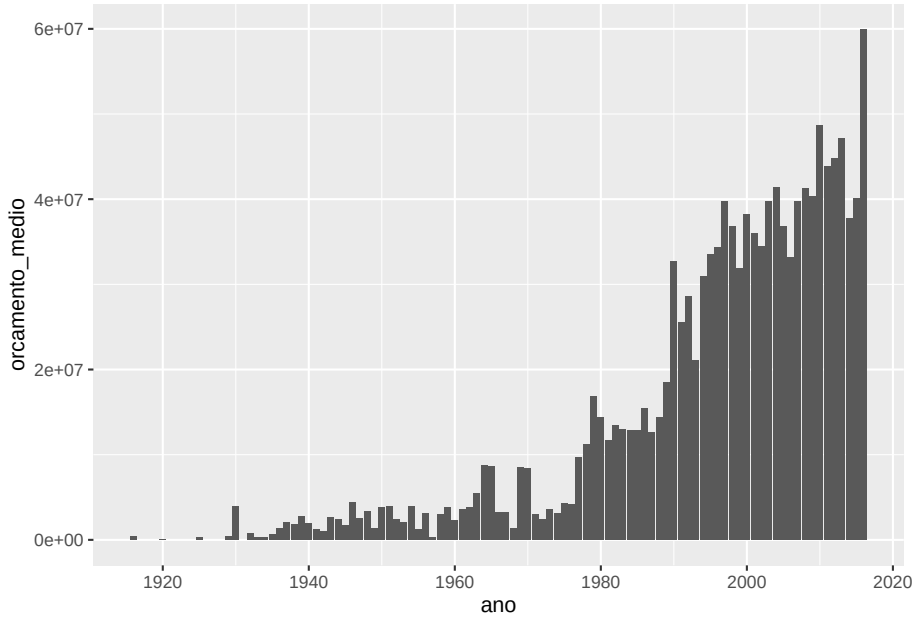


---

8. Transforme o gráfico do exercício anterior em um gráfico de barras.

```
imdb %>%
  group_by(ano) %>%
  summarise(orcamento_medio = mean(orcamento, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = orcamento_medio)) +
  geom_col()
```

```
## Warning: Removed 1 rows containing missing values (position_stack).
```



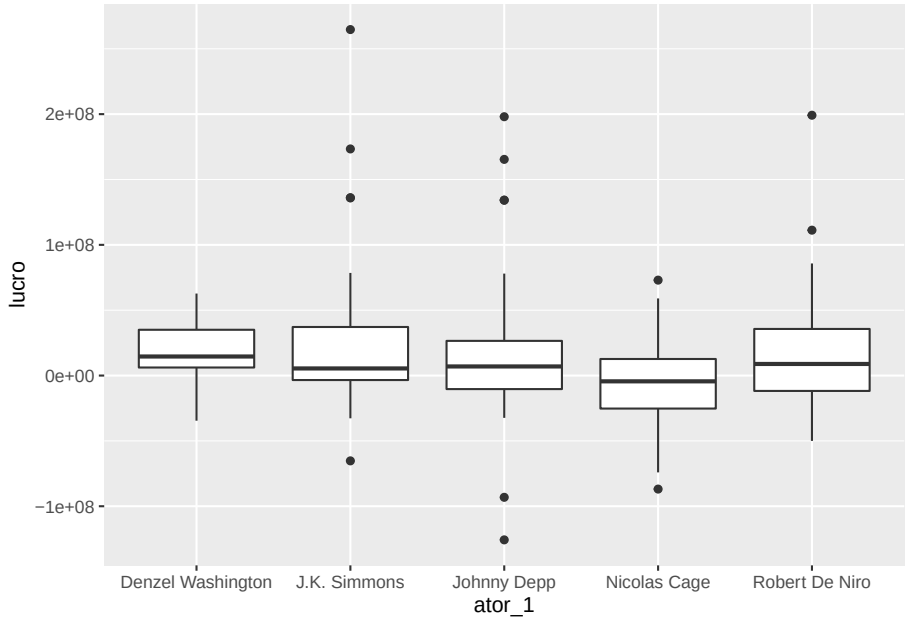
---

9. Descubra quais são os 5 atores que mais aparecem na coluna ator_1 e faça um boxplot do lucro dos filmes desses atores.

```
atores <- imdb %>%
  count(ator_1) %>%
  top_n(5, n) %>%
  pull(ator_1)

imdb %>%
  filter(ator_1 %in% atores) %>%
  mutate(lucro = receita - orcamento) %>%
  ggplot(aes(x = ator_1, y = lucro)) +
  geom_boxplot()
```

```
## Warning: Removed 6 rows containing non-finite values (stat_boxplot).
```



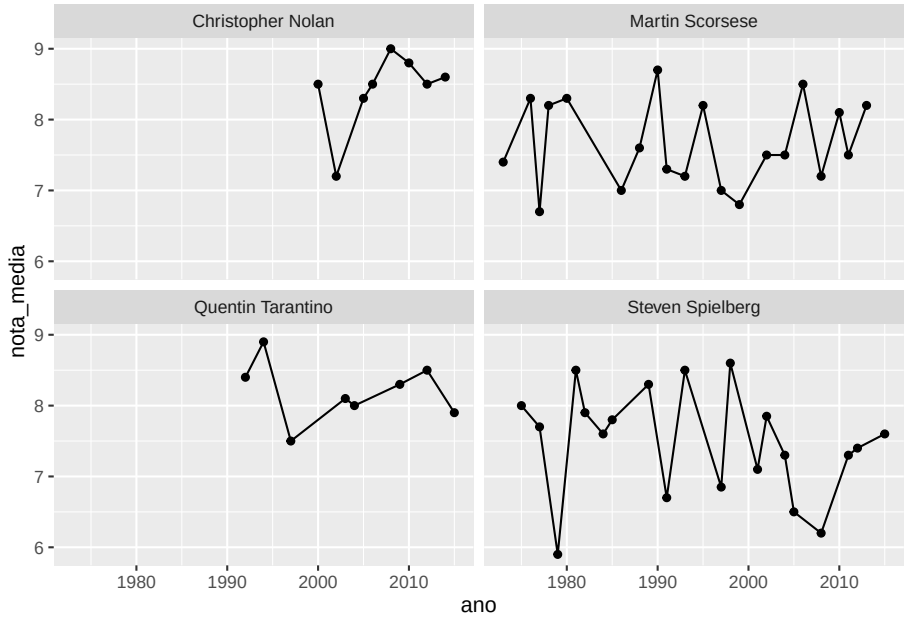
---

10. Com base no código abaixo, resolva os itens a seguir.

```
diretores <- c(
  "Steven Spielberg",
  "Quentin Tarantino",
  "Christopher Nolan",
  "Martin Scorsese"
)

imdb %>%
  filter(diretor %in% diretores) %>%
  group_by(ano, diretor) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = nota_media)) +
  geom_point() +
  geom_line() +
  facet_wrap(vars(diretor))
```

```
## `summarise()` has grouped output by 'ano'. You can override using the `.groups` argu
```



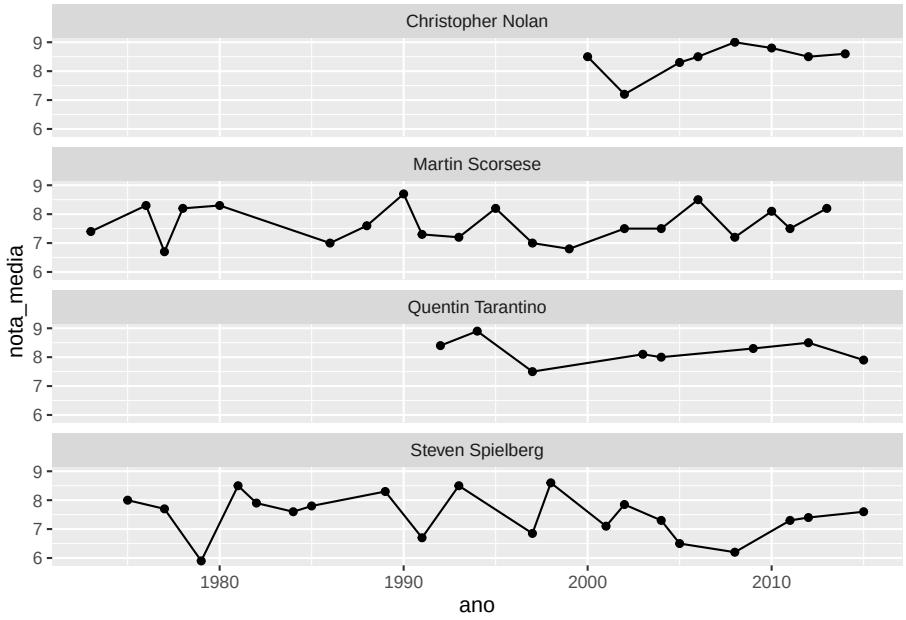
a. Analisando o gráfico gerado, descreva o que a função `facet_wrap()` faz.

A função `facet_wrap()` replica o gráfico especificado para cada categoria de uma coluna.

.b Utilize os argumentos nrow e `ncol` da função `facet_wrap()` para colocar os quatro gráficos em uma única coluna.

```
imdb %>%
  filter(diretor %in% diretores) %>%
  group_by(ano, diretor) %>%
  summarise(nota_media = mean(nota_imdb, na.rm = TRUE)) %>%
  ggplot(aes(x = ano, y = nota_media)) +
  geom_point() +
  geom_line() +
  facet_wrap(vars(diretor), ncol = 1)
```

```
## `summarise()` has grouped output by 'ano'. You can override using the `.groups` argument.
```



---

11. Resovla os itens a seguir para fazer um gráfico de barras da frequência de filmes com nota maior que 8 ao longo dos anos.

a. Crie uma nova coluna na base IMDB indicando se a nota de um filme é maior que 8 ou não.

```
imdb_notas_8 <- imdb %>%
  mutate(nota_maior_que_8 = ifelse(nota_imdb > 8, "Nota maior que 8", "Nota menor que 8
```

b. b. Utilizando a coluna criada em (a) crie uma tabela com o número anual de filmes com nota maior 8.

```
tabela <- imdb_notas_8 %>%
  filter(nota_maior_que_8 == "Nota maior que 8") %>%
  count(ano)

tabela
```

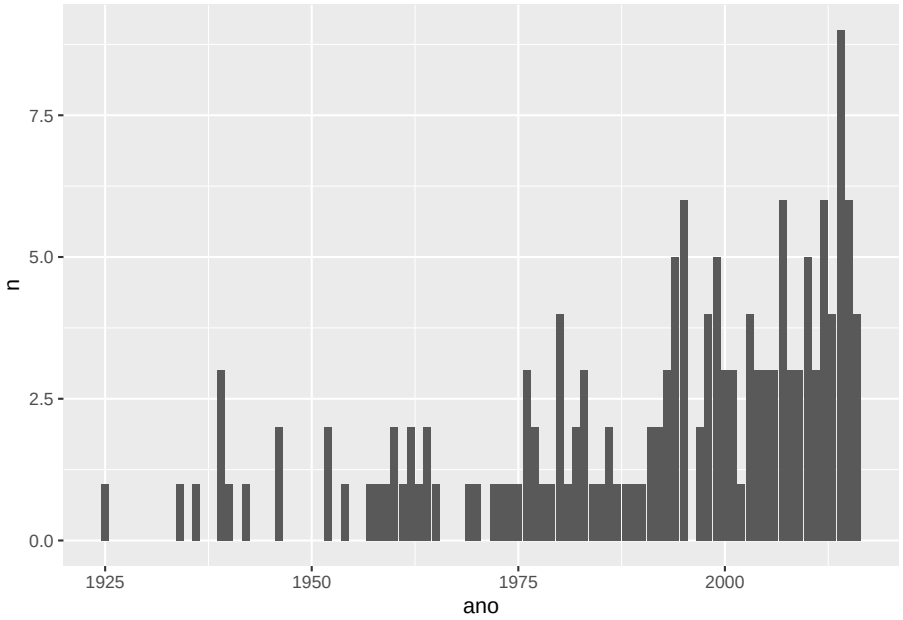
```
## # A tibble: 65 x 2
##      ano     n
##    <dbl> <int>
##  1  1925     1
```

```
##  2  1934     1
##  3  1936     1
##  4  1939     3
##  5  1940     1
##  6  1942     1
##  7  1946     2
##  8  1952     2
##  9  1954     1
## 10  1957     1
## # ... with 55 more rows
```

c. Utilize a tabela criada em (b) para fazer um gráfico de barras do número de filmes com nota maior que 8 ao longo dos anos.

```
tabela %>%
  ggplot(aes(x = ano, y = n)) +
  geom_col()
```

```
## Warning: Removed 1 rows containing missing values (position_stack).
```



13.9 Programação funcional (purrr)

```
library(dplyr)
library(purrr)
library(ggplot2)
```

1. Utilize a função `map()` para calcular a média de cada coluna da base `mtcars`.

```
map(mtcars, mean)
## $mpg
## [1] 20.09062
##
## $cyl
## [1] 6.1875
##
## $disp
## [1] 230.7219
##
## $hp
## [1] 146.6875
##
## $drat
## [1] 3.596563
##
## $wt
## [1] 3.21725
##
## $qsec
## [1] 17.84875
##
## $vs
## [1] 0.4375
##
## $am
## [1] 0.40625
##
## $gear
## [1] 3.6875
##
## $carb
## [1] 2.8125
```

2. Use a função `map()` para testar se cada elemento do vetor `letters` é uma vogal ou não. Dica: você precisará criar uma função para testar se é uma letra é vogal. Faça o resultado ser (a) uma lista de `TRUE/FALSE` e (b) um vetor de `TRUE/FALSE`.

```
testar_vogal <- function(x) {
  if(x %in% c("a", "e", "i", "o", "u")) {
    return(TRUE)
  } else {
    return(FALSE)
  }
}

# retornando uma lista
map(letters, testar_vogal)
## [[1]]
## [1] TRUE
##
## [[2]]
## [1] FALSE
##
## [[3]]
## [1] FALSE
##
## [[4]]
## [1] FALSE
##
## [[5]]
## [1] TRUE
##
## [[6]]
## [1] FALSE
##
## [[7]]
## [1] FALSE
##
## [[8]]
## [1] FALSE
##
## [[9]]
## [1] TRUE
##
## [[10]]
## [1] FALSE
##
## [[11]]
```

```
## [1] FALSE
##
## [[12]]
## [1] FALSE
##
## [[13]]
## [1] FALSE
##
## [[14]]
## [1] FALSE
##
## [[15]]
## [1] TRUE
##
## [[16]]
## [1] FALSE
##
## [[17]]
## [1] FALSE
##
## [[18]]
## [1] FALSE
##
## [[19]]
## [1] FALSE
##
## [[20]]
## [1] FALSE
##
## [[21]]
## [1] TRUE
##
## [[22]]
## [1] FALSE
##
## [[23]]
## [1] FALSE
##
## [[24]]
## [1] FALSE
##
## [[25]]
## [1] FALSE
##
## [[26]]
```

```
## [1] FALSE

# retornando um vetor (2 soluções equivalentes)
map(letters, testar_vogal) %>% flatten_lgl()
##  [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [13] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [25] FALSE FALSE
map_lgl(letters, testar_vogal)
##  [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [13] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [25] FALSE FALSE
```

---

3 Faça uma função que divida um número por 2 se ele for par ou multiplique ele por 2 caso seja ímpar. Utilize uma função `map` para aplicar essa função ao vetor `1:100`. O resultado do código deve ser um vetor numérico.

```
operacao <- function(x) {
  if (x %% 2 == 0) {
    return(x / 2)
  } else {
    return(x * 2)
  }
}

operacao(2)
## [1] 1
operacao(3)
## [1] 6

map_dbl(1:100, operacao)
##   [1]   2   1   6   2  10   3  14   4  18   5  22   6  26   7  30   8  34   9
##  [19]  38  10  42  11  46  12  50  13  54  14  58  15  62  16  66  17  70  18
##  [37]  74  19  78  20  82  21  86  22  90  23  94  24  98  25 102  26 106  27
##  [55] 110  28 114  29 118  30 122  31 126  32 130  33 134  34 138  35 142  36
##  [73] 146  37 150  38 154  39 158  40 162  41 166  42 170  43 174  44 178  45
##  [91] 182  46 186  47 190  48 194  49 198  50
```

---

4. Use a função `map()` para criar gráficos de dispersão da receita vs orçamento para os filmes da base `imdb`. Os filmes de cada ano deverão compor um gráfico diferente. Faça o resultado ser (a) uma lista de gráficos e (b) uma nova coluna na base `imdb` (utilizando a função `tidyr::nest()`).

```
imdb <- readr::read_rds("imdb.rds")

# gerando uma lista de gráficos

fazer_grafico <- function(tab, ano_) {
  tab %>%
    filter(ano == ano_) %>%
    ggplot(aes(x = orcamento, y = receita)) +
    geom_point()
}

anos <- unique(imdb$ano)

graficos <- map(anos, fazer_grafico, tab = imdb)

# gerando uma coluna na tabela imdb
fazer_grafico2 <- function(tab) {
  tab %>%
    ggplot(aes(x = orcamento, y = receita)) +
    geom_point()
}


imdb_com_graficos <- imdb %>%
  group_by(ano) %>%
  tidyr::nest() %>%
  mutate(
    grafico = map(data, fazer_grafico2)
  )
```

---

5. Utilize a função `walk` para salvar cada ano da base `imdb` em um arquivo `.rds` diferente, isto é, o arquivo `imdb_2001.rds`, por exemplo, deve conter apenas filmes do ano de 2001.

```
imdb <- readr::read_rds("imdb.rds")

salvar_base <- function(tab, ano_) {
  tab %>%
    filter(ano == ano_) %>%
    readr::write_rds(file = paste0("teste/imdb_", ano_, ".rds"))
}
```

```
anos <- imdb %>%
  filter(!is.na(ano)) %>%
  pull(ano)

walk(anos, salvar_base, tab = imdb)
```

13.10 NSE

```
library(dplyr)
library(ggplot2)
```

1. Escreva uma função que recebe uma *tibble* e uma coluna qualquer e devolve uma *tibble* sem as linhas contendo `NA` na coluna especificada.

```
tabela_qualquer <- tibble::tibble(
  x = c(1, 2, 3, NA, 6),
  y = c("a", "b", NA, "d", "e")
)

filtrar_na <- function(tab, coluna) {
  tab %>%
    filter(!is.na({{coluna}}))
}

filtrar_na(tabela_qualquer, x)
## # A tibble: 4 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     2 b
## 3     3 <NA>
## 4     6 e
filtrar_na(tabela_qualquer, y)
## # A tibble: 4 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     2 b
## 3    NA d
## 4     6 e
```

```
# string como argumento
filtrar_na <- function(tab, coluna) {
  tab %>%
    filter(!is.na(.data[[coluna]]))
}

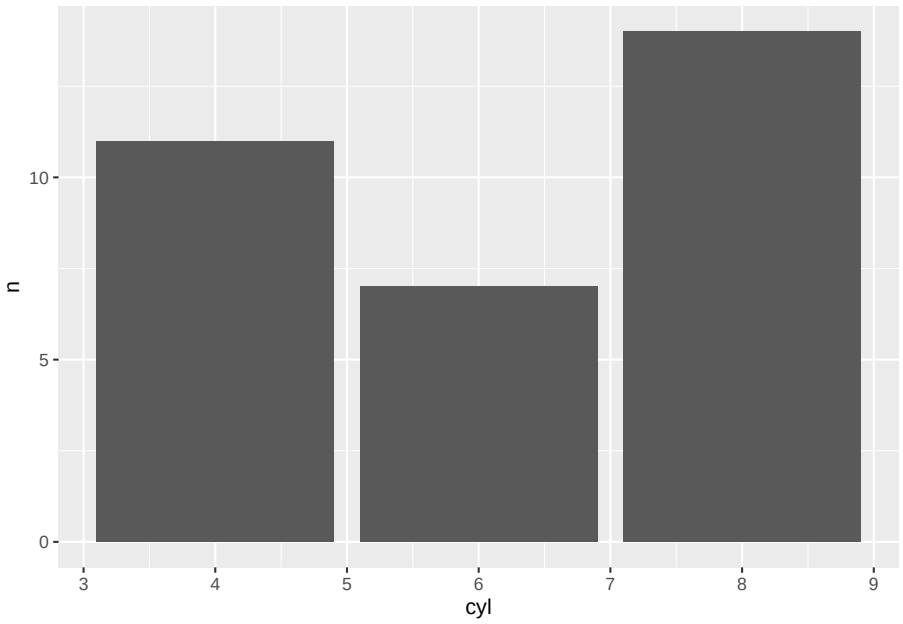
filtrar_na(tabela_qualquer, "x")
## # A tibble: 4 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     2 b
## 3     3 <NA>
## 4     6 e
filtrar_na(tabela_qualquer, "y")
## # A tibble: 4 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     2 b
## 3    NA d
## 4     6 e
```

---

2. Escreva uma função que recebe uma *tibble* e uma coluna e faz um gráfico de barras da frequência dessa coluna.

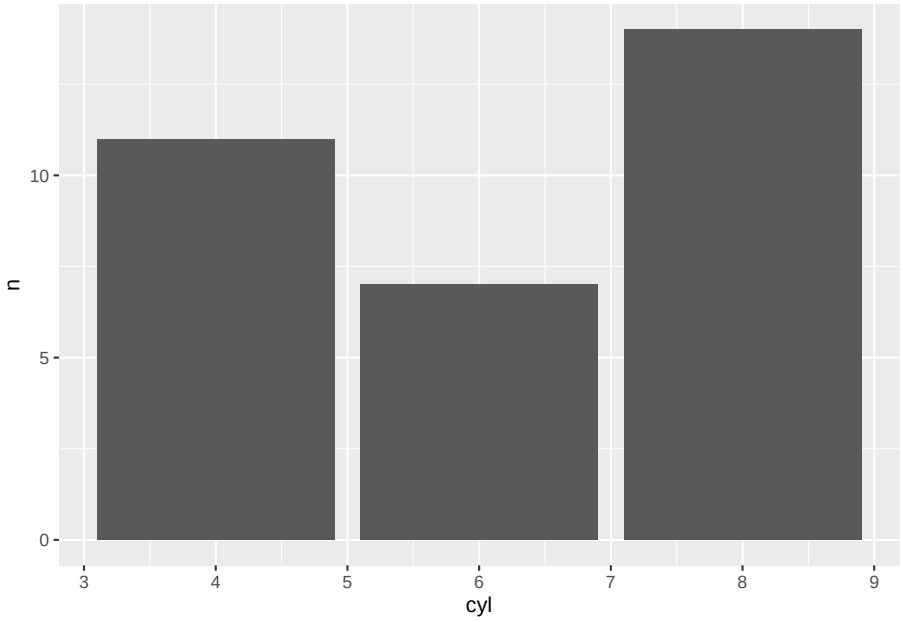
```
criar_grafico <- function(tab, coluna) {
  tab %>%
    count({{coluna}}) %>%
    ggplot(aes(x = {{coluna}}, y = n)) +
    geom_col()
}

criar_grafico(mtcars, cyl)
```




```
# string como argumento
criar_grafico <- function(tab, coluna) {
  tab %>%
    count(.data[[coluna]]) %>%
    ggplot(aes(x = .data[[coluna]], y = n)) +
    geom_col()
}

criar_grafico(mtcars, "cyl")
```

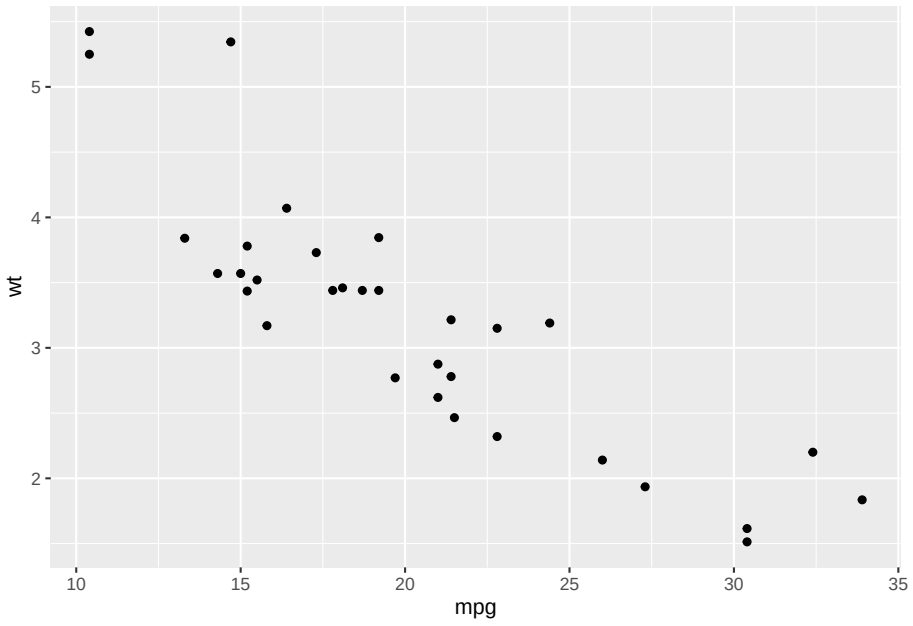


---

3. Escreva uma função que recebe uma *tibble* e o nome de duas colunas numéricas e faz um gráfico de dispersão dessas colunas.

```
fazer_grafico_disp <- function(tab, col1, col2) {
  tab %>%
    ggplot(aes(x = {{col1}}, y = {{col2}})) +
    geom_point()
}

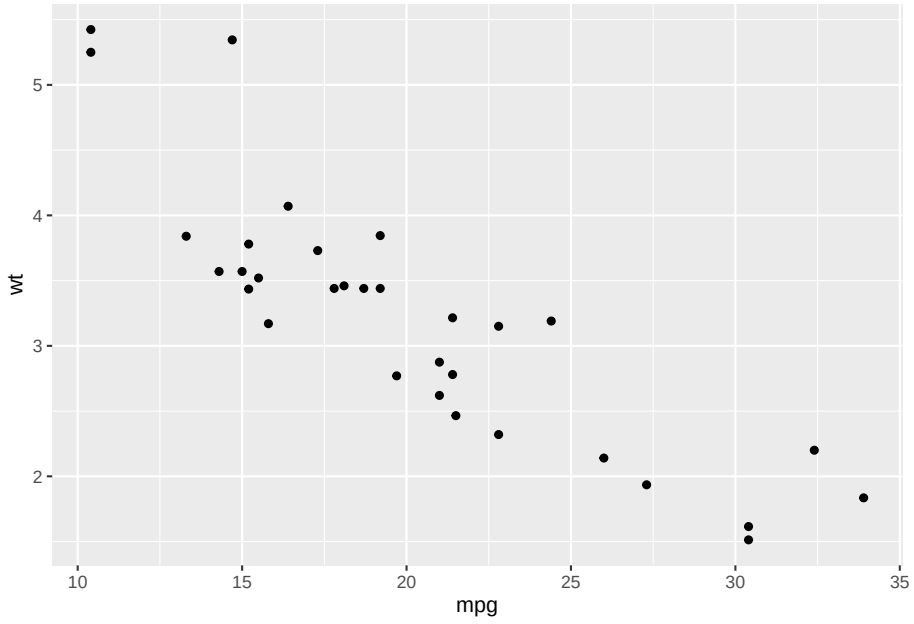
fazer_grafico_disp(mtcars, mpg, wt)
```




```
# string como argumento

fazer_grafico_disp <- function(tab, col1, col2) {
  tab %>%
    ggplot(aes(x = .data[[col1]], y = .data[[col2]])) +
    geom_point()
}

fazer_grafico_disp(mtcars, "mpg", "wt")
```



---

4. Escreva uma função que recebe uma *tibble*, o nome de uma coluna categótica e o nome de uma coluna numérica e devolva uma tabela com a média da coluna numérica para cada categoria da coluna categórica.

```
calcular_media <- function(tab, col_grupo, col_num) {
  tab %>%
    group_by({{col_grupo}}) %>%
    summarise(media = mean({{col_num}}, na.rm = TRUE))
}

calcular_media(mtcars, cyl, mpg)
## # A tibble: 3 x 2
##     cyl media
##   <dbl> <dbl>
## 1     4  26.7
## 2     6  19.7
## 3     8  15.1


# string como argumento
calcular_media <- function(tab, col_grupo, col_num) {
  tab %>%
```

```
    group_by(.data[[col_grupo]]) %>%
    summarise(media = mean(.data[[col_num]], na.rm = TRUE))
}

calcular_media(mtcars, "cyl", "mpg")
## # A tibble: 3 x 2
##     cyl media
##   <dbl> <dbl>
## 1     4  26.7
## 2     6  19.7
## 3     8  15.1
```

Referências

Abaixo, listamos todas as referências utilizadas na construção deste livro.

Allison Horst

As ilustrações contidas nesse livro são da Allison Horst.

- Acompanhe o trabalho dela: https://github.com/allisonhorst
- Repositório com as ilustrações: https://github.com/allisonhorst/stats-illustrations
- Siga a Allison no Twitter: https://twitter.com/allison_horst

Programação em R

- R for Data Sciente, do Garrett Grolemund e Hadley Wickham
- Advanced R, do Hadley Wickham
- R packages, do Hadley Wickham